MORGAN & CLAYPOOL PUBLISHERS

Microcontroller Education

Do it Yourself, Reinvent the Wheel, Code to Learn

Dimosthenis E. Bolanakis

SYNTHESIS LECTURES ON MECHANICAL ENGINEERING

Microcontroller Education

Do it Yourself, Reinvent the Wheel, Code to Learn

Synthesis Lectures on Mechanical Engineering

Synthesis Lectures on Mechanical Engineering series publishes 60–150 page publications pertaining to this diverse discipline of mechanical engineering. The series presents Lectures written for an audience of researchers, industry engineers, undergraduate and graduate students.
Additional Synthesis series will be developed covering key areas within mechanical engineering.

Microcontroller Education: Do it Yourself, Reinvent the Wheel, Code to Learn
Dimosthenis E. Bolanakis
2017

Unmanned Aircraft Design: A Review of Fundamentals
Mohammad Sadraey
2017

Introduction to Refrigeration and Air Conditioning Systems: Theory and Applications
Allan Kirkpatrick
2017

MEMS Barometers Toward Vertical Position Detecton: Background Theory, System Prototyping, and Measurement Analysis
Dimosthenis E. Bolanakis
2017

Vehicle Suspension System Technology and Design
Avesta Goodarzi and Amir Khajepour
2017

Engineering Finite Element Analysis
Ramana M. Pidaparti
2017

Microcontroller Education: Do it Yourself, Reinvent the Wheel, Code to Learn

Dimosthenis E. Bolanakis

www.morganclaypool.com

ISBN: 9781681731919 paperback
ISBN: 9781681731933 ebook

DOI 10.2200/S00802ED1V01Y201709MEC009

A Publication in the Morgan & Claypool Publishers series
SYNTHESIS LECTURES ON MECHANICAL ENGINEERING

Lecture #9
Series ISSN
Print 2573-3168 Electronic 2573-3176

Microcontroller Education

Do it Yourself, Reinvent the Wheel, Code to Learn

Dimosthenis E. Bolanakis

SYNTHESIS LECTURES ON MECHANICAL ENGINEERING #9

MORGAN & CLAYPOOL PUBLISHERS

ABSTRACT

Microcontroller education has experienced tremendous change in recent years. This book attempts to keep pace with the most recent technology while holding an opposing attitude to the *No Need to Reinvent the Wheel* philosophy. The choice strategies are in agreement with the employment of today's flexible and low-cost *Do-It-Yourself (DYI)* microcontroller hardware, along with an embedded C programming approach able to be adapted by different hardware and software development platforms. Modern embedded C compilers employ built-in features for keeping programs short and manageable and, hence, speeding up the development process. However, those features eliminate the reusability of the source code among diverse systems. The recommended programming approach relies on the motto *Code More to Learn Even More*, and directs the reader toward a low-level accessibility of the microcontroller device. The examples addressed herein are designed to meet the demands of *Electrical & Electronic Engineering* discipline, where the microcontroller learning processes definitely bear the major responsibility. The programming strategies are in line with the two virtues of C programming language, that is, the adaptability of the source code and the low-level accessibility of the hardware system. Some accompanying material of the book can be found at `http://bit.ly/mcu-files`.

KEYWORDS

microcontroller education, embedded C, ANSI C, microcontrollers, PIC, AVR

I've never seen any of the 7 wonders of
the world and from any available list.
I believe there are more though…
I see perfection and plenty in number 10
everyday…

To my wife, son, and daughter

Contents

Preface

Microcontroller education experienced tremendous change in recent years. For a few decades, the tutoring was mainly orientated toward:

(a) innovative teaching approaches for realizing the unstructured low-level programming techniques (e.g. the development of new simulator tools and/or methodologies for making the parallelization between the assembly and a higher-level programming language); and

(b) custom-designed hardware for avoiding focusing students' attention on the laboratory equipment instead of the coding processes (which would eventually direct students to illustrate the inner workings of the processor model).

For over a decade now, microcontroller programming in embedded C has effectively eliminated the assembly-level coding. Over the years, the associated C compilers have been (significantly) improved to generate more compact object code in sufficiently low execution time. Moreover, the emergence of several third-party companies focusing on the development of microcontroller-based systems and add-on boards has eliminated the need for custom-designed educational platforms.

Nowadays, microcontrollers' tutoring is experiencing the widespread dissemination of *Do-It-Yourself (DIY)* culture (such as the Arduino development platform). The idea of sharing open-source libraries and prototypes, which are addressed not only for engineers but also for hobbyists, has undoubtedly brought new directions in microcontroller education. Embedded C has been replaced with higher-level styles of programming, addressed to encrypt low-level practices with the microcontroller's hardware. And along with the available and ready-to-use collection of code and hardware modules, this culture sustains the development of microcontroller-based system within minutes. Based on the philosophy of avoiding the reinvention of the wheel while using the open-source examples available on the Internet, academic disciplines other than Electrical and Electronic Engineering can straightforwardly implement several microcontroller applications (able to serve the needs of their education).

This book attempts to keep pace with the most recent microcontroller technology while holding an opposing attitude to the *No Need to Reinvent the Wheel* philosophy. The choice strategies are in agreement with the employment of today's flexible and low-cost (DYI) microcontroller hardware, along with an embedded C programming approach, able to be adapted among different hardware and software development platforms. Modern embedded C compilers employ an abundance of built-in functions and pre-processor commands toward keeping programs short and manageable and, hence, speeding up the development process. However, those features eliminate one of the major virtues of C language, that is, the reusability of the source code

among diverse systems (a choice that serve well the needs of enterprises to encourage users to keep supporting their development products). The recommended programming approach relies on the motto *Code More to Learn Even More*, and directs the reader toward a low-level accessibility of the microcontroller device, which constitutes an additional virtue of C language programming.

Microcontroller applications nowadays involve several more tasks than just developing the firmware for the hardware device. In many cases the design of an application requires a software program, which is addressed to interface with the microcontroller device. For instance, a firmware obtaining data from a sensor device would—in many cases—be controlled by a graphical user interface (GUI) toward a real-time monitoring of the acquired data. The examples addressed herein are designed to meet the demands of Electrical and Electronic Engineering and allied disciplines, where the microcontroller learning processes definitely bear the major responsibility. The book could be considered a three-part project: the first part introduces readers to the recent trends and technologies of small (8-bit) microcontroller devices; the second part emphasizes hardware and software programming issues toward the code's adaptability among diverse development platforms; the final and concluding part focuses on a practical approach to modern microcontroller-based instrumentation systems, in consideration of the hardware and software co-design processes of a microcontroller application. The practical examples are orientated to the most popular PIC and AVR microcontroller units as well as to freeware development tools.

Dimosthenis E. Bolanakis
October 2017

Acknowledgments

I wish to thank the publisher Joel Claypool for the opportunity to write this book. Special thanks go to my editor Paul Petralia and his efficient team (CL Tondo, David Schlangen, Brent Beckley, and Melanie Carlson) for all the great work and cooperation toward the completion of this book.

Dimosthenis E. Bolanakis
October 2017

CHAPTER 1

Historical Review

This chapter provides a historical review to microcontroller (μC) education in consideration of the major hardware and software advancements of technology over the years.

1.1 THE ADVANCEMENT OF μC TECHNOLOGY AND ITS EFFECT IN EDUCATION

In recent decades, microcontroller (μC) education (as in the majority of technology-orientated course) has been strongly influenced by the advancement of technology. Microcontrollers constitute single-chip (complete) computer systems incorporating processor, memory, input/output (i/o) peripheral devices, as well as a custom-designed firmware code for the control of hardware. Hence, technology advancement can be divided into two primary categories: (a) hardware-related advancement and (b) software-related advancement.

The former has to do more about the progress in embedded systems design over the years, which nowadays experiences the widespread dissemination of *do-it-yourself* (DIY) culture, and the several available and ready-to-use hardware development platforms. The latter is associated with the progress in programming languages and software tools, and their subsequent effect on the source code development method for programming the *microcontroller unit* (MCU). Historically, microcontroller programming can be viewed as having progressed in three periods, that is (a) the era of *assembly-level programming*, (b) the era of *high-level programming* (*C*, *Pascal*, *Basic*), and (c) the era of *object-oriented programming* (*Arduino*, *MicroPython*).

The assembly-level programming approach has (already from the previous decade) been eliminated, while the high-level and object-oriented programming approaches, mainly *embedded C* and *Arduino*, respectively, are the dominant microcontroller programming methods of our age. It could be said that these two programming approaches have divided today's community into engineers and hobbyists, respectively, according to the share they hold in the employed programming methodology. However, both groups experience the utilization of today's DIY hardware culture of the abundant deliverable mezzanine cards, which are plugged to the microcontroller-based development platforms, and render feasible the implementation of sophisticated embedded systems within minutes.

Until the early 2000s the non-volatile[1] memory of many MCUs was still of the *erasable programmable read-only memory* (EPROM) type, recognized by the identical transparent quartz

[1]The *non-volatile* term refers to the type of memory that is able to retain its data when the power supply is turned off and, hence, it is addressed to hold microcontroller's firmware/object code (aka *program memory*).

window at the top of the package (Figure 1.1a). The firmware update to the microcontroller device was a particularly time-consuming procedure, as it required exposing the microcontroller's transparent window to *ultraviolet* (UV) light for about 30 min. Due to this particular erasing method, this type of memory is also known as UVPROM.

While working with real hardware increased students' interest for the course and gave them a sense of accomplishment [1], the processes related to the earlier microcontroller technology prevented instructors from deciding on hands-on experiments for the laboratory training. The time-consuming updates of the firmware code created the need for several available microcontroller devices in the classroom, as well as an additional UV lamp long enough to accommodate, and simultaneously erase, many EEPROM devices (Figures 1.1b,c). The workload and laboratory cost were further increased by a separate board system for programming the microcontroller device. Accordingly: an error in the firmware (that would need revision and re-examination of the code) required to: (a) removal of the MCU from the target board; (b) insertion of the MCU into the EEPROM eraser for about 30 min; (c) connection of the MCU to the programmer board system (Figure 1.1d) for loading the updated firmware in memory (regularly through an RS232 serial interface); and (d) attachment of the MCU back to the student's hardware experimentation platform for testing the updated firmware code. Obviously, such procedures were impossible with the time constraints of a two/three-hour laboratory session.

Those issues were eliminated when EPROM was replaced with the Flash type of microcontroller memory. This type of technology (constituting the dominant program memory of today's MCUs) not only sped up the firmware updating procedure, but also eliminated the need for external programmers. The *in-system programming* (ISP) feature of microcontroller devices allowed firmware updates to be performed directly to the target board. However, the building up of a customized microcontroller circuit (possibly to a breadboard) during the lesson was still a time-consuming procedure that could draw students' attention away from the goal of the course. Therefore, instructors were either selecting a microcontroller learning system based on commercial and/or customized simulators [2], or they were orientated toward the design of a flexible educational board systems (Figure 1.2a) with ISP capability, able to assure a potential number of experiments on microcontrollers [3]. Of course, there was always the possibility of selecting a commercial board system for the lab training activities. Yet, the available hardware platforms of that period were rather costly and difficult to operate [4].

Today's commercial experimentation platforms on microcontrollers are much more flexible and less expensive. The designer regularly spends only a few tens of dollars to purchase a microcontroller-based development system, which is delivered with a bootloader,[2] and receives power from the host PC through a regular *universal serial bus* (USB) cable (Figures 1.2b,c). Successive firmware updates via the USB port can be achieved within seconds, while the header connectors that have been generally installed onto the microcontroller board admit miscellaneous

[2]The *bootloader* term refers to the piece of firmware in microcontroller's program memory that is capable of updating the application firmware in the MCU. Updates in the bootloader firmware are only possible through a separate programmer board.

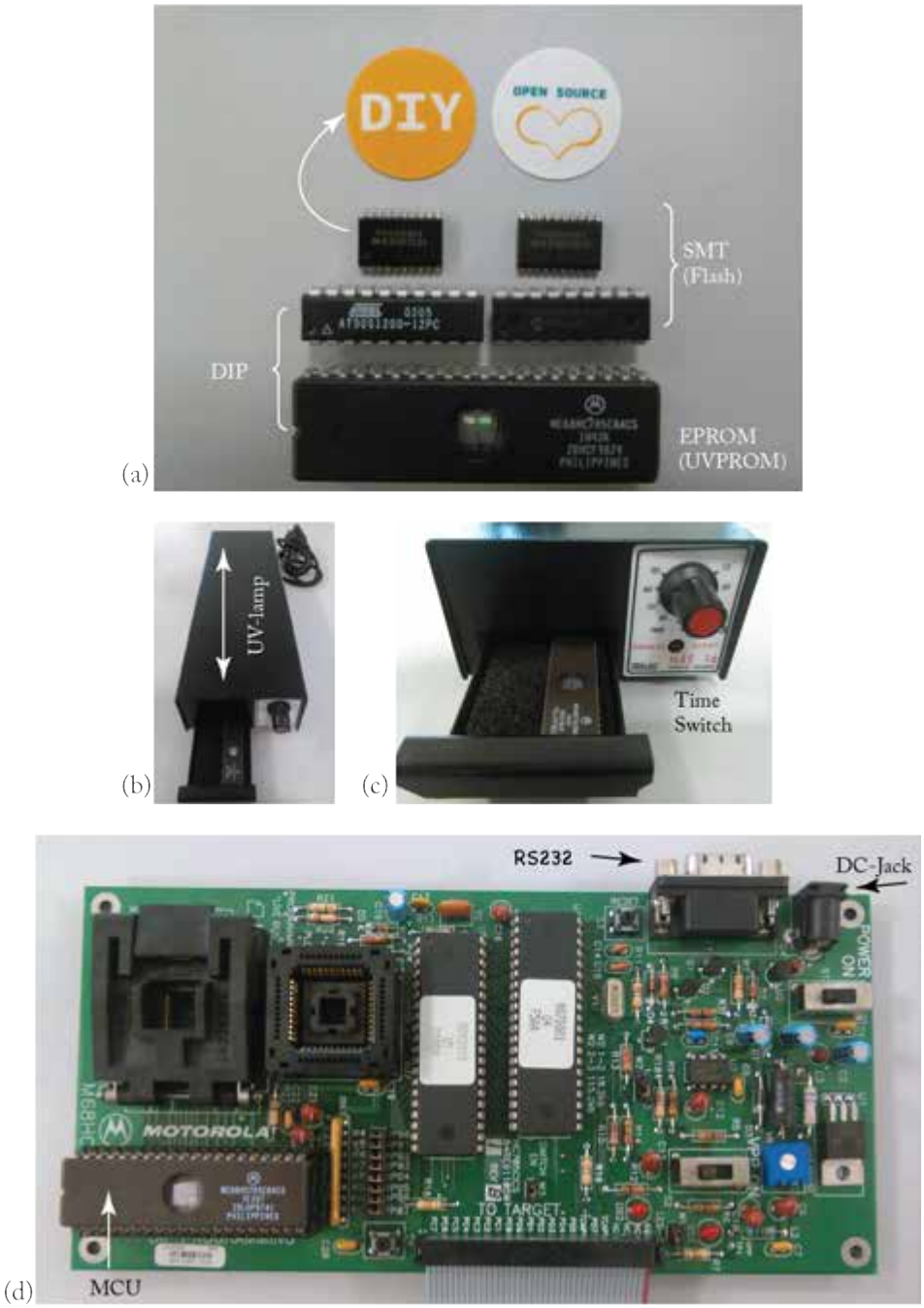


Figure 1.1: Hardware advancement of micontroller technology: from UVPROM to Flash.

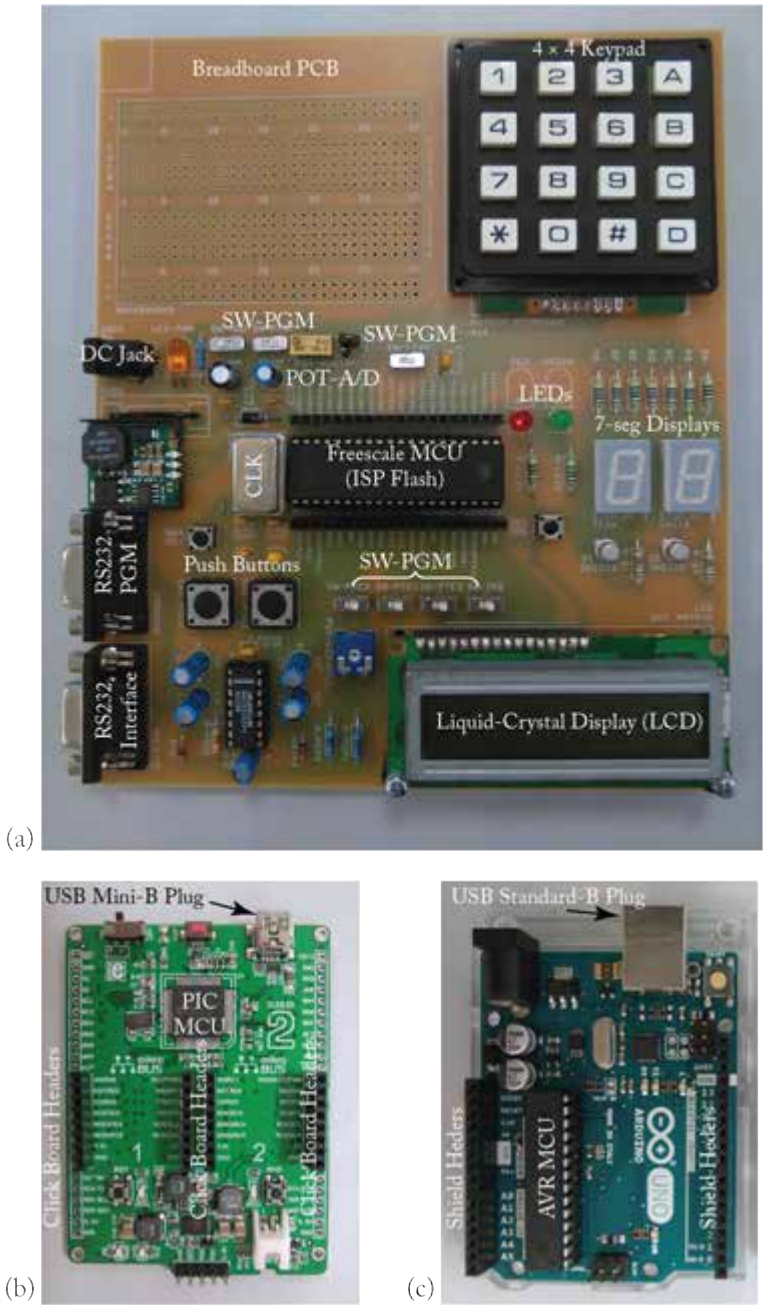


Figure 1.2: Hardware advancement of MCU technology: from serial ISP to USB bootloader.

mezzanine cards (for tasks related to sensing, storage, interfacing, displaying, etc.) toward the implementation of DYI applications.

In addition, the passage from *dual in-line package* (DIP) to *surface-mount technology* (SMT) method of producing *integrated circuits* (ICs) considerably decreased the dimensions of IC packages (Figure 1.1a). Therefore, modern microcontroller and mezzanine board systems may occupy several electronic components that guarantee the implementation of cutting-edge systems, applying to the *Internet of Things* (IoT), robotics, smart-home applications, industrial automations, etc. Accordingly, the recent trends in microcontroller technology exhibit no concern about conducting hands-on experiments in the classroom. The only worry one may have would be to make an accurate and efficient selection of the hardware systems, from the abundance of the today's available boards and add-on modules [5, 6].

Figure 1.3 presents five of today's most popular vendors/suppliers of microcontroller-based systems. Positive and negative issues are in agreement with the selection of one of the alternatives. MikroElektronika's hardware tools can be considered a rather increase in cost, but the corporation offers an abundance of peripheral devices (somewhat 300 different modules, aka *click boards*), requiring no extra involvement in the hardware connections. It also delivers the so-called *click shields* which allow connectivity of the click boards onto third-party microcontroller and single-board computers (such as the Arduino Uno, Rasberry Pi, etc.). MikroElektronika [7] is orientated to the Microchip PIC microcontrollers [8] and provides appropriately authored libraries and example codes for their software tools (e.g., microC, microBasic, etc.).

Olimex [9], on the other hand, covers diverse vendors of MCUs (e.g., PIC, AVR, STM, MSP430), but it is based on the philosophy of *Development* & *Proto* boards. The former incorporate the microcontroller device along with the proper peripheral subsystem(s) toward the implementation of an application-orientated system (like the RF board system, depicted in Figure 1.3, intended for wireless communications). The latter incorporates the MCU along with a proto-board embedded onto the same *printed-circuit board* (PCB), thereby requiring extra hardware involvement (i.e., soldering) for the implementation of a microcontroller-based system. The hardware tools delivered by Olimex could be considered of low cost, however, little description is provided with the accompanied documentation. It could generally be said that Olimex and MikroElektronika are two corporations that are primarily addressed for engineers, where the former constitutes a cost-effective option and the latter a more flexible alternative.

Arduino [10] constitutes perhaps the most popular hobbyists-orientated environment for microcontroller applications. It delivers microcontroller development platforms and peripheral board systems (aka *shields*) of middle cost. However, one may find an abundance of third-party compatible board systems of considerably low cost (such as the *Funduino Uno* which is identical to the *Arduino Uno* version). The hardware tools apply to the AVR[3] MCUs [11], while the Arduino *integrated development environment* (IDE) can be addressed both for embedded C and Arduino (i.e., object-oriented) programming. Several third-party corporations are orientated

[3]It is worth mentioning that Microchip's acquisition of Atmel concluded in 2016.

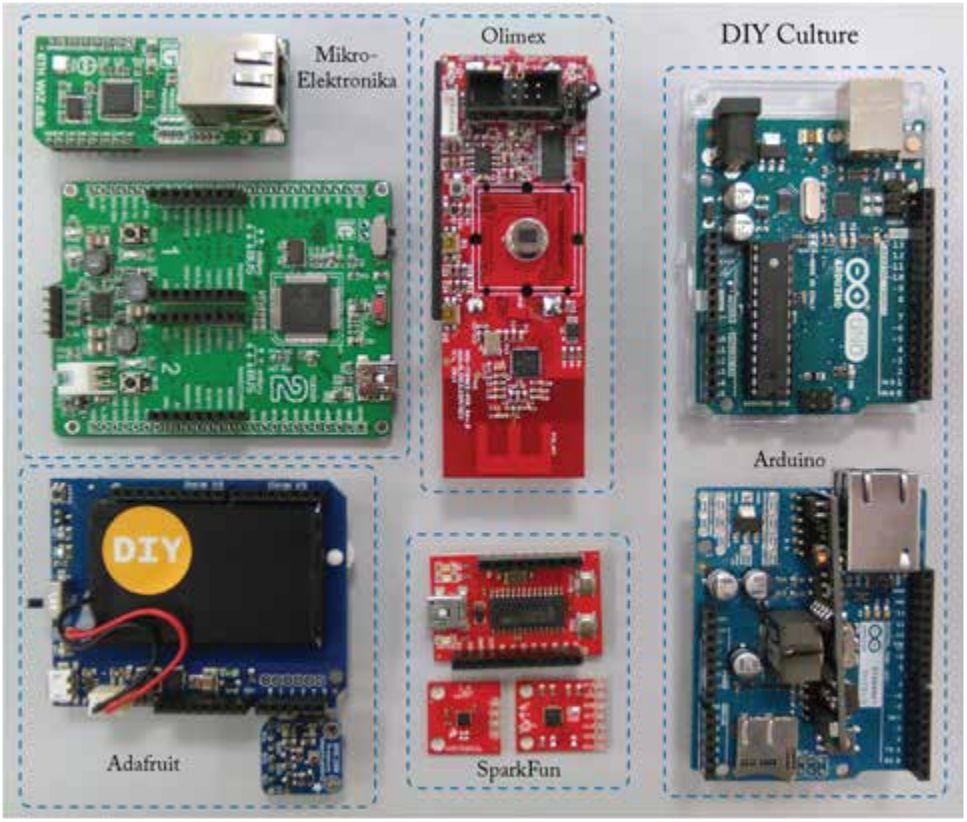


Figure 1.3: Modem microcontroller (hardware) tools: the DIY culture.

to the design and/or distribution of Arduino-based hardware tools. Two comparable examples are the Adafruit [12] and SparkFun [13]. Both companies deliver low-to-middle cost breakout boards that require (in a few cases) hardware involvement, while the former supplies additional customized (and Arduino-IDE programmable) boards, as well as an adequate number of particularly descriptive notes for the sample codes and products.

The application examples addressed herein apply to MikroElektronika and Arduino hardware tools. The motivation is in line with the flexibility and availability of the so-called *click boards*, as well as the flexibility of (the freeware) Arduino IDE in developing both embedded C and object-oriented code. The microcontroller programming language addressed for the book examples is *embedded C*, and the programming approach is directed toward (a) the code's adaptability among diverse development platforms (for both PIC and AVR devices), and (b) the low-level practices with the microcontroller's hardware. The flexibility in switching between PIC and AVR microcontroller devices with the utilization of MikroElektronika's *click board* and *click*

shields is illustrated in Figure 1.4. The adaption of the click shield (i.e., the blue color PCB) on the top of the Arduino Uno allows the latter development system to interface click boards toward a communication with a MEMS sensor, a microSD memory, or over Ethernet, WiFi, 802.15.4, and Bluetooth protocols (all of which are depicted in Figure 1.4), as well as the implementation of several other challenging applications.

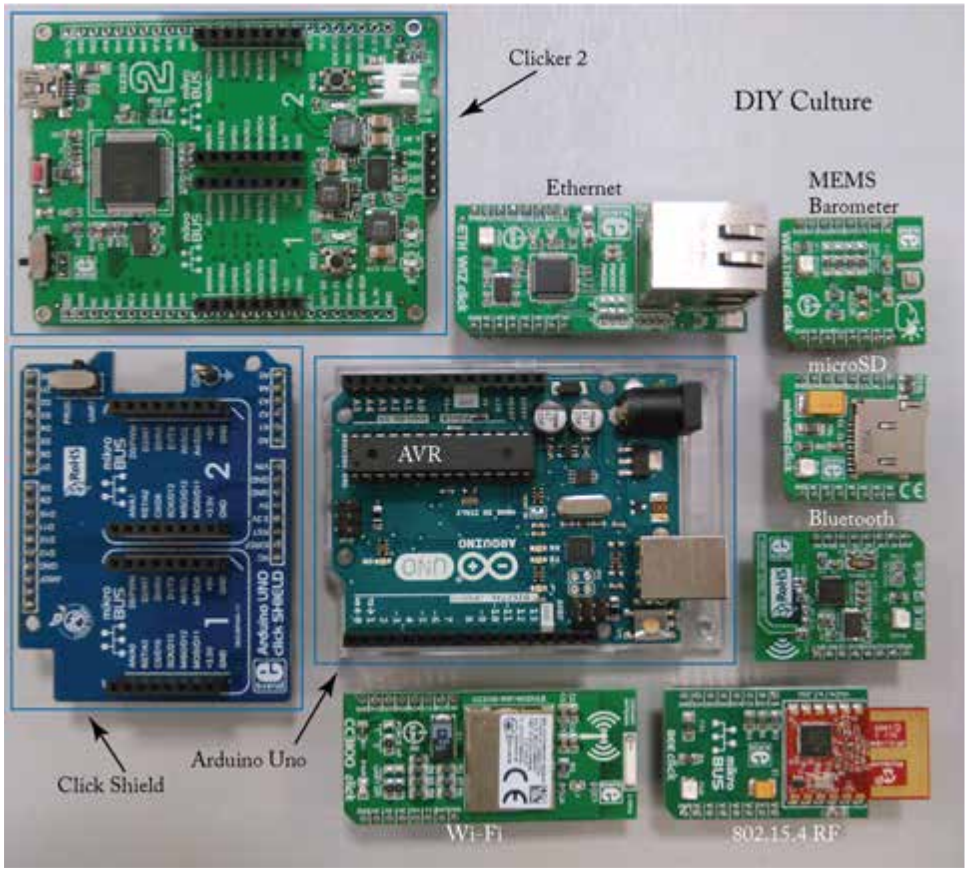


Figure 1.4: MikroElectronika and Arduino hardware boards: switch between AVR and PIC devices.

1.2 PREVIOUS AND RECENT RESEARCH STUDIES IN μC EDUCATION

Despite the endeavor on the design and development of flexible educational board systems for the laboratory training, research in microcontroller education in recent decades was orientated

toward facilitating the assembly language learning processes. Because the assembly-level programming allows total control over each machine instructions, the associated tutoring provided a deep insight into the inner workings of the processor model [14, 15]. However, the lack of structures in this programming approach rendered the assembly language a difficult-to-follow tutoring. Thereby, instructors often worked toward (a) the designing of customized simulators which helped in clarifying the assembly-level programming concepts [2] and (b) the devising of teaching approaches for bridging the gap between low-level and higher-level programming [16, 17].

The low-level system accessibility of the processor hardware leaves enough room for scientific as well as educational research. Indicative examples of scientific research are in line with: (a) the measurement of energy consumption according to the assembly instruction being executed [18, 19]; and (b) the identification of malicious executable files through the comparison of an assembly instruction sequence (aka *malware signature*) [20]. In consideration of the educational research, the assembly-level programming sustained the practical study of several interesting issues, such as the *endianness* representation in arithmetic techniques [21], the addressing modes of the processor's *instruction set architecture* (ISA) model [22], etc.

As the assembly language gave its place to higher-level methods of programming, the low-level practices with the processor model and their subsequent research practices in education were also eliminated. Today's research in microcontroller education can be classified into three categories; as outlined in the subsequent three sub-sections.

1.2.1 PEDAGOGIES IN MICROCONTROLLER LEARNING

Because of their technological nature, microcontroller-based tutoring systems incorporate the design processes of embedded systems and instructors often engage students in topics for solving design problems. Thereby, several propositions in the literature are organized around a *project-based learning* (PBL) approach toward microcontroller learning [23, 24, 25]. Based on the *active learning* method of asking questions and sharing ideas around their own problem-solving approach, students are directed toward a deeper understanding of the involved topics through this particular pedagogy [26].

1.2.2 MICROCONTROLLER EDUCATION IN DIVERSE DISCIPLINES/LEVELS OF EDUCATION

Traditionally, microcontroller education is intended for Electrical and Electronic engineers. However, as a consequence of its potential utilization in miscellaneous systems and applications, microcontroller technology often migrates to serve the particular needs of education in other diverse disciplines. Indicative examples are in agreement with the implementation of:

(a) equipment for chemical laboratories [27, 28] (e.g., photometers, phH meters, etc.);

(b) instrumentation systems for analyzing phenomena in physics education [29, 30];

(c) mechatronic prototype designs for the education of mechanical engineers [31, 32]; and

(d) hardware and software tool chain toward teaching signal processing and analog electronics [33]. It is worth noting that the DYI culture has undoubtedly played an important role in facilitating the adoption of microcontroller technology in disciplines other than Electrical and Electronic Engineering [34]. In addition, the minimization of time/effort needed to develop a MCU-based application (because of the advancement of today's hardware tools), as well as the collaborative (DYI) spirit of sharing open-source firmware examples, has rendered feasible the migration of technology in secondary education as well [35, 36]. This opportunity is further supported by the simplicity of the *drag-and-drop* programming methods for embedded systems (based on the popular *Scratch*[4] educational language [37]), which provide an appealing interface to the children [38]. Particular examples are the (a) *Ardublock* [39] plug-in application for Arduino IDE and (b) *S4A* [40] Scratch modification for the programming of the Arduino hardware (Figure 1.5).

1.2.3 MICROCONTROLLERS AS AN INTEGRAL PART OF TEACHING EMBEDDED SYSTEMS SOLUTIONS

Another category of research in education applies to the tutoring of microcontroller technology as an integral part of modern embedded systems solutions. Some of the contemporary applications are associated with the education of IoT [41, 42] and robotics for universities [43] and schools [44, 45]. Because of the widespread dissemination of robotics in education, several development tools applying to the *drag-and-drop* method of programming are available to the designer. For instance, *12Blocks* [46] constitutes a development environment for programming popular robots, while *Google Blockly* [47] library adds a visual code editor to a web browser and can export blocks to JavaScript code, Python code, etc.

One reasonable conclusion to be drawn from the above categories is that while microcontrollers' tutoring is primarily intended for the education of Electrical and Electronic engineers, lately little or no research is being conducted exclusively on this domain. Rather, the corresponding research thrives in allied science and engineering institutions of higher education (and lately in secondary schools, as well) mainly because of the advancement and widespread dissemination of DIY culture in microcontroller technology. One possible reason relies on the need to bear the major responsibility in a microcontroller-based tutoring system suitable for Electrical and Electronic engineers, which is opposed to the *No Need to Reinvent the Wheel* philosophy. This philosophy is not only present in the modern programming methods that tend to hide the low-level practices with the microcontroller's hardware, but is also found in the built-in features of today's commercial C compilers for microcontrollers (i.e., template functions and pre-processor commands), which eliminate the code's portability among different software packages. To this end the choice strategies of this book are in agreement with employment of today's flexible and low-cost DYI hardware for the application examples, along with an embedded C programming

[4]The *Scratch* drag-and-drop programming language developed by the Lifelong Kindergarten Group at the Massachusetts Institute of Technology (MIT) is addressed as a first step in programming (mainly in K-12 schools).

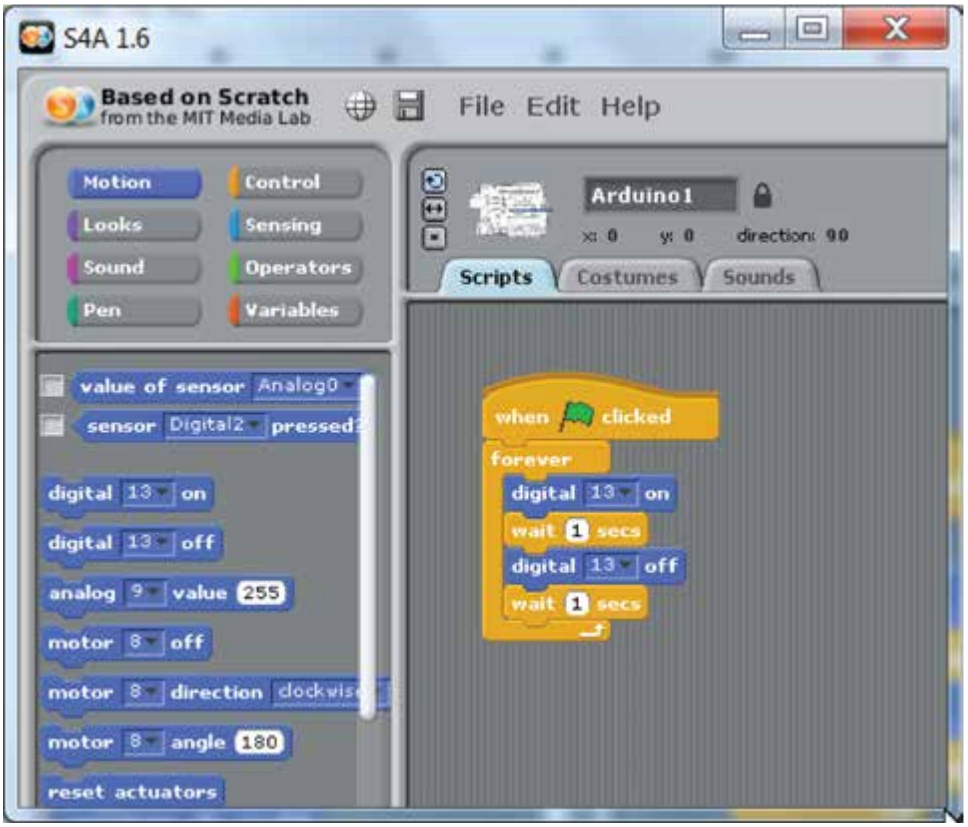


Figure 1.5: Drag-and-drop programming with Scratch 4 Arduino (S4A): blinking LED.

approach devised toward (a) adaptability among diverse hardware and software development platforms, and (b) low-level accessibility of the microcontroller device (which constitutes a very good example of effective practice in microcontroller education). Based on the programming perspective of *Code More to Learn Even More*, the book's viewpoint is shaped as *Do it Yourself, Reinvent the Wheel, Code to Learn*.

CHAPTER 2

Micro-Controller Fundamentals

This chapter introduces readers to the basic elements that compose microcontroller's hardware as well as to the role they hold in the system, without focusing on a particular vendor's architecture.

2.1 DEEP INSIGHT INTO THE UNDERLYING MECHANISMS OF HARDWARE

Microcontrollers constitute single-chip computer systems and, as in every computer system, they consist of the (a) *central processing unit* (CPU), (b) memory, and (c) i/o peripheral devices. Small MCUs (i.e., 8-bit devices) regularly do not run a *real-time operating system* (RTOS). An application firmware developed by the designer is able to obtain total control over the hardware elements. The firmware is held in program memory, and because we want to retain the application code in memory even when the power is turned off, this type of memory is also known as *non-volatile memory* (NVM). NVM inside modern microcontroller devices is of Flash technology, which can be electrically erased and reprogrammed using either a different hardware device (that is, a stand-alone programmer attached to the application board system), or a small portion of firmware that has been pre-installed in Flash memory. The latter firmware code is regularly referred to as *bootloader* and can be installed, removed, or updated in Flash memory, only through the stand-alone programmer device.

Figure 2.1 presents PICkit3 [48] stand-alone programmer delivered by Microchip. The programmer is controlled through the USB port, of a host PC running MPLAB X IDE [49] on Windows *operating system* (OS). In this figure example, the programmer is connected to the *in-circuit serial programming* (ICSP) connector of MOD-ZIGBEE-PIR [50] development module (delivered by Olimex). It is worth noting that PICkit3 is also an *in-circuit debugger* (ICD), i.e., it uses the special debug hardware of the target chip which allows us to run the program code while at the same time inspecting the values of internal registers and, hence, it renders feasible a debugging process of the application firmware.

The firmware execution inside of the Flash memory is in agreement with the conventional and familiar *sequential programming* approach. The sequential paradigm relies on the execution of statements according to their textual order in the source code. Since the machine instructions held in *program memory* are sequentially (and not randomly) fetched and executed by the

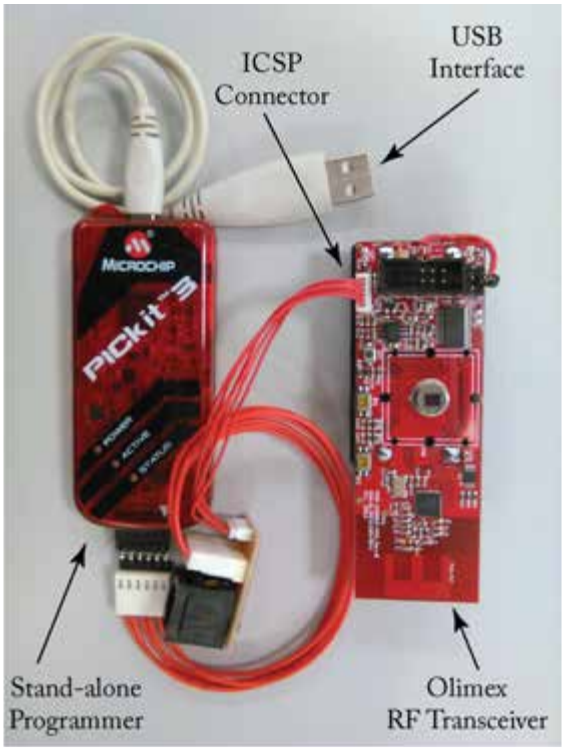


Figure 2.1: Stand-alone programmer/debugger (by Microchip) connected to an Olimex board.

CPU, and cannot be dynamically modified during the code execution, this type of memory is also known as *read-only memory* (ROM). When the application firmware forces register data to change value during the execution of code, then this kind of information is stored to the so-called *data memory*. Because the information held in data memory does not necessarily need to be performed in sequential order (as it could be of random arrangement), microcontroller's data memory is also known as *random-access memory* (RAM).

Another type of RAM memory that is employed by the MCUs is the i/o memory. The latter is addressed to control the configuration and information held by the internal peripheral devices. There are three kinds of registers peculiar to the i/o RAM memory. These are the *control registers*, *status registers*, and *data registers*. The first type of registers controls the configurable options of the corresponding internal subsystem (e.g., set the subsystem to work on *low-power* or *normal* operating mode). The second provides information about the current status of the subsystem (e.g., when a value has been obtained and is available to the data register of the system, such a result returned from an *analog-to-digital* conversion). The third holds data obtained from the subsystem, like for example the digital information obtained from the internal *analog-to-*

digital converter (ADC) inside of the MCU. Control, status, and data registers are of RAM type as they can change their content during the execution of a program (and are randomly accessed).

Additional RAM-type registers are also present in the microcontroller's CPU. Every CPU consists of (at least) the following registers: the *accumulator* (ACC), *program counter* (PC), *stack pointer* (SP), and *status register* (SR). The latter consists of separate flag bits influenced by the in-progress executed instruction, where the most important are the *carry flag* (CF) and *zero flag* (ZF). For instance, loading a zero value to the (general purpose) accumulator register will cause the corresponding zero flag to be raised (i.e., ZF=1), while shifting a RAM register one position to the right or left will, respectively, load into carry flag the value of *least/most significant bit* (LSB/MSB) of that register.

The ACC register is addressed for arithmetic/bitwise operations through the *arithmetic logic unit* (ALU) but also operates as general purpose register for obtaining/assigning data from/to RAM registers of the MCU. The PC register is always assigned to the memory location of the subsequent machine opcode that will be executed. The term *opcode* (abbreviated from the *operation code*) refers to the portion of the machine instruction describing the operation to be performed (e.g., addition, subtraction, memory assignment, etc.). Every machine instruction consists of an opcode and optionally one or more arguments. For instance, a machine instruction that alters the regular program flow of control, it admits an argument identical to memory location at which the control flow will be directed.

Since we made a reference to the alternation of the program flow of control, it would be wise to deepen to the role of PC register in the MCU, as well as to the way CPU executes firmware instructions from the program memory. Every machine instruction reserves one or more locations (i.e., registers) in memory, according to the incorporated arguments. For instance, the *No operation* (NOP[1]) instruction (a common instruction among different microcontroller cores) admits no arguments and, hence, it reserves one register in program memory. To make things simple, we assume to be working on an 8-bit MCU accessing 1 Kilobyte (KB) memory, while every machine instruction reserves one Byte (B) in memory. The units regularly used for counting microcontroller's memory are given in Table 2.1. The subsequent examples address pseudocode so as to help the reader clarify the basic operations of MCU at the machine level.[2]

Figure 2.2 presents a pseudocode example uploaded to microcontroller's memory locations addr8:11. It is noted that counting of memory locations is represented in the decimal numeral system, with values starting from zero. The former two memory addresses (addr0, addr1) constitute the microcontroller's i/o memory. The subsequent six locations (addr2:7) comprise the general purpose RAM registers, available for data write and read operations during the code execution. Memory locations addr8:1023 constitute the Flash (program) memory of the MCU

[1]The NOP instruction takes one or more clock cycles to be executed (according to the microcontroller architecture) and does nothing but delaying the code execution.

[2]It is noted that the MCU architecture of the examples presented herein is of Von Neumann type, as the program and data memory share the same address and data bus. Another popular MCU architecture is the Harvard type, where data and program memory use separate address and data buses. This particular implementation allows simultaneous access to both code instructions and data and, thus, it is of optimized performance.

Table 2.1: Memory units: bit, nibble, byte, kilobyte, megabyte, gigabyte

Unit	Value
1 Bit	Binary digit is depicted by logical 0 or 1 (denoting the presence of low-/high-voltage level)
1 Nibble	4 bits (regularly addressed for illustrating hexadecimal digits)
1 Byte (B)	8 bits
1 Kilobyte (KB)	2^{10} B = 1,024 B (=1,024*8 = 8,192 bits)
1 Megabyte (MB)	2^{10} KB = 1,024 KB = 2^{20} B = 1,048,576 B
1 Gigabyte (GB)	2^{10} MB = 1,024 MB = 2^{20} KB = 2^{30} B = 1,073,741,824 B

where all but the latter two addresses (denoted *Reset Vector*) are indented to hold the application code. The so-called reset vector holds the earlier machine instruction of the code, and this value is loaded to the PC register of the CPU when the microcontroller device is powered up.

The value of PC register is incremented by a number equivalent to the memory locations reserved by the executed instruction. If all instructions reserve 1 B in memory (like in this example code) the PC would always increment by one so as to point to the subsequent machine instruction. Thus, the PC value equals to nine (PC=9) when the CPU executes the first instruction (Figure 2.3), then this value is unary incremented (PC=10) when the CPU executes the second instruction (Figure 2.4), and so forth. In the last cycle (Figure 2.6) the PC would normally change its value from PC=11 to PC=12. However, the GOTO instruction change the value of PC to the address specified by the instruction argument and thus, the execution resumes from the second line of code (Figure 2.4). Such instructions alter the regular (sequential) execution of the code in order to meet the application's requirements. In this particular example, the GOTO instruction generates an unconditional branch that forces CPU to an endless execution of code lines 2–4 (i.e., addr9:11).

At this point it is worth noting that while all registers in CPU and memory are 8-bit long (to form an 8-bit MCU), the PC is 10-bit long. This is because the CPU should be able to access every single register in memory and since the latter consists of 1024 memory locations (that is, addr0:1023), the PC must necessarily be 10-bit long (i.e., $2^{10} = 1024$). Because all memory registers are in 8-bit arrangement, the reset vector reserves two memory location to hold the starting (10-bit) address to be loaded to PC. The sequential arrangement in memory, of large data separated in bytes, is referred to as *endianness* and is represented either as big-endian or little-endian byte ordering. The MCU of this particular example requires big-endian scheme for the assignment of reset vector. Figures 2.7a,b presents, respectively, an example of big-endian and little-endian byte arrangement in memory, of the 32-bit hexadecimal number 0x08090A0B. The counting scheme of the dominant numeral systems that are used in microcontroller applications is illustrated in Table 2.2.

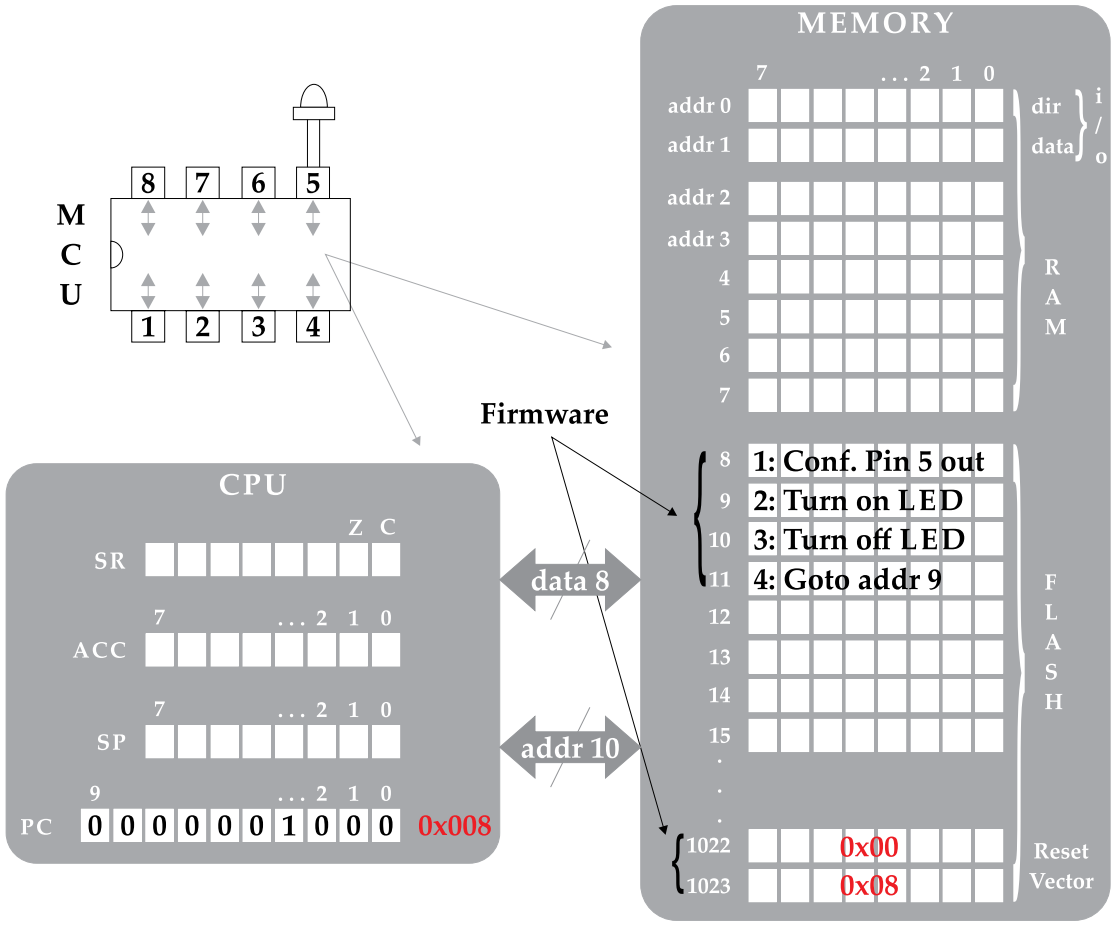


Figure 2.2: Firmware (pseudocode) execution in microcontroller's memory: step 1.

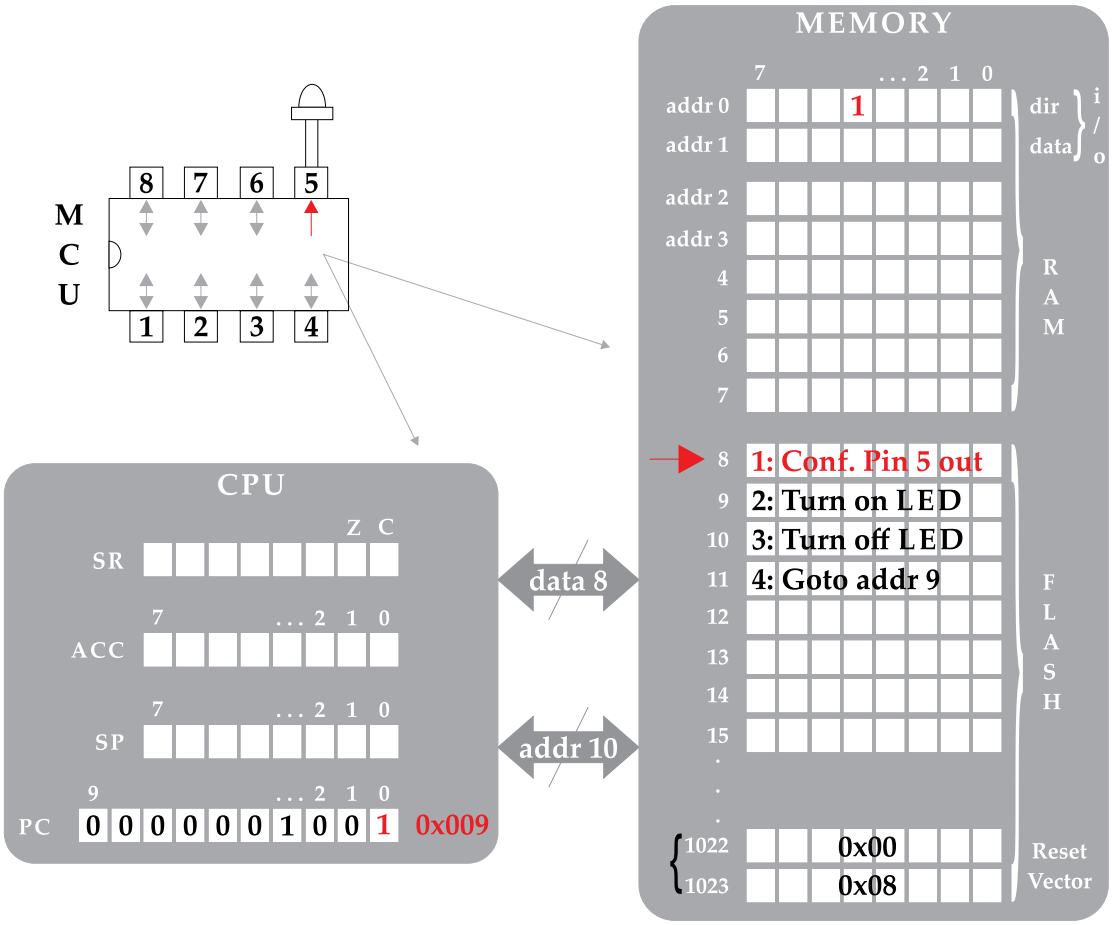


Figure 2.3: Firmware (pseudocode) execution in microcontroller's memory: step 2.

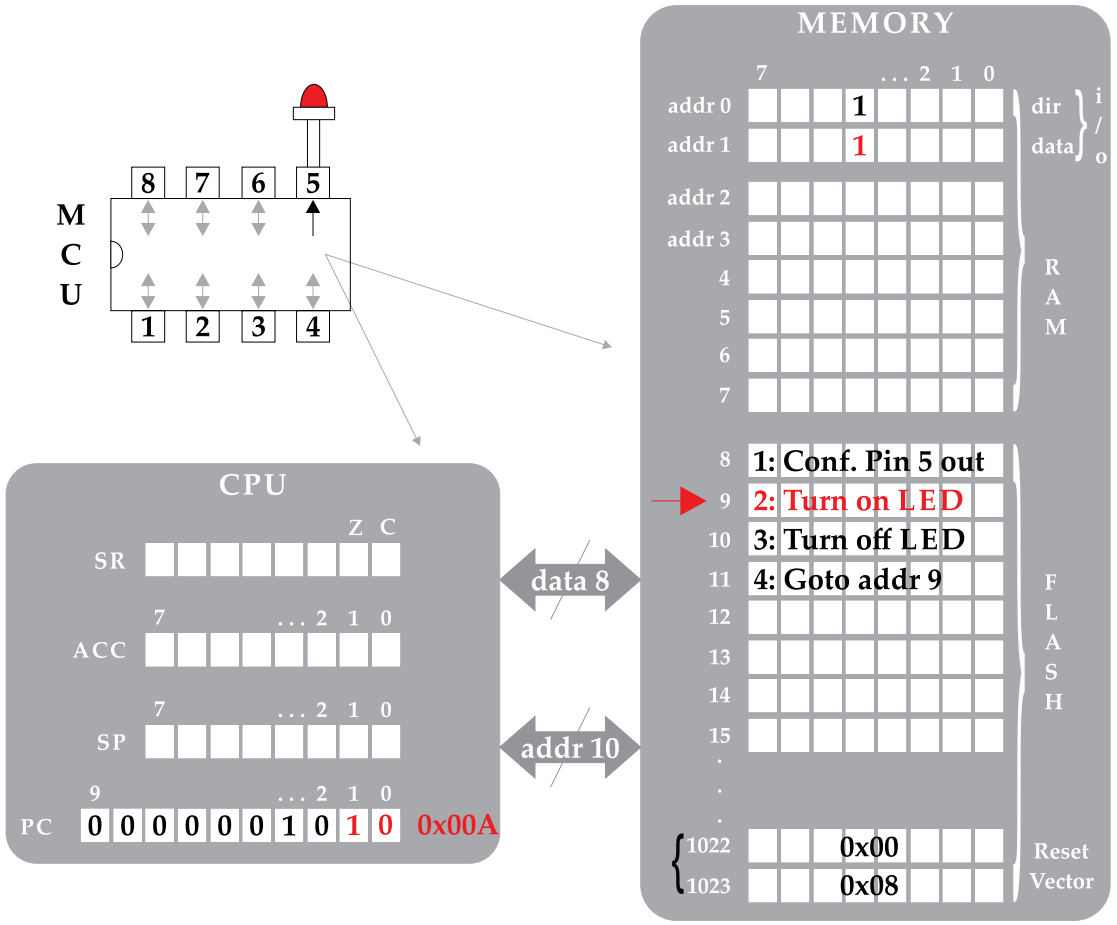


Figure 2.4: Firmware (pseudocode) execution in microcontroller's memory: step 3.

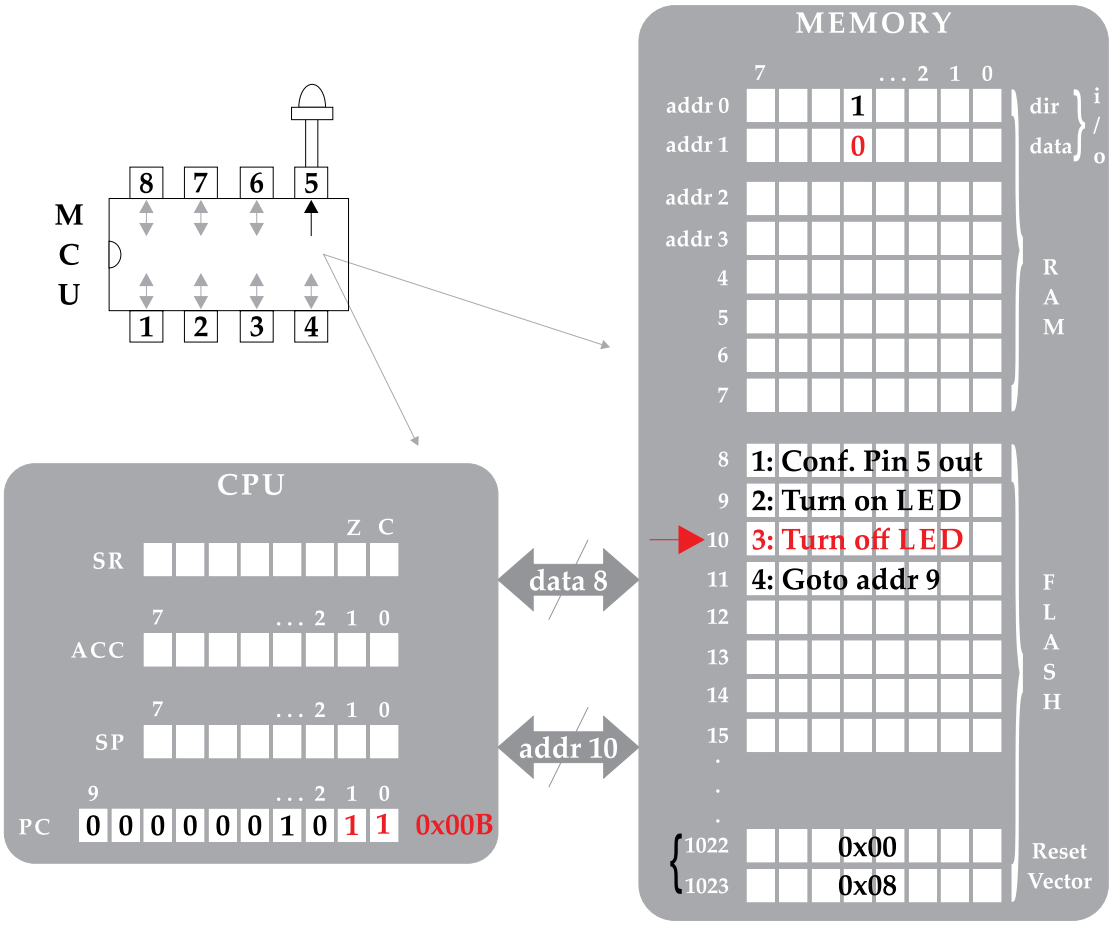


Figure 2.5: Firmware (pseudocode) execution in microcontroller's memory: step 4.

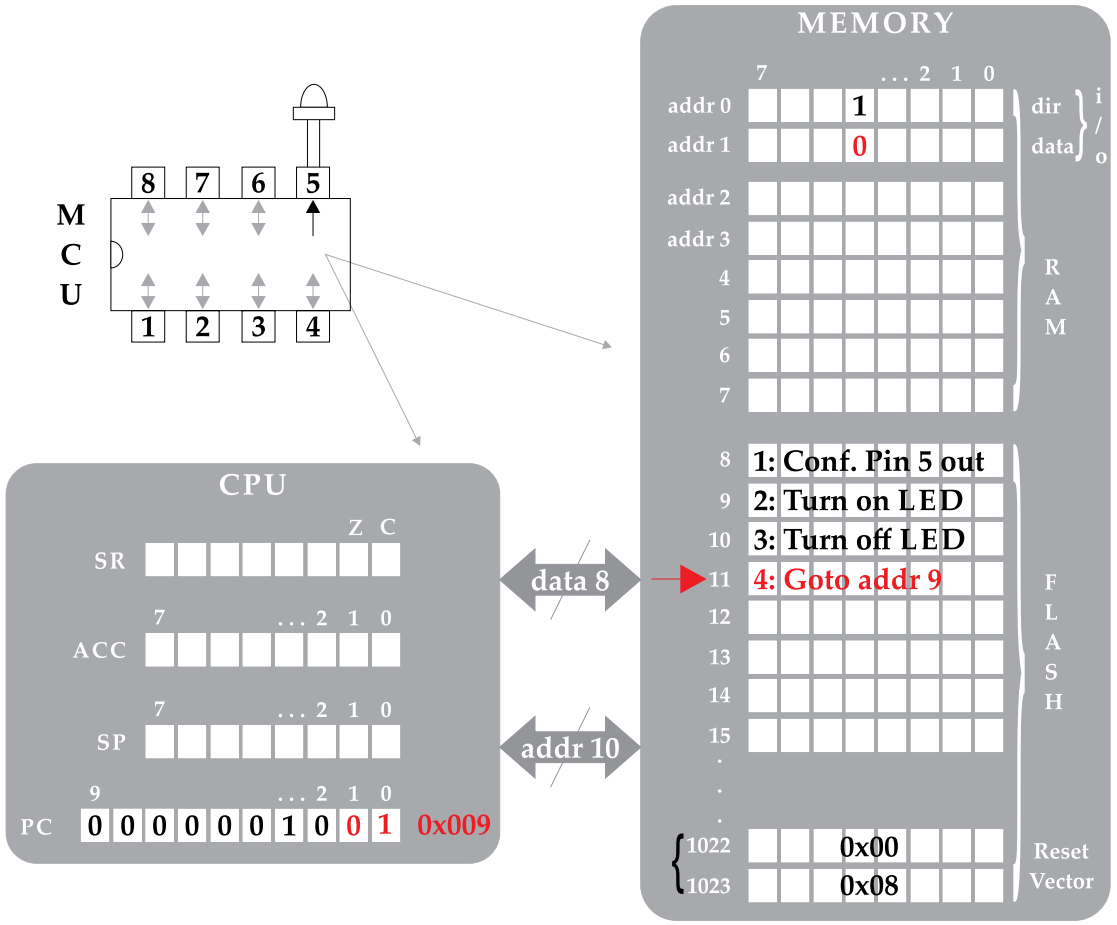


Figure 2.6: Firmware (pseudocode) execution in microcontroller's memory: step 5.

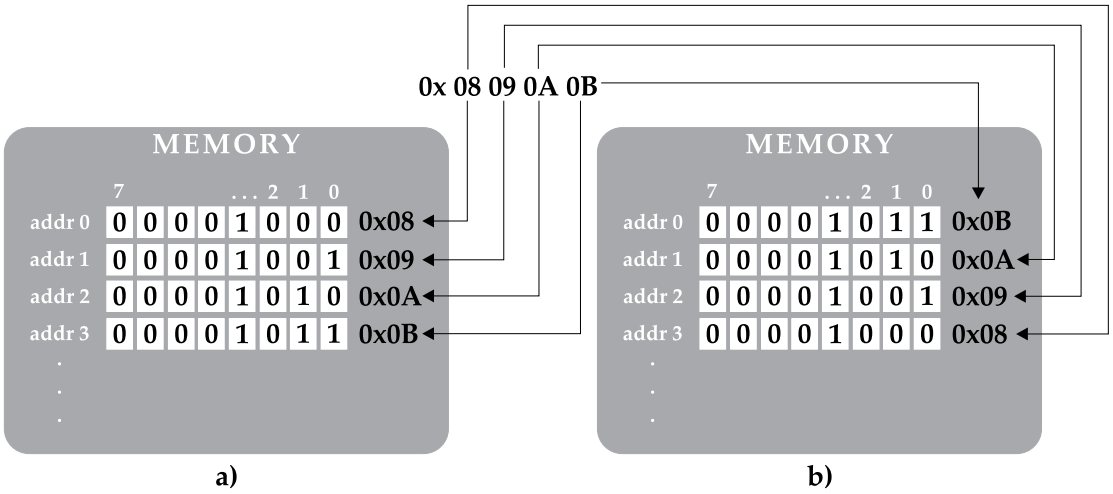


Figure 2.7: Endianness representation using (a) big-endian and (b) little-endian byte ordering.

Table 2.2: Prevailing numeral systems in microcontroller applications: binary and hexadecimal

DEC	BIN (0b…)	HEX (0x…)	DEC	BIN (0b…)	HEX (0x…)
0	0	0	**8**	1000	8
1	1	1	**9**	1001	9
2	10	2	**10**	1010	A
3	11	3	**11**	1011	B
4	100	4	**12**	1100	C
5	101	5	**13**	1101	D
6	110	6	**14**	1110	E
7	111	7	**15**	1111	F

The microcontroller's interface to the outside world is handled by the pins of the device. In 8-bit MCUs the pins are grouped into ports of eight or fewer terminals. Those terminals are regularly bidirectional, i.e., they can be declared either input or output port pins. The pin direction is configured by the corresponding bit in the *direction* (dir) i/o register, while accessing to the port pin is achieved by the corresponding bit of *data* register. The MCU of the preceding example consists of a single 8-pin port, and the first code instruction assigns a logical 1 to the 4^{th} bit of dir register (Figure 2.3). Thereby, pin 5 is configured output. Then, the second code instruction assigns a logical 1 to the 4^{th} bit of data register and consequently, a high-level voltage appears on pin 5, which results in turning the LED on (Figure 2.4). Thereafter, the third code

instruction assigns a logical 0 to the 4^{th} bit of data register and a low-level voltage appears on pin 5, which results in turning the LED off (Figure 2.5). Several different i/o units can be connected to the microcontroller's port pins so as to provide an i/o interface with the outside world. Some of the most popular ones are presented in Figure 2.8. Despite the i/o units that provide a digital signal (Figure 2.9a) interface,[3] an analog signal (Figure 2.9b) can be inputted to the microcontroller device through an ADC (such as a signal obtained from a temperature sensor), or outputted to the outside world through an *digital-to-analog converter* (DAC).

Up until this point, we have discussed the sequential programming approach and how the content of PC register can be modified (through particular machine instructions) in order to alter the normal execution of the code. Hereafter, we explore the role of SP register and how it is utilized by the MCU toward an automated process for resuming the program flow of control, subsequently to a subroutine call.

Figure 2.10 presents an alternative implementation of the previous pseudocode, addressed to eternally change the state of a LED between on/off. The current pseudocode incorporates the control of the LED within a subroutine (addr11:13), which is invoked by the CALL instruction of addr9. The very last instruction of the subroutine, named RETURN, resumes the program flow of control immediately below the last CALL instruction. The internal mechanisms toward automating the process of resuming control flow are presented hereafter.

In Figure 2.11, the execution of the first instruction (i.e., addr8) configures pin 5 as output and the content of PC register is increased so as to point to the subsequent instruction (i.e., addr9). The execution of the CALL instruction causes a series of tasks to be executed. At first the content of PC register is increased (i.e., PC=0x00A) so as to point to the subsequent, in row, instruction (Figure 2.12). This value will be pushed onto the stack in order to resume control flow, as soon as the execution of the subroutine finishes (Figure 2.13). Because the MCU memory incorporates 1024 registers, the PC value reserves two memory locations in the stack memory, while the stack assignment is carried out in big-endian ordering (i.e., 0x00 and 0x0A). As soon as the stack assignment finishes, the CALL instruction loads to the PC the starting address of the subroutine (Figure 2.14) and, hence, the PC now points to the very first instruction of subroutine (PC=0x00B). The execution of the latter instruction (Figure 2.15) turns the LED on and PC content increases so that it points to the subsequent subroutine instruction. The execution of the latter instruction (Figure 2.16) forces the LED to turn off and content of PC to be increased so that it points to the final subroutine instruction. The execution of RETURN instruction (Figure 2.17) pops the (returning from subroutine) effective address from the stack and assigns this value to the PC register. Therefore, the control flow of the program resumes from the GOTO instruction in addr10 (Figure 2.18), which is found immediately below the CALL instruction. This particular procedure renders feasible the callings to a subroutine from

[3]The high-level voltage of a digital signal inputted to/outputted from a microcontroller device is regularly of 3V3 and 5V.

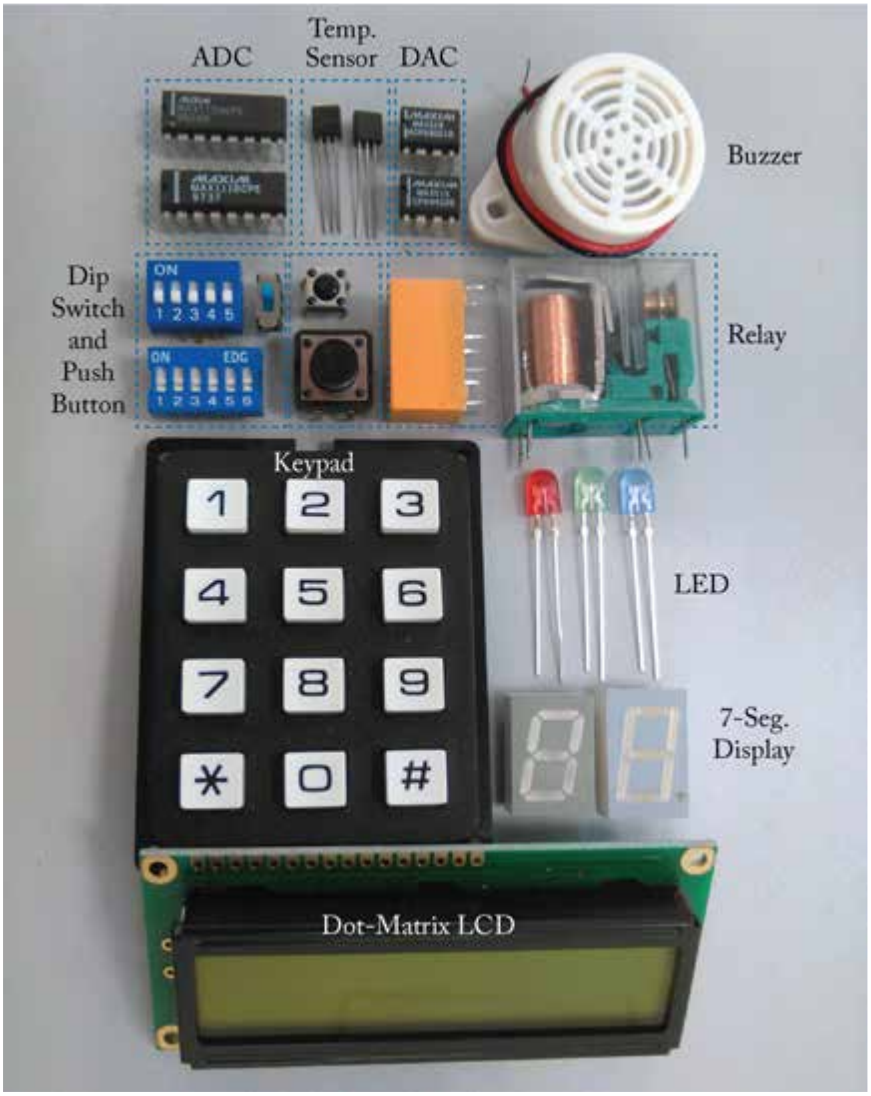


Figure 2.8: Popular i/o units for interfacing microcontroller with the outside world.

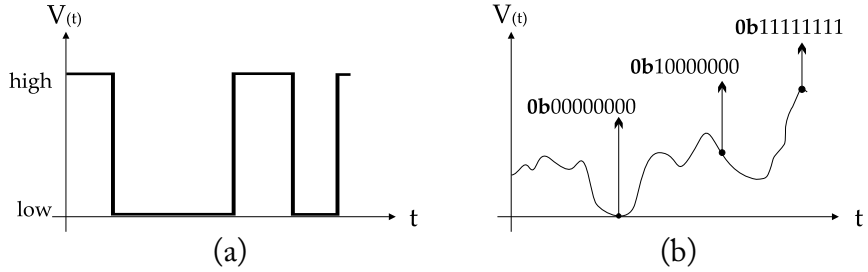


Figure 2.9: Illustrating (a) digital and (b) analog type of signals.

any particular location of the application code and, consequently, it supports the *top-down* and *bottom-up*[4] programming approach.

The term *stack* refers to the data (RAM) memory (i.e., addr2:7 in this MCU example) and its utilization as *last-in-first-out* (LIFO) memory. Through particular machine instruction the CPU can push/pull data onto/from the stack with the utilization of the SP register. The content of the latter register points to the very last location in data memory (i.e., SP=0x07 in this example). Every time an 8-bit value is pushed onto the stack, the SP is unary decreased so that it points to the next available free space in data memory. Accordingly, when the CPU obtains an 8-bit value from the stack, it forces SP to be unary increased. The value still remains to the data memory but the processor considers it as a free memory location (which is the first that will be written upon the next push operation). The last value pushed onto the stack is the first to be read and, hence, the stack is regularly referred to as LIFO memory.

Nested subroutine calls are possible, but the designer should keep in mind not to overlap the values in data memory (generated during the execution of code). The example in Figure 2.19 blinks the LED as before, but this time it incorporates a nested call to subroutine. The arrows in the upper left area of figure illustrate the alternations of the normal (sequential) flow of control. The eight lines of the code are executed in this particular order: the CPU executes only once the first line and then repeats endlessly the execution of code lines 2, 4, 5, 7, 8, 6, 3.

Figure 2.19 depicts the execution of the second call to subroutine. In detail, the CPU has finished the execution of the first CALL (i.e., code line 2) and therefore, the effective address of the subsequent instruction (i.e., 0x000A) has already been pushed onto the stack. The effective address of the instruction that follows the second CALL, and it is currently pointed by the PC register (i.e., 0x000D), is about to be pushed onto the stack. However, we observe that the program code uses three registers in RAM (i.e., addr2:4), which apparently hold data generated previously in the code. The CPU cannot foresee this particular information and thereby the execution of the second CALL overlaps the data of the user-defined register at memory location

[4]In the *top-down* approach, the designer starts with the main procedure of the code and subdivides it into the sub-processes. In the *bottom-up* approach, the designer considers the low-level processes of the code and then it ties them up to implement the main code procedure.

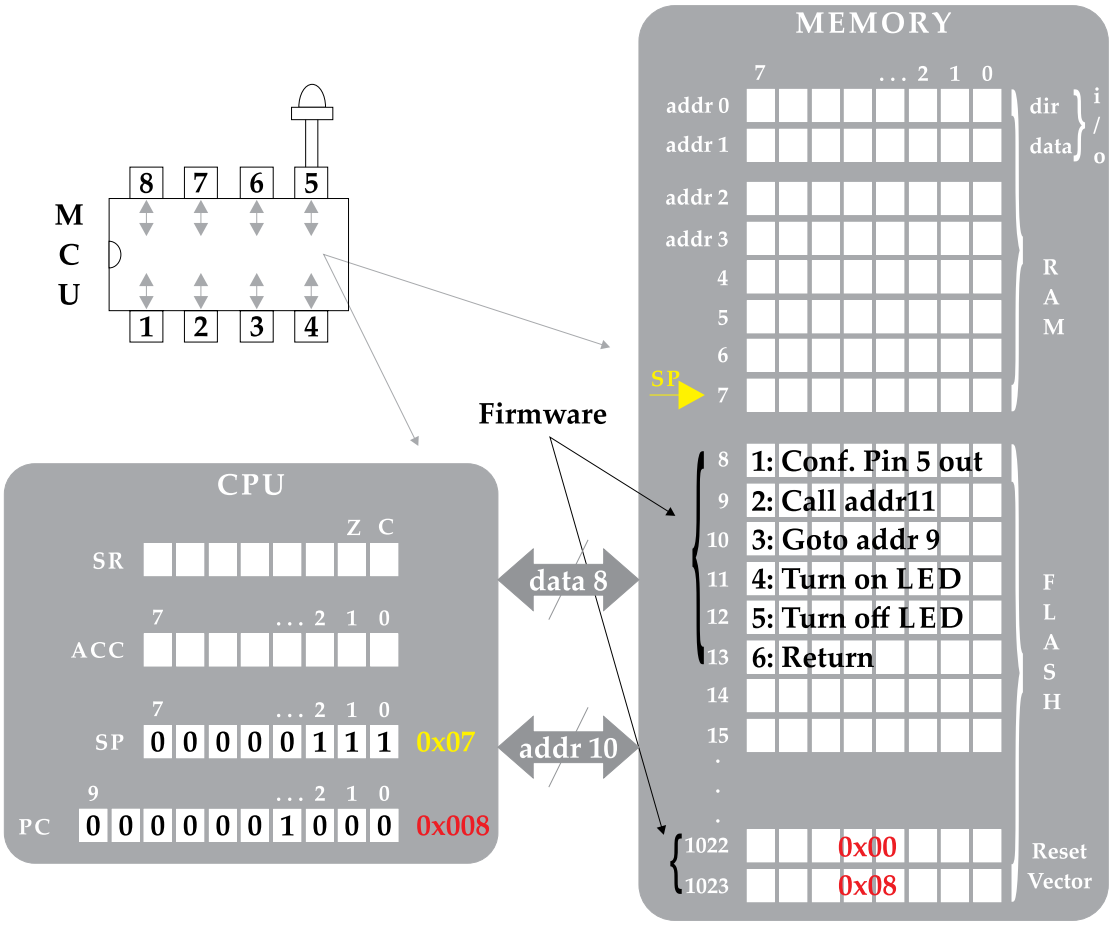


Figure 2.10: Firmware execution in microcontroller's memory: the role of stack (step 1).

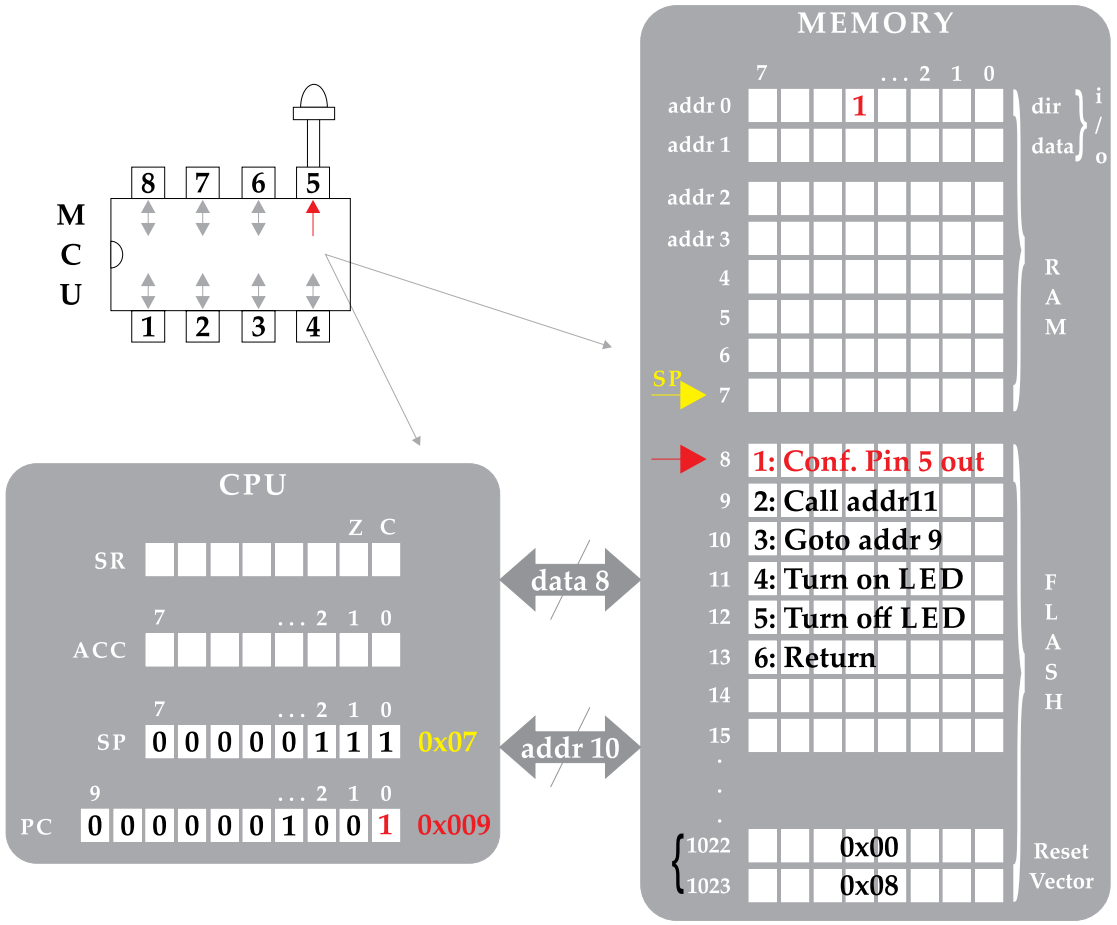


Figure 2.11: Firmware execution in microcontroller's memory: the role of stack (step 2).

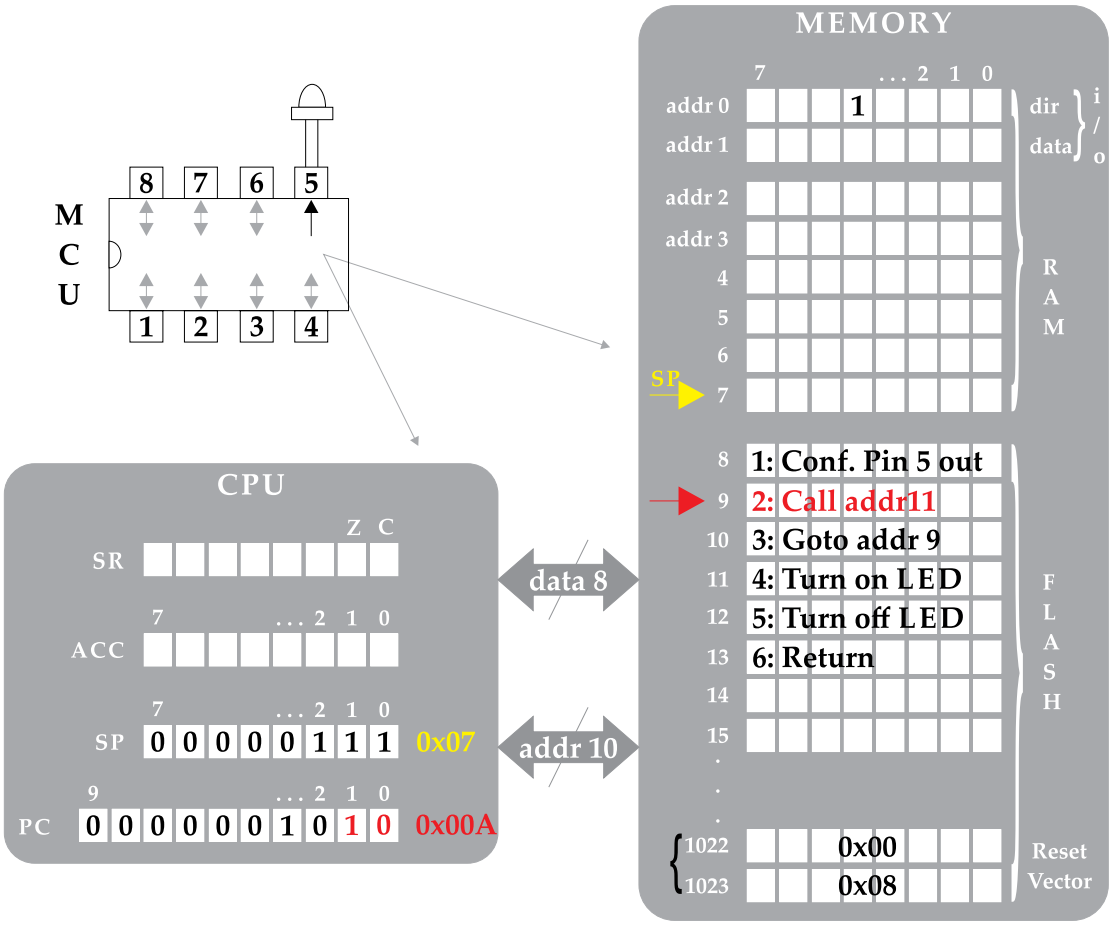


Figure 2.12: Firmware execution in microcontroller's memory: the role of stack (step 3).

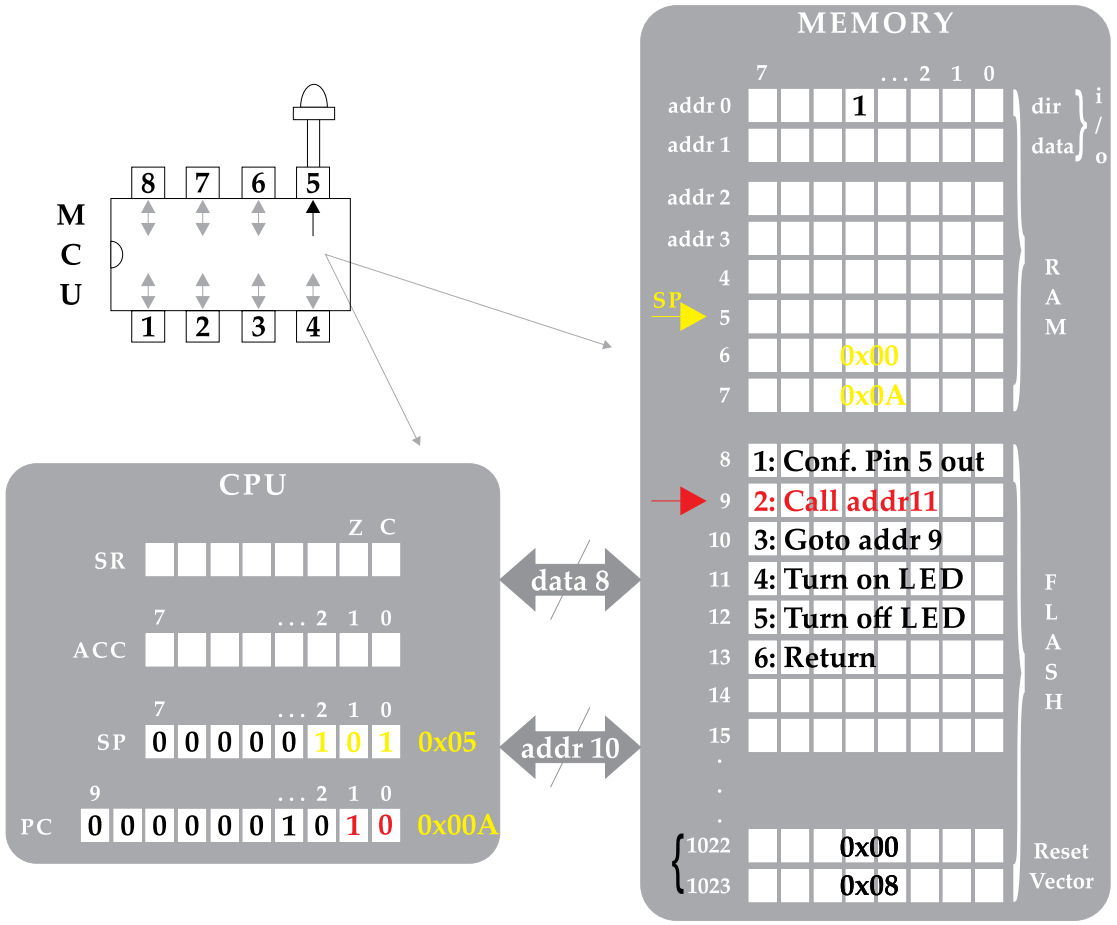


Figure 2.13: Firmware execution in microcontroller's memory: the role of stack (step 4a).

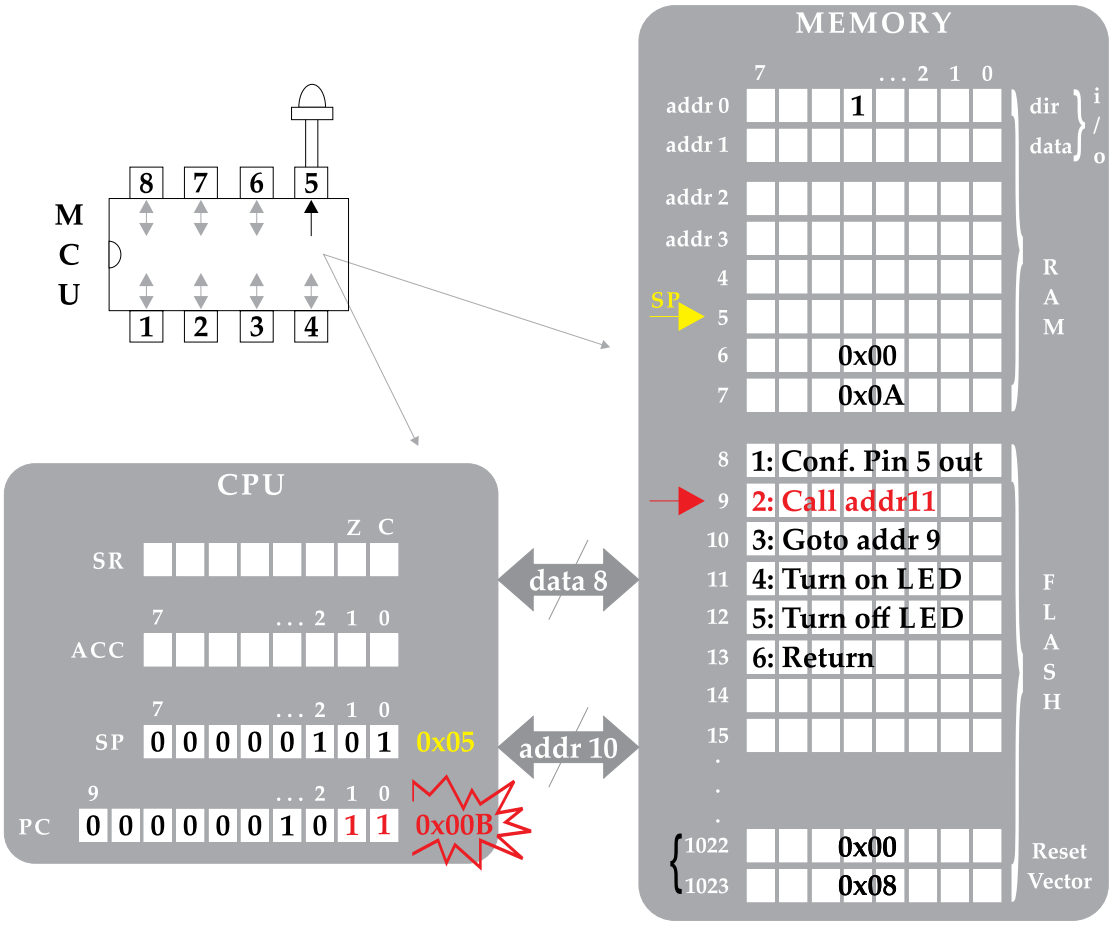


Figure 2.14: Firmware execution in microcontroller's memory: the role of stack (step 4b).

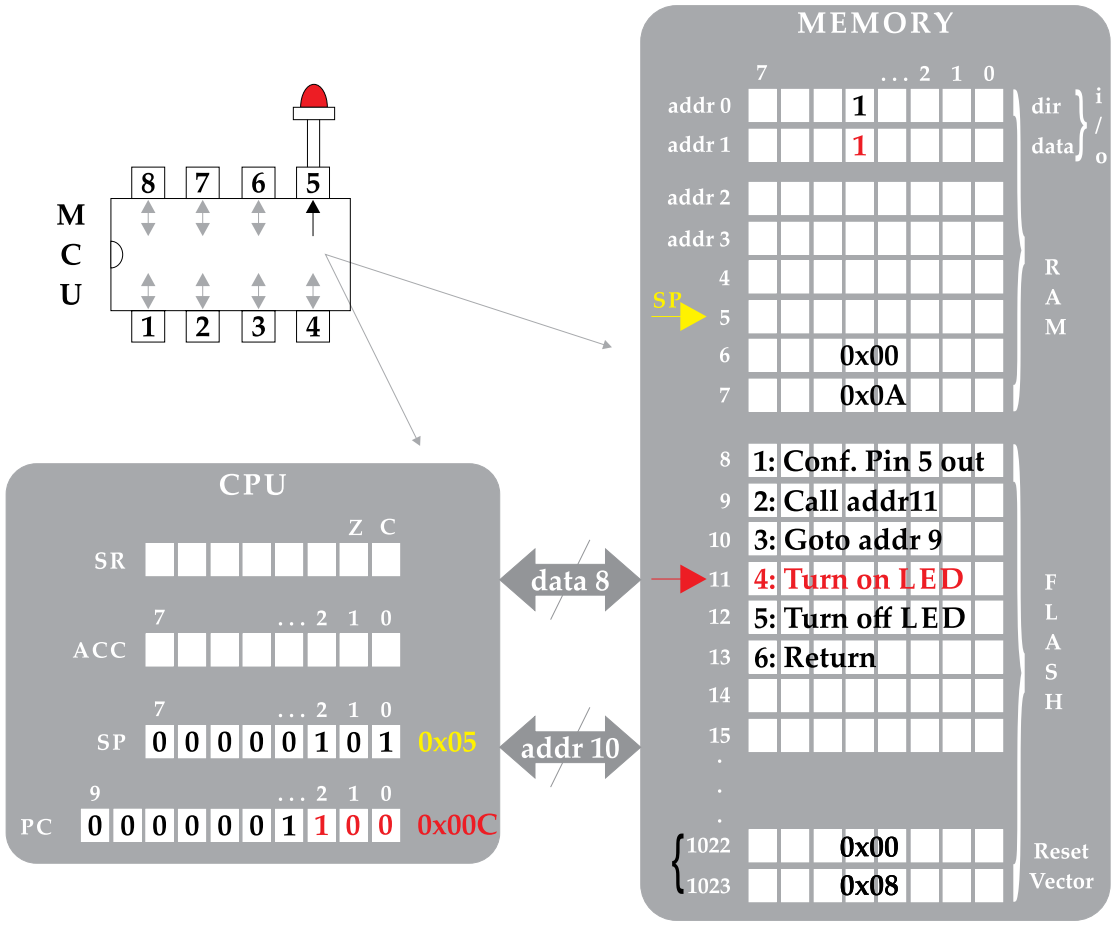


Figure 2.15: Firmware execution in microcontroller's memory: the role of stack (step 5).

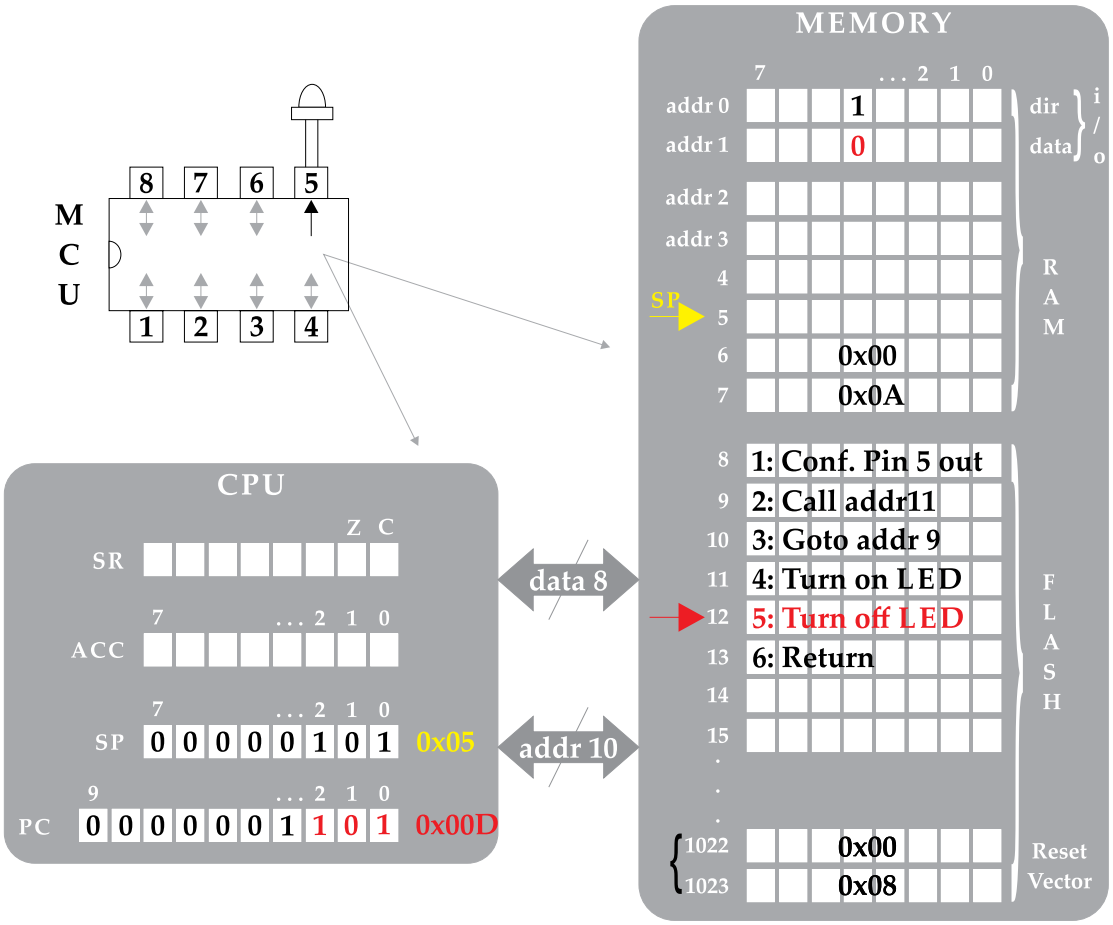


Figure 2.16: Firmware execution in microcontroller's memory: the role of stack (step 6).

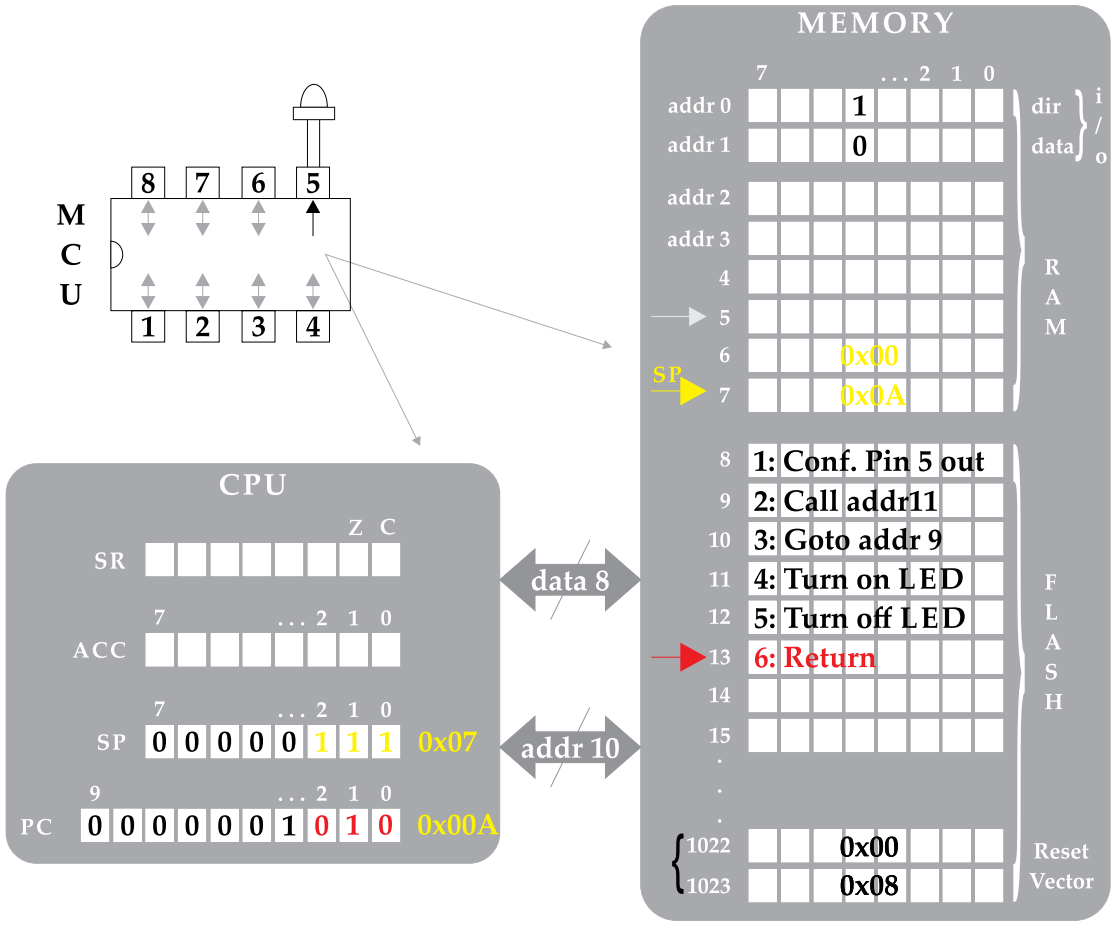


Figure 2.17: Firmware execution in microcontroller's memory: the role of stack (step 7).

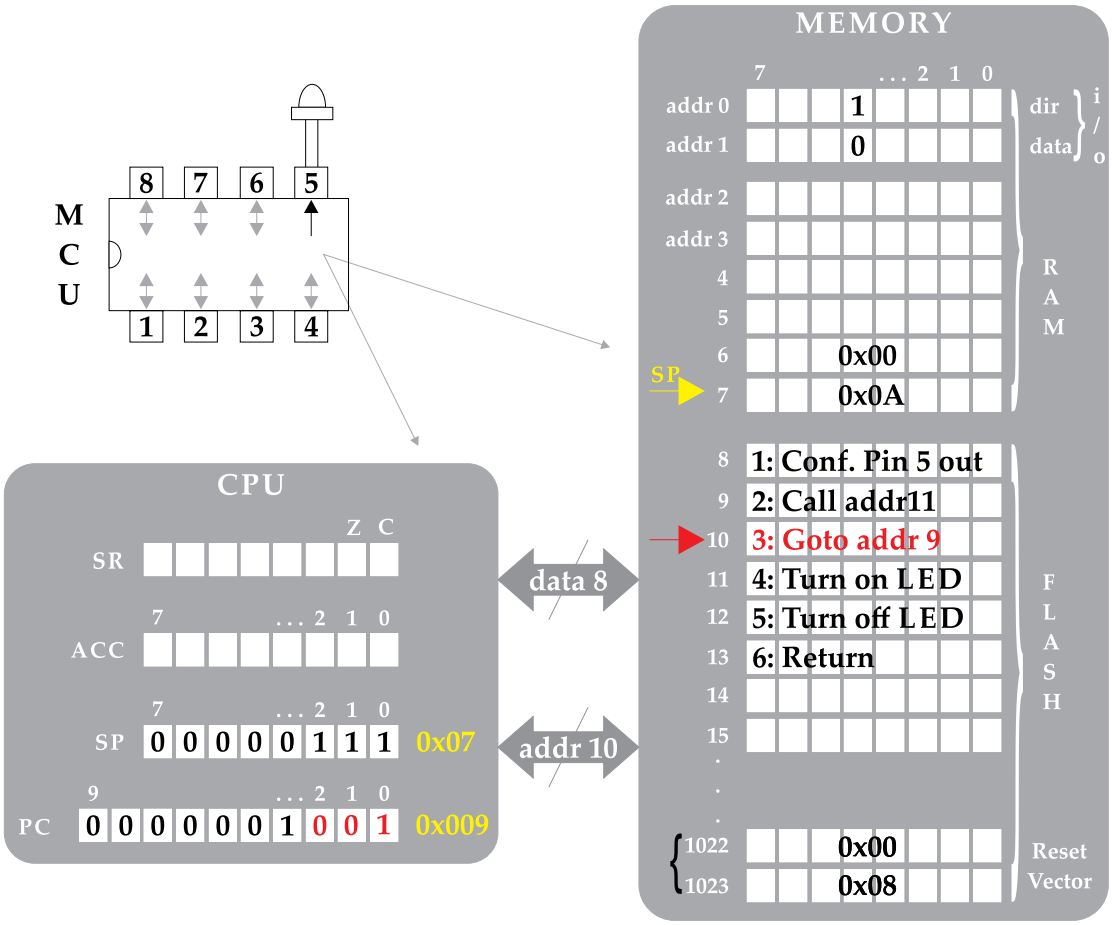


Figure 2.18: Firmware execution in microcontroller's memory: the role of stack (step 8).

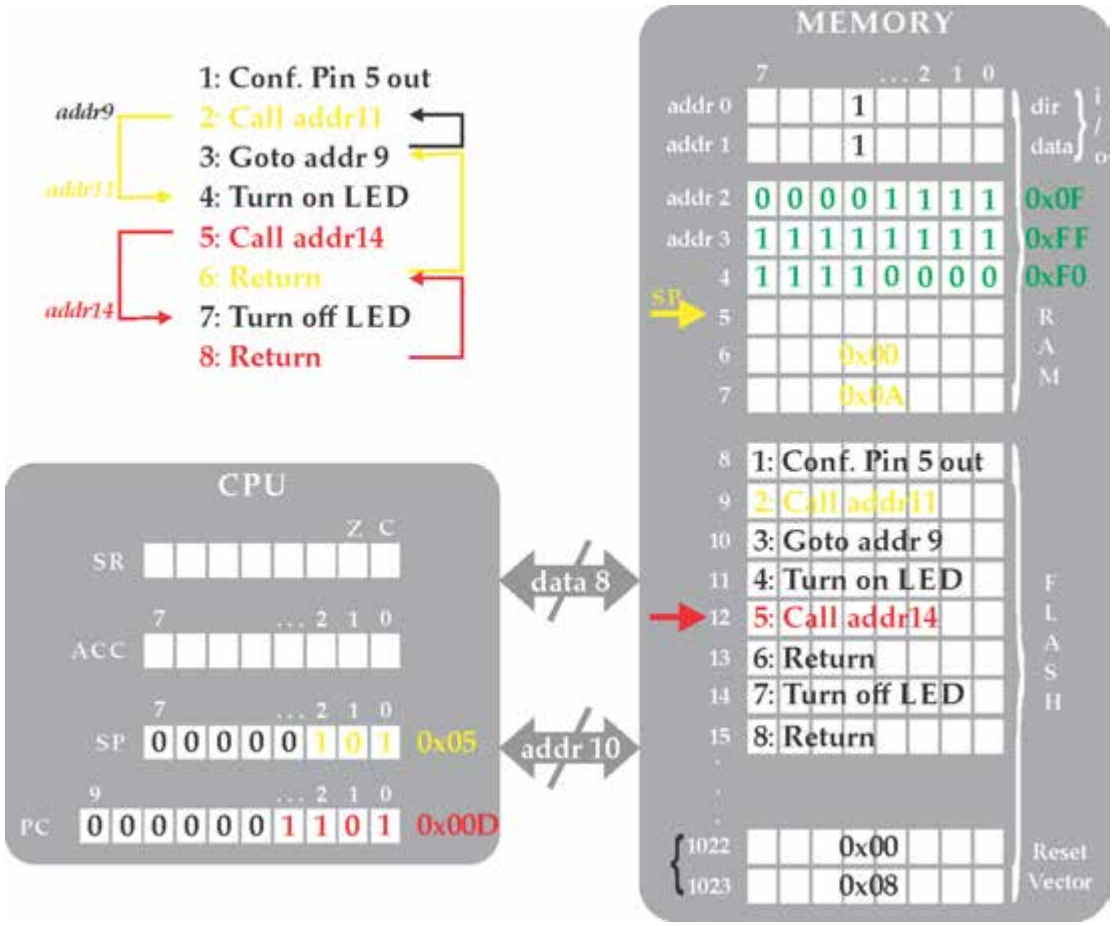


Figure 2.19: Firmware execution in microcontroller's memory: nested calls (step 1).

add4 (Figure 2.20). It is worth noting that modern MCUs reset the device upon faulty actions that attempt to either keep using the stack when the latter is being full of data or when the firmware mistakenly decreases SP register and the stack is already empty (overflow/underflow resets). A mistaken attempt on stack overflow/underflow is prevented because it attempts to extend the lower/upper register of RAM.

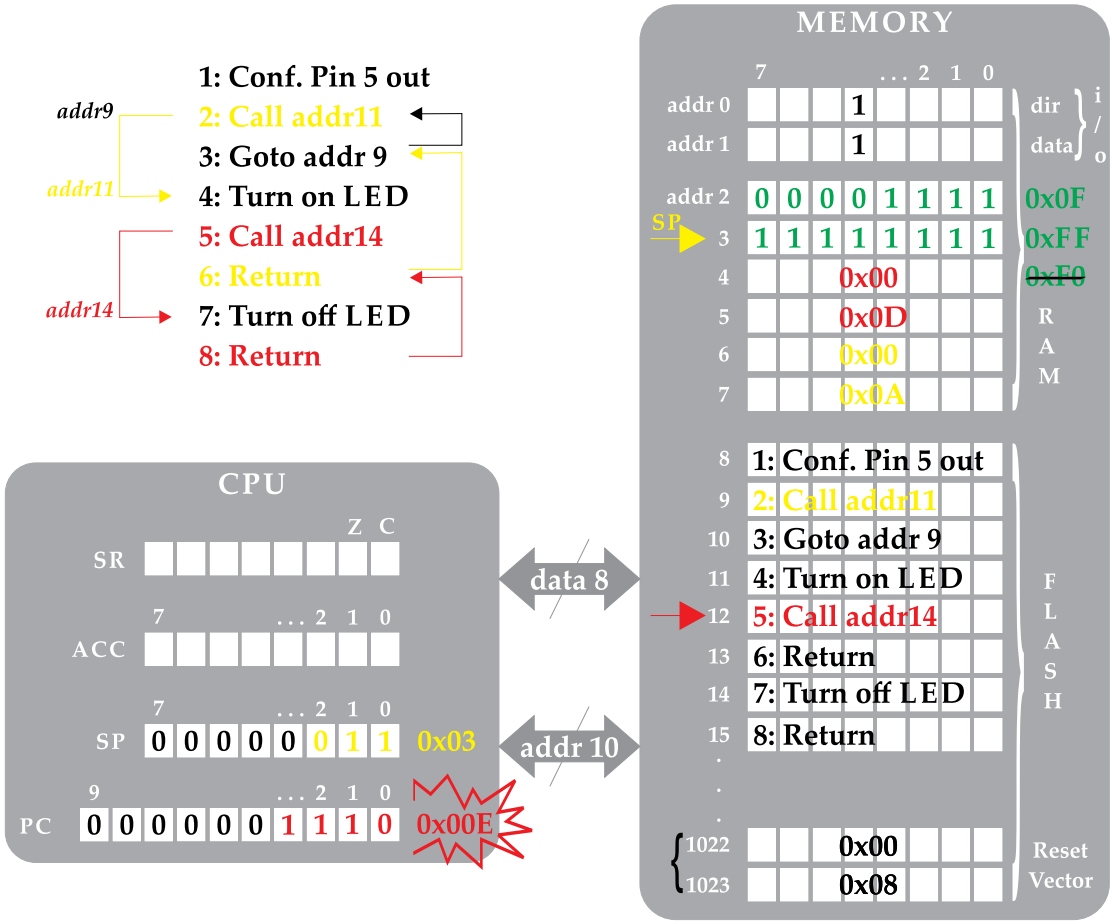


Figure 2.20: Firmware execution in microcontroller's memory: nested calls (step 2).

Having explored the outputting data process from the MCU to the outside, along with the internal mechanisms related to the sequential execution of code and to the methods of alternating the normal flow of control, we next explore the inputting data processes (from the outside world to the MCU). In addition, the following example explores the role and accessing process of SR and ACC registers of the CPU as well.

Figure 2.21 depicts a pseudocode for inputting data to the MCU through pin 8. The execution of the first command (addr8) writes a logical 1 to bit 7 of the i/o direction register, and thus pin 8 is configured input. What this means is that a high-level voltage applied to pin 8 will automatically appear as logical 1 to the bit 7 of i/o data register. Similarly, a low-level voltage on pin 8 will appear to the same bit as logical 0. Consequently, there is the possibility of connecting a mechanical switch to pin 8 of the MCU, and perform a particular operation according the state of the switch (i.e., determine when the user has pressed or released the switch).

In Figure 2.22, the execution of the second command (addr9) loads the content of i/o data register into ACC, where bit 7 is set to logical 1 (as the switch is held closed). The X value in the rest of the accumulator's bits expresses a don't-care term (i.e., bits can be either of logical 1 or 0). In Figure 2.23, the execution of the third command (addr10) shift all bits in the ACC register one position to the left. The *most significant bit* (MSB) is inserted into C flag of SR register, while a logical 0 is loaded to the *least significant bit* (LSB) of the ACC register. This shifting operation is particularly useful to many aspects in microcontroller applications. One of them is the implementation of math operations extending the 8-bit range of the employed architecture (16-bit, 32-bit operations, etc.).

2.2 TIMING ISSUES IN MICROCONTROLLER APPLICATIONS

Among the most important factors that regulate the performance of a microcontroller-based system are the CPU clock and range of registers. For instance, if the application system requires a manipulation of large numbers, then it might be wise to select an MCU of 16-bit or 32-bit architecture (over the 8-bit architecture discussed herein). Moreover, if the arithmetic outcome needs to be generated quickly enough, then a device of short response time would be more appropriate. The latter feature is determined by the maximum clock frequency,[5] which is used to synchronize the internal operation of the CPU. The CPU clock source derives either from a *crystal* (XTAL) or from an *oscillator* (OSC). Typical footprints of both devices are presented in Figure 2.24a. The former requires two extra components, that is, two capacitors, and reserves two special (clock) pins of the MCU (Figure 2.24b). The latter requires no additional components and generates the *free-running clock* (CLK) signal of precise frequency and 50% duty cycle, which is connected directly to a single clock pin of the MCU (Figure 2.24c). Modern MCUs incorporate an internal OSC and, hence, no external CLK source is required (Figure 2.24d).

The constituent parts of the processor model (aka *computer organization* or *microarchitecture*) cooperate to form the *instruction set architecture* (ISA), which is the machine language peculiar to the CPU family of microcontroller devices. The microarchitecture of the CPU (at the digital design level) comprises a sequential logic circuit, the response time of which determines the CPU clock frequency. If we consider a CPU admitting 1 MHz maximum clock frequency,

[5]In 8-bit MCUs the regular clock frequency varies among a few tens of MHz.

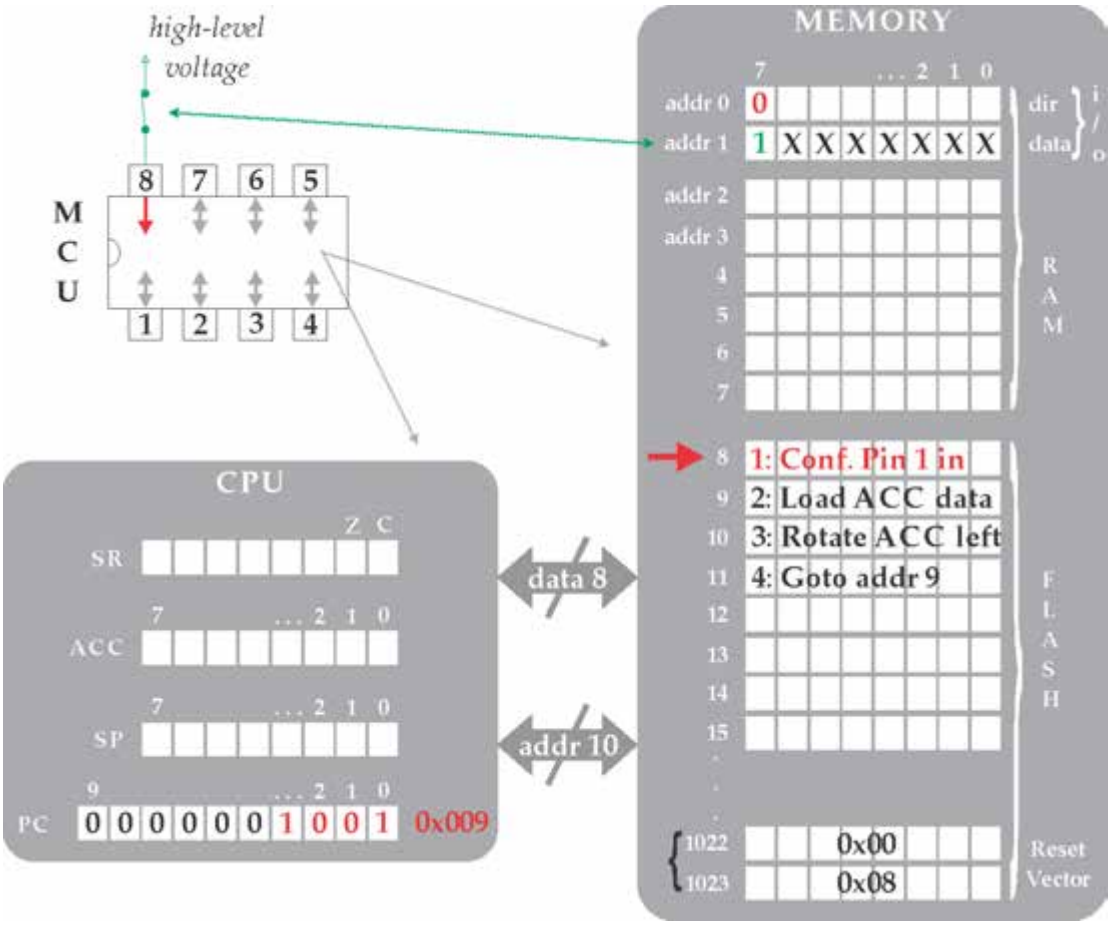


Figure 2.21: Firmware execution in microcontroller's memory: inputting data (step 1).

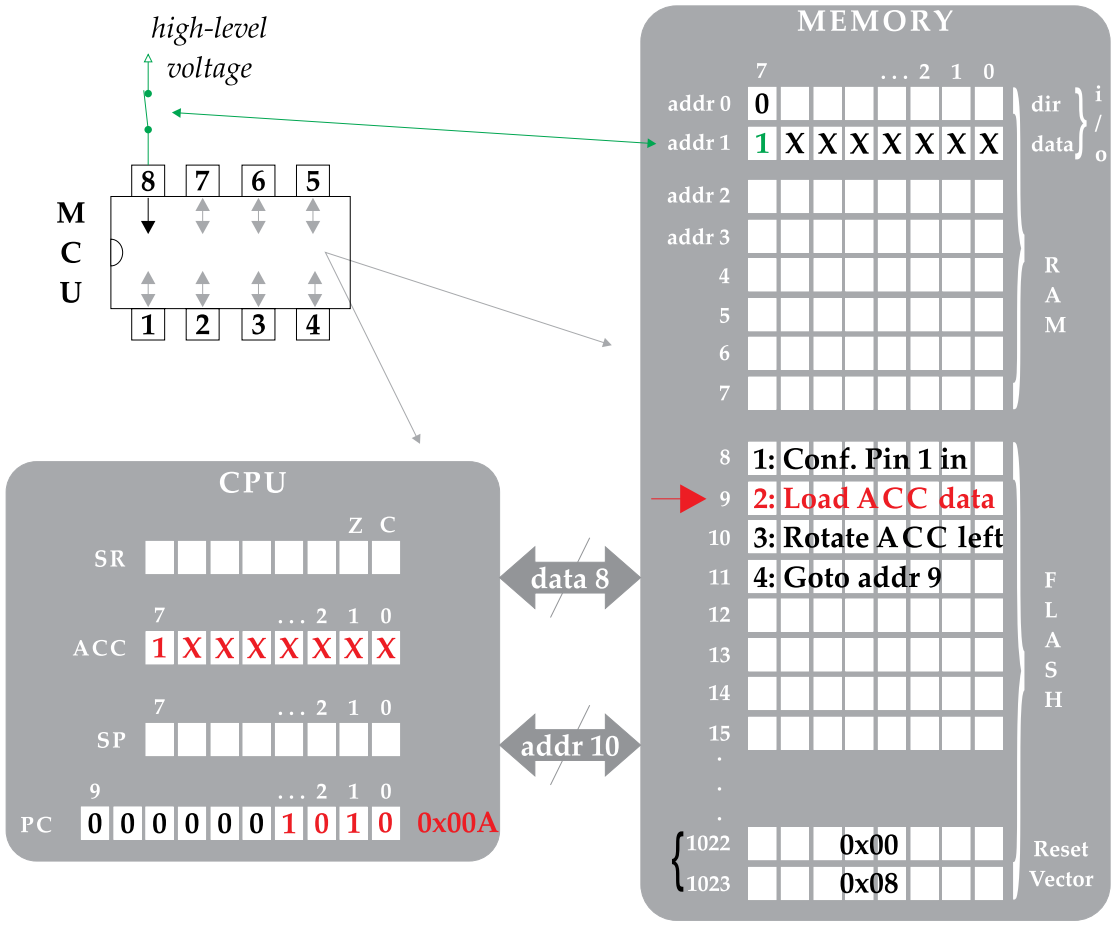


Figure 2.22: Firmware execution in microcontroller's memory: inputting data (step 2).

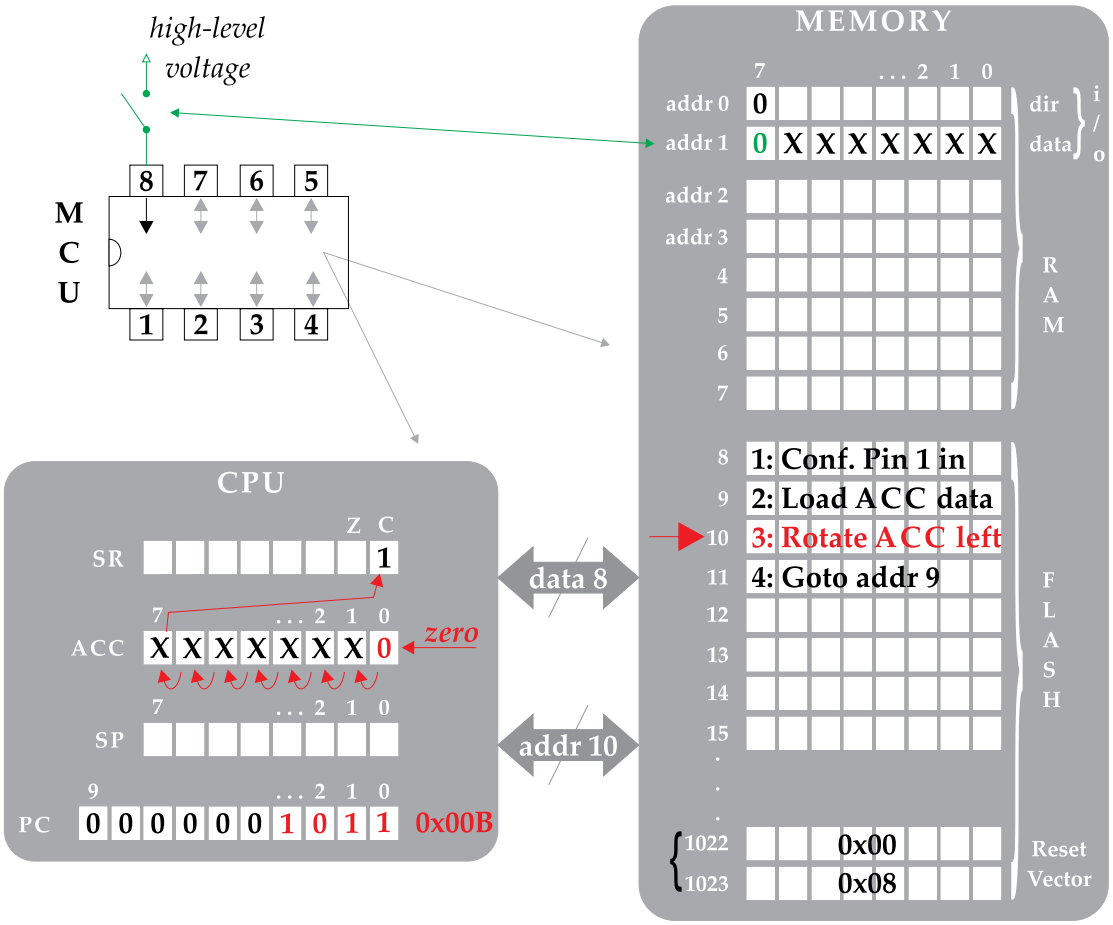


Figure 2.23: Firmware execution in microcontroller's memory: inputting data (step 3).

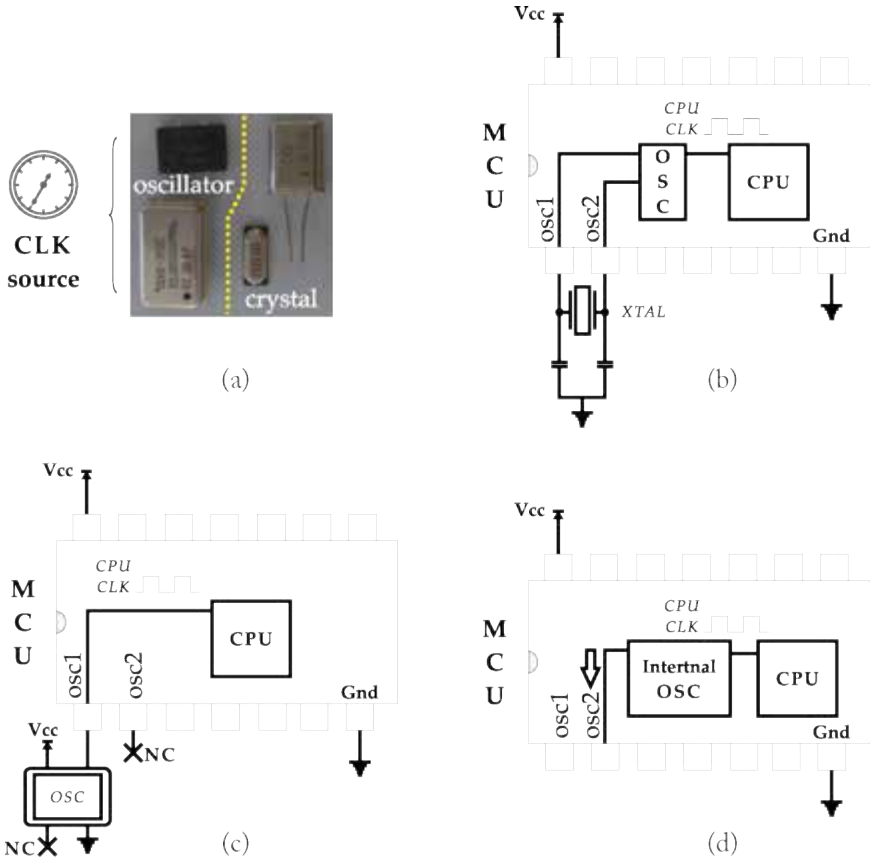


Figure 2.24: MCU clock sources.

while every machine instruction is executed within a single clock cycle (which is not far from reality), then the minimum time needed by the CPU to perform a single operation is 1 μsec. What this means in practice is that the one-time execution of the aforementioned pseudocode blinking the LED, is accomplished within 3 μsec (among successive iterations).

Figure 2.25 depicts the enumeration of the CPU clock periods during the execution of the blinking LED example. The blinking process every 3 μsec is observed by the user as being always on. For that reason, a delay among the on/off states should be considered so as to provide the user with the ability of identifying the LED blinking process. The timing issues hold an important share in microcontroller applications. In order to understand how delays are realized at the machine level, we need to keep in mind that time delays are generated when the CPU executes instructions, and address programming methods that would keep the CPU into iterations (i.e., a loop) for a predefined period.

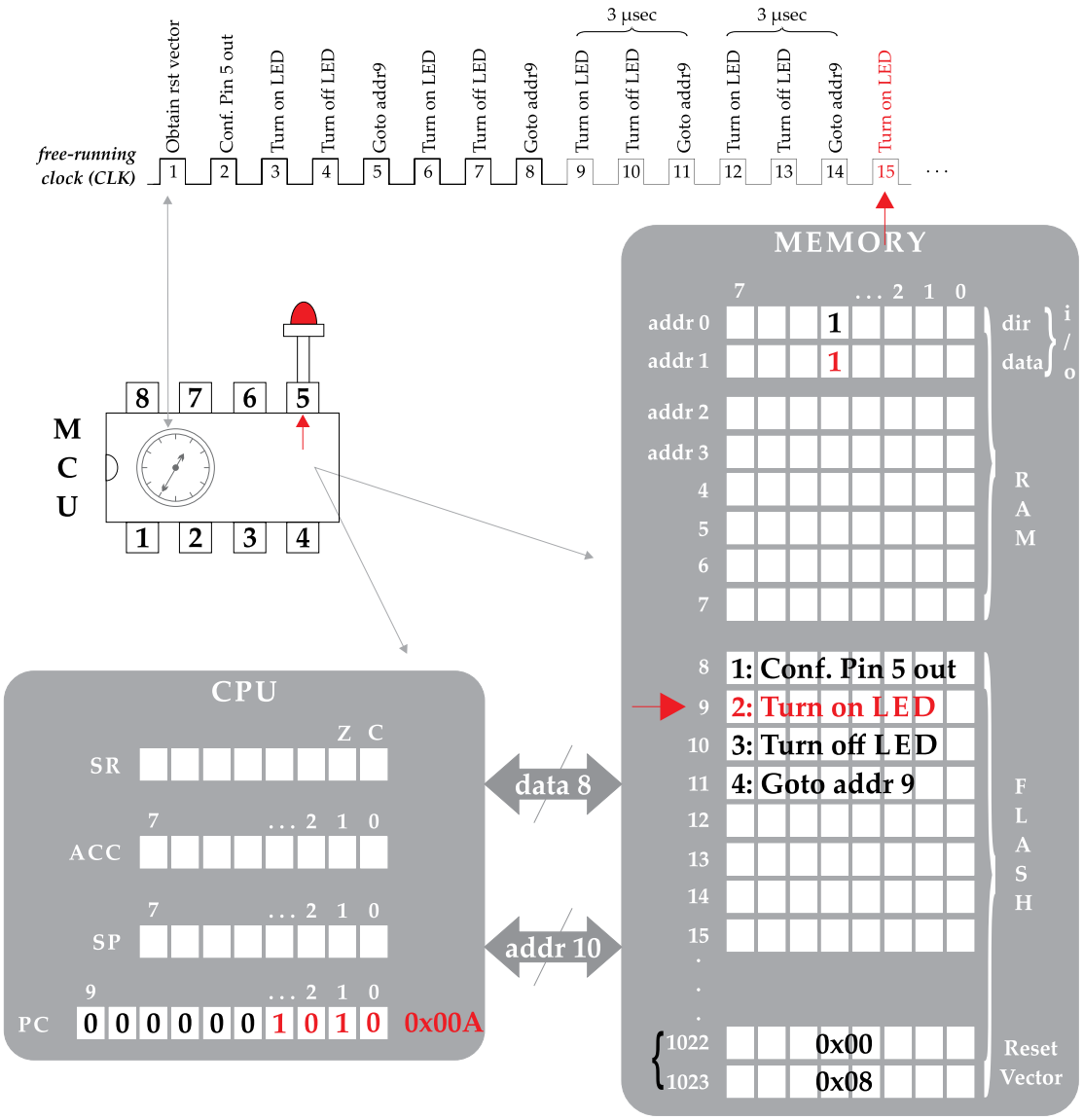


Figure 2.25: The CPU free-running (internal) clock.

The coding approach that should be addressed toward the implementation of a delay depends on the type of the machine instructions and the architecture of the MCU. The ISA of any microcontroller device can be separated into three fundamental groups, that is: (a) memory assignment; (b) flow of control; and (c) arithmetic and bitwise operations [17]. To implement a repeated process at the machine level we need a counter that would be consecutively increased/decreased up-to or down-to a predefined value, and an alternation of control flow at the beginning of the loop, in case the counter has not yet reached the desired value. The givens of the described process are as follows.

(a) Since the counter changes its content during the code execution it should be of RAM type.

(b) The range of values that admits of an 8-bit register is $2^8 = 256$ and, therefore, a single loop cannot be repeated more than 256 times.

(c) The kinds of instructions that without a doubt are needed for implementing the code iteration should be arithmetic and flow-of-control machine instructions. The former would normally provide unary increment/decrement of the loop counter. The latter would evaluate the content of the register and would either continue or alter the program flow if some conditions are (or are not) met.

The maximum delay that can be achieved through the repetition of two machine instructions (i.e., an arithmetic and an control-flow instruction) for 256 times, when each one of them delays the CPU one clock period (that is, 1 μsec in this examples), equals to 512 μsec. Again, this time delay is too short for the blinking LED example. What can be done is to make use of a double-nested loop. The double loop requires one additional counter (i.e., register), whose value will be unary incremented/decrement up-to/down-to a level, every time the first counter reaches the desired point value. In this way, the delay process is augmented up to the amount of $512 \star 512 \cong 262$ msec. While this value may be satisfactory for the blinking LED example, the user might further increase the delay up to seconds (sec) with the employment of a third-nested loop. The latter is capable of providing time delay up to $512^3 \cong 134$ sec.

The following example depicts the execution of the single delay loop. In detail, Figure 2.26 depicts the execution of the machine instruction that turns the LED on upon the CPU clock period 3. All registers are initially supposed to be of zero value and, therefore, Figure 2.27 depicts the unary increment of C1 register from zero to one, upon clock period 4. The subsequent instruction execution in Figure 2.28 evaluates the content of C1 register (currently equal to one) and because C1$\neq$0, the control flow moves to the preceding assembly instruction (clock period 5). Thereby, the subsequent unary increment to the content of C1 register in Figure 2.29 causes the latter to reach number two (clock period 6).

The evaluation of C1 content (i.e., if C1=0 or C1$\neq$0) is actually performed through an examination of the zero flag. If ZF=0, as it is in Figure 2.28, the delay loop is repeated (i.e., the control flow moves to code line 3). Otherwise, the control flow continues the sequential execution of the code, that is, from code line 5 in this case. It is worth noting that arithmetic

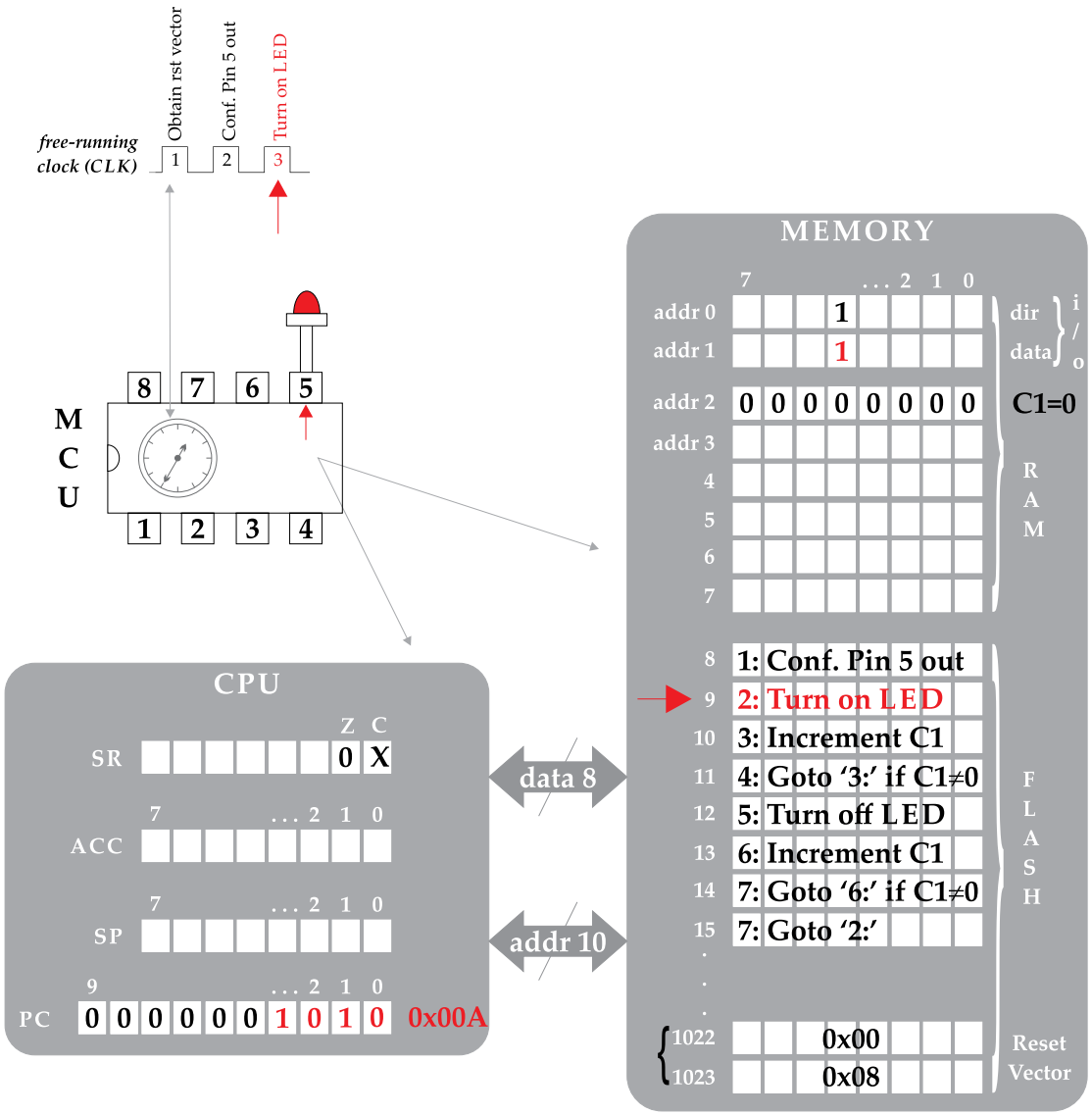


Figure 2.26: CPU delay (step 1).

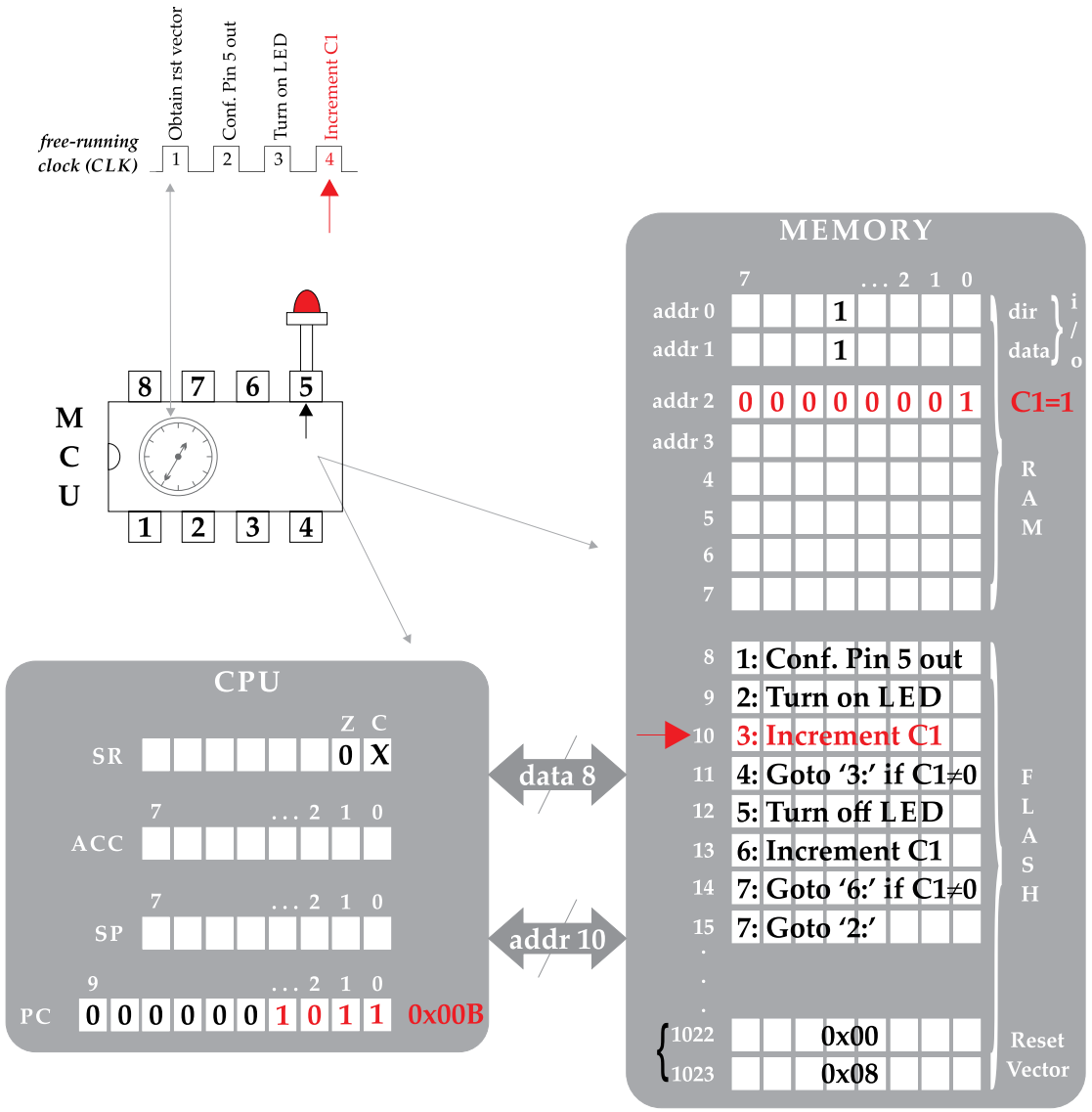


Figure 2.27: CPU delay (step 2).

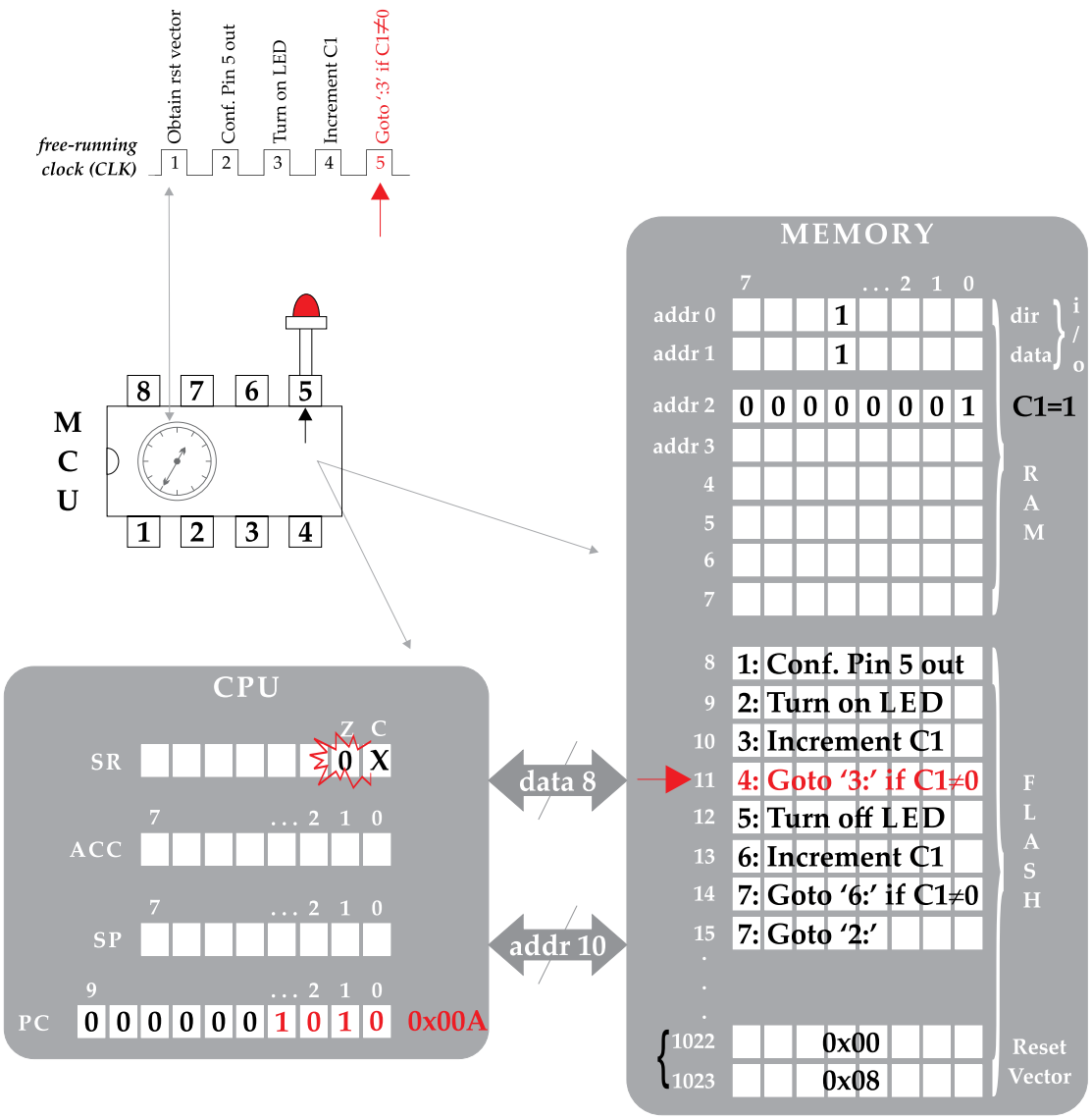


Figure 2.28: CPU delay (step 3).

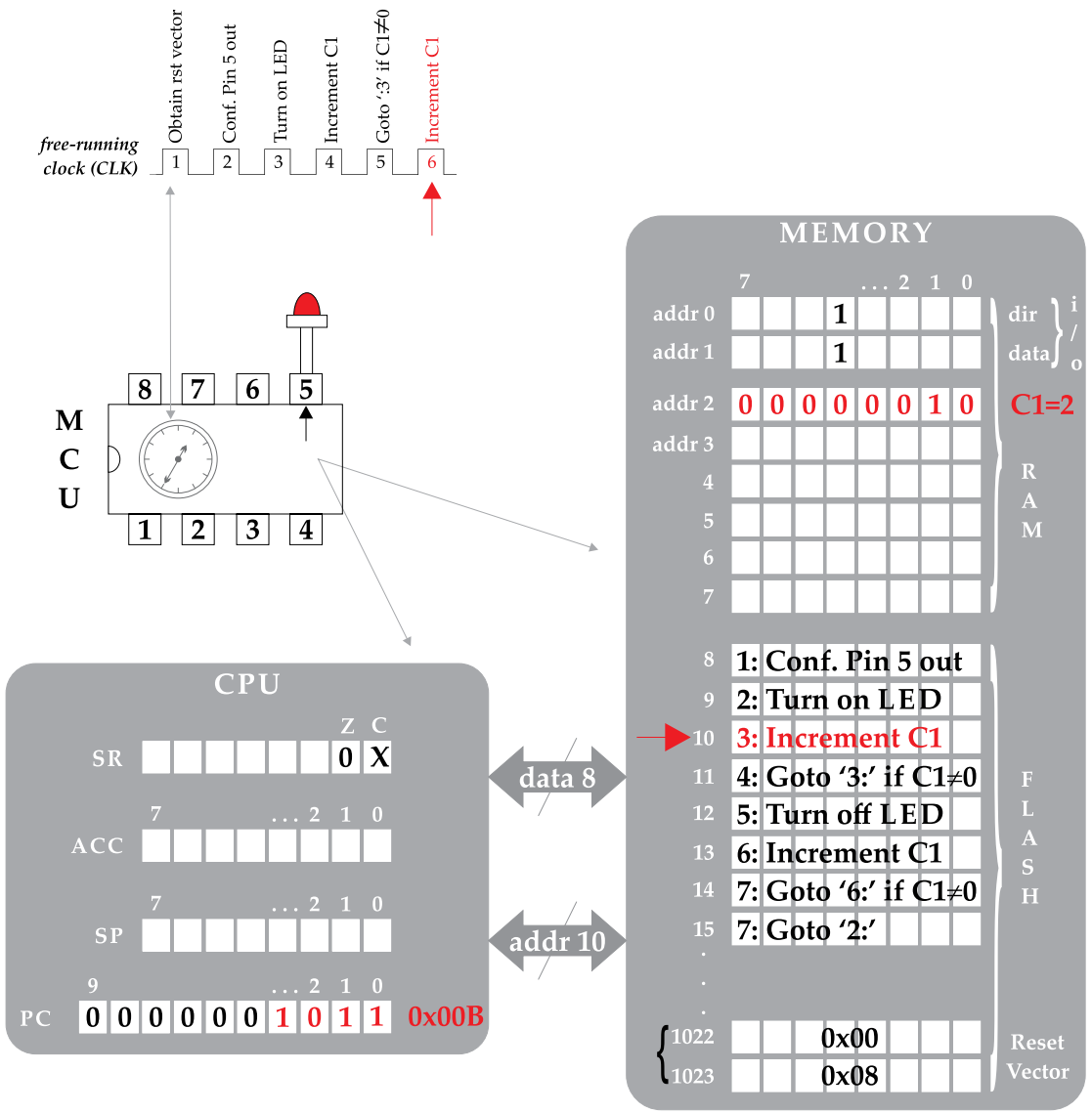


Figure 2.29: CPU delay (step 4).

machine instructions regularly affect the status register. Accordingly, an increment of the C1 content when the latter holds the maximum value (that is, $2^8 = 255_{10} = 11111111_2$) will cause an overflow of the C1 value. The latter process resets the value in C1 register and sets the carry flag to point out an *overflow* condition.[6] Because the register value resets to zero the ZF flag is normally raised as well.

The aforementioned procedure is depicted in Figures 2.30–2.33. In detail, the C1 register has reached the maximum value (i.e., C1=255) during the clock period 513, therefore the code line 4 sends the control flow to code line 3 (Figure 2.30). The unary increment to the content of C1 register upon clock period 514 resets register to zero and, hence, the ZF and CF are raised (Figure 2.31). The former flag is evaluated upon clock period 515 and because ZF=1, the content of the program counter increases (i.e., PC=0x00C) so as to point out to the next available instruction in memory (Figure 2.32). Thus, the control flow resumes the sequential execution and the CPU executes the instruction that turns the LED off in the clock period 516 (Figure 2.33). We may now observe 512 clock periods in between the instructions that turn the LED on and off, because of the consecutive execution of code lines 3 and 4 (which increase and evaluate the content of C1 register) for 256 times. It is worth noting that these two lines will repeat after the LED is turned off and, therefore, delays are regularly implemented as subroutines and invoked by the main program.

Delay routines are regularly addressed to support the human's interface with the microcontroller-based system. Another regular example is the delay routine addressed to eliminate the bouncing phenomenon which occurs when a mechanical switch is pressed and released. Figure 2.34 illustrates the phenomenon when the switch is pressed on. Normally it is expected a clear transition from high to low voltage upon pressing and holding the switch on. Yet, the switch signal alternates between the on and off states for a few msec. Those glitches are perceived by the CPU clock and hence, a method for the elimination of this undesirable feature should be considered. A regular approach is to delay the CPU for about 50 msec in order to assure the switch debounce. It is worth noting that delay routines are also useful during the configuration of subsystems used by the microcontroller application, which need some time to be initialized when, for example, a value is assigned to a register of the subsystem.

2.3 INTERRUPTS, PERIPHERALS, AND REGULAR HARDWARE INTERFACES

A microcontroller system regularly incorporates some input/output units and peripheral IC devices where the MCU (i.e., the brain of the system) is separately interfacing with each one of them. When the microcontroller waits to receive an input signal from an external device (e.g., pressing a button on the keypad of an alarm system), it performs either a *polling* operation to

[6]The corresponding operation that decrements the content of a register, when the latter value equals to zero, is referred to as *underflow* condition and causes the register to be loaded to its maximum value (i.e., $255_{10} = 11111111_2$ for an 8-bit register). Again, the carry flag will normally be raised (i.e., CF=1) to reveal the underflow condition.

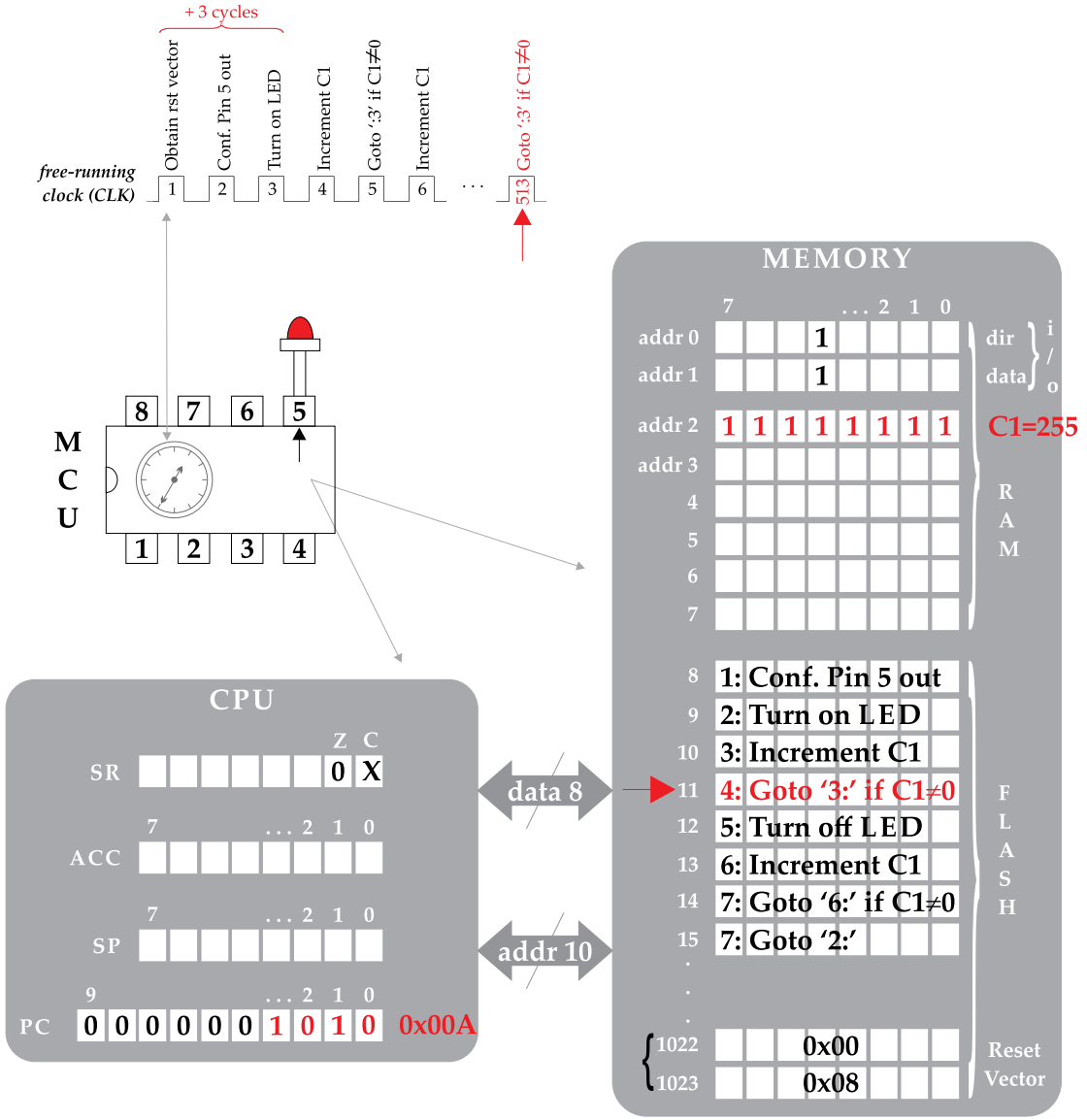


Figure 2.30: CPU delay (step 5).

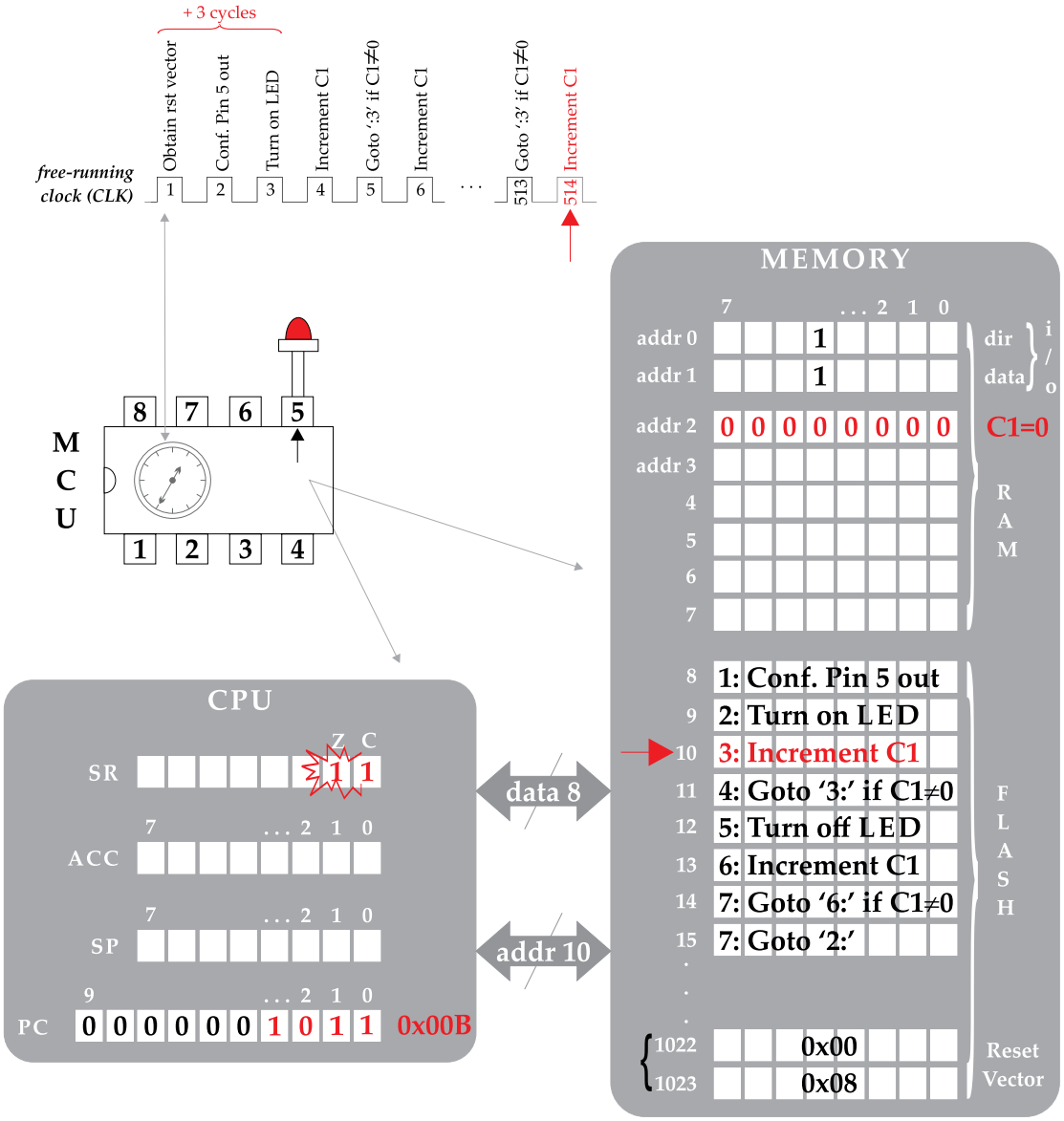


Figure 2.31: CPU delay (step 6).

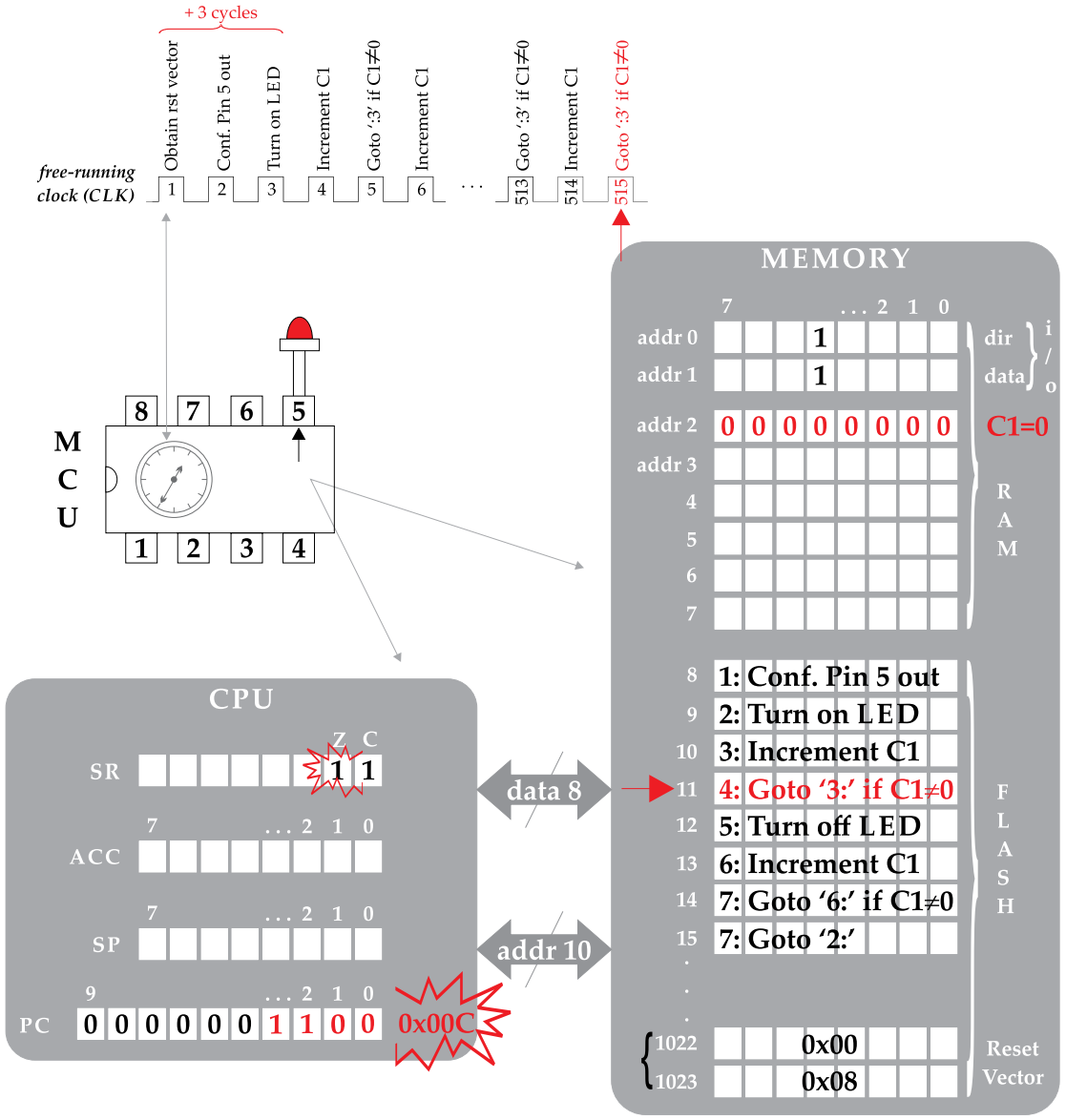


Figure 2.32: CPU delay (step 7).

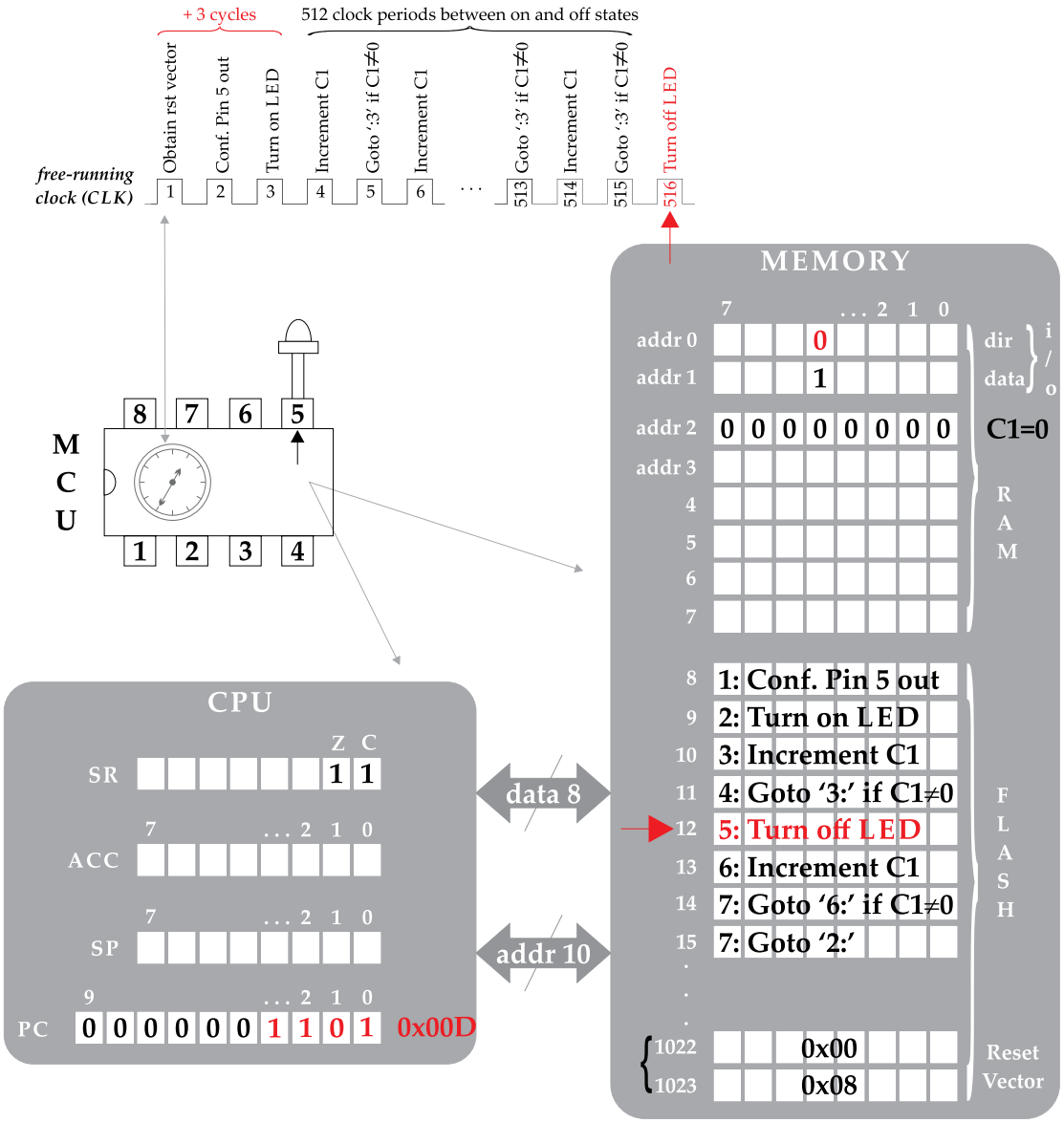


Figure 2.33: CPU delay (step 8).

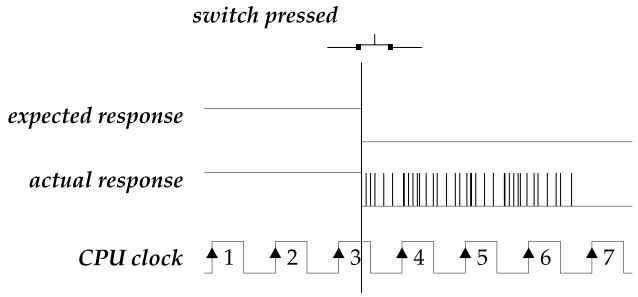


Figure 2.34: Switch bounce effect.

this signal or automatically responds to an internal mechanism called *interrupt*. In the former case the CPU executes commands for inspecting the state of a signal (e.g., a high-to-low voltage level transition in one of the MCU pins). In the latter case, the CPU may execute commands of some other operation (e.g., inspecting the state of the door/windows in the alarm system), or be suspended in a low-power mode,[7] and then respond automatically to an interrupt request.

The fetching of an interrupt request during the code execution is similar to the way the reset vector is fetched when the MCU is powered up. The firmware should configure the so-called *interrupt vector* with the effective address of an *interrupt service routine* (ISR), to which the control flow is forced upon the activation of the corresponding interrupt mechanism. The interrupt mechanism utilizes the RAM stack to assign the returning address for resuming the control flow as soon as the ISR execution finishes. The very last instruction of the ISR (i.e., return from interrupt) retrieves the returning addresses from the stack and resumes the control flow to the machine instruction that would have been executed if the interrupt was not activated.

The following example illustrates the execution on an external interrupt request. Microcontroller devices regularly incorporate a special function pin (denoted *irq* in this example), capable of activating an interrupt upon a high-to-low voltage level transition to this particular pin. Interrupt mechanisms can be considered internal peripheral subsystems of the MCU and, hence, their configuration is achieved through a control (i/o) register. It is worth noting that microcontrollers may incorporate several interrupt sources where each one of holds a particular priority in the system. Thereby, when multiple interrupt mechanisms are at the same time activated, the CPU arbitrates requests according to the priority of each interrupt source.

Figure 2.35 presents the servicing of an external interrupt request through a push-button connected to the irq pin. The circuit depicted in the figure illustrates a more realistic approach of the i/o units interconnection to the MCU. In more detail, the LED is connected in a scheme of positive logic, that is, through a resistor (R) pulling the cathode of LED down to earth, while the push button asserts the irq pin to common earth (i.e., logical 0) when it is pressed. When the button is released, the irq pin is connected to power (i.e., logical 1) through a pulled-up resistor.

[7]The CPU is entered into low-power mode of operation through particular assembly mnemonics (e.g., STOP, WAIT, etc.).

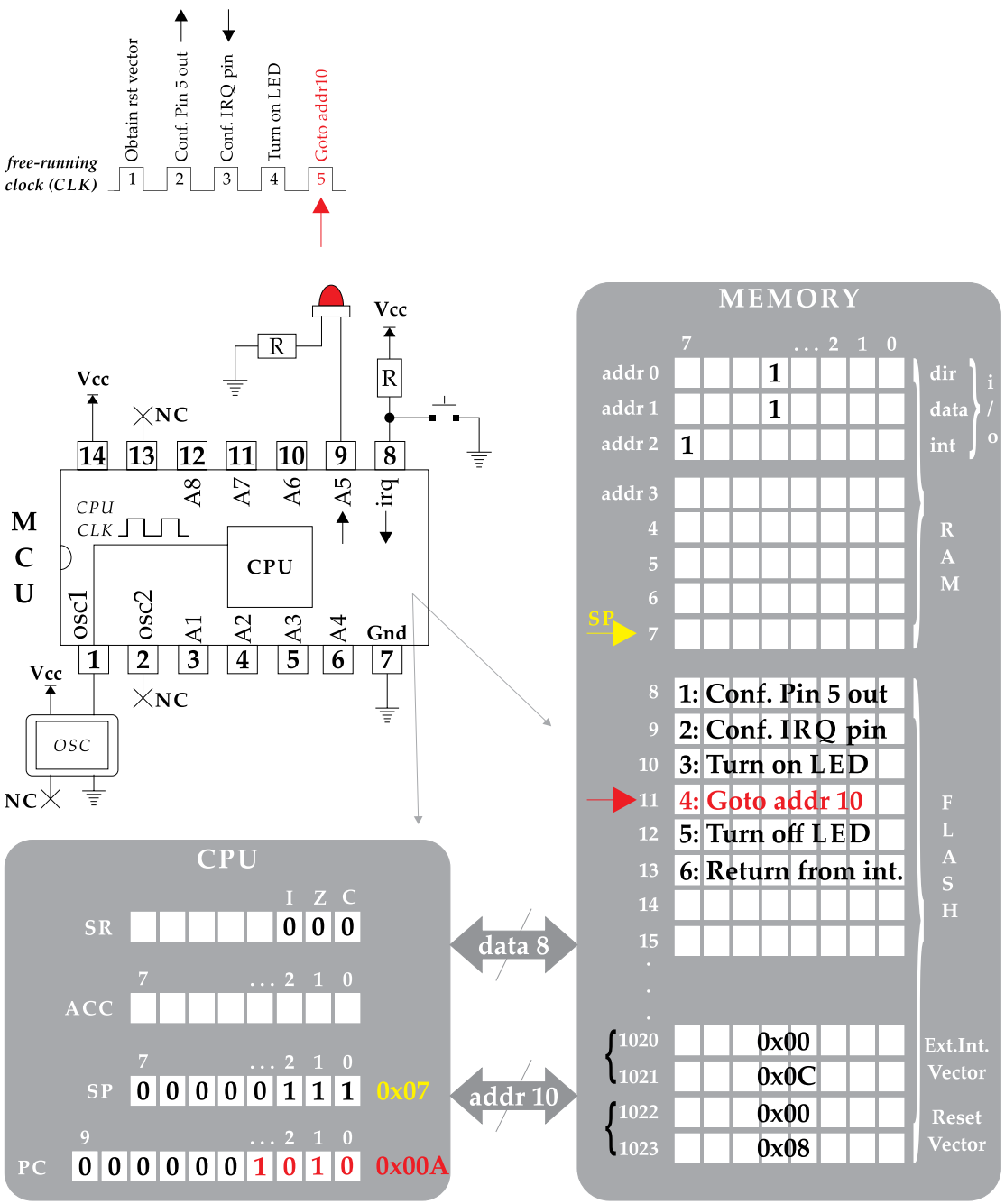


Figure 2.35: External interrupt (step 1).

In Figure 2.35, the CPU has already executed the former three firmware instructions (that configure pin 5 output, activate the external interrupt mechanism through the irq pin, and turn on the LED), and now executes the fourth instruction that sends the control flow to code line 3 for the repetition of the command that turns the LED on. The execution of the latter instruction, in Figure 2.36, has no particular effect on the LED as it has already been turned on. The execution of code lines 3 and 4 are repeated until the activation of the external interrupt in Figure 2.37, through the pressing of the push button. In more detail, upon the execution of the "Goto addr10" instruction (clock period 17) the push button connected to the irq pin is switched on. To preserve simplicity in the description, we consider that no bouncing effects take places upon pressing/releasing the switch.

The CPU finishes the execution of the current instruction and performs a series of actions before fetching the ISR. In Figure 2.37, a particular bit of the status register, called *interrupt flag* (IF), is set to logical 1 so as to prevent subsequent interrupts (that might occur) to take place during the servicing of the current interrupt mechanism. In Figure 2.38, the effective address for resuming the control flow (after the servicing of the interrupt instructions) is pushed onto the stack (clock period 18) and, hence, the SP register is decreased to point to the next available (free) space on the stack. In Figure 2.39, the content of PC is loaded to the launching address of the ISR (clock period 19). Thereby, the control flow is moved to the very first instruction of the ISR in Figure 2.40, while the execution of the current instruction forces the LED to turn off. The next instruction is the concluding one of the ISR and hence, a series of actions are performed in Figure 2.41 in order to return from the interrupt routine. In detail, the returning-from-interrupt address is retrieved from the stack and loaded to the PC register, the SP register increases so as to point out the current available free stack space and, finally, the ZF resets to zero so as to allow other interrupts to take place (clock period 21). Thereby, the execution of the instruction in Figure 2.42 turns again the LED on. It is worth noting that the external interrupt mechanisms will (normally) be re-activated as soon as the user release and re-press the push button.

As mentioned before, the interrupt mechanisms can be considered internal peripherals of the MCU. Modern MCUs embed several (internal) peripheral devices such as, *ADC*, *DAC*, *EEPROM*, *timers*, *pulse width modulation* (PWM), *analog comparators*, *real-time clock* (RTC), *USB controller*, *universal asynchronous receiver/transmitter* (UART), *serial peripheral interface* (SPI), *Inter-Integrated Circuit* (I2C), etc. All these internal modules require particular configuration and handling operations through the corresponding *control*, *data*, and *status* registers for their utilization. Because of the inconsistency of the available modules and associated memory registers among different MCUs, we will avoid using the internal subsystems in this book. The DIY culture of our age allows us to straightforwardly connect a (low-cost) mezzanine card that employs one or more subsystems (peculiar to the application under development) to the microcontroller development board; a choice addressed to support the code adaptability among diverse microcontroller devices.

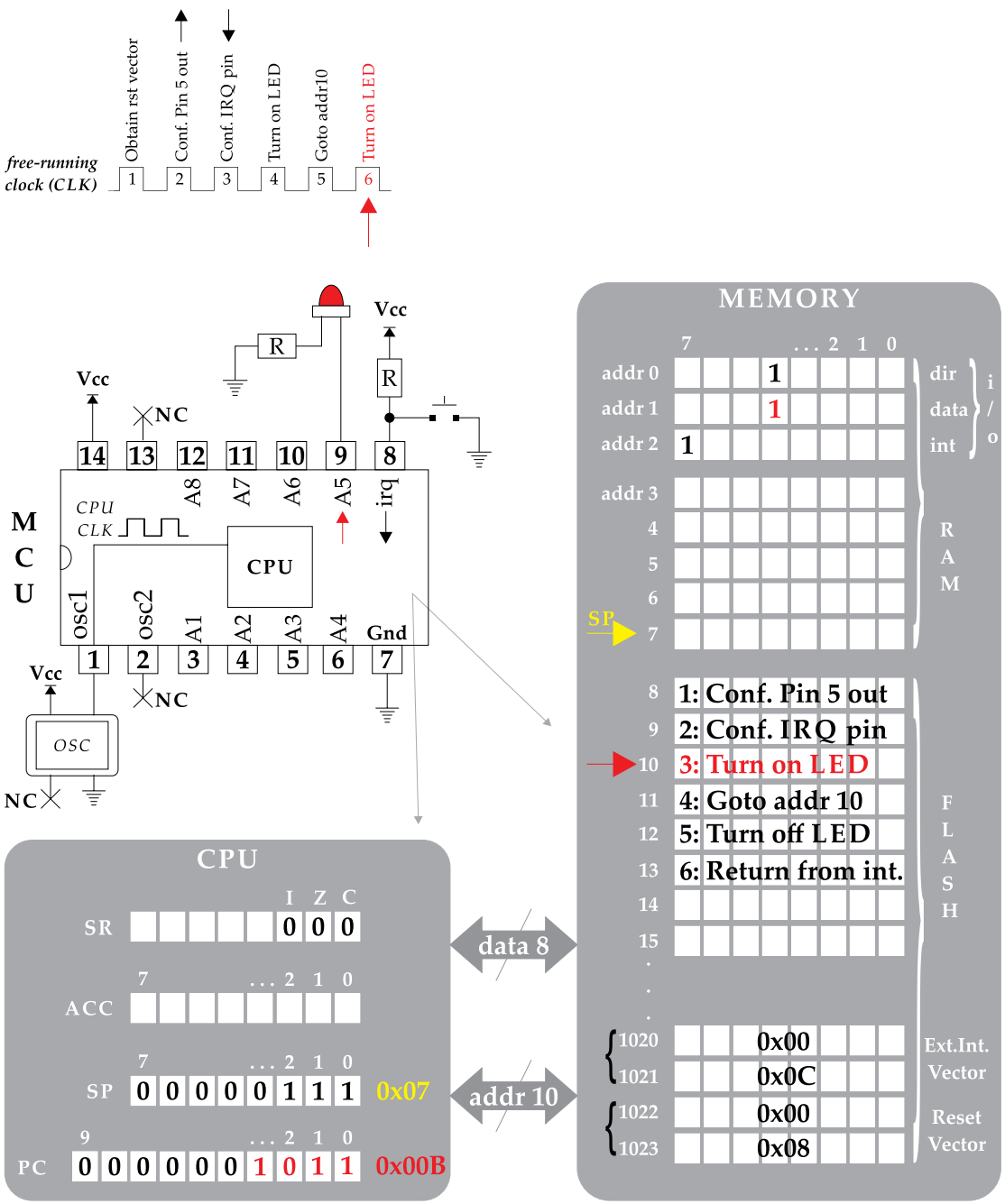


Figure 2.36: External interrupt (step 2).

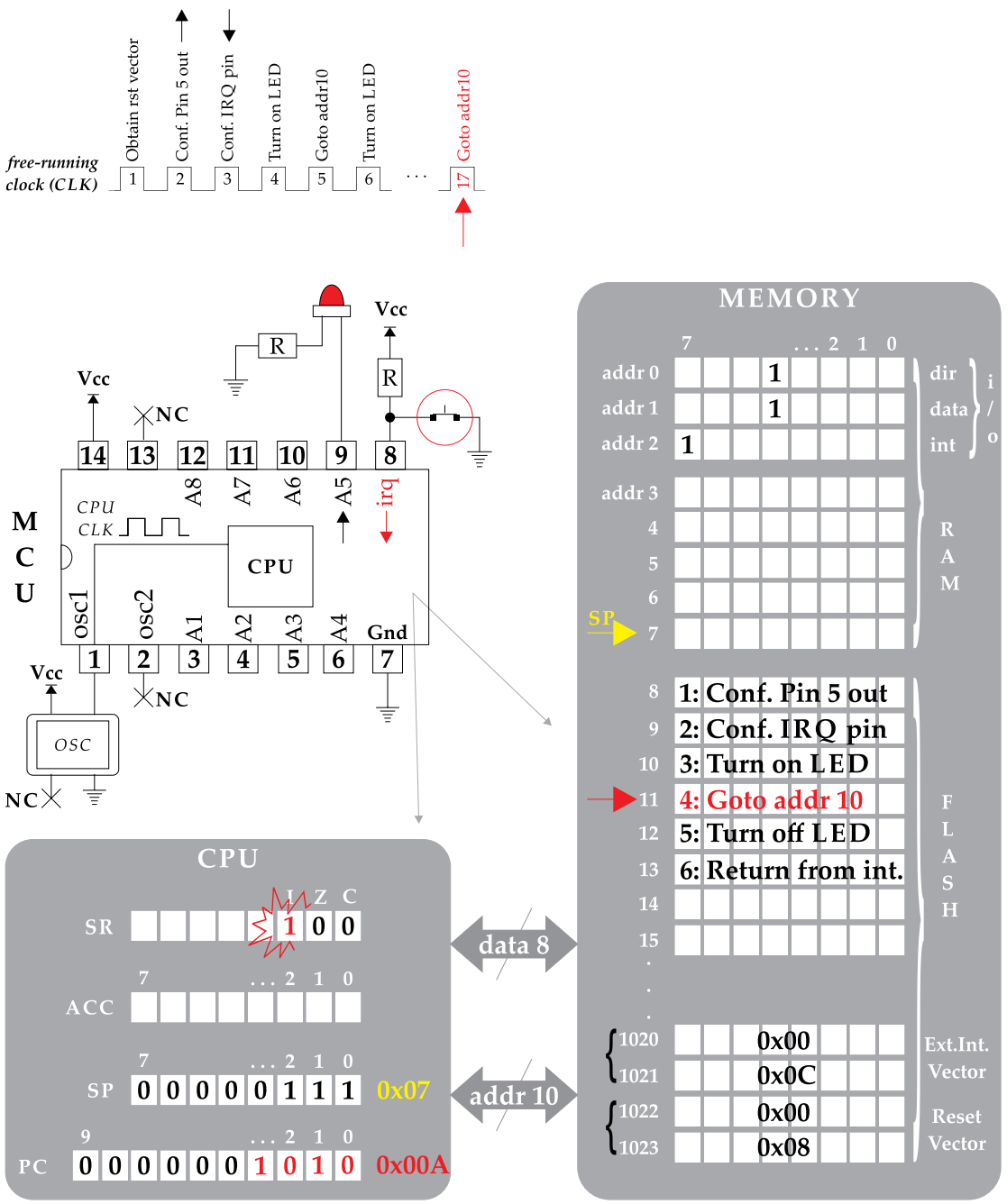


Figure 2.37: External interrupt (step 3).

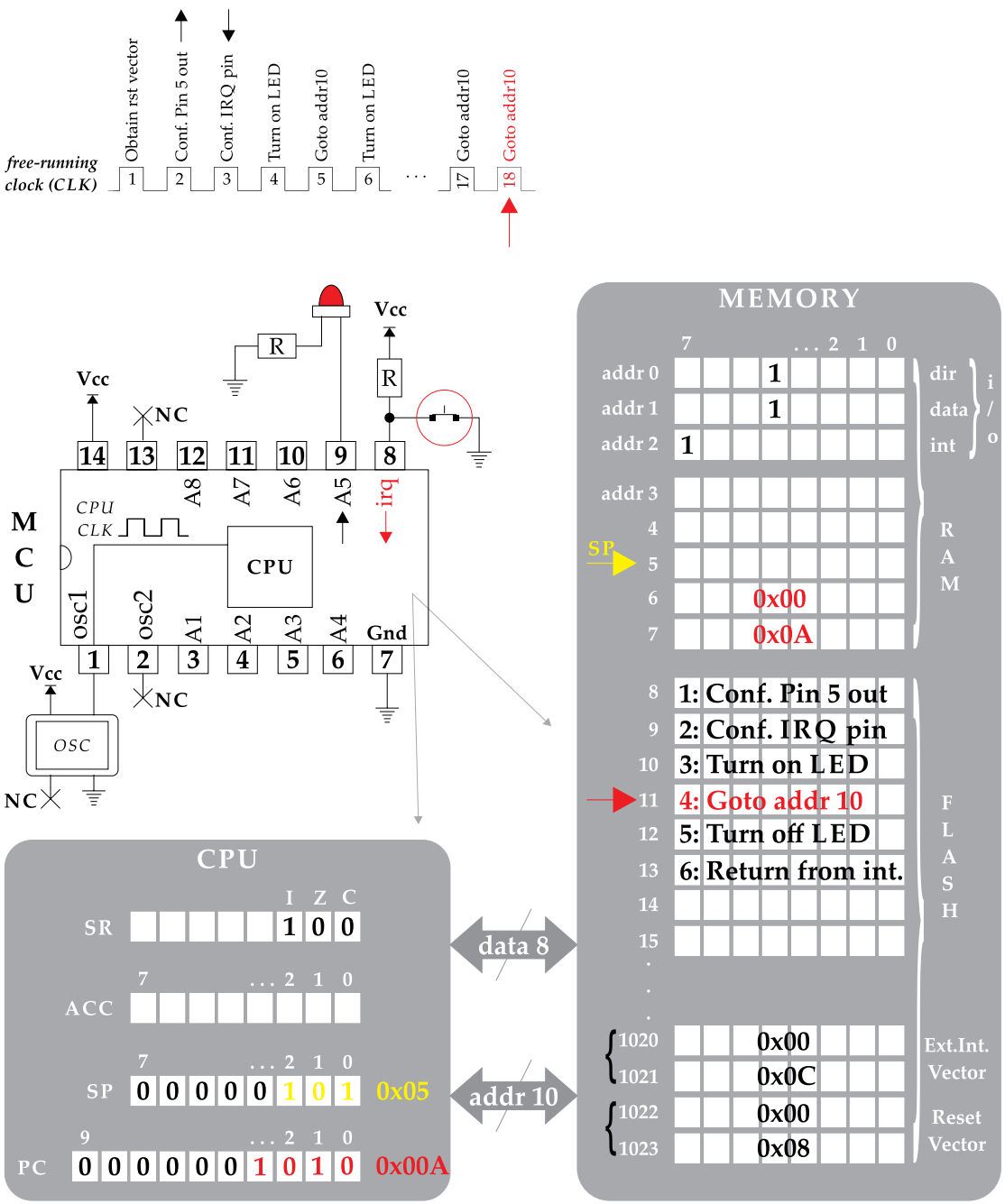


Figure 2.38: External interrupt (step 4).

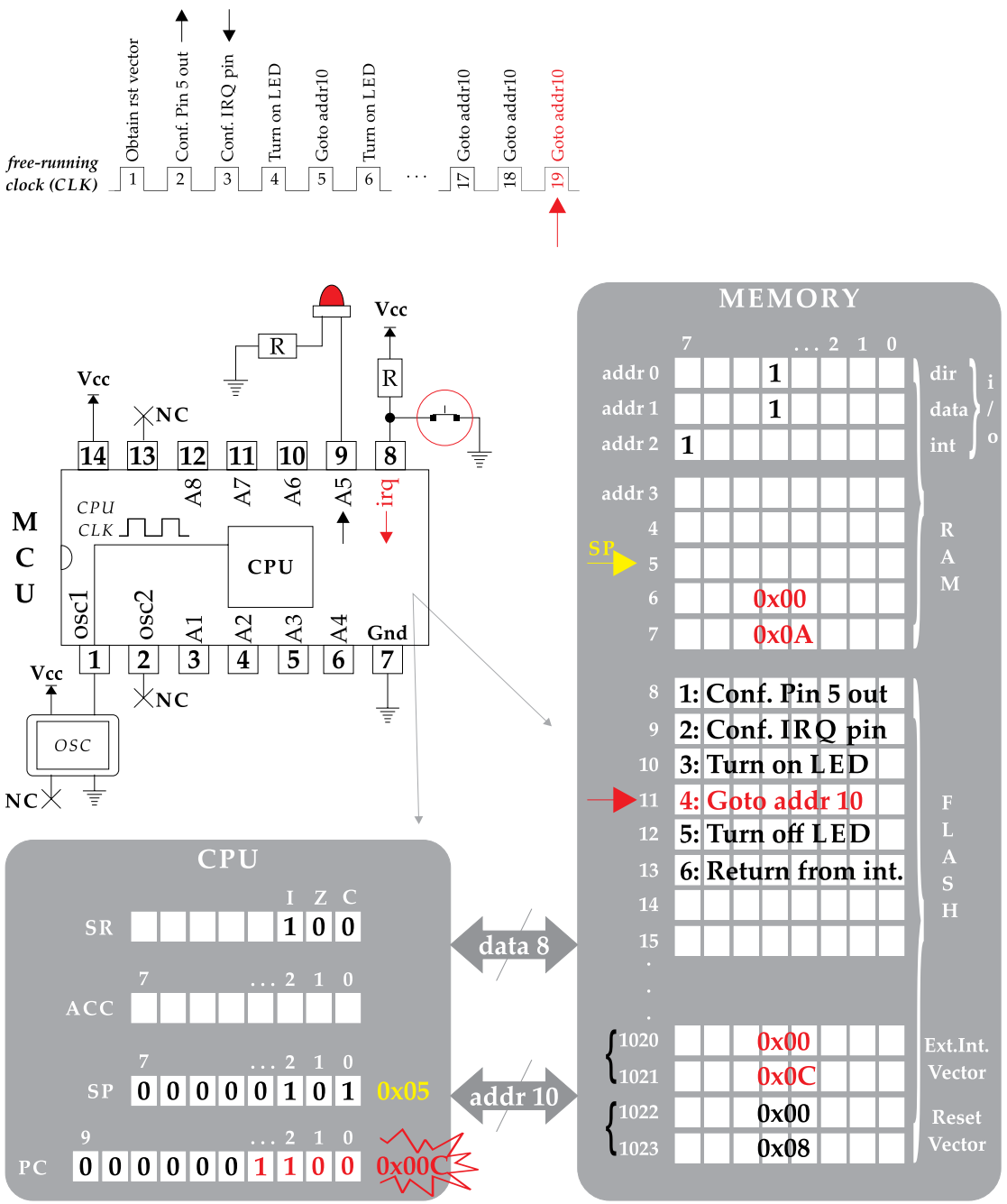


Figure 2.39: External interrupt (step 5).

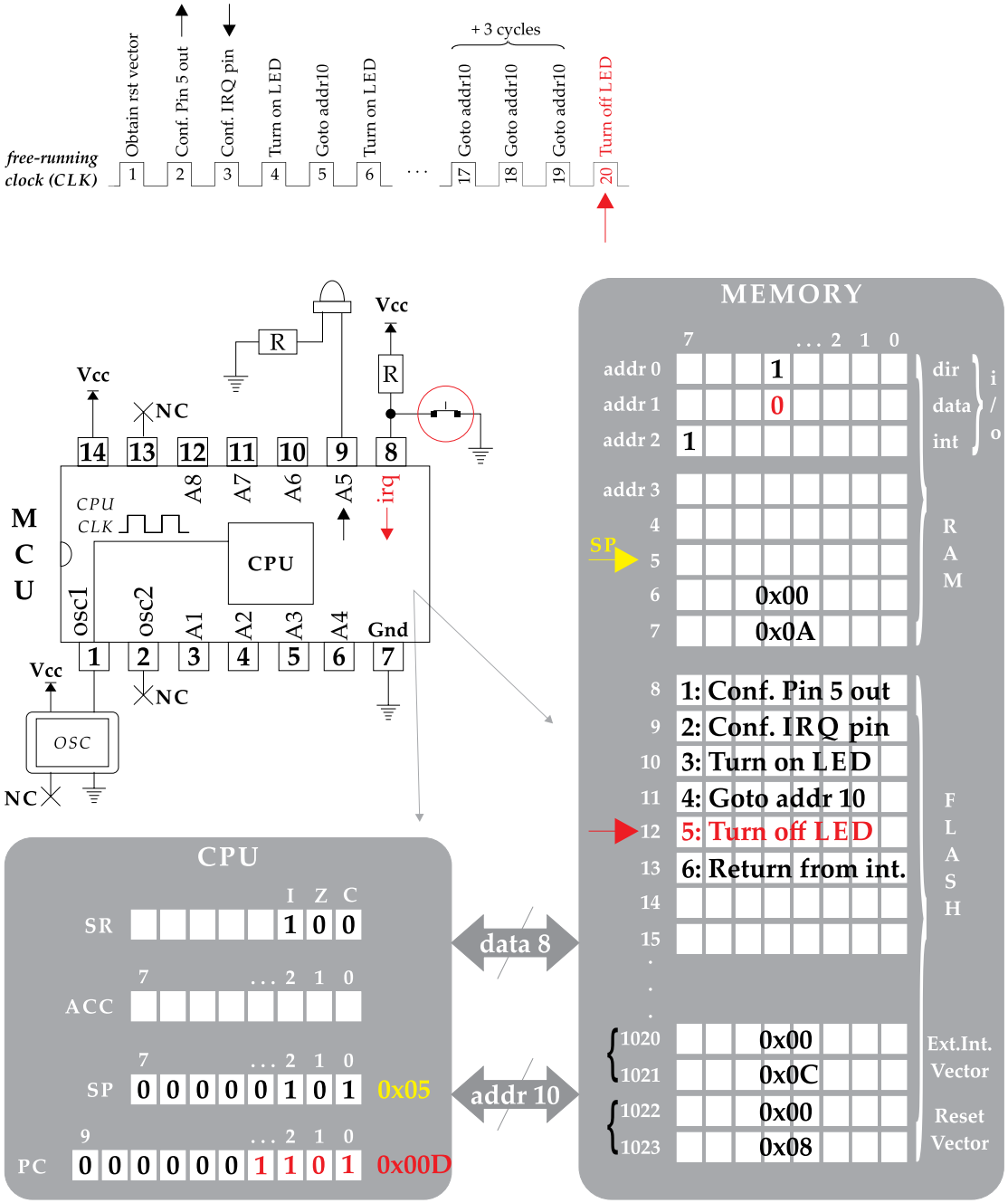


Figure 2.40: External interrupt (step 6).

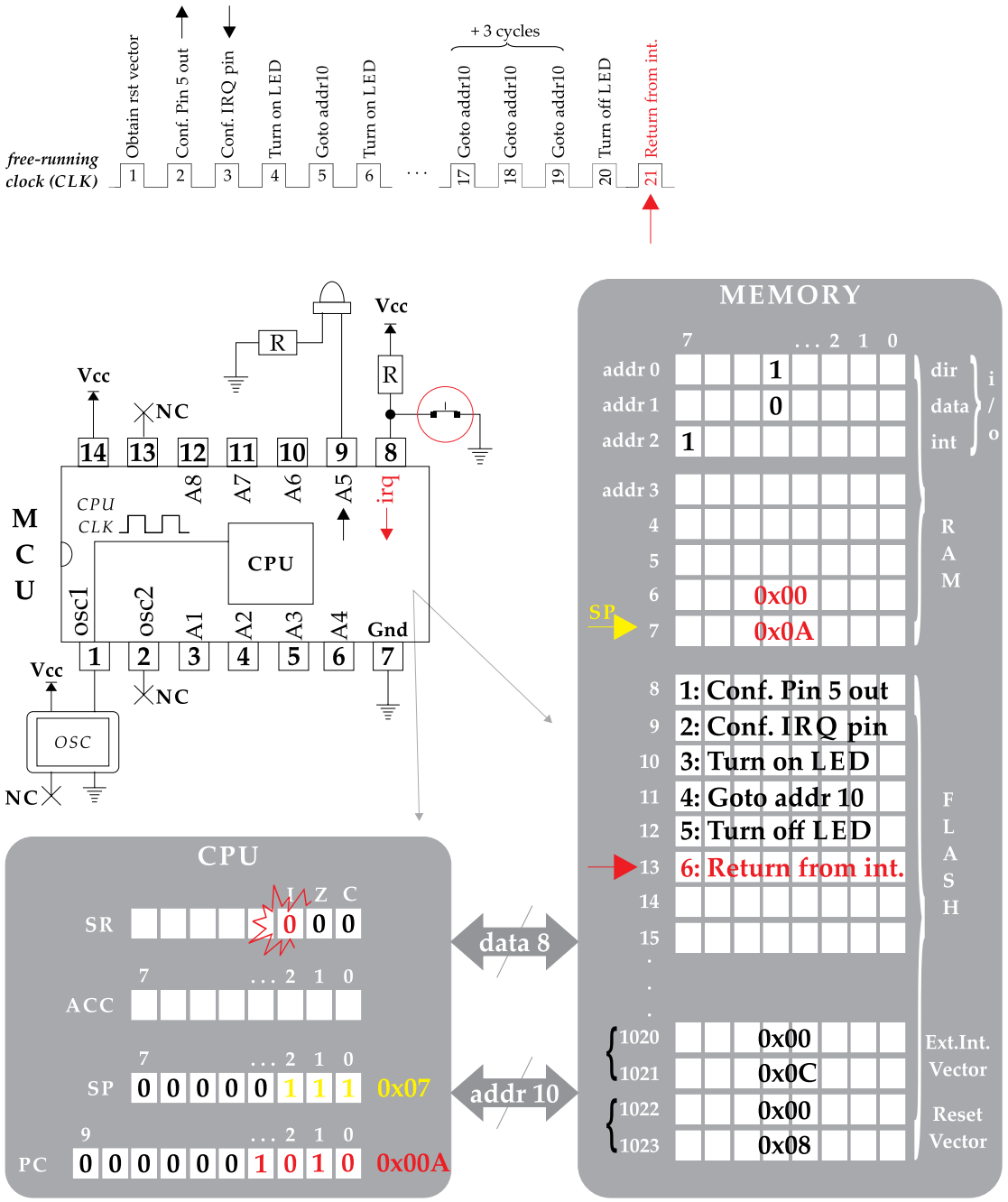


Figure 2.41: External interrupt (step 7).

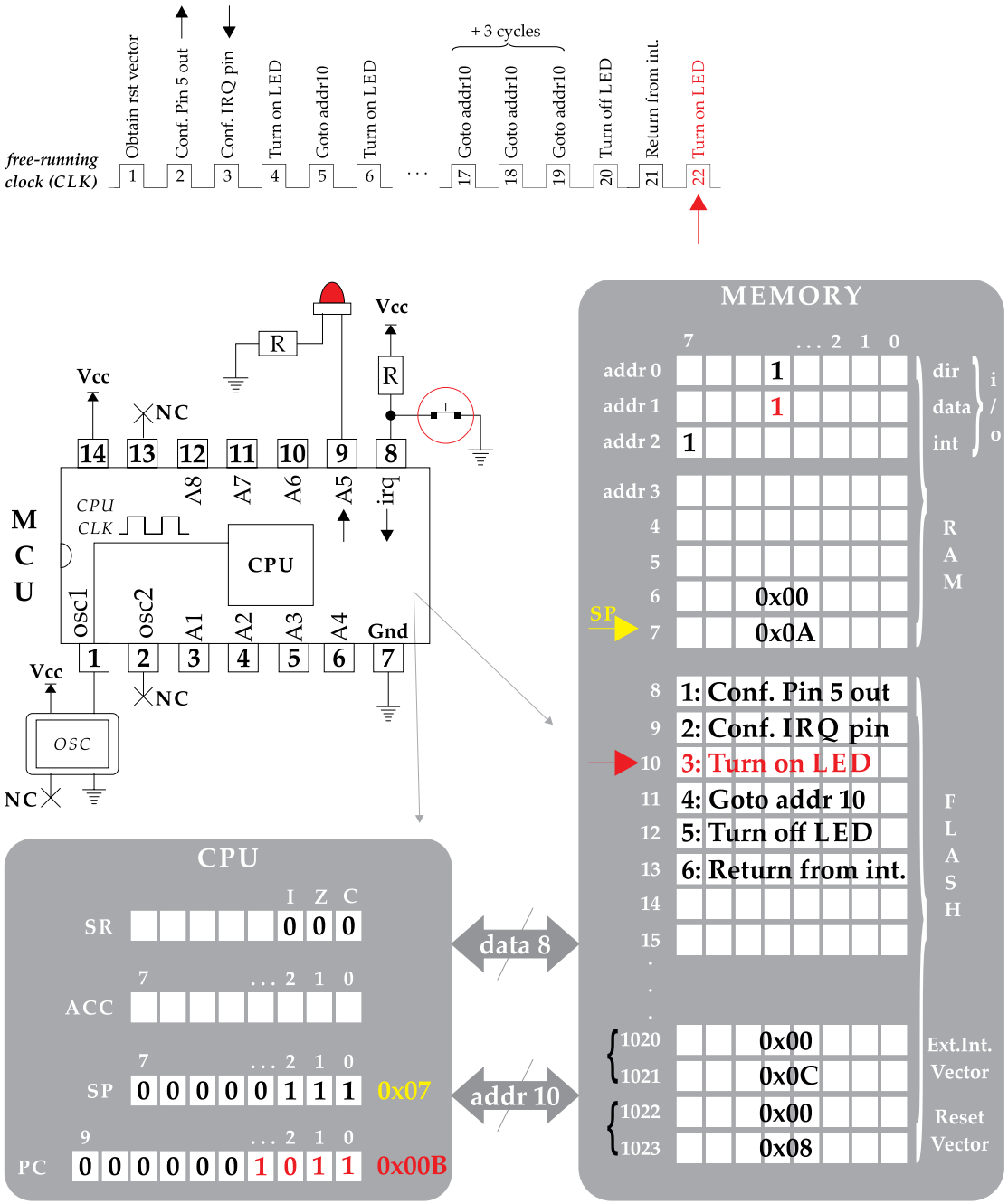


Figure 2.42: External interrupt (step 8).

The external peripheral modules can be controlled through a parallel or serial interface. The former approach might be more easily considered in the firmware development process, however, it reserves several port pins in the MCU. Thereby, the focus of the book is placed on serial interfaces for the control of the external modules. The most popular are the Motorola's SPI and the Phillip's I2C, which are often considered as "little communication" (synchronous) protocols, over the Ethernet, USB, and other sophisticated protocols meant for "outside the box communication" [51]. The SPI reserves four port pins, while the I2C reserves only two pins. These protocols, along the UART that will be used for exchanging data with a computer, are analyzed later in the book.

CHAPTER 3

Micro-Controller Programming

While the programming approach of this book is in line with embedded C programming, this chapter first provides an overview to the assembly language programming possibilities, which could be found assistive to the designers when directing the compiler toward efficient machine code generation, and/or during the debugging process of the application program. Thereafter, it presents a brief overview to C language, as well as a programming strategy that is in line with the two virtues of C; that is, the adaptability of the source code among different development platforms as well as the low-level accessibility of the hardware system.

3.1 HIGH-LEVEL VS. LOW-LEVEL PROGRAMMING

High-level languages are built around a set of symbols, keywords, punctuation marks, etc., which are familiar to humans and are addressed to provide a clear link between our reflection and the tasks performed by a computer. On the other hand, the assembly language consists of a predefined set of mnemonics, where each one of them is capable of providing total control over each machine instruction. Hence, it could be said that programming in assembly language requires conforming the designer's thought to the way tasks are performed at the machine level. Due to the need to go down into the machine level and think like the processor acts, this approach is regularly referred to as *low-level programming*. On the other hand, the *high-level programming* approach reduces the distance that humans must cover from their thought down to the underlying hardware principles and, hence, there no need to know much about the processor's model in order to make it operational over an assignment.

Because the "short distance" found in-between the assembly-level programming and the hardware tasks, the program that translates the assembly mnemonics into machine code is referred to as *assembler*. However, the "long distance" that should be covered for the translation of a high-level application program into machine language creates the need for a different program translator, aka *compiler*. Because no compiler knows the application code better than the designer does, it is often important to understand how the computer works and direct the compiler toward the generation of an efficient machine code [52]. For that reason, we next explore the basic assembly inventory (common among different MCUs), as well as the composition of the fundamental programming possibilities at the assembly level.

The separation of the assembly language mnemonics, independent of the employed MCU, can be addressed with reference to the fundamental tasks performed at the machine level, that is: (a) memory/register assignment, (b) program flow of control, and (c) arithmetic and bitwise operations [17]. Table 3.1 presents the basic assembly mnemonics for the popular 8-bit AVR [53] and PIC18 [54] microcontrollers (where both of them now belong to Microchip [55]), according to aforementioned separation. In consideration of the memory assignment mnemonics, the latter either assign a number or the content of a memory location to the CPU registers (and vice versa), or affect the flags of status register. In regard to the flow of control mnemonics, we observe tasks related to unconditional and conditional branch. The former tasks (analyzed in the previous chapter) perform an unconditional jump to a memory address, a call to subroutine, or respond to an interrupt. Finally, the basic arithmetic mnemonics perform and unary increment/decrement to a register or add/subtract a value (or memory content) to/from a register, while the bitwise mnemonics either rotate or generate an `AND`, `OR`, `XOR` operation to the content of a register.

To demonstrate the differences between high-level and low-level programming we next provide the composition of the basic programming possibilities, while also make a parallelization to the C language and AVR/PIC assembly programming. For someone having no previous experience on C and assembly language, it requires a significant endeavor to get started with microcontrollers programming, because of the influx of the new information related to this exciting technology. The DIY culture has left aside all the background theory, and initiated to the community a new programming approach for MCUs, which has rendered microcontroller technology usable to hobbyists with no particular experience in this field.

However, this book aims at educating readers through an approach that puts the necessary effort on simplifying the background theories and separating the involved topics into only some fundamental categories. We started earlier with the separation of the assembly mnemonics into a three-group inventory. It is worth noting that almost every program in assembly language can be implemented with those simple instructions, and the designer deciding to get involved with a different MCU may merely identify the corresponding mnemonics of the proposed inventory. We hereafter explore the fundamental programming possibilities that are also separated into a few fundamental categories, which can be addressed for the implementation of any particular application code. All that is needed is to take the first step, while the designer that has obtained the fundamental background theory is capable of self-enhancing and self-enriching their knowledge toward more advanced issues of microcontroller-based applications (which clearly cannot be administrated by hobbyists and DIY-only fanatics).

Earlier in the book we discussed the concept of sequential programming and focused on pseudocode techniques and to the way they are realized at the machine level. The examples illustrated the implementation of an unconditional branch (to a memory location) for the eternal execution of a code section, as well as the implementation of subroutines and ISRs calls that sustain the top-down and bottom-up design of the application code. We next explore how the

Table 3.1: Basic assembly inventory for AVR and PIC microcontrollers

Mnemonic (AVR)		Mnemonic (PIC)	
Memory/Register Assignment			
LDI	Load immediate	MOVLW	Move literal value to WREG
LDS	Load direct from data space	MOVF	Move *file register* (*f*) to WREQ/f
STS	Store direct to SRAM	MOVWF	Move WREQ to f
CLC	Clear carry flag (CF)	BCF	Bit clear f
CLZ	Clear zero flag (ZF)		
CLI	Clear global interrupt flag (IF)		
SEC	Set CF	BSF	Bit set f
SZF	Set ZF		
SEI	Set IF		
Program Flow of Control			
BRCC	Branch if carry cleared (CF=0)	BNC	Branch if not carry (CF=0)
BRNE	Branch if not equal (ZF=0)	BNZ	Branch if not zero (ZF=0)
BRBC	Branch if bit in status is cleared	BTFSC	Bit test f; Skip next if clear
BRCS	Branch if carry set (CF=1)	BC	Branch if carry (CF=1)
BREQ	Branch if equal (ZF=1)	BZ	Branch if zero (ZF=1)
BRBS	Branch if bit in status is set	BTFSS	Bit test f; Skip next if set
JMP	Jump to memory address	GOTO	Go to address
CALL	Long call to a subroutine	CALL	Call subroutine
RET	Return from subroutine	RETURN	Return from subroutine
RETI	Return from interrupt	RETFIE	Return from interrupt enable
Arithmetic/Bitwise operations			
DEC	Decrement (reg minus 1)	DECF	Decrement f
INC	Increment (reg plus 1)	INCF	Increment f
ADC	Add with carry	ADDWFC	Add WREG and carry to f
SBC	Subtract with carry	SUBWFB	Sub WREG from f with borrow
ROL	Rotate left through carry	RLFC	Rotate left f through carry
ROR	Rotate right through carry	RRFC	Rotate right f through carry
ANDI	Logical AND with immediate	ANDLW	AND literal to WREG
AND	Logical AND	ANDWF	AND WREG with f
ORI	Logical OR with immediate	IORLW	Inclusive OR literal with WREG
OR	Logical OR	IORWF	Inclusive OR WREG with f
EOR	Exclusive OR	XORLW	Exclusive OR literal with WREG
		XORWF	Exclusive OR WREG with f

CPU can be directed through particular mnemonics toward the implementation of *conditional brunching* tasks, which along with the unconditional branches cover a significant (if not the utmost) percent of the coding style regularly met in microcontroller's firmware code.

At this point it would be wise to explore the basic flowchart symbols that will be used to describe the code examples of the book. Figures 3.1a,b present the descriptive symbols for opening and closing the code, respectively. Inputting data from, as well as outputting data to, the outside world is illustrated by the symbol of Figure 3.1c, while Figure 3.1d denotes a data processing task performed by the MCU. Changing the regular flow of a program according to a particular decision that is taken is depicted by Figure 3.1e. The sequential execution of instructions is sustained when the evaluated condition if found to be *true* (T), otherwise the control flow follows the path of the *false* (F) condition, or vice versa (according with the designer's requirements). Finally, the two symbols of Figure 3.1f can be used to decompose an extended flowchart to smaller pieces, using a *number* (n) inside of the circle to denote the conceivable connections of the diagram.

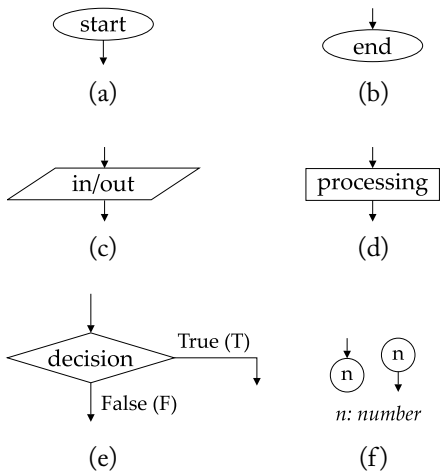


Figure 3.1: Basic flowchart symbols.

Figure 3.2 depicts the C language clauses that are used to provide a conditional branch to the program flow. Each clause starts with an identical *keyword* which goes ahead of an *expression*. The expression is regularly placed within *parentheses* and if the evaluated condition is true, the program flow (continues the sequential execution of the code and it) is inserted into the *compound statement* of the clause within the identical *braces* (`{..}`). The *double quotes* (`"`) used in the figure are referred to a statement.

The identical keywords of C language for implementing a conditional branch clause, so as to alter the control flow, are summarized in Table 3.2. The `if` (and `if-else`) clause decides on executing or skipping (one of either) statements (Figure 3.2a). The `switch` (as well as the

```
if ("test expression")
{
 "compound statement 1"
}

else
{
 "compound statement 2"
}

"subsequent statements"
```

(a)

```
switch ("integer variable")

{
 case "n1": { "compound statement 1"}
    break;

 case "n2": { "compound statement 2"}
    break;

 case "n1": { "compound statement 3"}
    break;

 default: { "compound statement 4"}
}

"subsequent statements"
```

(b)

```
"integer variable"

for ("init";"test";"inc/dec")
{
 "compound statement"
}

"subsequent statements"
```

(c)

```
while ("test expression")
{
 "compound statement"
}

"subsequent statements"
```

(d)

```
do
{
 "compound statement"
}
while ("test exression");

"subsequent statements"
```

(e)

Figure 3.2: Conditional branching clauses in C (`if-else`, `switch-case`, `for`, `while`, `do-while`).

`else if`) clause provides a multi-way conditional branching preference according to the evaluated `case`, which follows the `switch` keyword (Figure 3.2b). The `for`, `while`, and `do-while` clauses are addressed to implement iterative loops. However, the former clause realizes a predefined number of iterations (Figure 3.2c), while the latter two are more suitable for undefined number of iterations (Figures 3.2d,e), e.g., when the MCU waits for an incoming event (such as, a key button to be pressed by the user).

The test expression that decides on the branching action regularly incorporates a *rational operator*, and the available operators in C language are also summarized in Table 3.2. On the other hand, the branching instructions in assembly language regularly evaluate the flag bits of status register, and commonly the *zero* and *carry* flags (Table 3.1). Accordingly, if we want to evaluate an expression such as *lower*, *higher*, *same*, etc., we regularly prepare an arithmetic operation just before the evaluation of the ZF and CF. Among the most important factors that define the barriers of understanding in the low-level programming techniques is the unavoidable employment of "go to" conditional and unconditional statement. The disastrous effects of

Table 3.2: Keywords of conditional branching clauses and associated operators in C language

Keywords	Rational Operators	
if	<	lower
else (or else if)	>	higher
switch	<=	lower or same
case	>=	higher or same
for	==	equal
while	!=	not equal
do		
Additional control flow keywords in C: ***break***, ***continue***, and ***goto***		

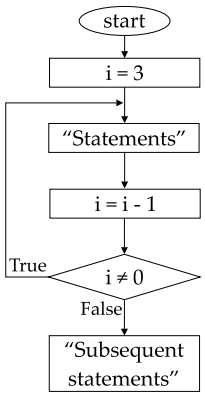
such statements have been identified in the early years of programming languages and, hence, its usage is being eliminated from the high-level programming techniques of nowadays [56].

The example of Figure 3.3 presents an iterative loop in C (Figures 3.3b,c) and assembly (Figures 3.3d,e) programming, using `for` and `do-while` clauses and a pre-decrement operation (as depicted by the flowchart of Figure 3.3a). The test expression of the `for` clause incorporates three parts separated by a *semicolon* (`;`) character (Figure 3.3b). The first (aka *initialization expression)* is executed only once when the loop is fetched and it is addressed to assigns an initial value to a variable[1] (herein assigning the number 3 to `i` variable). The second part holds the condition that is evaluated every time the loop is executed, and determines whether the *compound statement* inside of the loop will be repeated (or the CPU will exit the loop) in the upcoming cycle. The test expression of this example decides on repeating the loop as long as the content of `i` register (aka *loop counter*) is not equal to zero. The latter expression provides unary[2] decrement to the content of loop counter every time the `for` clause is executed. When the unary operator precedes the variable name (as in this particular example) the decrement operation is executed before evaluating the test condition (i.e., the subtraction $i - 1$ is executed before the evaluation of $i \neq 0$). The latter task is clearer with the syntax of the equivalent `do-while` clause

[1]The term `variable` refers to a register reserved in data memory and, hence, its contents can change during the execution of code.

[2]The decrement operation does not have to be unary; for instance, the expression $i = i - 2$ will subtract number 2 from the loop counter.

in Figure 3.3c. At this particular example it does not matter if we syntax `"--i"` or `"i--"` as the condition `"i!=0"` proceeds the unary decrement operation.



(a)

```
for (i=3;i!=0;--i)
{
 "compound statement"
}

"subsequent statements"
```

(b)

```
i=3;
do
{
 "compound statement"
 i--;
} while (i!=0);

"subsequent statements"
```

(c)

```
        movlw   0x03
        movwf   i,1
Loop1:  "statements"
        decf    i,1,1
        bnz     Loop1
        "subsequent statements"
```

(d)

```
        ldi     r16,$03
        sts     i,r16
Loop1:  "statements"
        dec     i
        brne    Loop1
        "subsequent statements"
```

(e)

Figure 3.3: Iterative loop in C and PIC/AVR assembly (pre-decrement operation).

The equivalent PIC and AVR assembly language is depicted by Figures 3.3d,e, respectively. A few things that would be wise to make clear about the assembly-level programming is as follows: the code is separated into three columns where the first holds the labels, the second holds the assembly mnemonics, and the third holds the argument(s) of the mnemonic, in case the latter accepts argument(s). Optionally, the designer may use a fourth column for comments, provided that a semicolon character (;) is inserted immediately before the comment line. The labels of the first column are associated with memory locations in program, as well as data memory. In the former case a "go to" mnemonic that accepts a label as argument, will send the control flow to the effective memory address associated to this particular label. In the latter case, particular assembler directives (such as the EQU abbreviation of the word *equate*) are used to associate descriptive names to a memory location. In this particular example it is assumed that the descriptive name `i` has been previously assigned to a register from data memory. Finally, the immediate values assigned to a register are regularly performed in hexadecimal notation, where the regular prefixes denoting this system are the `0x` and `$` (and sometimes the `h` suffix is used instead).

The former two instructions in the assembly code load number three (3), denoted in hexadecimal numeral system, into `i` register (Figures 3.3d,e). This particular operation is achieved with the utilization of the CPU register(s). The number is initially loaded into the CPU register (code line 1), and then the content of the CPU register is assigned to `i` register (line 2). In detail, the PIC devices incorporates a *general purpose register* (GPR) in the CPU, which is

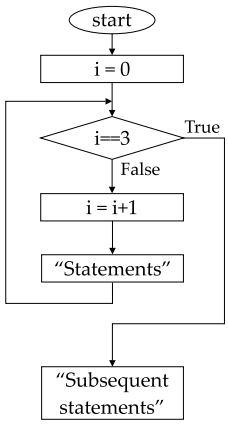
called *working register* (WREG). On the other hand, the AVR devices hold 32 GPRs named *Rd*, where *d* admits values from 0–31. However, when loading an immediate value to an AVR CPU register through LDI mnemonic (Figure 3.3e), only registers R16-R31 are allowed to be used. The (unary) pre-decrement operation is performed in code line 4. In PIC devices (Figure 3.3d), the DECF mnemonic admits three arguments, where the former represents the data register to be decreased. The second argument defines whether the result will be loaded back into the data register (if set to 1) or into WREG (if set to 0). The third argument is peculiar to the architecture of PIC devices and in simple words, when it is set to 1 it allows accessing more content of the available internal data memory (while the accessible data memory is kind of limited when this argument is set to 0).

In AVR devices (Figure 3.3e), the DEC mnemonic admits only a single argument referring to the data register to be decreased. In both devices, the decrement operation affects the status register and when the outcome is of zero value, the corresponding flag is raised. Subsequently, the BNZ & BRNE (Figures 3.3d,e) mnemonics send the control flow to the memory location labeled `loop`, as long as ZF=0. Otherwise, the program flow resumes the sequential execution of instructions, from the pseudocode line denoted `"subsequent statements"`. It is worth noting that all but STS (which is a two-cycle) mnemonic are one-cycle instructions. However, the branch instructions require two successive cycles when they alter the control flow (otherwise they are executed in one cycle only).

The pre-decrement is the most optimal task in consideration of the generated machine code. In the next example, we explore the iteration implemented with a post-increment task to the content of loop counter. In this particular procedure the counter should be initially reset to zero and then increased up to a value. This task is the least optimal implementation, as it overloads the assembly code with extra mnemonics and, hence, the code reserves more bytes in memory and its execution requires extra cycles.

A `for` loop with a post-increment (Figure 3.4b) or post-decrement task is equivalent to a `while` clause (Figure 3.4c). We may straightforwardly observe that both PIC (Figure 3.4d) and AVR (Figure 3.4e) assembly incorporate extra mnemonics, compared to the previous (pre-decrement) example. When the evaluated condition is other than zero, a subtraction should be performed between the existing value of loop counter and the desired number of iteration (that is, three for this particular example). Thereafter, the status flags are evaluated so as to decide on the program flow of control.

The mnemonics that perform subtraction are regularly avoided in such examples because the store the result to (one of) the involved register(s). Such conditions are regularly evaluated by a *compare* instruction which performs *on the fly* subtraction (i.e., it does not store the result to the involved register). Compare instructions, as well as compound instructions that perform on the fly subtraction and then a branching task, are presented in Table 3.3. The CPFSEQ mnemonic which is used in the PIC example of Figure 3.4d (line 4) performs the following on the fly subtraction (f)-(WREG), and skips the subsequent instruction in case the result is zero (i.e.,



(a)

```
for (i=0;i!=3;i++)
{
 "compound statement"
}

"subsequent statements"
```

(b)

```
i=0;
while (i!=3)
{
 i++;
 "compound statement"
}

"subsequent statements"
```

(c)

```
        movlw   0x00
        movwf   i,1

Loop1:  movlw   0x03
        cpfseq  i,1
        goto    repeat
        goto    end

repeat: incf    i,1,1
        "statements"
        goto    Loop1


end:    "subsequent statements"
```

(d)

```
        ldi     r16,$00
        sts     i,r16

Loop1:  cpi     i,3
        breq    end

        inc     i
        "statements"
        jmp     Loop1

end:    "subsequent statements"
```

(e)

Figure 3.4: Iterative loop in C and PIC/AVR assembly (post-increment operation).

Table 3.3: On the fly subtraction and compound mnemonics for AVR and PIC

Mnemonic (AVR)		Mnemonic (PIC)	
On the fly arithmetic operations (i.e., subtractions) and compound mnemonics			
CP CPC CPI	Compare two registers (Rd, Rr) Compare two registers with CF Compare with immediate		
"*compound instructions: compare & branc*"		"*compound instructions: compare & branc*"	
CPSE	Compare and skip if equal	CPFSEQ CPFSGT CPFSLT	Compare f with WREG, skip if = Compare f with WREG, skip if > Compare f with WREG, skip if <

the two registers are of equal value). On the other hand, the CPI mnemonic of AVR example of Figure 3.4e (line 3) compares `i` register to the (decimal) value 3 and, hence, the subsequent instruction evaluates the value of zero flag.

While compilers are designed to generate the optimum machine code, it is always wise to apply an effective coding style that would direct them toward an improved compilation of the high-level application code [57]. The two-way and multi-way conditional branching at

the assembly level is based on the same philosophy as the iterative loop described before. It is worth noting that the iterative loops apply to an abundance of tasks within the microcontroller's firmware, such as, memory allocation processes, (that is, initialization and accessing of one-dimensional arrays in memory), delay loops, etc.

3.2 A BRIEF OVERVIEW TO C PROGRAMMING LANGUAGE

Before exploring embedded C programming and the hardware/software issues that one should be concerned with toward the adaptability of the code among different development platforms, we provide a summary of the C language programming possibilities. It is now clear that the MCUs constitute complete computer systems, but are of limited abilities. Thereby, it is our intention to keep the programming techniques as simple as possible so as to sustain the code's adaptability.

3.2.1 OPERATORS, DATA TYPES, CONSTANTS, AND VARIABLES

The fundamental areas that will be discussed herein[3] are as follows: (a) data types (and size of) variables and constants; (b) control flow for suspending/alternating the sequential execution of code (where some issues have already been introduced to the reader); (c) functions and preprocessor directives for decomposing a program into smaller segments (and hence, supporting top-down/bottom-up programming methods); and (d) arrays and pointers (in association with flow-of-control/loop statements for initializing and accessing the array elements).

Table 3.4 presents the overall operators that can be found in a C language program. Those operators regularly co-operate with variables and either affect, or evaluate, their content. In the latter case, the content of a variable can be assessed to a constant value. The type and size of *variables* and *constants* is determined by one of the *data type* keywords presented in Table 3.5. When the `signed` qualifier precedes the `char` or `int` type, then the corresponding variable/constant admits positive and negative values (commonly expressed in a *two's complement* notation). On the other hand, the `unsigned` qualifier forces the variable/constant to positive-only values (including zero), which are in line with *modulo* 2^n arithmetic. For instance, an `unsigned char` variable obtains numbers 0–255 (i.e., from 0 to 2^8), while the corresponding signed variable admits decimal values from −128 to +127 (*2's complement* notation). Some syntax examples associated with C language constants are also given in Table 3.5. It is worth noting that the optimal machine code is generated when using the smallest applicable data type for a variable/constant [57]; that is, the byte-size length for 8-bit MCUs.

[3]The C language input/output operations for a personal computer are quite different from the corresponding operations addressed for an MCU-based system and, therefore, they are not explored by this section.

Table 3.4: C language operators

Arithmetic Operators		Rational Operators		Assignment Operators	
++	Unary increment	==	Equal	=	Assign (from right to left)
--	Unary decrement	!=	Not Equal	+=	Add and Assign
+	Addition	<	Lower	-=	Subtract and Assign
-	Subtraction	<=	Lower or Same	*=	Multiply and Assign
*	Multiplication	>	Higher	/=	Divide and Assign
/	Division	>=	Higher or Same	%=	Modulus and Assign
%	Modulus and Remainder out of Integer Division			&=	Bitwise AND and Assign
				|=	Bitwise OR and Assign
Bitwise Operators		**Logical Operators**		^=	Bitwise XOR and Assign
&	Bitwise AND	&&	Logical AND	<<=	Shift Left and Assign
|	Bitwise OR	||	Logical OR	>>=	Shift Right and Assign
^	Bitwise XOR	!	Logical NOT		
~	One's Complement	**Ternary Operator**		**Address-of and Indirection Operators**	
<<	Binary Shift Left	?:	Conditional Expression, e.g.: x = (a>b)? a : b;	&	Point-to the Address Variable
>>	Binary Shift Right			*	Obtain the Content of the Memory Address

3.2.2 ARRAYS AND INDEXING TECHNIQUES

Among the most common tasks performed by an MCU firmware are the initialization and accessing of array elements in memory. In consideration of the initialization process this can only be performed in data memory, while a regular firmware code for microcontrollers incorporates single-dimensional arrays (i.e., as the elements are actually assigned into the MCU memory). We have already discussed the C language clauses for alternating the program flow and we will henceforth explore how to associate loops with arrays, and how to construct functions that cooperate with arrays (in order to decompose the code into smaller segments).

Figure 3.5a presents the initialization of a single-dimensional array in program memory. The selection of the latter type of memory is identified by the keyword `const`. This *type qualifier* can be applied to (i.e., qualify) any C declaration in terms of a non-modifiable object. An optional value within the *square brackets* (`[ ]`) may be addressed so as to declare the number of elements held by the array, while omitting this value (like in this example) will cause the compiler to adjust the array size in accordance with the employed elements. Accessing the array

Table 3.5: Data types and constants in C language

Data Type	Description
char	Byte size: *ideal type for holding the 'printable / ASCII' characters, as well*
int	Integer of 16-bit or 32-bit long (determined by the compiler): *regularly the natural size of registers found in a computer system*
float	Single-precision floating point
double	Double-precision floating point
short int (or *short*)	Different, but no longer, than *int* type (and at least 16-bit long)
long int (or *long*)	Equal or longer than *int* type (and at least 32-bit long)
Constant Example	**Description**
1987	A plain number refers to an *int* type of constant; herein represented in decimal numeral system (and is 16-bit long)
0x07C3 / 0b1010	Hexadecimal (prefix 0x or 0X) / Binary notation (0b or 0B)
17101987L	Upper-case or lower-case (L or l) suffix declares a long constant
17101987UL	Unsigned Long notation (UL or ul)
'@'	Character constant placed within *single quotes*
'\0x0D'	Syntax of non-printable characters (e.g., *carriage return*)
"Hello"	String constant placed within *double quotes*, while also holding a null character '\0' (i.e., zero value) at the end of the string
1987.17	Floating point / *double type* constant, unless suffixed (exponent notations are also allowed): ✓ *f* or *F* suffix denotes a *float* constant ✓ *l* or *L* suffix denotes long double
Constants are declared using the ***#define*** *preprocessor directive or the* ***const*** *qualifier (where the latter can be used with array arguments as well), e.g.:* ✓ *#define MASK 01* ✓ *const char arrayExample[] = "hello world";* ✓ *int functionExample (const char[]);*	

elements in C language is commonly achieved by indexing the array name through `subscripts`. A subscript regularly refers to a number that is placed within square brackets and immediately after the array name, which is used to identify a particular array element. The first array element is identified by number zero and, hence, the expression `x=romARRAY[0]` (where `x` an unsigned char type of variable) will result in assigning x with character `'h'` in this particular example, `x=romARRAY [1]` will result in assigning `x` with character `'e'`, and so forth.

```
const unsigned char romARRAY[] = {"hello world"};
```
(a)

```
int i;
unsigned char x;

for (i=0; i<=10; i++)
{
  x=romARRAY[i];
}
```
(b)

```
int i=0;
unsigned char x;

while ( romARRAY[i] != '\0')
{
  x=romARRAY[i];
  ++i;
}
```
(c)

```
int i=0;
unsigned char x;
do
{
  x=romARRAY[i];
}while(romARRAY[++i] != '\0');
```
(d)

Figure 3.5: Array indexing in C language (access data in program memory).

Figure 3.5b addresses a `for` loop which uses a subscript to access the elements of the aforementioned array in program memory. The string "hello world" holds 11 characters and, therefore, the array indexing runs from 0 up to 10 (i.e., $i = 0, 1, 2, \ldots, 10$) and successively loads each particular array element into `x` variable (i.e., `x='h'`, `x='e'`, `x='l'`, …, `x='d'`). When we declare a string array in C language, the compiler automatically appends a terminating (null) character of zero value at the end of the string. Thereby, the romARRAY of this example consists of the following (individual) elements: `'h'`, `'e'`, `'l'`, `'l'`, `'o'`, `'<space>'`, `'w'`, `'o'`, `'r'`, `'l'`, `'d'`, `'\0'`. The zero value at the end of the string allows us to employ a `while` or `do-while` loop for the array indexing process. The benefits of addressing `while` and `do-while` clauses for loops of undefined number of iterations relies on the fact that we can modify the number of array elements without needing to adjust the number of iterations inside the loop. Thereby, the `while` loop of Figure 3.5c waits for the terminating character to be received in order to break the execution of the loop, while providing an unary increment to the `i` counter of the loop so as to successively obtain each individual array element.

It is also possible to insert the unary increment within the test expression (and optimize further the source code), just like in the `do-while` example given by Figure 3.5d. However, it should be paid attention to the position of the unary (`++`) operator. Herein, the unary operator precedes the `i` variable and the test expression is evaluated after the unary increment of the loop counter. Thereby, the control flow exits the `do-while` loop without loading the null character into `x` variable. The syntax `while(romARRAY[i++]!='\0')` would result in loading null character into `x` variable and then exiting the loop. It is worth noting that if we put the unary

operator in the test expression of the `while` clause (Figure 3.5c), either as `++i` or `i++` syntax, we will omit loading the first array element (i.e., `romARRAY[0]`) into `x` variable. We could, however, insert the post-decrement task within the while loop, that is, `x=romARRAY[i++]`. It is noted that the pre-decrement task (i.e., `x=romARRAY[++i]`) will not work as it will also skip the first array character.

Table 3.6 presents the *printable* and *control* characters of the *American standard code for information interchange* (ASCII). In C language, the *backslash* (\) symbol followed by one or more characters is used to denote an *escape sequence*, which could be addressed for the representation of a control (non-printable) character. Some of the most commonly used escape sequences in C are also given by this table.

Besides the use of subscripts, the array indexing in C language is also possible through the use of *pointers*. A pointer refers to a variable, which is declared with the *indirection operator* (`*`). Then, the *address-of operator* (`&`) is used to assign a memory address to the pointer variable. For instance, the `char *ptr;` syntax declares a variable pointing to a `char` type of data, the `ptr=&y;` command line assigns the memory address of `y` to `ptr` variable, and the `x=*ptr;` loads into `x` variable the content of memory address pointed by `ptr` (i.e., the content of `y` is copied into variable `x` in this example).

Figure 3.6 addresses array indexing examples in C language using pointers. In detail, the initial code line of Figure 3.6b declares a variable pointing to the beginning of an array in program memory (declared by `const`) of `unsigned char` data type (i.e., to the array of Figure 3.6a). Then, the test expression of the `while` clause evaluates then content of the current array element pointed by `ptROM` variable, and if the latter value in not equal to the null character it first assigns the current array element into `x` variable, and then provides unary increment to the content of pointer. Thereby, the next time the loop is invoked by the MCU the `ptROM` variable will point to the subsequent array element, and the loop is terminated as soon as the pointer indicates the very last array element of zero value (i.e., the element holding null character).

The example of Figure 3.6c addresses pointers toward the initialization and accessing of arrays in data memory. The first code line declares an array of seven elements in data memory and the second line declares `ptRAM` pointer, assigned to the memory address of the first array element. The first time the `for` loop is invoked it assigns `'a'` character (i.e., hex value `0x061`) into the first array element (pointed by `ptRAM`), as well as the null character into the second array element and then, the pointer value in unary incremented. The second time the `for` loop is invoked it assigns `'b'` character (i.e., hex value `0x062`) into the second array element (pointed by `ptRAM`), as well as the null character into the third array element, and so forth. Thereby, after six successive iterations the content of ramARRAY is as follows: `'a'`, `'b'`, `'c'`, `'d'`, `'e'`, `'f'`, `'\0'`. Then, the `while` loop is addressed for accessing the array in data memory and successively loading its elements into `x` variable (until null character is obtained).

Table 3.6: ASCII table and common escape sequences in C language

Low Nibble	High Nibble							
	0	1	2	3	4	5	6	7
0	NUL	DLE	space	0	@	P	`	p
1	SOH	DC1	!	1	A	Q	a	q
2	STX	DC2	"	2	B	R	b	r
3	ETX	DC3	#	3	C	S	c	s
4	EOT	DC4	$	4	D	T	d	t
5	ENQ	NAK	%	5	E	U	e	u
6	ACK	SYN	&	6	F	V	f	v
7	BEL	ETB	'	7	G	W	g	w
8	BS	CAN	(	8	H	X	h	x
9	HT	EM	)	9	I	Y	i	y
A	LF	SUB	*	:	J	Z	j	z
B	VT	ESC	+	;	K	[	k	{
C	FF	FS	,	<	L	\	l	|
D	CR	GS	-	=	M	]	m	}
E	SO	RS	.	>	N	^	n	~
F	SI	US	/	?	O	_	o	DEL

Control Characters

(HEX value) Abbreviation: Description

(0x00) NUL: Null	(0x10) DLE: Data link escape 1
(0x01) SOH: Start of heading	(0x11) DC1: Device control 1
(0x02) STX: Start of text	(0x12) DC2: Device control 2
(0x03) ETX: End of text	(0x13) DC3: Device control 3
(0x04) EOT: End of transmission	(0x14) DC4:Device control 4
(0x05) ENQ: Enquiry	(0x15) NAK: Negative acknowledgement
(0x06) ACK: Acknowledge	(0x16) SYN: Synchronous idle
(0x07) BEL: Bell	(0x17) ETB: End of transmitted block
(0x08) BS : Backspace	(0x18) CAN: Cancel preceding message/block
(0x09) HT: Horizontal tab	(0x19) EM: End of medium
(0x0A) LF: Line feed	(0x1A) SUB: Substitute for invalid character
(0x0B) VT: Vertical tab	(0x1B) ESC: Escape
(0x0C) FF: Form feed	(0x1C) FS: File separator
(0x0D) CR: Carriage return	(0x1D) GS: Group separator
(0x0E) SO: Shift out	(0x1E) RS: Record separator
(0x0F) SI: Shift in	(0x1F) US: Unit separator

Escape Sequences in C Language

\n	new line
\r	carriage return
\t	horizontal tab
\v	vertical tab
\xhh	any hex value (e.g., \x0D represents the carriage return)

```
const unsigned char romARRAY[] = {"hello world"};
```

(a)

```
const unsigned char *ptROM = &romARRAY[0];
unsigned char x;

while ( *ptROM != '\0')
{
  x=*(ptROM++);
}
```

(b)

```
unsigned char ramARRAY[7];
unsigned char *ptRAM = &ramARRAY[0];
unsigned char x;
int i;

for (i=0;i<=5;i++)
{
  *ptRAM = 0x61 + i;
  *(ptRAM+1) = '\0';
  ++ptRAM;
}

ptRAM = &ramARRAY[0];
while ( *ptRAM != '\0')
{
  x=*(ptRAM++);
}
```

(c)

Figure 3.6: Array indexing in C language using pointers (data in program/data memory).

3.2.3 USER-DEFINED FUNCTIONS

Functions in C language are used for decomposing an application program into smaller manageable and readable segments codes, which could be reused in other programs, too. While C language incorporates an abundance of *standard library functions*, we will only talk about *user-defined functions* because of our purpose to sustain the code's portability.

Figure 3.7a presents the general form of a function declaration starting with type of data that returns when executed, through the indicative `return` keyword at the end of the code. The `void` keyword is used when the function returns no value, while in this particular case the `return` keyword is omitted. Optionally, a function incorporates arguments of particular data type which are placed within parentheses, while the main code of the functions follows the parentheses and it is inserted within braces. Figure 3.7b presents an example of a function which admits two arguments of `unsigned char` type, while also returning `unsigned char` type of data. The function evaluates the content of arguments `a` and `b` and returns that character `'='`, `'<'`, or `'>'`, in case `a` is equal to `b`, `a` is lower than `b`, or `a` is higher than `b`, respectively.

Figure 3.7c presents the calls to the `compare` (user-defined) function using character literals. The first call forces `f` variable to be loaded to the character `'<'`, while the second and third calls load to `f` variable the ASCII characters `'='` and `'>'`, respectively. Figure 3.7d presents similar function calls, but it incorporates variable arguments instead of literals. The content of variables `a` and `b` are passed to the function and hence, the first call assigns `f` variable to `'='` character, while the second call assigns `'<'` character to `f`. It is noted that the variables used in the function call do not have to be of same name as the arguments used in the function declaration. In both programming approaches of Figure 3.7 the C passes arguments to functions by value; a method that is regularly referred to as *call by value* [58].

```
return-type function-name(arguments)
{
  "declarations" and "statements"
 return "data";
}
```

(a)

```
unsigned char f;

f=compare('1','7');
f=compare('7','7');
f=compare('7','1');
```

(b)

```
unsigned char compare(unsigned char a, unsigned char b)
{
  if (a==b)
    return '=';
  else if (a<b)
    return '<';
  else
    return '>';
 }
```

(c)

```
unsigned char a,b,f;

a=b='7';
f=compare(a,b);

a=1;
f=compare(a,b);
```

(d)

Figure 3.7: User-defined functions in C language (call by value).

It is also possible to perform a call to a function through another function just like the example of Figure 3.8. A new advanced function named `compareAdv` is declared in Figure 3.8a which performs a call to the aforementioned function (which is also presented in the figure example). The new function addresses the *logical* `AND` (`&&`) operator to evaluate if the `a` and `b` arguments admit ASCII characters from `'0'` to `'9'`. If the condition is true the `compareAdv` performs a call to the `compare` function and afterwards, it returns `f` value (as in the previous example). Otherwise, the `f` variable is assigned to the ASCII character `'#'` in order to denote that variables `a`, `b` incorporate characters other than the ASCII-encoded decimal. Accordingly, the consecutive calls to `compareAdv` function in Figure 3.8b assign to the following characters to `f` variable (and with this particular order): `'#'`, `'<'`, `'='`, `'#'`, `'>'`.

Previously in the chapter we explored how to associate subscripts and pointers with loops toward the accessing and/or initializing process of an array. Hereafter, we explore how we can integrate these tasks inside a function, which is a particularly useful process for the MCU firmware and the practices covered next in the book.

Figure 3.9b describes the `accessROM` function, which addresses subscripts and successively loads into *x* variable the array elements of Figure 3.9a. The same process is achieved by the `pointROM` function of Figure 3.9c, except this time pointers are used instead of subscripts. When an array name is passed to a function through its argument parameters, what is actually passed is the location of the initial array element. Consequently, the definitions `char Array[ ]` and `char *Array` as function arguments are equivalent. Since a pointer is a variable, it is legal to increment the pointer inside the function, as in this example where we provide a post increment to the pointer (i.e., `ArrayPtr++`) before loading into `x` variable the array element pointed by `ArrayPtr`. Figures 3.9c,d describe the `initRAM` and `pointRAM` functions, which, respectively, initialize an array in data memory and the access its elements. A call process to all the above

```
unsigned char compare(unsigned char a, unsigned char b)
{
 unsigned char f;
  if (a==b)
    f= '=';
  else if (a<b)
    f= '<';
  else
    f= '>';
 return f;
}

unsigned char compareAdv(unsigned char a, unsigned char b)
{
 unsigned char f;
  if (  ( (a>='0')&&(a<='9') )  &&  (  (b>='0')&&(b<='9') )  )
  {
   f=compare(a,b);
  }
  else
  {
    f= '#';
  }
 return f;
}
```

(a)

```
unsigned char x,y,f;

x='1';
y='@';
f = compareAdv(x,y);
f = compareAdv('1','7');
f = compareAdv('7','7');
f = compareAdv('7','!');

x='7';
y='1';
f = compareAdv(x,y);
```

(b)

Figure 3.8: Function call inside another function.

functions is illustrated in Figure 3.9e. Here is noted that every program in C language incorporates at least one function (regularly referred to as `main()` function) which holds the application code.

3.2.4 PREPROCESSOR DIRECTIVES

The C language provides certain directives that take place in the first step of compilation. Those directives start with the *hash* (`#`) symbol[4] and are addressed to further assist the designer in decomposing the program code to smaller segments, as well as to provide a more readable and manageable code. The most common tasks related to the processor directives are associated with (a) *file inclusion*, (b) *conditional inclusion*, and (c) *macro substitution* processes.

Functions declaration could be included in a separate file and invoked by the directive `#include`. Such files are regularly referred to as `header` files and are of `.h` extension (i.e., `filename.h`), instead of the `.c` extension (i.e., `filename.c`) that is addressed for the source file(s). File inclusion in a source code is generated be the following statements: `#include <filename>` and `#include "drive:\path\filename"`. The former tells the compiler to search in the default location path where the given filename is expected to be found. The

[4]Such a directive is the `#define` which has already been used for the declaration of constants.

```
const unsigned char romARRAY[] = {"hello world"};
```

(a)

```
void accessROM(const unsigned char ArraySubscript[])
{
 int i=0;
 unsigned char x;
  do
  {
    x=ArraySubscript[i];
  }while(ArraySubscript[++i] != '\0');

}
```

(b)

```
void pointROM(const unsigned char *ArrayPtr)
{
 unsigned char x;

  while ( *ArrayPtr != '\0')
  {
    x=*(ArrayPtr++);
  }

}
```

(c)

```
void initRAM(unsigned char elements, unsigned char *ptRAM)
{
 int i;
  for (i=0;i<elements;i++)
  {
    *ptRAM++ = 0x31 + i;
  }
  *(ptRAM+1) = '\0';
}

void pointRAM( unsigned char *ptRAM)
{
 unsigned char x;
  while ( *ptRAM != '\0')
  {
    x=*(ptRAM++);
  }
}
```

(d)

```
unsigned char ramARRAY[7];

accessROM(romARRAY);
pointROM(romARRAY);

initRAM(6,ramARRAY);
pointRAM(ramARRAY);
```

(e)

Figure 3.9: Functions with subscripts and pointer arguments in C language.

latter incorporates the full location path[5] the compiler should use for the given filename. It is noted that file inclusion within a different included file is legal.

Conditional inclusion in C programming is referred to the control of preprocessor toward selectively including a segment of code in the compilation process. The conditional inclusion is achieved by the following directives: `#if`, `#elif`, `#else`, `#endif`, `#ifdef`, `#ifndef`, and it is strongly associated with the `#define` directive. For example, every command line that is included in between the directives `#if` and `#endif` will be compiled in case the evaluated condition is true (e.g., if we have previously wrote `#define __linux__` and next we evaluate the variable as `#ifdef __linux__`).

Despite the employment of functions, an alternative programming approach that allows us to make tasks within a code less repetitive is referred to as *macro* and is declared as follows: `#define macro-name replacement-code`. Contrary to a function code that is stored in predefined segment of program memory and invoked by the main program as many time as needed, the entire macro code is copied to the position invoked by the main program every single time. Thereby, macros are more efficient in terms of the time required for their execution, but they reverse more space in program memory.

[5] Some compilers search in the working folder where the source file is found, in case the location path is not provided within the double quotes.

Figure 3.10 presents three different declarations and calls to macros. In detail, a new word defined `endlessLoop` is addressed in Figure 3.10a so as to substitute the generation of an infinite loop through the `for` statement, while invoking of this particular macro is illustrated in Figure 3.10d. On the other hand, Figures 3.10b,c are presented for calculating the two's complement of a variable as well as for returning the maximum variable, respectively. The latter macro is "read" as follows: *is x greater than y? if true return x, else return y*. The corresponding calls to these macros are given in Figure 3.10e, where line 1 loads to variable `N1` number `-3`, line 2 loads into `N1` the corresponding positive value (i.e., `N1=3` after this call), line 3 loads again the corresponding negative number of the latter result (i.e., `N1=-3` after this call), while the latter two instructions assign to `N1` variable the maximum value of the two given arguments (i.e., `N1=7` and `N1=13`, respectively).

```
#define endlessLoop for(;;)
```
(a)

```
#define twosComplement(x) ( (x^0xFF) + 1)
```
(b)

```
#define max(x,y) ( (x) > (y) ? (x) : (y) )
```
(c)

```
endlessLoop
{
 "compound statement
}
```
(d)

```
signed char N1 = -3;

N1=twosComplement(N1);
N1=twosComplement(N1);

N1=max(7,5);
N1=max(5,13);
```
(e)

Figure 3.10: Macro examples in C language.

The C language reserved keywords that cannot be reused for the declaration of macros, functions, etc., are given in Table 3.7. Most of them have previously been explored, and a few more keywords that are worth mentioning are as follows. The `volatile` identifier forces the compiler to suppress optimization of the code and keep (during compilation) any particular statement of the code referring to an "object" declared `volatile`. This keyword is regularly used for values that may change without the compiler knowing it, e.g., a value inserted into an i/o register from the outside world. The `typedef` identifier creates a new data type name to be used for declarations. For instance, the declaration `typedef unsigned char ubyte;` will create the *ubyte* type, which would be synonym of *unsigned char* data type. The `continue` and `break` statements are useful inside loops when there is a need to satisfy particular conditions. The former forces the CPU to immediately fetch the next iteration from the beginning of the loop, while the latter causes the innermost enclosing loop to be exited immediately. In case of nested loops, the `goto` keyword can be used for immediately exiting all loops (although its utilization is not recommended in high-level programming).

Table 3.7: Keywords in C language

C Identifiers			
auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

3.3 EMBEDDED C PROGRAMMING AND ISSUES TOWARD ADAPTABILITY

As mentioned earlier, the current DIY culture is in agreement with the *No Need to Reinvent the Wheel* philosophy that aims at hiding the low-level practices with the microcontroller's hardware from the designer's view. This section presents a programming strategy that opposes to the aforesaid philosophy, and is also directed toward the adaptability of the code among different software and hardware development platforms. Moreover, the proposed strategy aims at helping the reader to get initiated and learn how to exploit an MCU without needed to study the overall architecture peculiar to a microcontroller device, which definitely requires serious endeavor. However, the book does intend to oppose today's DIY culture, which has undoubtedly introduced a new direction in microcontroller applications. For that reason, we first begin with an introduction to the i/o operations in MCUs toward the code verification, through examples that apply to Arduino development platform. The examples reveal the flexibility the new trends have brought in microcontroller applications as well as education.

3.3.1 I/O OPERATIONS IN MCUS TOWARD CODE VERIFICATION

Hereafter we provide an example code which incorporates the previously explored programming methods and is customized to work on Arduino IDE. The only things needed for the code verification is a physical connection of an Arduino board (herein, the Arduino Uno) to the USB port of a host PC (Figure 3.11) along with the Arduino IDE installed on the PC. It would also be helpful to employ an RS232 terminal console with the possibility of a hexadecimal view of the received data (for the examination of the information sent from the microcontroller device to the host PC), such as the freeware *Termite* console provided by CompuPhase [59].

The program given in Figure 3.12 starts with the initialization of an array in program memory which encompasses the string "hello world." Subsequent to the array initialization is

Figure 3.11: Physical connection of Arduino Uno board to the host PC through the USB port.

the declaration of three functions, where the first is addressed for accessing arrays in program memory, the second initializes an array in data memory using the ASCII character `'1'`, `'2'`, `'3'`, and so on, and the final is used for accessing arrays in data memory. The next code lines employ the two mandatory functions for every Arduino program written in C language, where the `setup()` function holds any possible initialization performed within the MCU, and the `loop()` function comprises the main application code.

The code inside the main function first declares an array of seven elements (and of `unsigned char` type) in data memory and then performs a call to the following functions: `accessROM`, `Serial.println`, `initRAM`, and `accessRAM`. Then, it executes infinitely the `while` clause which admits a test condition equal to 1 (i.e., an always true condition). The functions in red color, that is `Serial.begin`, `Serial.write`, and `Serial.println`, constitute standard library functions which are integrated into the Arduino IDE. Because we have previously discussed the above user-defined functions, we will talk about the standard library functions that implement a serial/RS232 interface between the MCU and a host PC.

Microcontrollers regularly embed a universal asynchronous receiver/transmitter (UART) peripheral, capable of providing an RS232 compatible communication between the MCU and a host PC. While this interface protocol was the dominant PC-MCU communication strategy for many years, it became obsolete from the moment the USB serial communication made its appearance. However, the USB interface is considered a complicated standard compared to the simplicity of RS232 communication and, hence, most of today's MCUs incorporate a UART peripheral instead of USB transceiver, such as, the ATmega328P microcontroller [60] attached to the Arduino Uno R3 board [61]. For that reason, the latter board employs an ATmega16U2

Figure 3.12: i/o operations in MCUs toward the code verification (Arduino IDE example).

microcontroller [62] with an embedded USB controller that along with an accompanying driver for host PC, implements a *USB to Serial* converter (inserted between the USB port of the host PC and the UART peripheral of the prime MCU). Through this particular setup, the prime ATmega328P microcontroller is capable of updating the application code in its program memory (through a bootloader firmware that is loaded to the Arduino Uno by the manufacturer), as well as communicating with the host PC (through USB) using *Serial* library that is included in the Arduino IDE.

In order to examine the data arriving to the host PC from the MCU, an RS232 terminal console is needed. Figure 3.13 presents the built-in Arduino IDE *Serial Monitor* terminal, while Figure 3.14 presents the freeware *Termite* console. Both figures illustrate the data send from the MCU and received by the host PC during the execution of the example code (Figure 3.12), while *Termite* console is capable of depicting hexadecimal values of the received characters. The application code runs as follows. The microcontroller reads from program memory and transmits to the host PC the "hello world" string and afterward it invokes the `Serial.println` function. The latter function admits a string argument to send to the host PC, and appends carriage return (`0x0D`) and line feed (`0x0A`) characters at the end of the string (thereby, the next string will be printed in the first column and subsequent row of the terminal console). The function herein is syntax with no string and therefore, it only sends CR and LF characters to the host PC. Following the `Serial.Println` function, the microcontroller assigns to the `ramARRAY` the characters from `'1'` to `'6'` and afterward it obtains from data memory those characters and sends them to the host PC. The `Serial.Println` function is the reason the two string "hello world" and "123456" are vertically aligned into two subsequent rows, as illustrated by the built-in terminal console of Arduino Uno (Figure 3.13).

It is noted that the `Serial.write` function of the example admits two arguments: (a) an array to read from and (b) a number revealing the array elements that will be transmitted upon the execution of the function. On the other hand, the `Serial.begin` function admits the baud rate of the serial communication (herein defined 9600 baud) and the same rate should be used in the terminal console, as well. This function is inserted into the `setup()` of the application code because it configures the internal UART peripheral device of the MCU.

This particular example is addressed to reveal the benefits of the new trends in microcontroller applications and education as well. Through the employment of the Arduino Uno board system (with the embedded bootloader code), as well as the use of `Serial` library and the built-in terminal console of Arduino IDE, the designer is able to verify the development firmware within a few minutes. Before that, the designer would normally need significant endeavor just for the configuration of the UART peripheral device and the underlying code from transmitting (and receiving) data. This would also result in an extended, and less readable, code than the one presented by this example (Figure 3.12). Similar functions are also embedded in modern embedded C compilers.

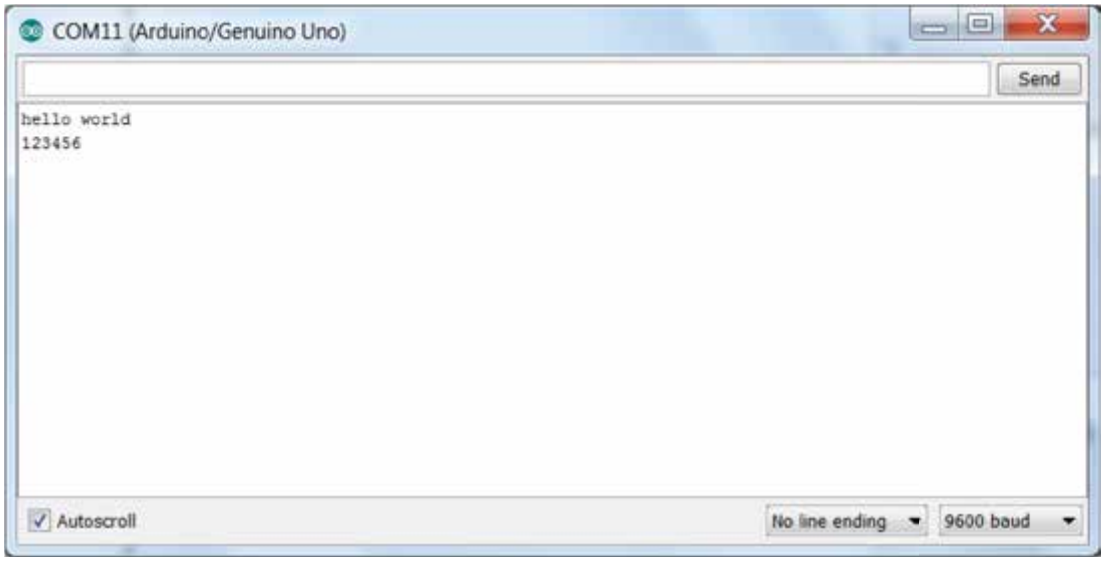


Figure 3.13: Serial monitor in Arduino IDE.

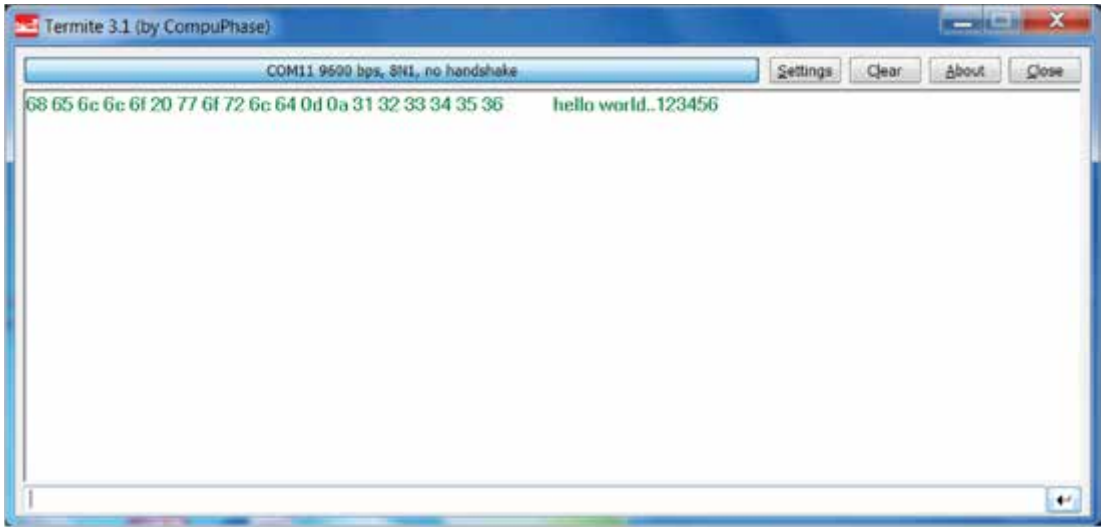


Figure 3.14: Hexadecimal view of the received data in Termite console.

A good practice for the reader would be the experimentation on changing the post-increment task of the pointers (used by the user-defined functions of the example) to pre-increment and then exploring the output result on the terminal console (through a hexadecimal observation, as well)

3.3.2 PROGRAMMING ISSUES TOWARD THE CODE'S ADAPTABILITY

The issues surrounding the adaptability of an embedded C code that the designer should take into account can be separated into (a) hardware and (b) software considerations [63]. The former are related to the employed manufacturer of MCUs as well as the selected device/processor model. The latter are peculiar to the employed embedded C compiler for the firmware development.

The hardware considerations are relevant to the microcontroller's communication with the outside world through the available internal subsystems. As mentioned earlier, the complex internal subsystems (ADCs, Timers, I2C/SPI interfaces, etc.) require particular configuration and handling operations through their internal registers, and the inconsistency of such processes among different MCUs (as well as the different associated addresses in memory) is opposed to the code's adaptability. Therefore, we will only work with simple i/o operations through the regular i/o pins, that is, inputting/outputting a logical signal (i.e., logical 0 or 1) from/to the outside world. The corresponding internal mechanisms of the MCU that are related to these i/o processes have been thoroughly examined in the first chapter of the book, through part-and-steps figure examples [64, 65]. Such figure examples are proved to tackle comprehension difficulties in microcontroller education and make a clearer link between the firmware and hardware [66, 67, 68]. The hardware communication of the MCU with external (and composite) peripheral devices toward the implementation of interfaces, such as wireless, Ethernet, USB, etc., will be based on these regular i/o processes along with user-defined UART, SPI and I2C interfaces (which constitute the dominant communication protocols among hardware devices).

The software considerations rely on the avoidance of using the built-in utilities delivered by the software package (which is addressed for the embedded C compilation into machine code). Those utilities might be ready-to-use functions and macros, preprocessor directives, constant declarations of the available memory addresses, etc. While such utilities speed up the code development process, they are opposed to the code adaptability as well as the low-level accessibility of the microcontroller device.

Due to these hardware and software demands, we hereafter begin with the identification of the memory addresses that are associated with the i/o register of the employed microcontroller devices, as well as with the development of macros toward the implementation of i/o regular operations. Those processes appear as the designer is *reinventing the wheel*, yet, such tasks constitute examples of very good practice in low-level tasks for microcontrollers and especially for a beginner. Thus, it is our intention to *reinvent the wheel* as much as it is to develop an adaptable code among different hardware and software development platforms. Despite the i/o register

addresses peculiar to the employed microcontroller device, the designer should also identify the corresponding i/o pins on the employed board system.

The examples presented within this book are addressed for the ATmega328P microcontroller [60] attached to Arduino Uno R3 board (Figure 3.15a) [61], as well as for the PIC18F87J50 [69] attached to Clicker2 for PIC18FJ board (Figure 3.15c) [70]. Figure 3.15b presents an add-on board for Arduino Uno, aka Click Shield [71], which renders feasible the connection of peripheral devices on Arduino hardware, thereby providing consistency between the two experimentation platforms (i.e., for AVR and PIC microcontroller devices). The peripheral ICs are attached to the so-called Click Boards delivered by MikroElektronika [72]. The Click Board adapters for AVR and PIC microcontrollers along with the available pinout are given in Figures 3.15d,e, respectively.

3.3.3 I/O REGISTERS DEFINITION AND USER-DEFINED MACROS

Every i/o port for the PIC and AVR devices has three memory-mapped registers for its operation. In PIC devices the three registers are named `TRIS`, `PORT`, and `LAT`, where the former controls the direction (i.e., either input or output) of the port pins, and the latter two read and write, respectively, values from/to the port pins (which are inputted/outputted from/to the outside world). Assigning logical 1 to a TRIS bit makes the corresponding pin input, while a logical 0 makes the pin output. In AVR devices the corresponding registers are named `DDR`, `PIN`, and `PORT` and, thus, `DDR` controls the direction of an i/o port, the pin value is read from `PIN` register, and the microcontroller outputs data to the outside world through writing values to the `PORT` registers. However, in AVR devices the pin is configured input when assigning a logical 0 to the corresponding bit of `DDR`, and output where assigning a logical 1. For that reason, we will develop two groups of macros (for the implementation of i/o operations), one for PIC and one for AVR devices.

Before proceeding to the implementation of macros, we hereafter declare the three registers for each available port on ATmega328P and PIC18F87J50 devices, according to the memory addresses identified by the corresponding AVR [60] and PIC [69] reference manuals. To preserve consistency in the register names of both devices, we use the names `dirPORTx`, `inPORTx`, and `outPORTx`, for the registers that configure port direction, read input data from a port, and output data to the outside world through a port, respectively. The `x` notation in the variable is replaced by the identification name of the port, which regularly admits letters of the alphabet (i.e., A, B, C, and so forth).

ATmega328P MCU incorporates three ports, that is, port B, C, and D. Despite the port C that consists of 7 pins, the rest two consist of 8 pins each. Table 3.8 presents the registers associated with i/o ports B, C, and D. The bits in red color illustrate the available (17) port pins through the Click Board headers (on Arduino Click Shield).

The registers declaration of the available i/o ports on ATmega328P is presented in Figure 3.16. It is noted that Notepad++ free software has been addressed for the authoring of

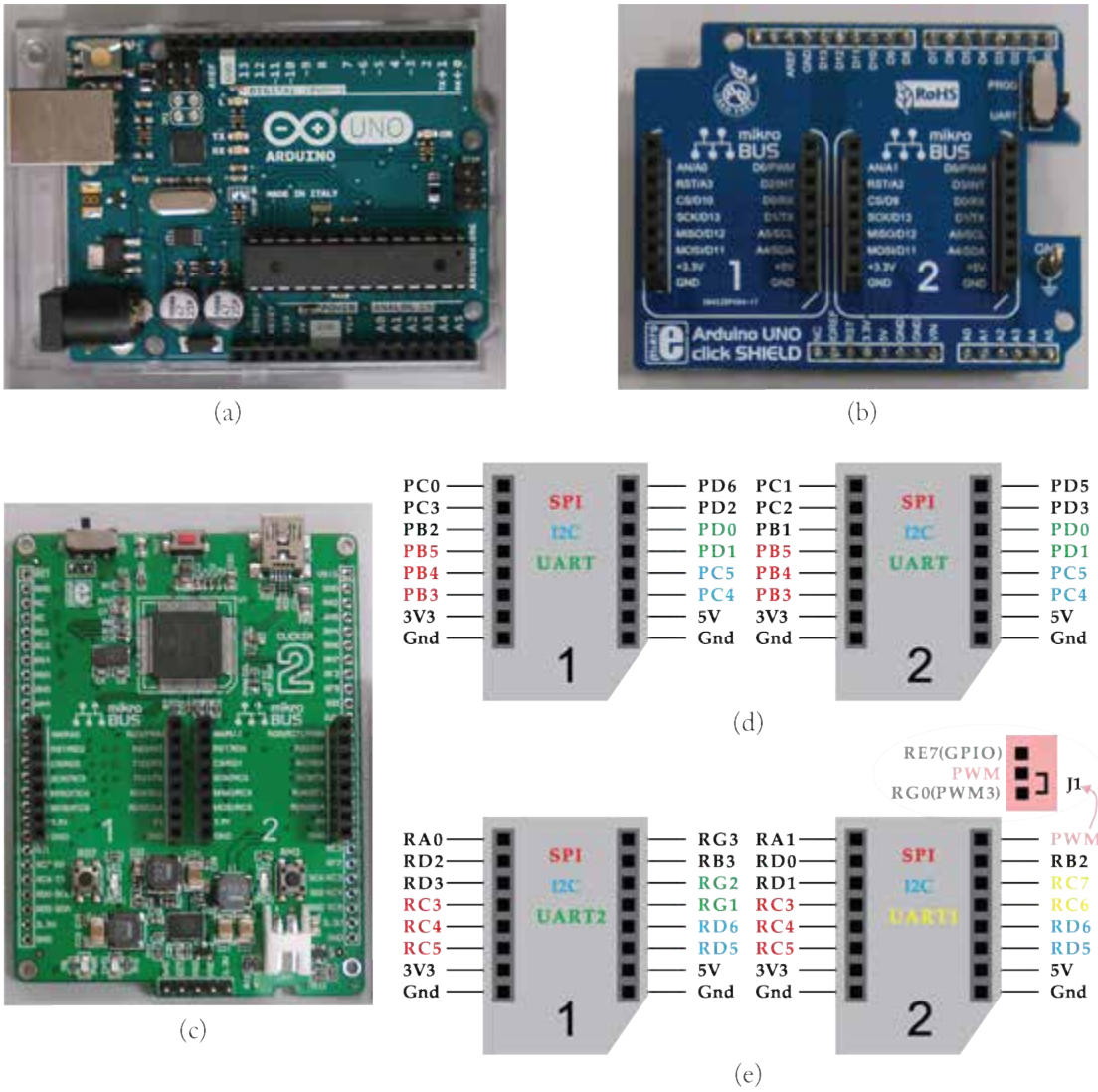


Figure 3.15: (a) Arduino Uno, (b) Click Shield, (c) PIC18FJ, Click Board headers for (d) AVR, and (e) PIC.

code [73]. The effective addresses of registers, as well as the macros that will be explored next in the chapter, can be integrated into a single header file of an descriptive name (such as, the REGS-MACROS.h used in the example). The first and last code lines perform a conditional inclusion through the `#if` and `#endif` directives. Thus, the inside code will be compiled in

Table 3.8: Registers of the available i/o pins on (a) ATmega328P and (b) Click Board headers (in red)

Memory Address	Register Name	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit
(0x2B)	PORTD	7	6	5	4	3	2	1	0
(0x2A)	DDRD	7	6	5	4	3	2	1	0
(0x29)	PIND	7	6	5	4	3	2	1	0
(0x28)	PORTC	-	6	5	4	3	2	1	0
(0x27)	DDRC	-	6	5	4	3	2	1	0
(0x26)	PINC	-	6	5	4	3	2	1	0
(0x25)	PORTB	7	6	5	4	3	2	1	0
(0x24)	DDRB	7	6	5	4	3	2	1	0
(0x23)	PINB	7	6	5	4	3	2	1	0

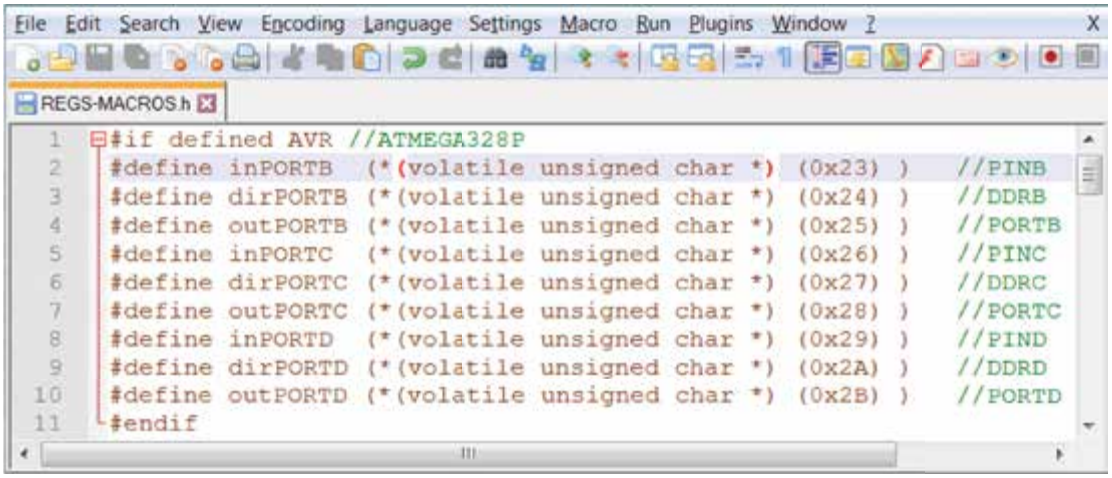


Figure 3.16: Registers declaration of the available i/o ports on ATmega328P (Notepad++).

case the evaluated condition is true, that is, if the designer has previously used the directive `#define AVR`. Every single line assigns a variable to a specific memory location. In order to understand how each code line is realized by the compiler, we hereafter break the assignment of `dirPORTB` register into the following segments (it is noted that the characters "//" denote a commented line):

1. `unsigned char *` declares a pointer that points to a byte-size memory location, which admits non-signed values (i.e., the values 0–255 in decimal system);

2. `(unsigned char *) (0x24)` the pointer now points to memory address 0x024;

3. `(* (unsigned char *) (0x24) )` the first * character from the left refers to the content of address indicated by the pointer and, hence, the content of address `0x24` is now modifiable;

4. `(*(volatile unsigned char *)(0x24))` the volatile identifier reveals that the content of 0x24 may change without the compiler knowing it and hence, it forces the compiler to suppress optimization of any statement in the code that is associated with `0x24` memory address; and

5. `#define dirPORTB (*(volatile unsigned char *)(0x24))` the overall expression associates the name `dirPORTB` to a variable of fixed address (i.e., to the variable located at `0x024` memory address in this example).

PIC18F87J50 MCU incorporates nine ports, that is, port A–H and J. Table 3.9 presents the registers associated with those i/o ports, while bits in red color illustrate the available (18) port pins through the Click Board headers (on Clicker2 for PIC18FJ). It should be noted that RE7 an RG0 pins are not at the same time available to the designer. The selection of the desired pin is available through a configuration jumber (J1) attached to the board (Figure 3.15). The registers declaration of the available i/o ports on PIC18F87J50 is presented in code lines 13–41 (of the same file) of Figure 3.17, using the same techniques as in the AVR device.

The macros for implementing the regular i/o operations on both AVR and PIC devices are given in Figure 3.18, and in code lines 45–52 and 57–64, respectively. The only difference between the two macro sets is that the former assigns logical 0 for declaring an input pin and logical 1 for declaring an output pin, while the latter follows opposite sense (i.e., the output pin is declared with logical 0 and the input pin with logical 1). For that reason, we hereafter look into the macros of AVR devices and the corresponding analysis is valid for PIC devices, too. At this point one is reminded that the direction register of a port should be configured before reading/writing a value from/to a pin. Such configurations can be performed only once in the beginning of the program, but the pin direction can also be reconfigured during the code execution, as many times as needed. To preserve simplicity of the application code, the proposed macro instructions configure the port/pin direction on each particular read/write operation.

The `pinSET` macro of code line 45 admits 3 arguments and (a) sets logical 1 to an output pin, and (b) configures that pin output. The `dir` argument refers to the port's direction register, the `data` argument refers to the port's data register, and the `pin` argument defines the corresponding pin that we would like to set to logical 1. To understand how this statement is realized by the compiler, we hereafter break it into the following segments (it is noted that the charac-

Table 3.9: Registers of the available i/o pins on (a) PIC18F87J50 and (b) Click Board headers (in red)

Memory Address	Register Name	Bit	Bit	Bit	Bit	Bit	Bit	Bit	Bit
(0xF80)	PORTA	-	-	5	4	3	2	1	0
(0xF81)	PORTB	7	6	5	4	3	2	1	0
(0xF82)	PORTC	7	6	5	4	3	2	1	0
(0xF83)	PORTD	7	6	5	4	3	2	1	0
(0xF84)	PORTE	7	6	5	4	3	2	1	0
(0xF85)	PORTF	7	6	5	4	3	2	-	-
(0xF86)	PORTG	-	-	-	4	3	2	1	0
(0xF87)	PORTH	7	6	5	4	3	2	-	-
(0xF88)	PORTJ	7	6	5	4	3	2	1	0
(0xF89)	LATA	-	-	5	4	3	2	1	0
(0xF8A)	LATB	7	6	5	4	3	2	1	0
(0xF8B)	LATC	7	6	5	4	3	2	1	0
(0xF8C)	LATD	7	6	5	4	3	2	1	0
(0xF8D)	LATE	7	6	5	4	3	2	1	0
(0xF8E)	LATF	7	6	5	4	3	2	-	-
(0xF8F)	LATG	-	-	-	4	3	2	1	0
(0xF90)	LATH	7	6	5	4	3	2	-	-
(0xF91)	LATJ	7	6	5	4	3	2	1	0
(0xF92)	TRISA	-	-	5	4	3	2	1	0
(0xF93)	TRISB	7	6	5	4	3	2	1	0
(0xF94)	TRISC	7	6	5	4	3	2	1	0
(0xF95)	TRISD	7	6	5	4	3	2	1	0
(0xF96)	TRISE	7	6	5	4	3	2	1	0
(0xF97)	TRISF	7	6	5	4	3	2	-	-
(0xF98)	TRISG	-	-	-	4	3	2	1	0
(0xF99)	TRISH	7	6	5	4	3	2	-	-
(0xF9A)	TRISJ	7	6	5	4	3	2	1	0

REGS-MACROS.h

```
10  #define outPORTD (*(volatile unsigned char *) (0x2B) )   //PORTD
11  #endif
12
13  #if defined PIC //PIC18F87J50
14  #define inPORTA   (*(volatile unsigned char *) (0xF80) ) //PORTA
15  #define inPORTB   (*(volatile unsigned char *) (0xF81) ) //PORTB
16  #define inPORTC   (*(volatile unsigned char *) (0xF82) ) //PORTC
17  #define inPORTD   (*(volatile unsigned char *) (0xF83) ) //PORTD
18  #define inPORTE   (*(volatile unsigned char *) (0xF84) ) //PORTE
19  #define inPORTF   (*(volatile unsigned char *) (0xF85) ) //PORTF
20  #define inPORTG   (*(volatile unsigned char *) (0xF86) ) //PORTG
21  #define inPORTH   (*(volatile unsigned char *) (0xF87) ) //PORTH
22  #define inPORTJ   (*(volatile unsigned char *) (0xF88) ) //PORTJ
23  #define outPORTA  (*(volatile unsigned char *) (0xF89) ) //LATA
24  #define outPORTB  (*(volatile unsigned char *) (0xF8A) ) //LATB
25  #define outPORTC  (*(volatile unsigned char *) (0xF8B) ) //LATC
26  #define outPORTD  (*(volatile unsigned char *) (0xF8C) ) //LATD
27  #define outPORTE  (*(volatile unsigned char *) (0xF8D) ) //LATE
28  #define outPORTF  (*(volatile unsigned char *) (0xF8E) ) //LATF
29  #define outPORTG  (*(volatile unsigned char *) (0xF8F) ) //LATG
30  #define outPORTH  (*(volatile unsigned char *) (0xF90) ) //LATH
31  #define outPORTJ  (*(volatile unsigned char *) (0xF91) ) //LATJ
32  #define dirPORTA  (*(volatile unsigned char *) (0xF92) ) //TRISA
33  #define dirPORTB  (*(volatile unsigned char *) (0xF93) ) //TRISB
34  #define dirPORTC  (*(volatile unsigned char *) (0xF94) ) //TRISC
35  #define dirPORTD  (*(volatile unsigned char *) (0xF95) ) //TRISD
36  #define dirPORTE  (*(volatile unsigned char *) (0xF96) ) //TRISE
37  #define dirPORTF  (*(volatile unsigned char *) (0xF97) ) //TRISF
38  #define dirPORTG  (*(volatile unsigned char *) (0xF98) ) //TRISG
39  #define dirPORTH  (*(volatile unsigned char *) (0xF99) ) //TRISH
40  #define dirPORTJ  (*(volatile unsigned char *) (0xF9A) ) //TRISJ
41  #endif
```

Figure 3.17: Registers declaration of the available i/o ports on PIC18F87J50 (Notepad++).

ters "/*" of code line 44 denote the start of a comment, which may span multiple lines, while the characters "*/" denote the end of the comment).

1. `(1<<pin)` because the `pin` argument refers to the port pin that will be set to logical 1, it admits values from 0–7 (that is, for 8-bit MCUs). Therefore, if the `pin` argument is replaced to number 5 upon the macro call (i.e., `1<<5`), a mask is generated for setting bit 5 of `data` argument to logical 1.

2. `(data) |= (1<<pin)` statement make use of the *bitwise OR and assign operator* (`|=`) and hence, it is equivalent to the following expression: `data = data | (1<<pin)`. Therefore,

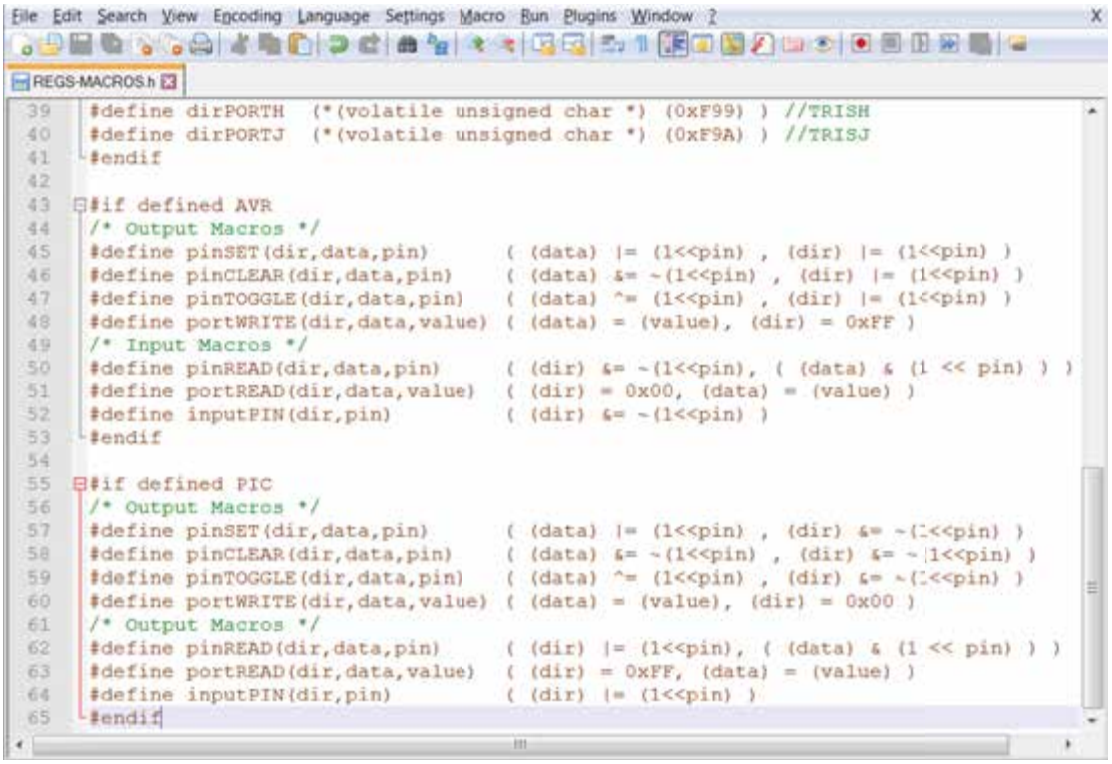


Figure 3.18: Macros for the implementation of regular i/o operations.

the mask is ORed with the content of *data* argument in order to set to logical 1 bit n of the latter register (that is, bit 5 in this example), and leaves the rest bits unchanged. The final result is assigned into the `data` register.

3. `(dir |= (1<<pin)` statements is equivalent to the aforementioned one and thus, it generates a mask for setting bit n of `dir` register to logical 1 (where $n = 0, 1, 2, \ldots, 7$), and hence, configuring the corresponding bit output.

4. `pinSET(dirPORTB,outPORTB,5);` this statement constitutes a possible call to `pinSET` macro, which configures pin 5 of `PORTB` output and sets its value to logical 1.

The term *mask* in computing discipline is used in association with bitwise operations toward forcing a bit to a logical state (i.e., to logical 0 or 1) or querying the status of a bit. The three basic bitwise operations `OR`, `AND`, `XOR` are given in Table 3.10. When two bits (i.e., the boolean variables P and Q of Table 3.10) are `ORed` together, the outcome is of logical 0 only when both of them hold zero value, otherwise the outcome is of logical 1. When P and Q are `ANDed` the outcome is of logical 1 only when both of them hold logical 1, otherwise the outcome

is of logical 0. Finally, when P and Q are of same value (i.e., P=Q=1 or P=Q=0) and XORed together, the outcome is of logical 0, otherwise the outcome is of logical 1. The latter bitwise operation could also be considered as follows: if a bit is XORed with logical 1 then its value is complemented, otherwise the bit remains unchanged, that is, P xor 1 = $\bar{P}$ and P xor 0 = P.

Table 3.10: Truth table of bitwise operations OR, AND, XOR

P	Q	OR	AND	XOR
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

Figure 3.19 presents the use of masks in association with the bitwise OR, AND, XOR toward forcing (Figures 3.19a,b,c) or querying the status (Figures 3.19d,e,f) of a bit. In detail, the bitwise OR to bit 5 of data register with the mask 0x20 forces the value of the latter bit to logical 1, while the rest bits remain unchanged (Figure 3.19a). The bitwise AND to bit 5 of data register with the mask 0xDF forces the value of the latter bit to logical 0, while the rest bits remain unchanged (Figure 3.19b). The bitwise XOR to bit 5 of data register with the mask 0x20 forces the value of the latter bit to be complemented (i.e., if the bit value is logical 0 it changes to logical 1, and vice versa), while the rest bits remain unchanged (Figure 3.19c). On the other hand, in order to extract the value of bit 5 through a bitwise OR, with address the mask 0xDF which forces (i.e., masks) the rest of bits to logical 1 (Figure 3.19d), while a bitwise AND with the mask 0x20 leaves bit 5 immutable and forces the rest of bits to logical 0 (Figure 3.19e). Finally, a bitwise XOR between two registers can be addressed for querying whether the registers are of same value, that is, when a zero outcome is generated (Figure 3.19f).

Following the aforementioned description, a call to pinCLEAR macro, like the pinCLEAR(dirPORTB,outPORTB,5); will initially perform a *bitwise AND assignment* to the *data* register (i.e., (data) &=), using the *complement* (~) of mask 0x20 (i.e., ~(1<<5)=~(0x20)=0xDF), which results in clearing bit 5 and leaving immutable the rest of bits in data registers. Then, the macro assigns the corresponding pin (i.e., pin 5) output. It is here noted that it is often advisable (by the MCU reference manuals) to assign a value to a port pin before configuring the pin output, in order to ensure that the pin will not be driven momentarily with an old data value (which may generate glitches on the port pin) [74].

Based on the same technique, the pinTOGGLE macro addresses a *bitwise XOR assignment* (^=) to the content of *data* register and hence, a call to the macro such as pinTOGGLE(dirPORTB,outPORTB,5); performs bitwise XOR to the outPORTB register with the mask 0x20. Thereby, if pin PB5=1 before the execution of the macro, then PB5=0 after the exe-

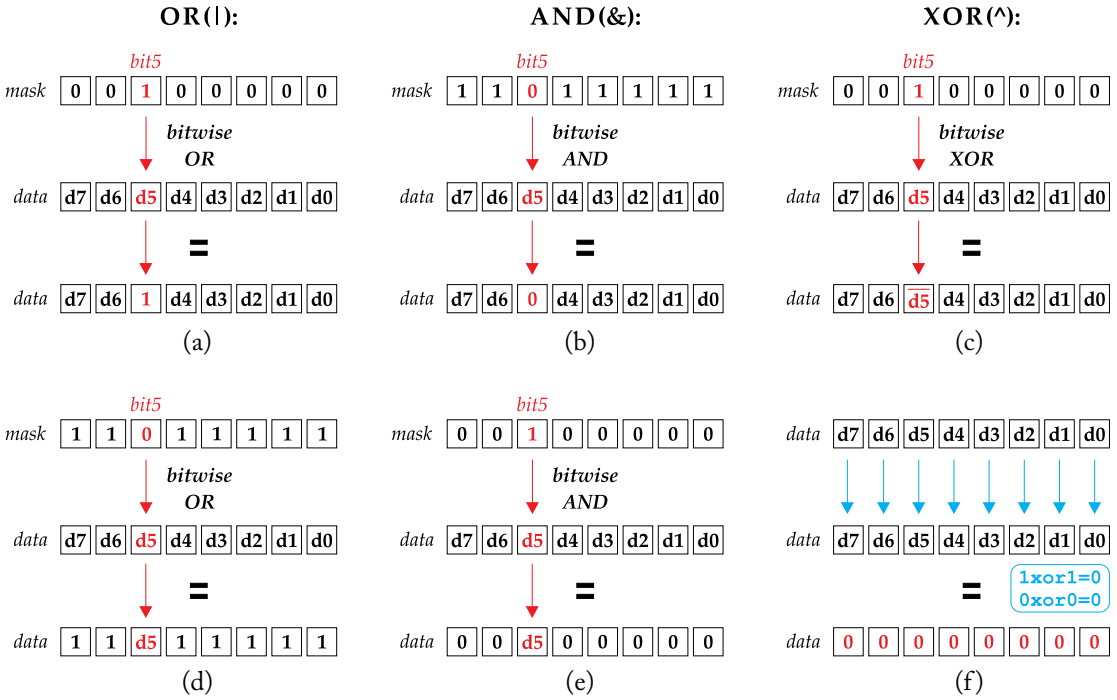


Figure 3.19: Bitwise operations and masks.

cution of the macro, and vice versa. The rest of the bits remain unchanged by the execution of the macro.

The description of the above three, as well as the rest of user-defined macros (which are based on the same coding style), is presented in Table 3.11. The former four macros are in line with outputting tasks, while the latter three involve with inputting operations. Hereafter, we explain the latter four call examples of the table, which employ macros that have not yet been discussed.

Up until this point we have explored macros that assign a value to an output pin. The `portWRITE` macro writes a value to the overall output port defined by the corresponding argument. Thereby, the macro call example of the table writes the value `0x020` to `PORTB` and then configures the port as output. The `pinREAD` macro call example configures `PB5` pin as input and then evaluates the value of the bit. Because the macro call is inserted into the test expression of a *while* clause, the execution of the latter statement is repeated as long as `PB5=1`. The loop is terminated upon the reception of a logical 0 on `PB5` pin (from the outside environment). The `portREAD` macro call example configures the entire `PORTB` as input and then assigns to `x` (8-bit) variable the content found in the port (which is obtained from the outside environment, as well). Finally, the `inputPIN` macro configures `PB5` as input.

Table 3.11: Description of the user-defined macros toward regular i/o operations

Macro Name (Arguments)	Macro Call Example	Description
pinSET (dir,data,pin)	pinSET(dirPORTB,outPORTB,5);	Set a logical 1 to pin 0..7 in data register; then configure pin as output.
pinCLEAR (dir,data,pin)	pinCLEAR(dirPORTB,outPORTB,5);	Set a logical 0 to pin 0..7 in data register; then configure pin as output.
pinTOGGLE (dir,data,pin)	pinTOGGLE(dirPORTB,outPORTB,5);	Complement logical state of pin 0..7 in data register; then configure pin as output.
portWRITE (dir,data,value)	portWRITE(dirPORTB, outPORTB,0x20)	Write a byte-size value to data register; then configure the port as output.
pinREAD (dir,data,pin)	while (pinREAD(dirPORTB,inPORTB,5));	Read pin value through data register; then configure pin as input.
portREAD (dir,data,value)	x = portREAD (dirPORTB,inPORTB,0x00);	Read port value through data register; then configure port as input.
inputPIN (dir,pin)	inputPIN (dirPORTB,5)	Configure pin as input.

3.3.4 PIN ASSIGNMENT, TYPE DEFINITIONS, AND TIMING OF EVENTS

Because of the pinout inconsistency among different MCUs, even if they are parts of the same manufacturer and of the same family of devices, it would be wise to keep pinout assignment in a separate header file (e.g., `PINOUT.h`). Thus, the pin location according to the employed board

system easily can be revised. The example of Figure 3.20 presents the pinout definition for the available LED on Arduino Uno and PIC18FJ boards, where the selection of the appropriate pinout is performed through a conditional inclusion.

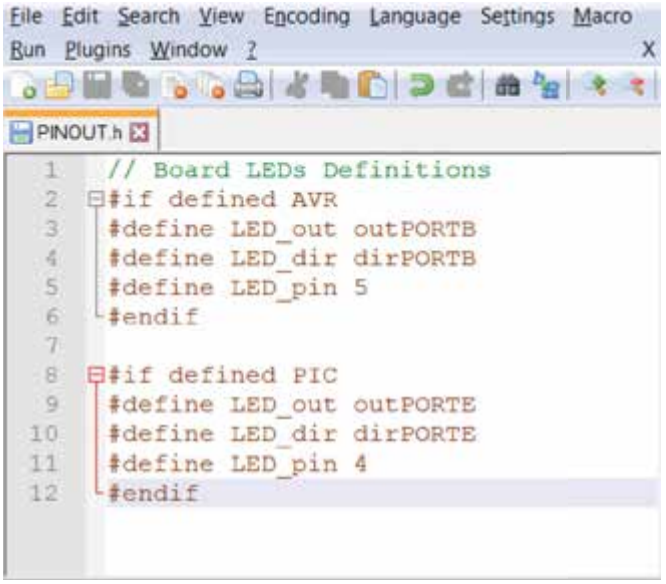


Figure 3.20: Pinout definition for the LED attached to Arduino Uno and PIC18FJ boards.

Another consideration that creates the need for an additional header file is in line with the features of the employed compiler. Different compilers incorporate different identifiers for the declaration of data types. For instance, the `unsigned char` type of data in `avr-gcc` compiler (incorporated by Arduino IDE) [75] is defined as `unsigned char` or `uint8_t`,[6] while `microC` compiler [76] addresses the identifier `unsigned char` or just `char`, and the *CCS C* compiler [77] makes use of the `unsigned int8`. Such differentiations, as it is understood, could be considered a politic of the enterprises developing embedded C compliers, so as to encourage users to keep supporting their mercantile tools. And, of course, this is one the main causes that embedded C code cannot be straightforwardly reused among different (software) development platforms. Figure 3.21 addresses `typedef` identifier in a separate header file (named `DATATYPES.h`) for bypassing non-reusability of the code because of the different data type definitions in between the aforementioned compilers. Lines 2–7 create new data types (i.e., `bit8` for `unsigned char`, `bit16` for `unsigned int`, and so forth) if the code has been previously defined either MICROC or AVRGCC name (as defined by the logical `OR` operator), while the definition of CCS forces the compilers to create the data types of code lines 11–16 (i.e., `bit8` for `unsigned int8`, `bit16` for `unsigned int16`, and so forth).

[6]Those alternate data type (e.g., `uint8_t`) are outlined in the C99 standard.

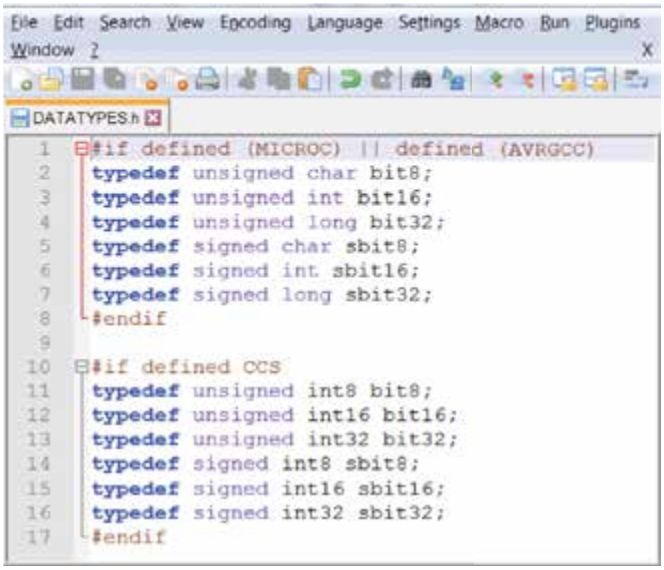
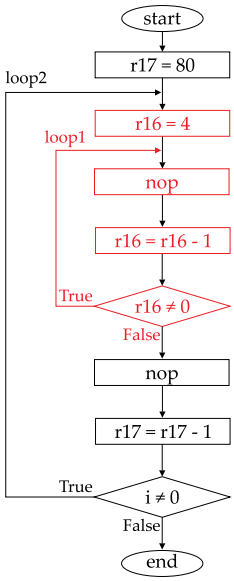


Figure 3.21: Data type definitions toward the resusability of the code among different C compilers.

Among the most demanding tasks in a microcontroller's firmware is the timing of events. The way time delays are generated at the machine level is already known from the previous chapter and hereafter we explore how to develop accurate delays using the programming techniques from our previous study. Unfortunately, the high-level (i.e., embedded C) programming is not able to provide accurate time delays through, for example, the development of an iterative loop. The only way to do this is to write assembly code, whose execution time can be precisely determined, and embed this code in the C language program.

Figure 3.22 presents the time delay code in assembly language for the AVR (Figures 3.22a,b,c,g) and PIC (Figures 3.22d,e,f,h) devices. The delay is implemented with a nested (double) loop, where the red color denotes the inner loop and the black color the outer loop. The time that will be used for delaying the CPU is 0.1 msec. To decide on the assembly mnemonics and the values of the loop counters which are able to implement this particular delay, we first need to know the frequency of the CPU synchronization clock of the employed board systems. Because Arduino Uno and PIC18FJ Clicker2 are delivered with a preloaded bootloader code, the corresponding fuses that control the CPU clock have already been configured during the uploading of the bootloader in microcontroller's memory, and they cannot be changed by the bootloader[7] itself. The free-running (internal) clock frequency of the Arduino Uno is 16 MHz, while the corresponding frequency of PIC18FJ Clicker2 board is 12 MHz.

[7]If there is a need to change the CPU clock frequency, an external programmer should be addressed for reloading the bootloader code to the MCU memory, using the revised configurations for the fuses that define the clock frequency.



(a)

```
for (j=80;j!=0;--j){
  for (i=4;i!=0;--i){
  }
}
```

(b)

```
        ldi r17, 80

loop2:  ldi r16, 4
loop1:  nop
        dec r16
        brne loop1

        nop
        dec r17
        brne loop2
```

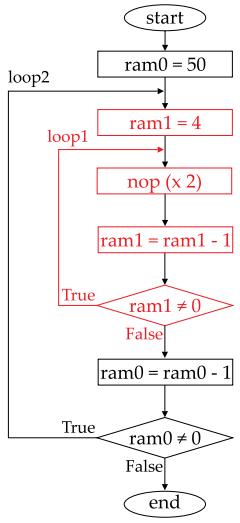
(c)

```
for (j=50;j!=0;--j){
  for (i=4;i!=0;--i){
  }
}
```

(d)

```
        movlw   50
        movwf   0x00

loop2:  movlw   4
        movwf   0x01
loop1:  nop
        nop
        decf    0x01,1
        bnz     loop1

        decf    0x00,1
        bnz     loop2
```

(e)



(f)

Arduino Uno

the final execution of brne (1 cycle)

```
1 cycle equals to 1/16MHz and for time delay equal to 100 µsec:

time-delay(usec) =  100 µsec = 1600cycles / 16MHz

t1(cycles) = ldi(1) + [nop(1)+dec(1)+brne(2)]*r16 - brne(1) = 4*r16

t2(cycles) = ldi(1) + [t1+nop(1)+dec(1)+brne(2)]*r17 - brne(1) = (4*r16+4)*r17

T=1600(cycles)=(4*r16+4)r17; if r16=4, then 1600 = 20*r17 and hence, r17=80
```

(g)

PIC18FJ

the final execution of bnz (1 cycle)

```
1 cycle equals to 1/12MHz and for time delay equal to 100 µsec:

time-delay(usec) =  100 µsec = 1200cycles / 12MHz

t1(cycles) = movlw(1) + movwf(1) + [nop(1)+nop(1)+decf(1)+bnz(2)]*ram1 - bnz(1) = 1+5*ram1

t2(cycles) = movlw(1) + movwf(1) + [t1+decf(1)+bnz(2)]*ram0 - brne(1) = 1+(1+5*ram1+3)*ram0

T=1200(cycles)=1+(1+5*ram1+3)*ram0; if ram1=4, then 1200=1+24*ram0 and hence,

if ram0=50 then T=1201(cycles) and T(µsec) = 100.08 µsec
```

(h)

Figure 3.22: Accurate time delays in assembly language for Arduino Uno and PIC18FJ boards.

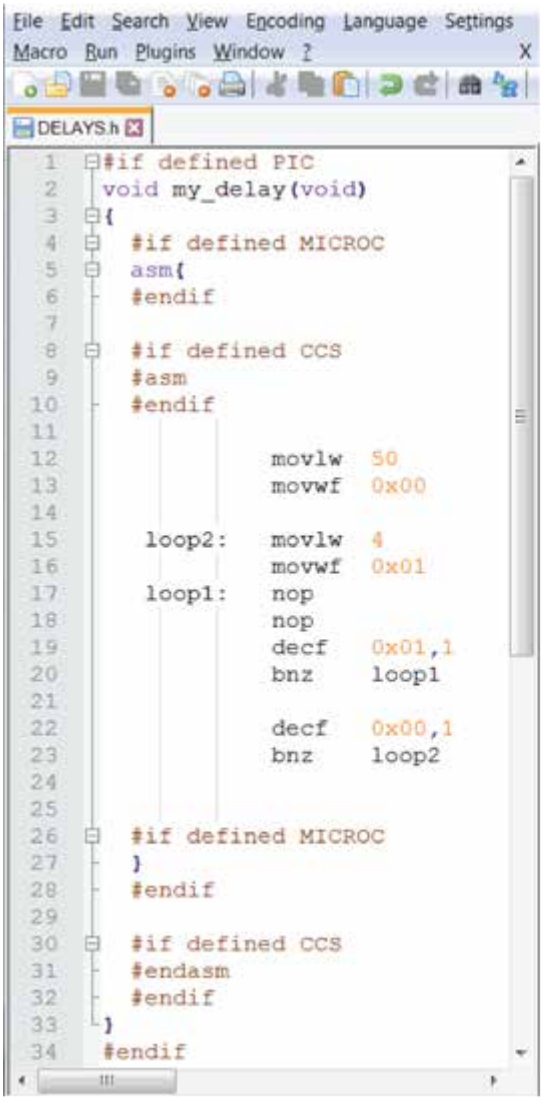
In consideration of the AVR assembly code (Figure 3.22c), we need to delay the CPU for 1600 clock cycles in order to generate 0.1 msec delay. Inside of the (inner/outer) loops we can incorporate NOP mnemonics to round up the generated number of clock cycles to the desired value. All the utilized mnemonics are executed in one clock cycle, except the conditional branching ones (that is, the *bnre* for AVR and *bnz* for PIC assembly) which are executed in 2 clock cycles when they perform a branch, otherwise they need 1 clock cycle.

According to the above information, the inner loop (in red color) of the AVR code delays the CPU for `t1=4*r16` clock cycles (where the `r16` CPU register admits decimal values 0–255). The latter result is due to the fact that the `ldi` instruction (which assigns `r16` register) is executed only once, while the three subsequent mnemonics (that is, `nop`, `dec`, `brne`) require 4 clock cycles and are repeated as many times as the value into `r16` register. However, during the last iteration of the loop the three mnemonics are executed in 3 clock cycles, as the `brne` requires one cycle instead or two (because it performs no branch at that particular moment). Based on the same scheme, the outer loop delays the CPU for `t2=(t1+4)*r17=(4*r16+4)*r17` clock cycles. Because we need to generate `t2=1600` clock cycles (which when divided to the 16 MHz synchronization clock of the Arduino Uno board generate 100 μsec delay) we search for two appropriate numbers to be assigned to `r16` and `r17` values. Assigning to `r16` register the value 4 causes the calculation inside the parentheses to produce a number equal to 20 (i.e., `4*r16+4=20`), and, hence `r16=4`. Subsequently, `r17=t2/20=1600/20=80`.

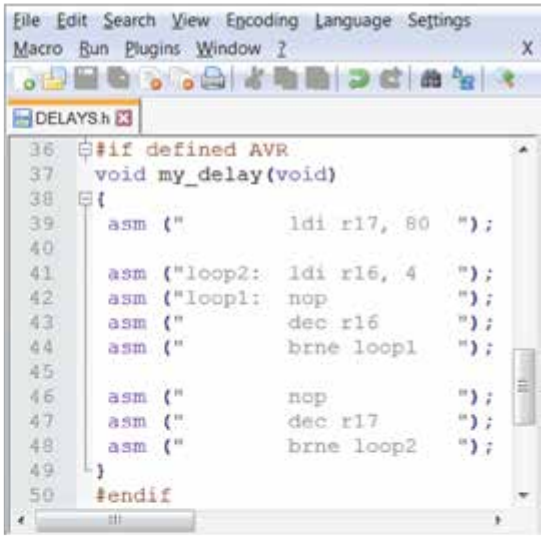
The flowchart of the AVR assembly code, the equivalent high-level scheme, as well as the aforementioned calculations are, respectively, given in Figures 3.22a,b,g. Similar procedure is followed toward the implementation of the PIC assembly code of Figure 3.22e, while the equivalent high-level scheme and the corresponding flowchart diagram and time calculations are, respectively, given in Figures 3.22d,f,h. Since the PIC device has only one CPU register, the example addresses two RAM registers as loop counters, that is, the RAM memory addresses `0x00` (`ram0`) and `0x01` (`ram1`). Because the synchronization clock of PIC18FJ board is 12 MHz, we now calculate 1200 cycles for delaying the MCU. According to the calculation addressed by Figure 3.22h, assigning to ram1 register the value 4 causes the calculation inside the square brackets to produce a number equal to 24 (i.e., `[(1+5*ram1)+3]=24`). Subsequently, if `ram0` is assigned to number 50 then `t=1+24*50=1201` clock cycles (which is very close to the desired 1200 number).

Figure 3.23 presents the PIC and AVR assembly[8] code (Figures 3.23a,b) for delaying the CPU 0.1 msec, as well as the C function (Figure 3.23c) that is used to invoke the proper code. The latter function incorporates a 16-bit variable that is addressed to repeatedly call the assembly code and generate longer delays, multiples of the 0.1 msec. The maximum time delay is determined by the range of the employed variable, that is, $(2^{16} - 1) * 0.1 \cong 6.5$ sec. The conditional inclusion in Figure 3.23a is required because of the differentiations in the MICROC and CCS compiler directives for embedding assembly in the source code.

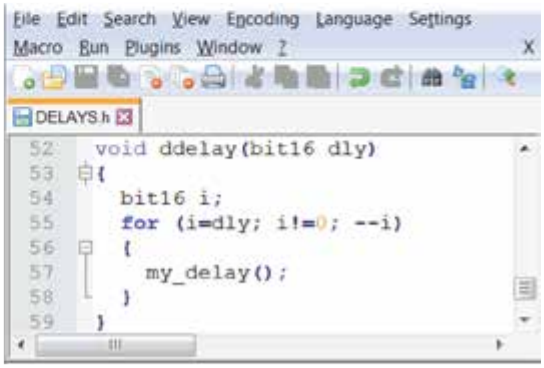
[8]It is here noted that the PIC assembly code is developed without considering the PIC18 extended instruction set enabled.



(a)



(b)



(c)

Figure 3.23: (a) PIC; (b) AVR assembly for 0.1 msec delay; and (c) function to invoke the assembly code.

3.4 GETTING STARTED WITH BLINKING LED

According to the proposed programming strategy presented above, the hardware-related issues certainly bear the major endeavor toward the code's adaptability (which is addressed with the construction of macros for the i/o operations). However, the compiler-specific features overload the tasks required for the generation of a portable application code.

Figure 3.24 illustrates the coding differentiations among different embedded C compilers, through a simple blinking LED example. In detail, the "_BV" macro of AVR GCC compiler (Figure 3.24a) builds a mask that contains either logical 1 or 0 (according to the corresponding assignment operator `''|= _BV''` and `''&= ~BV''`) to the bit location denoted by the macro argument within the parentheses. Thereby, this compiler-specific macro first declares PB5 as output and then, it alternates the state of the pin between ON and OFF. The call of the compiler-specific `''_delay_ms()''` function between the two states, causes the LED to preserve each logical state for 1 sec and, hence, the LED blinks. Moreover, the declaration of two header files is required for identifying the i/o register names and the utilized delay function, that is, `<avr/io.h>` and `<util/delay.h>`, respectively.

MicroC compiler (Figure 3.24b) assigns an 8-bit value to the `TRISE` compiler in order to declare `PORTE` as output port and then, it addresses a different macro (i.e., `PORTE.F4`) so as to assign a bit value to the corresponding output pin. The syntax of the utilized delay routine (i.e., `''delay_ms()''`) is quite different from the previous example. Finally, the `PIC C` compiler (Figure 3.24c) first requires the declaration of a header file identical to the employed micro-controller device (that is, `<18F87J50.h>`), as well as a macro defining the pre-divided frequency that synchronizes the CPU (which is four times the internal frequency for this particular MCU). Then, the code drives the pin between ON and OFF states through the corresponding functions `''output_high()''` and `''output_low()''`. The configuration of the port pin as output is skipped by this example, because the latter functions configure the direction of the pin before assigning it to a value.

Figure 3.24d presents the portable code that can run on both AVR and PIC devices, thanks to the *register definitions* and *i/o macros* within the `REGS-MACROS.h` file as well as the device-specific *delay code* within the `DELAYS.h` file. The additional pinout definition of each development board within the file `PINOUT.h` allows the utilization of common code lines for setting the state (i.e., ON/OFF) of the LED pin. The `DATATYPES.h`, as well as some conditional inclusion code for the "ASM" directives within the `DELAYS.h` file, is addressed to bypass the compiler-specific features which are opposed to the code's portability. Two definitions are required at the beginning of the portable code (Figures 3.24e,f,g). The former defines the hardware device type (i.e., PIC or AVR), while the latter defines the employed embedded C compiler. In the case of `PIC C` compiler the definition of the microcontroller device through the corresponding (compiler-specific) header file is mandatory in this development platform.

Figure 3.25 presents the voltage on the blinking LED (attached to the Arduino Uno board) using different time delays, which are generated with the aforementioned (portable) code.

Standard Coding Style

```
/* AVR GCC Compiler */

#include <avr/io.h>
#include <util/delay.h>


int main ()
{

 DDRB |= _BV(DDB5);

 while(1)
 {
  PORTB |= _BV(PORTB5);
  _delay_ms(1000);

  PORTB &= ~_BV(PORTB5);
  _delay_ms(1000);
 }

 return 0;
}
```

(a)

```
/* microC Compiler */

int main()
{

 TRISE = 0x00;

 while(1)
 {
  PORTE.F4 = 1;
  delay_ms(1000);

  PORTE.F4 = 0;
  delay_ms(1000);
 }

 return 0;
}
```

(b)

```
/* PIC C (PCWHD) Compiler */

#include <18F87J50.h>
#use delay(clock=48000000)

int main()
{

 while(1)
 {
  output_high(PIN_E4);
  delay_ms(1000);

  output_low(PIN_E4);
  delay_ms(1000);
 }

 return 0;
}
```

(c)

Code for Portability

```
#include "C:\MCUs\REGS-MACROS.h"
#include "C:\MCUs\PINOUT.h"
#include "C:\MCUs\DATATYPES.h"
#include "C:\MCUs\DELAYS.h"

int main()
{

 while(1)
 {
  pinSET(LED_dir,LED_out,LED_pin);
  ddelay(10000);

  pinCLEAR(LED_dir,LED_out,LED_pin);
  ddelay(10000);
 }

 return 0;
}
```

(d)

```
/* AVR GCC Compiler */

#define AVR
#define AVRGCC
```

(e)

```
/* microC Compiler */

#define PIC
#define MICROC
```

(f)

```
/* PIC C (PCWHD) Compiler */

#include <18F87J50.h>
#define PIC
#define CCS
```

(g)

Figure 3.24: Embedded C programming style among different compilers and coding for portability.

We observe a small deviation from the theoretical values and, in detail, Figure 3.25a illustrates the actual delay time of 212 μsec for a single iteration of the *while* loop, while the theoretical period would expected to be 200 μsec (plus the time needed to turn the LED ON and OFF and branch to the start of loop). Figures 3.25b,c illustrates the actual time delay of 204 msec and 2.02 sec, while the theoretical period is 200 msec and 2 sec, respectively. Finally, Figure 3.25d illustrates the maximum time delay of 6.6 sec, when the repetition counter is set to the value of 65535. It is noted that in this particular example the counter is set to the value of 6553 during the LED's OFF state, which results in the non-periodic waveform of the figure.

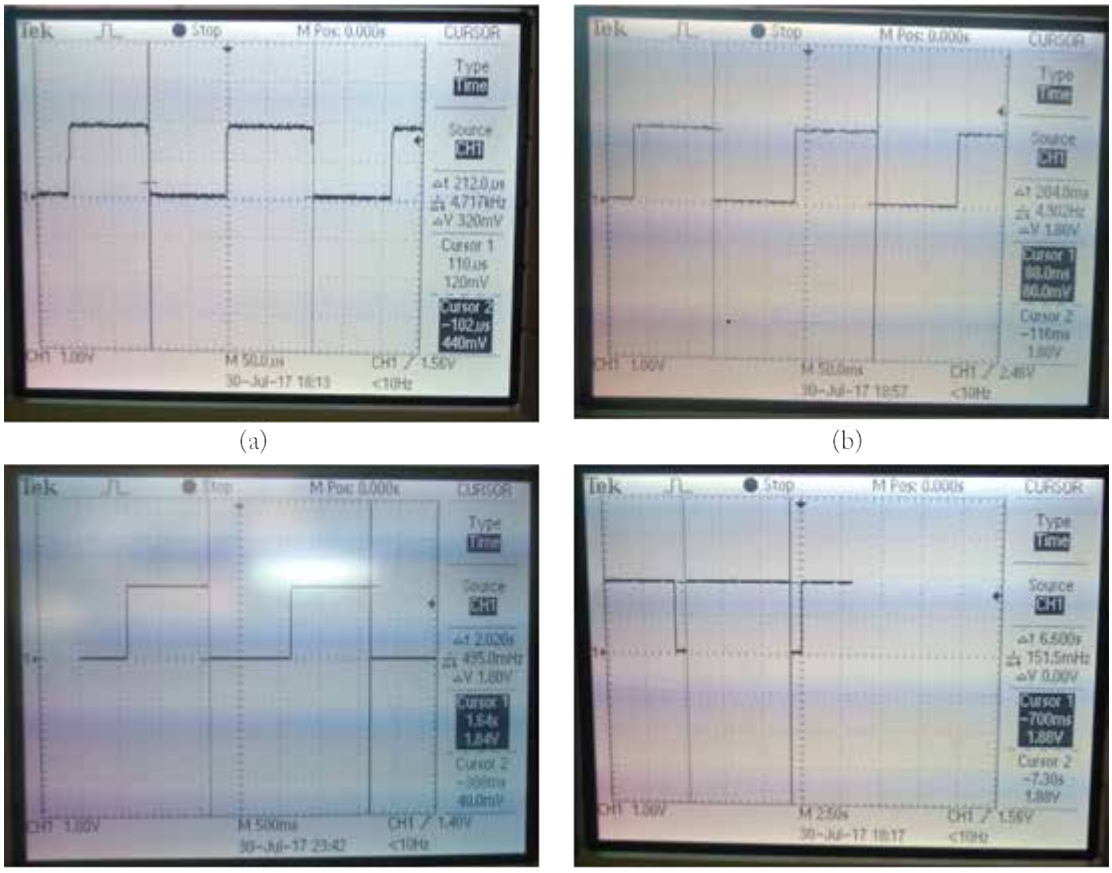


Figure 3.25: Data acquisition of the ON/OFF states of the blinking LED.

CHAPTER 4

Micro-Controller Applications

This chapter applies to microcontroller applications, while the development of the application firmware is in agreement with the proposed and portable programming strategy analyzed previously in the book.

4.1 DOMINANT COMMUNICATION PROTOCOLS FOR HARDWARE INTERFACING

Up until this point we have discussed how a microcontroller device interfaces with the outside world through its digital i/o pins. However, the establishment of a complicated interface, such as a communication link between the microcontroller and a personal computer through USB, is not trivial. For that reason the microcontroller suppliers, as well as many third-party corporations, deliver ready-to-use hardware modules that assure the successful establishment of an interface (e.g., USB), while also providing an easy communication protocol with the microcontroller device. The most popular communication protocols for establishing a link between the microcontroller and a hardware device module are the: (a) UART, (b) I2C, and (c) SPI. Because of their popularity, several microcontroller models incorporate internal peripheral devices conformed to the specifications listed by these three protocols. Yet, due to the portable programming strategy addressed by this book, we will explore how to manually implement a simplified version of these three protocols.

Next, we attempt to separate the available modules according to the type of protocol they incorporate, for the communication with a microcontroller device. Figure 4.1 depicts a possible microcontroller-based application, implementing a *data acquisition* (DAQ[1]) system. The MCU board communicates with four different modules via the aforementioned protocols. In detail, the MCU communicates with a USB module (denoted 1) via UART protocol in order to exchange data with a computer. Measurements of a physical phenomenon (e.g., atmospheric pressure) are obtained from a sensor module (denoted 2) via I2C protocol. Those measurements are available to the computer through a possible GUI acquiring data through the USB port, but could also be available through the Internet as the MCU communicates with an Ethernet module (denoted 3) via SPI protocol. Finally, the implementation of a *wireless personal area network* (WPAN) is possible since the MCU communicates with an RF module (denoted 4) via SPI protocol. With the employment of the latter module the microcontroller could, for instance, receive wireless

[1] A DAQ system measures a physical phenomenon (such as, temperature, humidity, etc.) and converts the acquired samples into digital values, capable of being manipulated by a computer system.

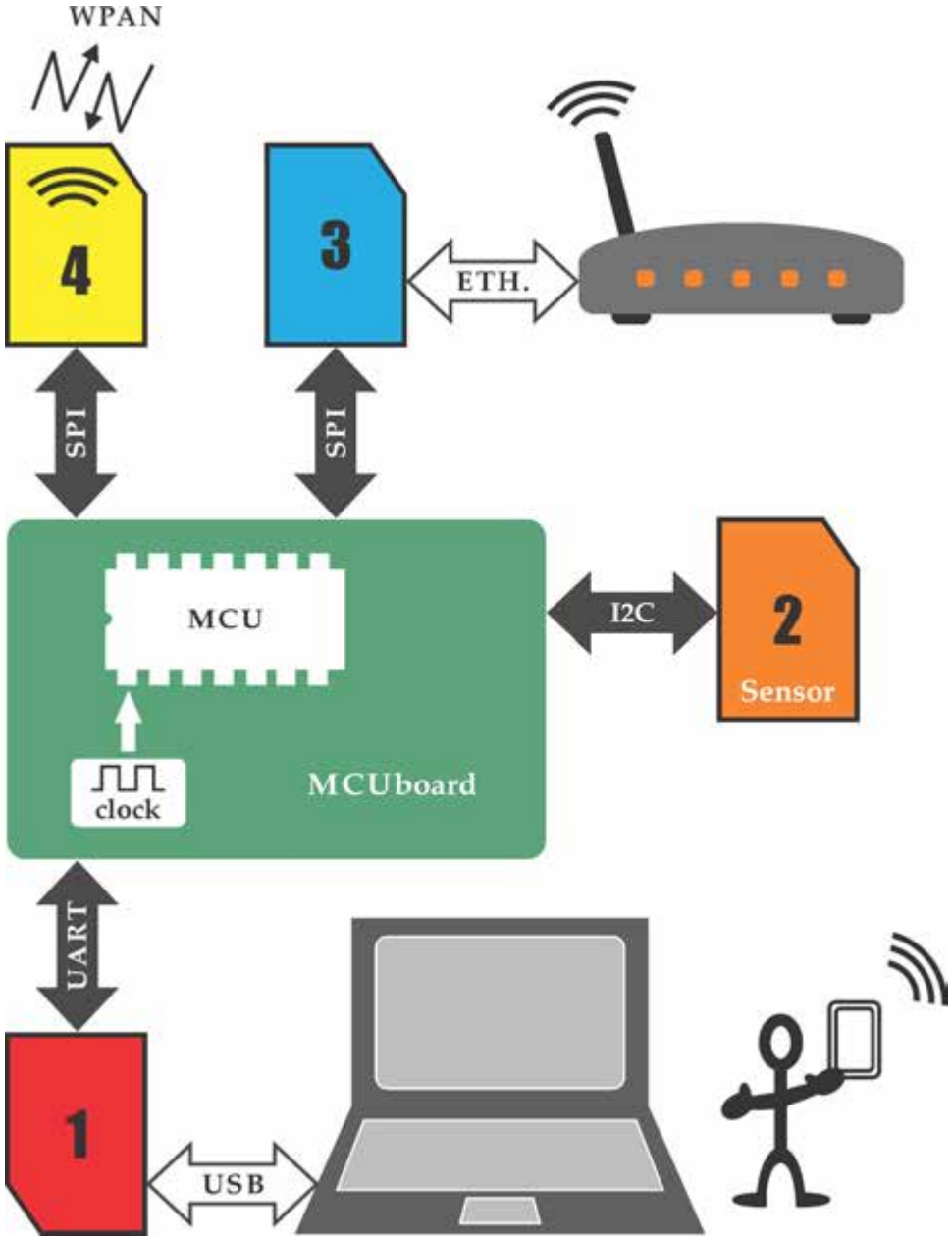


Figure 4.1: Data acquisition system.

measurements from a battery-operated system located in distant place (where mains electricity is not available). A possible implementation of the latter device is given in Figure 4.2. The latter system employs an MCU board, an RF module, a sensor module, and a battery and, hence, it applies to battery-powered autonomous wireless sensor devices.

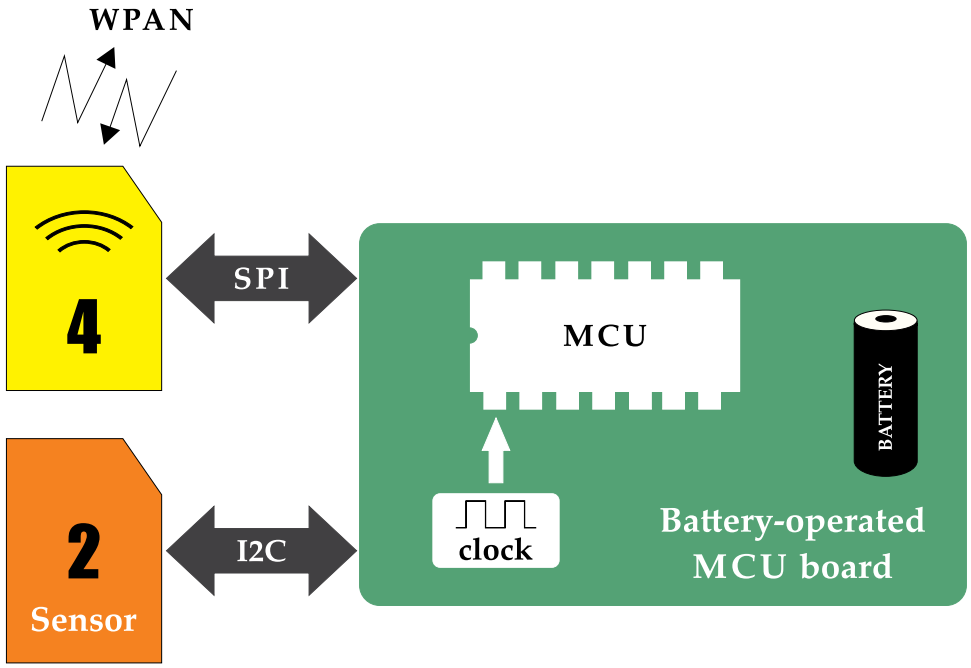


Figure 4.2: Wireless (battery-operated) sensor device.

The MCU-sensor communication is regularly performed through I2C. Because a microcontroller-based system may occupy several sensor devices, the latter protocol renders feasible the exchange of information among several ICs using only two signal lines. The SPI protocol, on the other hand, uses four signals and for each additional SPI module connected to the system, the microcontroller allocates one more pin for the selection of the device to be used in the upcoming communication. Yet, the SPI is considered a faster communication protocol compared to I2C, and therefore, it is commonly incorporated by modules that are meant for composite interfaces, such as Ethernet communication, wireless communication, etc. In addition, a microcontroller-based system would generally employ just a couple of such interfaces and thereby the total number of the allocated SPI pins in not an issue for the application.

For many years, the UART protocol was the standard communication interface between a microcontroller device and the RS-232 serial port of a personal computer. RS-232 is a simplified point-to-point signaling standard (i.e., only two devices can be connected to each other), having no dependency on a higher-level protocol. Hence, the interconnection between the

microcontroller and a computer system was straightforwardly performed through an RS-232 driver/receiver, such as the popular MAX232 [78]. The latter was needed because of the inconsistency of the (logical 0 and 1) voltage levels defined by the RS-232 standard (regularly in agreement with a bipolar ±12 V supply), and the logical levels commonly used by a microcontroller device (commonly in agreement with a single +5 V supply).

Nowadays, the simplified RS-232 serial port has been replaced by the composite USB serial port. Hopefully, there are plenty USB to UART ICs available in the embedded systems market, such as the FT2232H chip [79]. Those ICs provide a simplified UART interface for the microcontroller, while also undertaking the hard work for exchanging data compliant to the USB protocol specifications. Hence, they still render the UART one of the dominant communication protocols for information exchanging between the microcontroller device and a personal computer.

Hereafter is given a short description of the aforementioned serial protocols (UART, I2C, and SPI) along with an example firmware code providing a simplified implementation of them at the microcontroller level.

4.1.1 UART COMMUNICATION PROTOCOL

The most common interconnection between two UART devices (Figure 4.3) incorporates three signal lines, that is, the Tx and Rx lines for transmitting and receiving data, respectively, as well as the common ground line (Gnd) among the two devices. The separate Tx, Rx lines reveal that data can simultaneously flow in both direction, in terms of a full-duplex serial communication. However, because of the sequential execution of instructions within a conventional computer system, the UART communication is regularly performed in a half-duplex mode. That is, only one device at the time is set to transmit or receive data. In addition, the lack of a (synchronization) clock signal among the two devices illustrates an asynchronous type of serial communication. In particular, the two devices work on their own clock signal, but they are configured to exchange information based on a matching transaction rate, aka *baud rate*. The latter illustrates the period of each transmitted bit. For instance, when the baud rate is set to 9600, the period of each transmitted bit is approximately equal to 104 μsec (that is 1/9600 Hz).

Figure 4.3 presents the UART transmission/reception of ASCII character `'C'` (which equals to the hexadecimal value `0x40`). When no transaction is performed among the two devices the line is held to logical 1, thus denoting an *idle* line. The transmitter initiates a new transaction by pulling and holding the line low for one bit period, and then sends the data bits starting from the LSB. The first bit is regularly referred to as *start* bit and is always identified as logical 0. The most common communication is the 8N1, that is, 8 data bits, no *parity*[2] bit, and one *stop* bit (like the one presented in the figure example). The latter bit is always identified as logical 1 and denotes the end of the transmission. On the other side, the receiver identifies the start bit through the high-to-low transition of its Rx line, and then starts to acquire samples (regularly

[2]The parity bit is optionally used to announce an odd/even number of logical 1s sent by the transmitter.

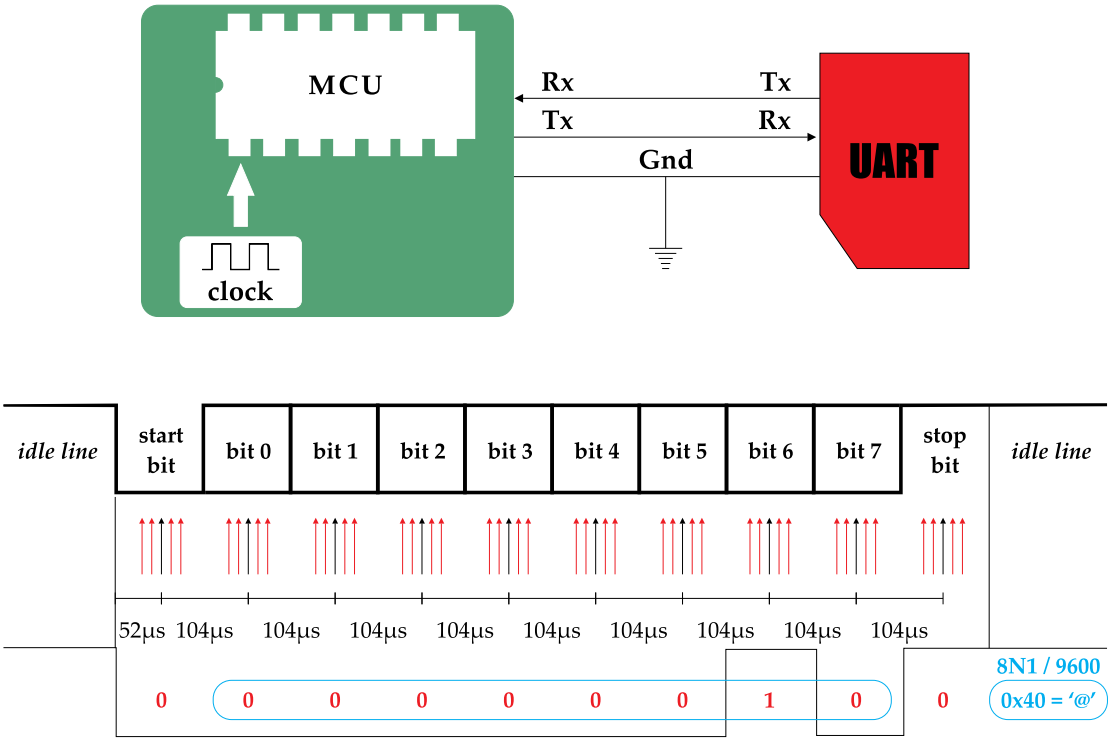
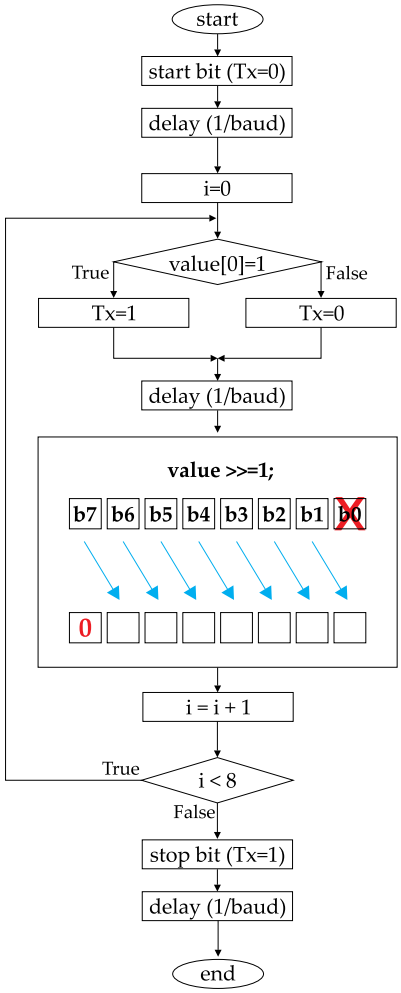


Figure 4.3: UART topology and communication.

16) so as to assure that a valid bit was received and not just an unwanted spike. If the start bit is confirmed, the receiver continues to receive the expected number of data bits based on the predefined baud rate and the type of communication (e.g., 9600/8N1).

Next, we examine two custom-designed UART functions which will be used later in the chapter for transmitting and receiving data conformed to the 9600/8N1 configuration. The functions invoke the i/o macros explored in the previous chapter. The flowchart and source code for the data transmission are given in Figures 4.4a,b, respectively. The function admits a byte-size argument (i.e., a variable named `value` in this example) and sends all 8 bits of that variable through the Tx signal line, starting from the LSB. This process is accomplished within a `for` loop which runs eight successive times. At first, the loop extract the LSB bit of the variable with the utilization of the bitwise `AND` along with the mask `0x01`, and clears Tx line if the latter outcome equals to zero, otherwise it sets Tx signal line. Then it shifts the variable one position to the right so as to evaluate the subsequent bit of the variable upon the upcoming iteration.

The flowchart and source code for the data reception are given in Figures 4.5a,b, respectively. The function admits no arguments and returns a byte-size value, which is formed by the serial bits identified in the Rx line. As soon as the receiver identifies the reception of a start bit



(a)

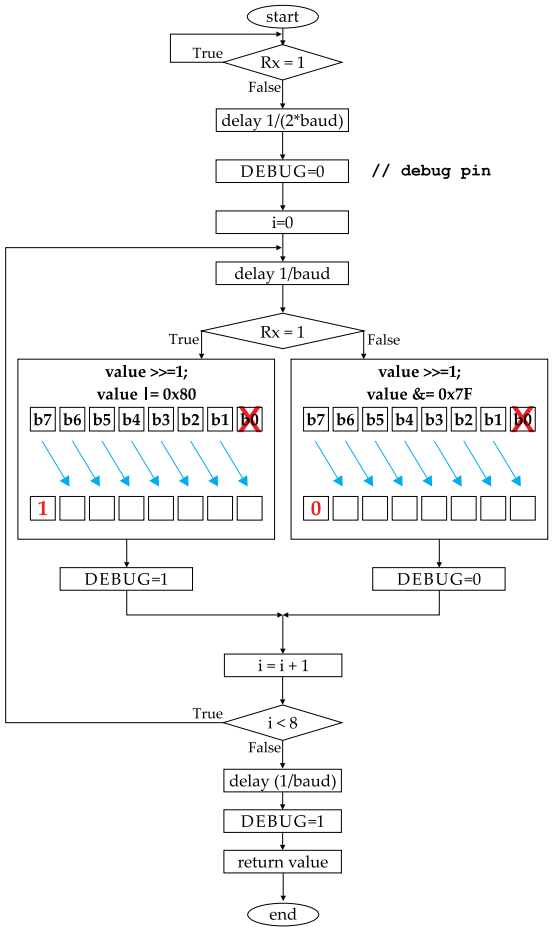
```
void putcharFTDI(bit8 value)
{

 bit8 i;

  pinCLEAR(dirUART,outUART,txUART); // start bit
  delay_baud(1);

  for (i=0;i<8;i++)
  {

    if( (value) & 0x01) // LSB first
    {
      pinSET(dirUART,outUART,txUART);
    }

    else
    {
      pinCLEAR(dirUART,outUART,txUART);
    }

    delay_baud(1);
    value >>= 1;

  }

  pinSET(dirUART,outUART,txUART); // stop bit
  delay_baud(1);
}
```

(b)

Figure 4.4: UART function for data transmission.

(i.e., a high-to-low transition) at the Rx line, it delays for half of the regular period of the serial bit in order to reach to the middle point of the start bit. Within a `for` loop that is repeated for 8 times, the receiver first delays for one period to reach to the middle of the next serial bit. Then it shifts the variable one position to the right and appends a logical 1 or 0 to the MSB bit of the variable, in accordance to the voltage level identified in the Rx line (and with the utilization of the appropriate bitwise operator and mask). After eight successive iterations the receiver delays

again one bit period so as to wait for the stop bit reception to be completed, and then returns the 8-bit variable.



(a)

```
bit8 getcharFTDI()
{

 bit8 i, value=0;

  while (pinREAD(dirUART,inUART,rxUART));

  delay_halfbaud(); // go to middle of start bit
  pinCLEAR(DEBUG_dir,DEBUG_out,DEBUG_pin); // debug

  for (i=0;i<8;i++)
  {

   delay_baud(1);  // delay first to discard start bit

   if (pinREAD(dirUART,inUART,rxUART))
   {
    value>>=1;
    value |= 0x80;
    pinSET(DEBUG_dir,DEBUG_out,DEBUG_pin); // debug
   }

   else
   {
    value>>=1;
    value &= 0x7F;
    pinCLEAR(DEBUG_dir,DEBUG_out,DEBUG_pin); // debug
   }

  }

  delay_baud(1);// delay to discard stop bit
  pinSET(DEBUG_dir,DEBUG_out,DEBUG_pin); // debug

 return value;

}
```

(b)

Figure 4.5: UART function for data reception.

The implementation of a user-defined UART communication constitutes perhaps the most risky attempt because of the asynchronous mode of transaction and, in particular, in the receiver side. In the aforementioned function we considered that the code inside the loop, which is addressed for generating the byte-wide variable according the received serial data, is of negligible time consumption. However, this is only an ideal situation and the time frame given in Figure 4.6 illustrates how this extra time can produce a non-errorless data reception (because of a successive time shift in the evaluation of each bit by the receiver).

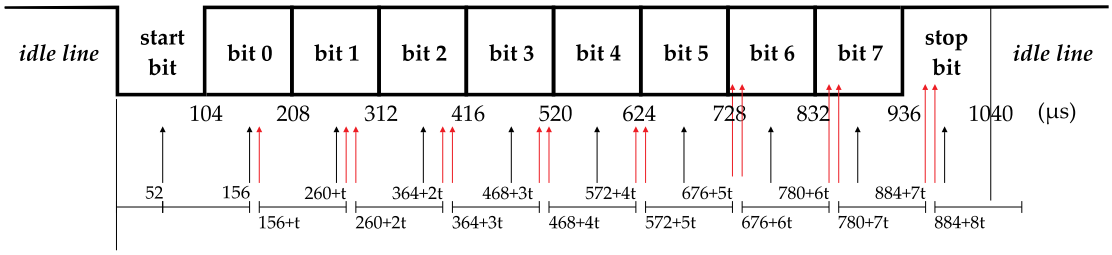


Figure 4.6: UART data reception with detection error.

Ideally, in the 9600/8N1 configuration of this example the data reception would evaluate the Rx line at $52 + 104 = 156$ μsec (bit 0) than at $156 + 104 = 260$ μsec (bit 1), and so forth (as depicted by the dotted-line black arrow). However, the additional *time t* which is needed for the formation of the byte-wide variable, forces the microcontroller to evaluate Rx line for bit 0 at 156 μsec and finish the formation of the variable at $156 + t$ μsec. Subsequently, the MCU evaluates bit 1 at $260 + t$ μsec (dotted-line red arrow) and finishes the formation of the variable at $260 + 2t$ μsec (solid-line red arrow), and so forth. Assuming that the time `t` is equal to 12 μsec, the additive delay causes the microcontroller to evaluate bit 5 at $52 + (6 * 104) + 6 * t = 736$ μsec. Yet, the transmitter initiates bit 5 transaction at 624 μsec and finishes at 728 μsec and hence, the receiver evaluates bit 6 instead of bit 5.

It is worth noting that this additive time depends on the machine code generated by the employed compiler and can vary among different compilers. One possible and simple solution (that will be used in the upcoming examples) is to start the evaluation of Rx line after a small delay following the high-to-low transition of the start bit. However, a more sophisticated solution would be the calculation of this additive time delay and its subtraction from the delay identical to a single bit period. The calculation of this extra time delay is possible by addressing assembly code for this part of the reception routine.

4.1.2 I2C COMMUNICATION PROTOCOL

The I2C protocol uses three signal lines, but contrary to the UART protocol it does not support full-duplex communication. The I2C is based on synchronous type of (half-duplex) serial communication synchronized by a clock signal called *serial clock* (SCL). On the other hand, the data are transferred through a bidirectional signal line called *serial data* (SDA). The third signal is the common ground line (Gnd) among the two devices. A typical I2C interconnection like the one depicted by Figure 4.7 incorporates one master device and one or more slaves. The SCL line is controlled by the master device which is capable of initiating a transfer, and the slaves respond to the master's call as soon as they identify a matching address. I2C slave devices (such as an I2C sensor module) commonly feature a 7-bit address which admits 128 different values

(that is, from 0–127), and which defines the maximum number of slaves that can be connected to an I2C bus.

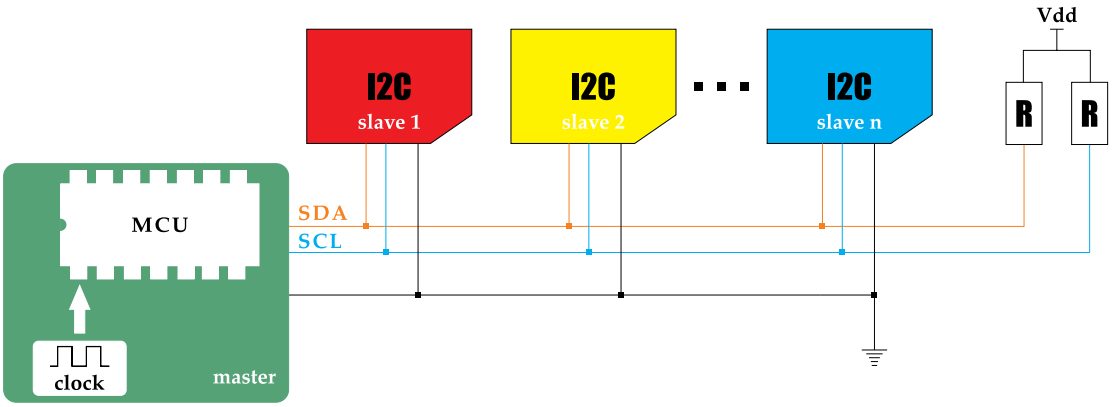


Figure 4.7: I2C topology.

The two pull-up resistors depicted in Figure 4.7 are needed because SDA and SCL are open-drain i/o pins. When the I2C device needs to transfer a logical 0 it pulls the line low 0, whereas releasing the line (i.e., letting the line float) is associated with a transfer of logical 1. This physical layer assures no conflict or damage of the associated devices when more than one units attempt to drive the I2C lines simultaneously, while also providing the possibility of detecting collisions. It is also worth referring to the 100 KHz *standard* clock speed as defined by the I2C specifications, as well to the 400 KHz and 3.4 MHz *fast* and *high-speed* mode of operation, respectively.

When no transaction is present on the I2C bus, the SCL and SDA lines are floating (i.e., a logical 1 is present on each signal line). To initiate a transfer the master pulls SDA low while SCL is held high and this operation is known as *I2C start condition*. Then it begins with the transmission of the I2C address of the device that desires to communicate with, starting from the MSB. Data are sent in sequences of 8 bits and, hence, after the 7-bit address the master appends a logical 1 or 0 in case a read or write operation, respectively, is about to be performed. Then the master releases SDA line and waits for the *acknowledge* (ACK) signal to be sent by the slave device. If SDA line is left floating on the 9^{th} clock then the slave did not respond (ACK=1). Otherwise, the master verifies the response of the slave device and starts the transfer of the second byte in row. Every transaction finishes with an *I2C stop condition* sent by the master; that is, a low-to-high transition of the SDA line after releasing SCL line.

Figure 4.8 presents a successful write operation to a single register of an I2C device. The overall transaction is performed by the master, except the serial data denoted in orange (that is, the ACK bit sent by the slave device). At first, the master initiates a start condition. Then it sends the first byte containing the 7-bit address of the I2C device (that is, `A6:0`) along with

the R/#W bit cleared to zero, denoting an I2C write process. This task is performed in 8 clock pulses (all controlled by the master) and on the 9^{th} pulse of the clock, the slave responds by clearing SDA line (i.e., ACK=0). Thereby, the master continues the process by transferring the second byte in row, which holds the address of the internal register to be written. As soon as the slave responds (on the 9^{th} clock pulse) with a zero ACK bit, the master sends the 8-bit data to be written in the internal register of the slave device. The slave responds again with ACK=0 and, finally, the master concludes the transaction with a stop condition.

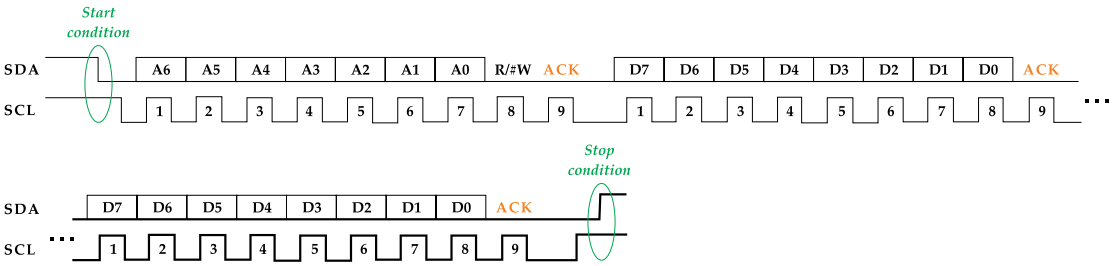


Figure 4.8: I2C communication: write data to a single register.

Figure 4.9 presents a successful read operation from a single register of an I2C device. The process of the first two transferred bytes is the same as before; that is, the master (a) sends the I2C address along with R/#W=0, (b) receives the first ACK=0, (c) sends the address of the internal register to be read, and (d) receives the second ACK=0. The read operation continues with transaction of two more bytes. In terms of third byte, the master (a) repeats a start condition, (b) resends the I2C address but this time a bit R/#W=1 is appended in the transferred byte, thus, denoted an I2C read process, and (c) receives the third ACK=0. In consideration of fourth and concluding byte, the master (a) receives the content of the slave's I2C register, (b) sends a NACK=1 to the slave in order to inform the latter device about a successful read procedure, and (c) sends a stop condition. It is here noted that the master sends ACK in I2C multiple byte read (except in the final byte where the master also send NACK).

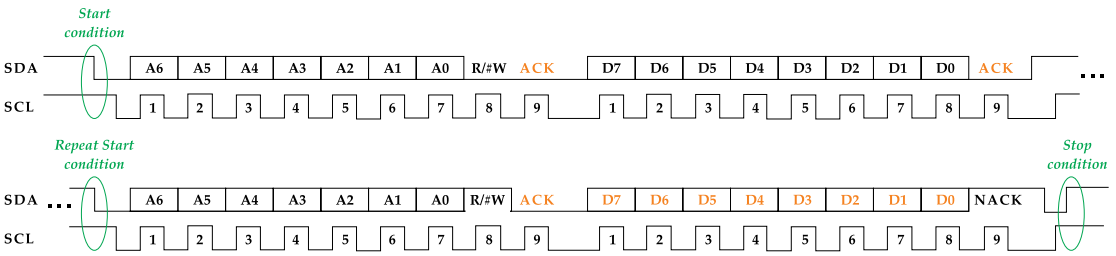


Figure 4.9: I2C communication: read data from a single register.

Figure 4.10 presents the top-level I2C functions and flowcharts for writing to and reading from an internal register of a slave device. The functions are in agreement with the abovementioned operation and transmit/receive a single byte on each function call.

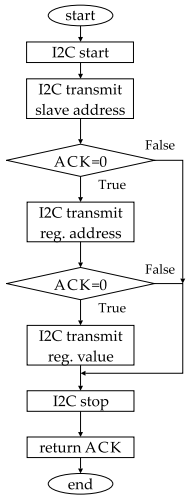
The low-level flowchart and function of the I2C serial write process are given in Figure 4.11. The function admits an 8-bit argument which is shifted left for 8 successive times within a loop, while the latter runs for 8 plus 1 times. On each particular shift the function generate a logical 1 or 0 in accordance to the MSB value found in the variable. In the last (9^{th}) iteration the code acquires the ACK bit from the slave device, then delays for one quarter of the SCL period, and finally returns the ACK value. The latter delay is addressed for distinguishing among successive byte transfers in case there is a need to inspect the timing diagrams on an oscilloscope.

The low-level flowchart and function of the I2C serial read process are given in Figure 4.12. The function admits an 8-bit argument incorporating the ACK/NACK bit (i.e., logical 0 or 1, respectively) that will be sent from the master device on the last transaction. The code addresses a loop that runs eight successive times. Each time the master generates a clock rise on the SCL line and then evaluates the data on the SDA line (sent by the slave). The LSB of the `I2C_readData` variable is set to logical 1 or 0 in accordance to the identified value on the SDA line, and then the variable is shifted one position to the left so as to leave room on the LSB position for the upcoming serial bit (denoted X, herein, as a *don't care* mark if the LSB is set or cleared by the shifting process). The code generates a high-to-low transition of the SCL line to force the slave extract then next serial bit on the SDA line. After eight consecutive times the master sends the NACK bit, delays one quarter of the SCL period, and finally returns the `I2C_readData` variable.

At this point it is worth noting that every time the master generates a clock rise on the SCL signal it waits until the slave releases the line. Because the I2C clock speed is determined by the master, the slave device is allowed to hold the line low if it is not able to cooperate with this clock and notifies, in this way, the master device that it needs to slow down the transfer rate. This mechanism is regularly referred to as *clock stretching*.

Figure 4.13 presents the functions and the generated timing diagrams of an `I2C start` and `I2Cstop` condition. The X notation on the SDA and SCL lines denotes that we don't need to know about the logical state of these lines when the function code is invoked. Start condition begins at `t1` and stop condition begins at `t2`. Both lines are cleared to zero when returning from the `I2Cstart` function, while they are set to logical 1 when returning from the `I2Cstop` function. It is here noted that delay code lines of all I2C functions are equal to the one quarter of the actual SCL period and, hence, this time delay should be conformed to the I2C clock speed specifications. For instance, if we address 100 KHz standard I2C clock speed, the one quarter delay should not be less than 2.5 μsec ($T = 4 * 2.5 = 10$ μsec $\rightarrow F = 1/T = 100$ KHz).

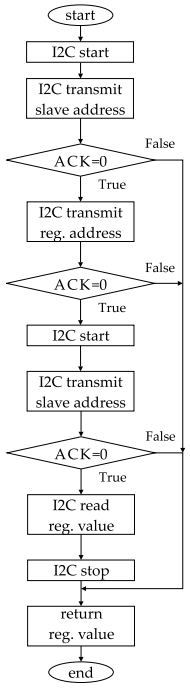
Figure 4.14 presents the actual timing diagram (based on the proposed I2C functions) which represents the reading process of an I2C sensor. That is, the MEMS barometric sensor



(a)

```
bit8 I2C_wrByte(bit8 reg, bit8 value, bit8 I2Caddr)
{
 bit8 ack=1;
  I2Cstart();
  ack=I2C_Tx((I2Caddr<<1) & 0xFE); // R/#W=0 (LSB)
  if(ack==0)
  {
   ack = I2C_Tx(reg);
   if(ack==0)
   {
    ack = I2C_Tx(value);
   }
  }
  I2Cstop();
 return ack;
}
```

(b)

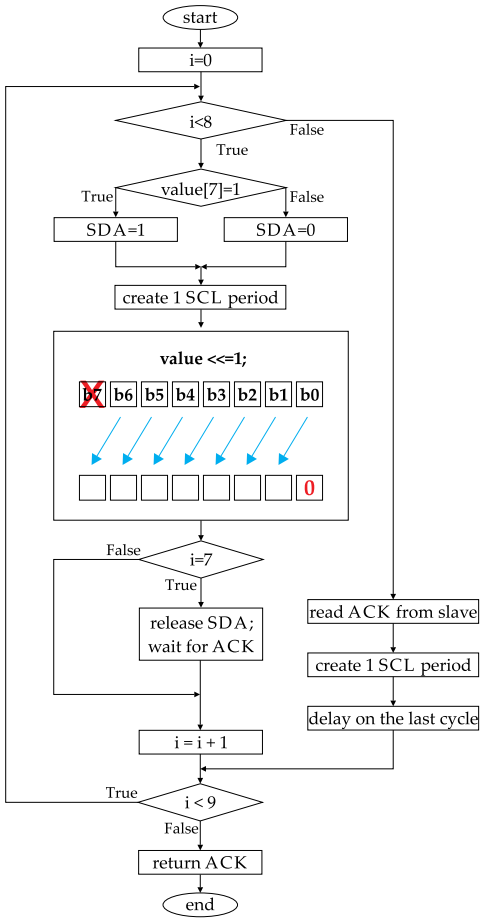


(c)

```
bit8 I2C_rdByte(bit8 reg, bit8 I2Caddr)
{
 bit8 ack=1;
 bit8 regValue=1; // regValue=NACK if slave not responding
  I2Cstart();
  ack = I2C_Tx((I2Caddr<<1) & 0xFE); // R/#W=0 (LSB)
  if(ack==0)
  {
   ack = I2C_Tx(reg);
   if(ack==0)
   {
    I2Cstart();
    ack = I2C_Tx((I2Caddr<<1) | 0x01); // R/#W=1 (LSB)
    if(ack==0)
    {
     regValue = I2C_Rx(1); //read data & send master NACK
    }
   }
  }
  I2Cstop();
 return regValue;
}
```

(d)

Figure 4.10: I2C top-level flowcharts and functions for (a), (b) writing and (c), (d) reading single bytes.

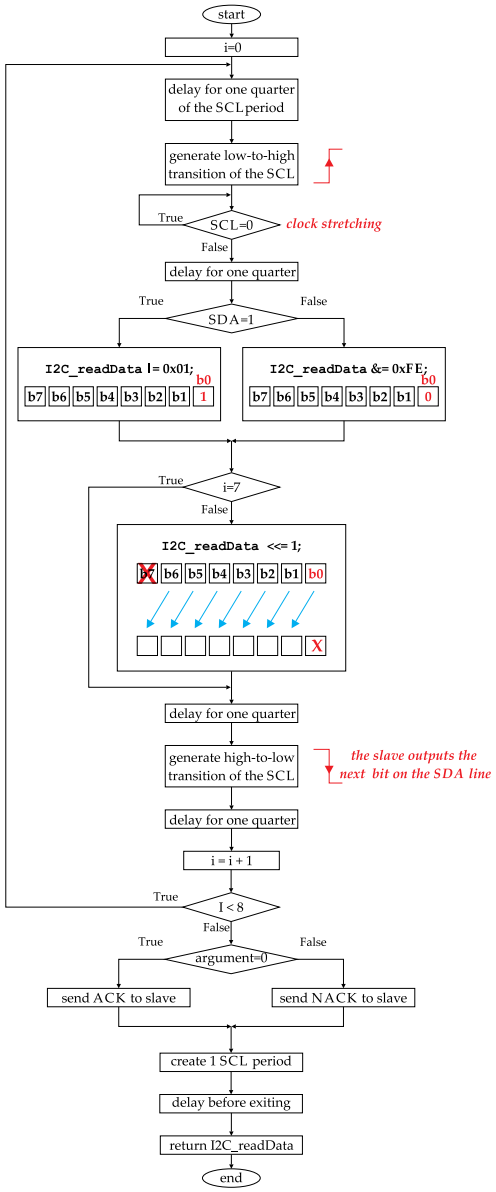



```
bit8 I2C_Tx(bit8 value)
{
 bit8 ack=0, i=0;
   for (i=0;i<9;i++)
   {
      if (i<8)
      {
         if(value & 0x80)
         {
          inputPIN(I2Csda_dir,I2Csda_pin);
         }
         else
         {
          pinCLEAR(I2Csda_dir,I2Csda_out,I2Csda_pin);
         }
         delay_i2C(I2Cscl_oneQuarter);
         inputPIN(I2Cscl_dir,I2Cscl_pin);
         delay_i2C(I2Cscl_oneQuarter);
         delay_i2C(I2Cscl_oneQuarter);
         pinCLEAR(I2Cscl_dir,I2Cscl_out,I2Cscl_pin);
         delay_i2C(I2Cscl_oneQuarter);
         value = value<<1;
         if (i==7)
         {
          inputPIN(I2Csda_dir,I2Csda_pin);
          delay_i2C(I2Cscl_oneQuarter);
         }
      }
      else
      {
       ack = pinREAD(I2Csda_dir,I2Csda_in,I2Csda_pin);
       inputPIN(I2Cscl_dir,I2Cscl_pin);
       delay_i2C(I2Cscl_oneQuarter);
       delay_i2C(I2Cscl_oneQuarter);
       pinCLEAR(I2Cscl_dir,I2Cscl_out,I2Cscl_pin);
       delay_i2C(I2Cscl_oneQuarter);
       delay_i2C(I2Cscl_oneQuarter);
       delay_i2C(I2Cscl_oneQuarter);
      }
    }
 return ack;
}
```

(b)

Figure 4.11: I2C flowchart and function for low-level serial write of a single byte.

LPS25HB [80] which will be used in an upcoming project. The slave (7-bit) address is the `101110X_2`, where the LSB is determined by the logical level on a device pin called SDO/SA0. The latter pin is attached to the power supply in this example and, hence, the 7-bit address is defined as follows: `1011101_2=5D_16`. The four bytes transferred through SDA line are as: (a) the—sent by the master—`0xBA` value, which is the slave address `0x5D` shifted one position to the left and appended to a `R/#W` bit equal to 0; (b) the—sent by the master—`0x0F` value, which is the address of the internal register to be read; (c) the—sent by the master—`0xBB` value, which is the slave address 0x5D shifted one position to the left and appended to a `R/#W` bit equal to 1; and (d) the—sent by the slave—`0xBD` value, which is the content of the register `0x0F`. The latter



(a)

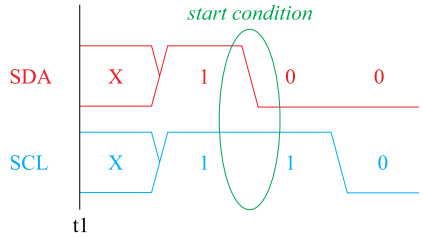
```
bit8 I2C_Rx(bit8 value)
{
 bit8 I2C_readData=0, i=0;
   for (i=0;i<8;i++)
   {
      delay_i2C(I2Cscl_oneQuarter);
      inputPIN(I2Cscl_dir,I2Cscl_pin);
      while(!pinREAD(I2Cscl_dir,I2Cscl_in,I2Cscl_pin));
      delay_i2C(I2Cscl_oneQuarter);
      if (pinREAD(I2Csda_dir,I2Csda_in,I2Csda_pin))
      {
       I2C_readData = (I2C_readData|0x01);
      }
      else
      {
       I2C_readData = (I2C_readData&0xFE);
      }
      if(i==7)
      {
       I2C_readData = I2C_readData;
      }
      else
      {
       I2C_readData = I2C_readData << 1;
      }
      delay_i2C(I2Cscl_oneQuarter);
      pinCLEAR(I2Cscl_dir,I2Cscl_out,I2Cscl_pin);
      delay_i2C(I2Cscl_oneQuarter);
   }
   if(value == 0)
   {
    pinCLEAR(I2Csda_dir,I2Csda_out,I2Csda_pin);
   }
   else
   {
    inputPIN(I2Csda_dir,I2Csda_pin);
   }
   delay_i2C(I2Cscl_oneQuarter);
   inputPIN(I2Cscl_dir,I2Cscl_pin);
   delay_i2C(I2Cscl_oneQuarter);
   delay_i2C(I2Cscl_oneQuarter);
   pinCLEAR(I2Cscl_dir,I2Cscl_out,I2Cscl_pin);
   delay_i2C(I2Cscl_oneQuarter);
   delay_i2C(I2Cscl_oneQuarter);
   delay_i2C(I2Cscl_oneQuarter);
 return (I2C_readData);
}
```

(b)

Figure 4.12: I2C flowchart and function for low-level serial read of a single byte.

```
void I2Cstart(void)
{
  delay_i2C(I2Cscl_oneQuarter);
  inputPIN(I2Csda_dir,I2Csda_pin);
  inputPIN(I2Cscl_dir,I2Cscl_pin);
  delay_i2C(I2Cscl_oneQuarter);
  pinCLEAR(I2Csda_dir,I2Csda_out,I2Csda_pin);
  delay_i2C(I2Cscl_oneQuarter);
  pinCLEAR(I2Cscl_dir,I2Cscl_out,I2Cscl_pin);
  delay_i2C(I2Cscl_oneQuarter);
}
```

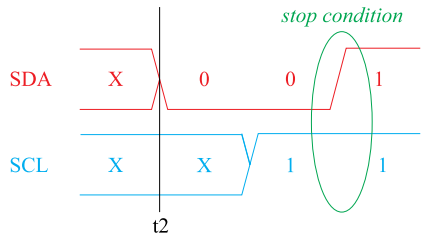
(a)



(b)

```
void I2Cstop(void)
{
  pinCLEAR(I2Csda_dir,I2Csda_out,I2Csda_pin);
  delay_i2C(I2Cscl_oneQuarter);
  inputPIN(I2Cscl_dir,I2Cscl_pin);
  delay_i2C(I2Cscl_oneQuarter);
  inputPIN(I2Csda_dir,I2Csda_pin);
  delay_i2C(I2Cscl_oneQuarter);
}
```

(c)



(d)

Figure 4.13: I2C functions and generated timing diagrams of (a), (b) start and (c), (d) stop condition.

register holds the descriptive name `WHO_AM_I` and it is used to validate the correct operation of the I2C device (as it is permanently assigned to the content `0xBD`).

It is worth noting the serial data received from the slave device, which all appear immediately after the high-to-low transition on the SCL line (exclusively controlled by the master device). This is also observable by the ACK bits, sent by the slave.

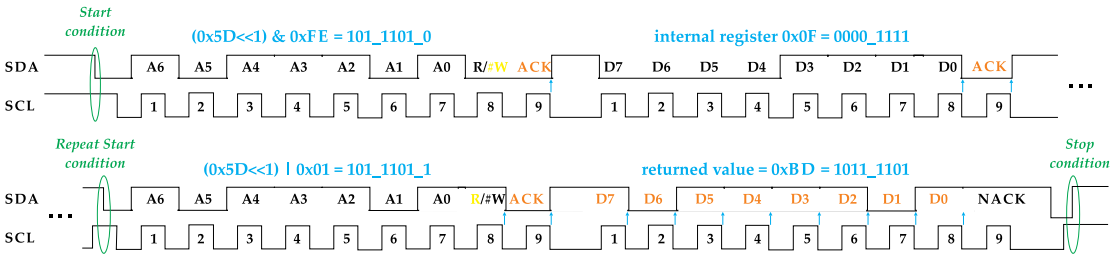


Figure 4.14: I2C read (from slave `0x5D`) the internal register `0x0F` and obtain the value `0xBD`.

4.1.3 SPI COMMUNICATION PROTOCOL

The SPI protocol uses four signal lines, but contrary to the I2C, it supports full-duplex communication. The SPI is based on synchronous type serial communication, synchronized by a clock signal regularly referred to as SCK (or SCLK). The SPI topology is given in Figure 4.15. When master desires to exchange information with a slave device, it pulls the corresponding *chip select* (CS) signal low. The master sends serial data to the slave device through the *master out-slave in* (MOSI) signal, while it receives data from the slave through the *master in-slave out* (MISO) signal. The SPI clock frequency generated by the master should not be higher than the maximum operating frequency specified by the slave datasheet.

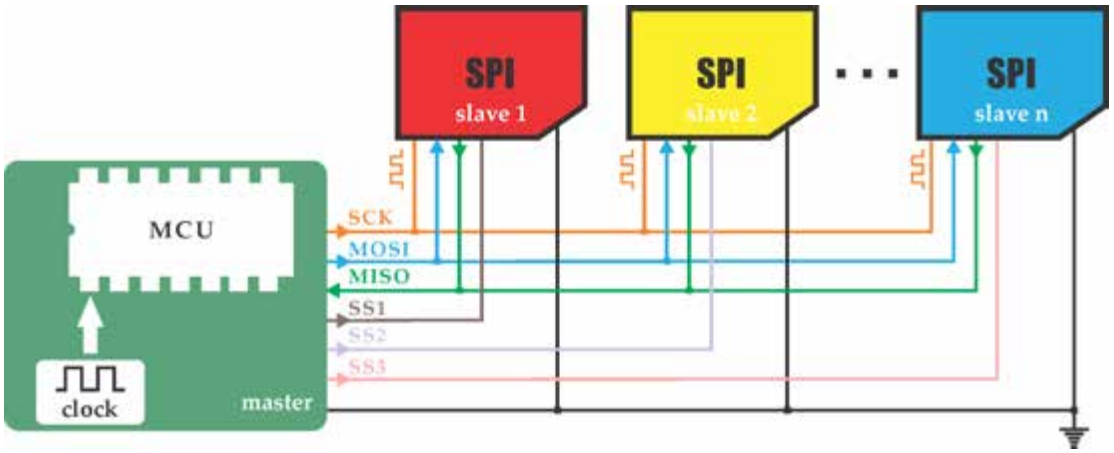


Figure 4.15: SPI topology.

The SPI protocol supports four different modes of communication defining (a) the clock edge where data will appear on the MOSI and MISO signals, (b) the clock edge that should be used for sampling data, and (c) the idle level of the SCK signal (when no transfer is performed). Figure 4.16 illustrates the four communication modes, as defined by the clock polarity (CPOL) and clock phase (CPHA) parameters.

Figure 4.17 presents a regular SPI timing diagram for writing and reading bytes to/from an internal register (of 8-bit address) of a slave device. The diagram illustrates mode 3 type of SPI communications, that is, (a) a high level for SCK idle line, (b) a bit toggle on the falling edge of the clock (i.e., when the next serial bit appears on the MOSI and/or MISO lines), and (c) a sampling of data from the master device on the rising edge of the SCK signal.

At first, the master initiates the SPI communication asserting CS signal low. After that, the master begins the transfer of the first byte by toggling the SCK line low and preparing the MSB (i.e., bit 7) on the MOSI line. Then, it toggles the SCK line high so as to force the slave device in accepting the (stable on the MOSI line) first bit of data. The same process is repeated for

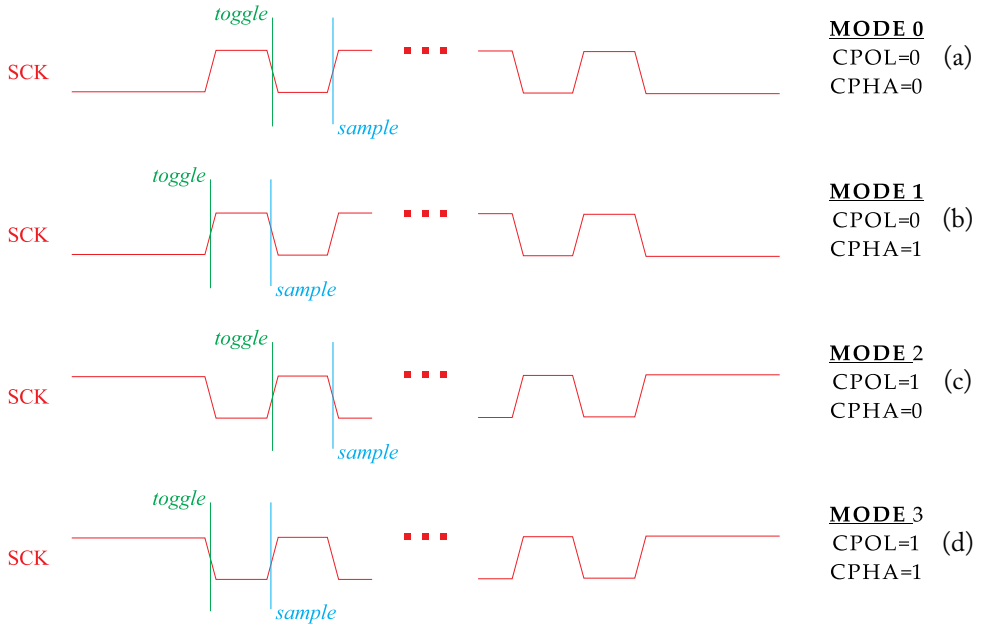


Figure 4.16: SPI modes of operation: clock polarity (CPOL), clock phase (CPHA), and idle level of the clock.

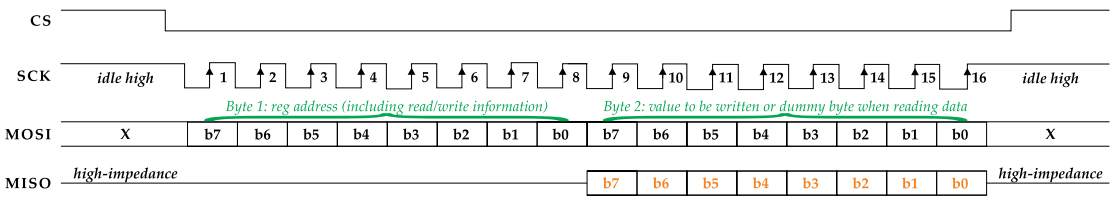


Figure 4.17: SPI communication: write/read byte to/from a single register (mode 3).

the rest 7 bits of data, while this byte incorporates a bit referring to the corresponding process of reading or writing data from/to a register. If the latter bit defines a read process, the slave reveals on the MISO line and on the falling edge of the 8^{th} clock period, the MSB of the register content. At the same time, the master prepares on the MOSI line the MSB of the second byte to be transferred to the slave device. In an SPI write process, the second byte holds the value to be assigned to the register address defined by the first transferred byte, while in an SPI read process this byte holds a "dummy" number just to accomplish the transaction. It should be noted that the slave device regularly provides a high-impedance signal to the MISO line when not transferring data to the master. At the end of the second byte transfer, the master retains the SCK line idle

(i.e., a high-level signal for this example) and concludes the SPI transaction by de-asserting CS line (and, hence, deactivating the slave device).

A more realistic timing diagram, obtaining the content (i.e., `0xBD`) of `WHO_AM_I` register (i.e., `0x0F`) within the LPS25HB MEMS sensor, is presented in Figure 4.18. The latter sensor supports I2C and SPI interfaces. A first glance at the SPI timing diagram reveals the benefits of the SPI communication protocol. Despite the fact that SPI does not define any maximum data rate, the full-duplex type of communication obtains the content of a register in a 2-byte transaction (while the corresponding I2C diagram, presented previously in Figure 4.14, required a 4-byte transaction). It is worth noting that the maximum SPI frequency of the LPS25HB sensor is 10 MHz, which is 25 times greater than the 400 KHz I2C fast mode supported by the sensor [80].

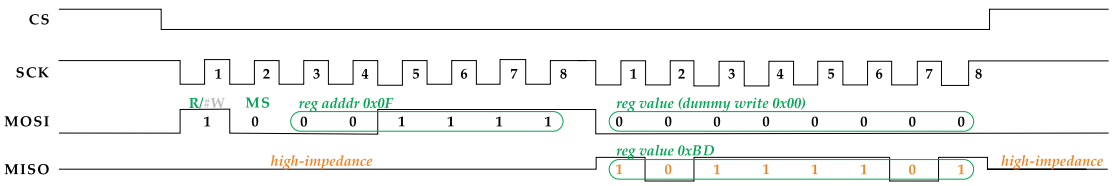
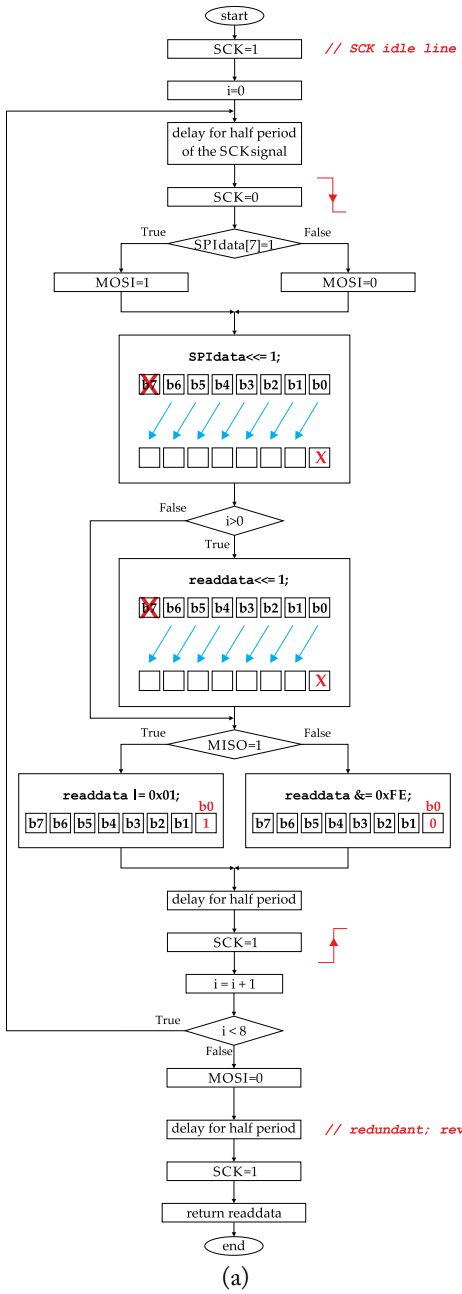


Figure 4.18: SPI read (from slave `0x5D`) the internal register `0x0F` and obtain the value `0xBD`.

The low-level flowchart and function of the SPI function for serial read/write of a single byte is given in Figure 4.19. The function is conformed to the SPI communication mode 3, admits an 8-bit argument (called `SPIdata`), and returns an 8-bit variable (called `readdata`). Within a `for` loop that runs eight successive times, the `SPIdata` variable is shifted left, and the MSB is extracted through the MOSI signal on each particular iteration. Within the same loop, the `readdata` variable is shifted left as well, and the LSB of the variable is repeatedly assigned to value identified by the MISO line. Thereby, the `readdata` variable is finally assigned to the byte transferred by the slave device. Top-level generic SPI functions are not addressed in this example because the top-level procedure may vary among different slave devices.

4.2 DRIVER DEVELOPMENT OF A MEMS BAROMETRIC SENSOR

Having developed the user-defined hardware drivers for the UART, I2C and SPI communication interface among the MCU and peripheral devices, the development of a microcontroller-based system consist of a straightforward procedure. According to the application under development, the designer should select the module(s) needed by the application system and develop the corresponding driver(s) for the control of the hardware device(s). Then, the designer builds the top-level code for the control of the latter devices and the decisions/tasks that need to be made in accordance to the specifications of the custom-designed application system.



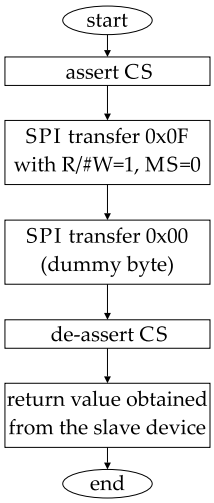

```
bit8 SPIbyte(bit8 SPIdata) // SPI Communication Mode 3
{
 bit8 i=0;
 bit8 readdata = 0x00;
 pinSET(SPIsck_dir,SPIsck_out,SPIsck_pin);

   for (i=0;i<8;i++)
   {
       delay_spi(SPIsck_halfPeriod);
       pinCLEAR(SPIsck_dir,SPIsck_out,SPIsck_pin);
       if (SPIdata & 0x80)
       {
        pinSET(SPImosi_dir,SPImosi_out,SPImosi_pin);
       }
       else
       {
        pinCLEAR(SPImosi_dir,SPImosi_out,SPImosi_pin);
       }
       SPIdata = SPIdata << 1;
       if(i>0)
       {
         readdata = readdata << 1;
       }
       if (pinREAD(SPImiso_dir,SPImiso_in,SPImiso_pin))
       {
       readdata = (readdata|0x01);
       }
       else
       {
        readdata = (readdata&0xFE);
       }
       delay_spi(SPIsck_halfPeriod);
       pinSET(SPIsck_dir,SPIsck_out,SPIsck_pin);
   }

 pinCLEAR(SPImosi_dir,SPImosi_out,SPImosi_pin);
 delay_spi(SPIsck_halfPeriod);
 pinSET(SPIsck_dir,SPIsck_out,SPIsck_pin); // redundant
 return (readdata);
}
```

(b)

Figure 4.19: SPI flowchart and function for low-level serial write/read of a single byte.

Hereafter, we give an example of building a driver for LPS25HB MEMS barometric pressure sensor and thereafter, we will examine the development of the top-level source code for building a real-time monitoring system. MEMS barometers apply to several interesting and contemporary applications, especially when they are configured as barometric altimeters [81]. For instance, they could be utilized for the floor detection inside large shopping centers and provide, possibly through a mobile application, information about the available stores on the identified floor.

The driver applies to I2C communication protocol, yet we have included two additional functions for testing both SPI and I2C protocol by reading the content of `WHO_AM_I` register (as described earlier in the chapter). The flowchart and source code of the SPI example are given in Figures 4.20a,b, respectively, while the minimized I2C code is given in Figure 4.20c.



(a)

```
bit8 LPS25HB_WhoAmI_SPI(void)
{
 bit8 SPI_value;
  pinCLEAR(SPIcs_dir,SPIcs_out,SPIcs_pin);
  SPI_value = SPIbyte ( (0x0F|0x80) ); // R/#W=1, MS=0
  SPI_value = SPIbyte(0x00); // Send Dummy byte
  pinSET(SPIcs_dir,SPIcs_out,SPIcs_pin);
 return SPI_value;
}
```

(b)

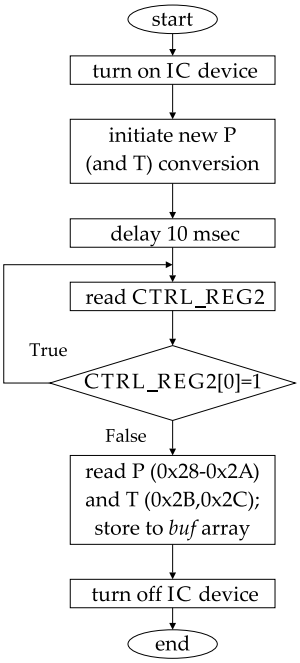
```
bit8 LPS25HB_WhoAmI(void)
{
 bit8 I2C_value;
  I2C_value = I2C_rdByte(0x0F, LPS25HB_ADDR);
 return I2C_value;
}
```

(c)

Figure 4.20: LPS25HB SPI and I2C functions for reading the content of `WHO_AM_I` register.

The main I2C function and flowchart for acquiring measurements by the sensor device are presented in Figure 4.21. When powered-up the sensor device is inserted into power-down mode and, hence, the code initially sets `CTRL_REG1[7]` bit to logical 1 so as to turn the device on. The code afterward writes a logical 0 to `CTRL_REG2[0]` bit to enable `ONE_SHOT` mode, and thereby a new dataset of atmospheric *pressure* (P) and *temperature* (T) is acquired. It is here noted that barometric sensor devices incorporate a temperature sensor used to provide temperature compensation to the sensed pressure (and which is regularly available to the designer). Next, the code delays 10 msec to leave time to the IC device to convert the new sample, and then evaluates bit 0 of `CTRL_REG2` to assure the conversion process has been finished before reading the new dataset. As soon as the later result is available, the code obtains the corresponding information

from the PRESS_OUT (0x28-0x2A) and TEMP_OUT (0x2B, 0x2C) registers. This information is stored to the to buf array incorporated by the function argument and, finally, CTRL_REG1[7] bit is cleared to logical 0 so as to turn the device off.



(a)

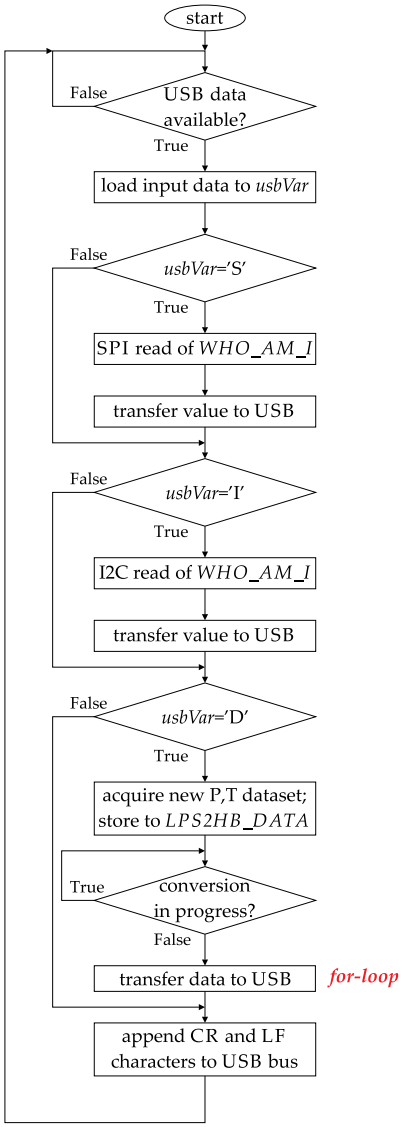
```
void LPS25HB_convert(bit8 *buf)
{
 bit8 i, I2C_value;
  I2C_wrByte(0x20, 0x80, LPS25HB_ADDR);
  I2C_wrByte(0x21, 0x01, LPS25HB_ADDR);
  delay_x100usec(100);
  do
  {
    I2C_value = I2C_rdByte(0x21, LPS25HB_ADDR);
  } while (I2C_value != 0);
  for(i=0;i<5;i++)
  {
    I2C_value = I2C_rdByte(0x28+i, LPS25HB_ADDR);
    buf[i]=I2C_value;
  }
  I2C_wrByte(0x20, 0x00, LPS25HB_ADDR);
}
```

(b)

Figure 4.21: LPS25HB I2C functions for acquiring atmospheric pressure and temperature data.

4.3 SYSTEM-LEVEL DESIGN OF A REAL-TIME MONITORING APPLICATION

The top-level source code and corresponding flowchart for building a real-time monitoring system is presented in Figure 4.22. This code is addressed for Arduino Uno board and AVR MCU, but revising the first two code lines (as explained in the previous chapter) allows us to upload the code in a PIC device, as well. The system obtains ambient air pressure, as well as temperature, samples from the LPS25HB barometric sensor and then transfers measurements to the USB port of a computer, through a USB to UART IC. The USB transaction is initiated by the computer system, which sends to the MCU the 'D' character (from the abbreviation *Data*) every time a new dataset is acquired. Despite the 'D' character, the MCU looks also for the characters I and S (from the abbreviations I2C and SPI), as well, and transfers to the computer the content of WHO_AM_I register when one of them is identified, in terms of an I2C/SPI debug task.



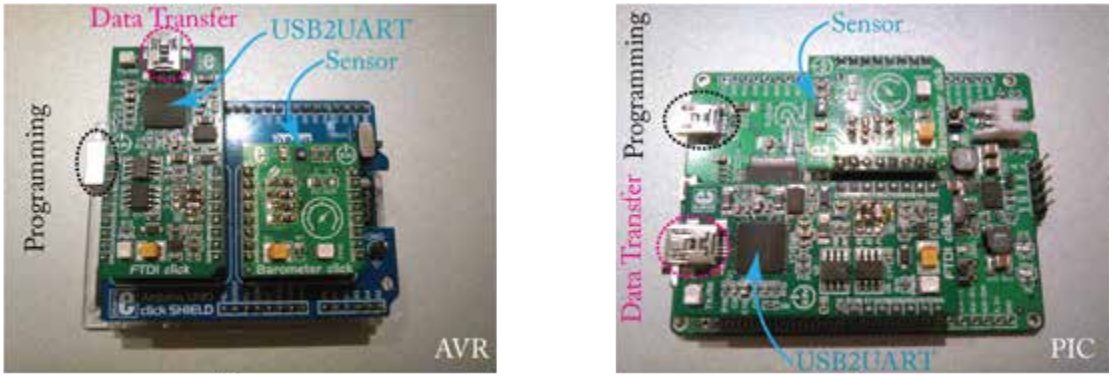
(a)

```
#define AVRGCC
#define AVR

#include "Drive:\..path..\PINOUT.h"
#include "Drive:\..path..\DATATYPES.h"
#include "Drive:\..path..\IO.h"
#include "Drive:\..path..\DELAYS.h"
#include "Drive:\..path..\hwINTERFACE.h"
#include "Drive:\..path..\drvLPS25HB.h"


int main()
{
 bit8 i, usbVar, sensorVar, LPS25HB_DATA[5];

 for(;;)
 {


  usbVar=0;
  usbVar=getcharFTDI();

  if(usbVar=='S') // SPI who_am_I test
  {
    sensorVar = LPS25HB_WhoAmI_SPI();
    putcharFTDI(sensorVar);
  }

  if(usbVar=='I') // I2C who_am_I test
  {
    sensorVar = LPS25HB_WhoAmI();
    putcharFTDI(sensorVar);
  }

  if(usbVar=='D') //I2C obtain dataset
  {
   LPS25HB_convert(LPS25HB_DATA);
   for(i=0;i<5;i++)
   {
    putcharFTDI(LPS25HB_DATA[i]);
   }
   putcharFTDI(0x0D);
   putcharFTDI(0x0A);
  }


 }

 return 0;
}
```

(b)

Figure 4.22: Flowchart and top-level source code of a real-time monitoring system.

The `#include` keywords of the top-level code should be complemented with the location path that holds the header C files. It is here noted that the code of these files is incorporated in the Appendix. Figure 4.23 depicts the setup for implementing the proposed real-time monitoring system in Arduino Uno and PIC18FJ hardware platforms. The peripheral devices employed by the system are the FTDI click and the Barometer click (delivered by MikroElektronika), which, respectively, communicate with the MCU via UART and I2C by default. This configuration may be changed by soldering the corresponding board jumpers in the appropriate position.



(a) (b)

Figure 4.23: (a) Arduino Uno and (b) PIC18FJ setup for implementing a real-time monitoring system.

Building a user-defined communication protocol (such as, the UART) constitutes an example of very good practice in microcontroller education. However, it is an attempt that involves several risks which may result in a non-working code. Hereafter, we present some photos taken by an oscilloscope, which could help designers in arranging a debugging process when working with UART, SPI, and I2C protocols.

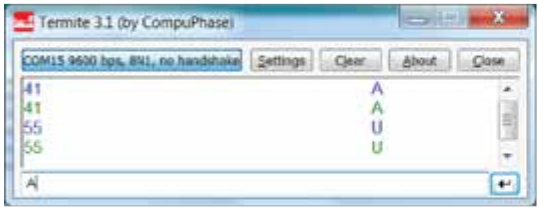
In Figure 4.24a we connected only the FTDI click in the Arduino Uno board and in Figure 4.24b we address a simple code that loops the received—from USB—data back to the computer. To run this example we need a terminal console connected to the appropriate USB Serial COM port using 9600/8N1 configuration. For every key pressed on the computer keyboard, the terminal transfers the corresponding ASCII character to the MCU through the USB2UART module, and then the MCU sends the same character back to the computer through this particular module, as well. In Figures 4.24c,d we see the results of transferring the characters `'A'` and `'U'`, where the first figure addresses also a *hex view* of the transferred data in the Termite console. The characters in blue color are sent from the personal computer, while the characters in green color are transmitted by the MCU.

Figures 4.24e,f illustrates the timing diagrams obtained from this particular UART transaction. Each figure depicts two waveforms where the upper one constitutes the signal captured

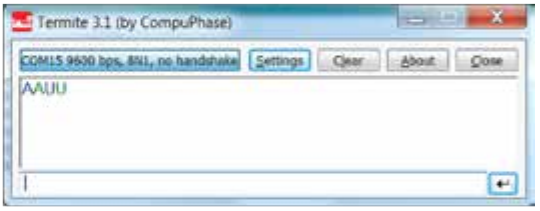
(a)

```
define AVRGCC
#define AVR

#include "Drive:\..path..\PINOUT.h"
#include "Drive:\..path..\DATATYPES.h"
#include "Drive:\..path..\IO.h"
#include "Drive:\..path..\DELAYS.h"
#include "Drive:\..path..\hwINTERFACE.h"
#include "Drive:\..path..\drvLPS25HB.h"

int main()
{
 bit8 usbVar;
 for(;;)
 {
  usbVar=0;
  usbVar=getcharFTDI();
  putcharFTDI(usbVar);
 }
 return 0;
}
```

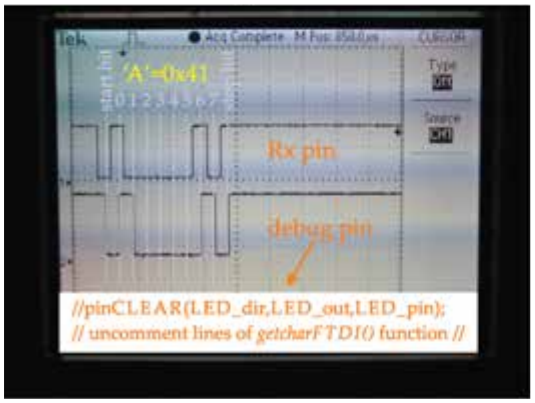
(b)



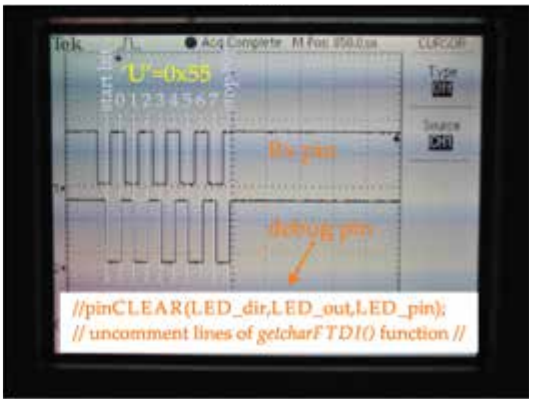
(c)



(d)



(e)



(f)

Figure 4.24: Debug UART communication protocol.

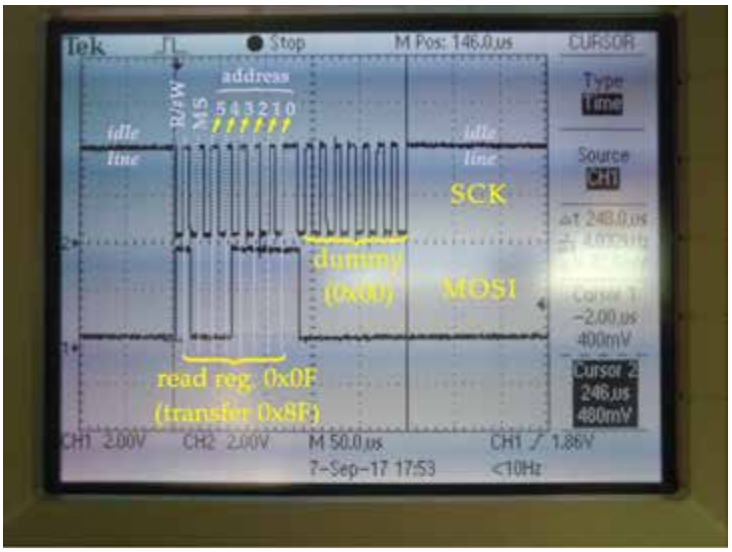
in the MCU Rx line (that is, the serial data transmitted by the computer), and the lower one is generated by the MCU (and it is used for debugging the UART receive process). To be able to capture the second signal, we need to uncomment the code line of the `getcharFTDI()` function (found in the `hwINTERFACE.h`), which is depicted in the bottom area of the figure. This *debug pin* is assigned to the PB5 pin (i.e., D13) which employs the board LED, as well. This line is originally commented because it conflicts with the SPI SCK pin connected to the sensor clock input.

The debug pin is set or cleared as soon as we prepare the variable incorporating the received data which, of course, are in accordance to the serial data identified in the Rx line. What we expect to see in this waveform is that a change to the signal's logical level is in agreement with the previously received serial bit in the Rx line, and that the signal toggling occurs before the appearance of the subsequent serial bit in the Rx line. The signal in the debug pin would preferably toggle in the middle the current received and the upcoming serial bit on the Rx line.

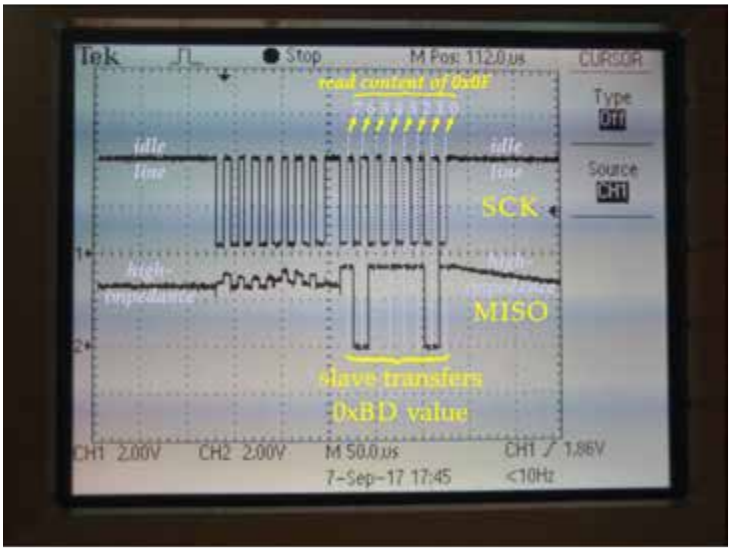
Figure 4.25 presents the timing diagrams of the SPI master reading `0xBD` content from `0x0F` internal register (of the MEMS barometric sensor device). To debug SPI protocol (as well as the I2C that will be presented next) we need to connect the LPS25HB sensor to the system and upload to code that implements the real-time monitoring application. In addition, we need to comment the code line of `getcharFTDI()` function that was activated before, so as to avoid conflict with the SCK signal.

In Figure 4.25a, the MCU transfers serially two bytes (i.e., `0x8F` and `0x00`) from MOSI line. The first byte is in agreement with the register address 0x0F, of which the MSB (i.e., the `R/#W` bit) is set to logical 1 in order to initiate an SPI read process. The second byte is a dummy value sent by MOSI line, while the master obtains the byte sent by the slave through the MISO line. The latter process is depicted by Figure 4.25b, where the MISO line is in high-impedance during the transmission of the first byte (as well as after the transfer of the second byte sent by the slave). The transfer is initiated by the MSB and the data are captured at the rising edge of the SCK (held at logical 1 when the line is idle).

The timing diagrams of the I2C read—from `0x0F` register—transaction are presented in Figures 4.26–4.30. In detail, Figure 4.26 presents the I2C start condition (sent by the master) along with the transfer of the first byte, that is, the slave address appended with `R/#W=0` to initiate a register write process. Figure 4.27 presents the transfer of the second byte, that is, the address (`0x0F`) of the slave's internal register. Figure 4.28 presents the repeat of the start condition along with the transfer of the third byte, that is, the slave address appended with `R/#W=1` to initiate a read process from register (i.e., from register `0x0F`). Figure 4.29 presents the transfer of the fourth byte, which is this time transmitted by the slave device and incorporates the content of register 0x0F, that is the value `0xBD`. Finally, Figure 4.30 presents the I2C stop condition (sent by the master device). At the end of each byte sent by the master device, the slave transmits an acknowledge bit, denoted *Slave ACK (S-ACK)*. On the other hand, as soon as the master receives



(a)



(b)

Figure 4.25: Debug SPI communication protocol.

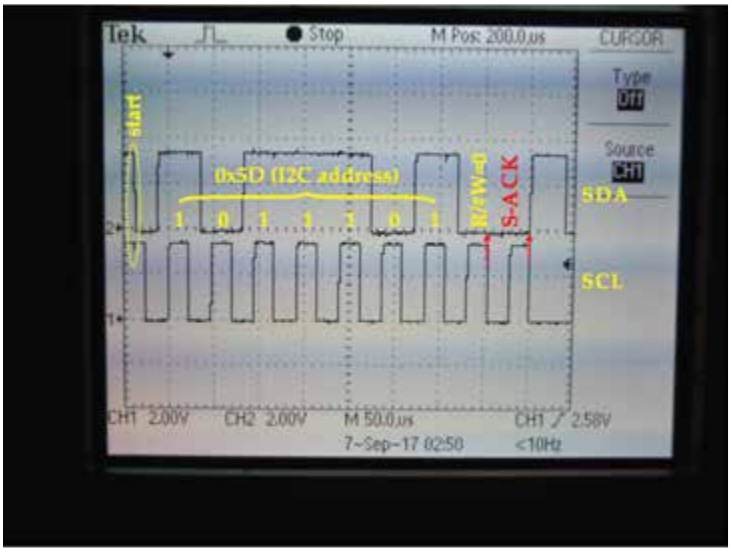


Figure 4.26: Debug I2C communication protocol: start condition and first byte transfer.

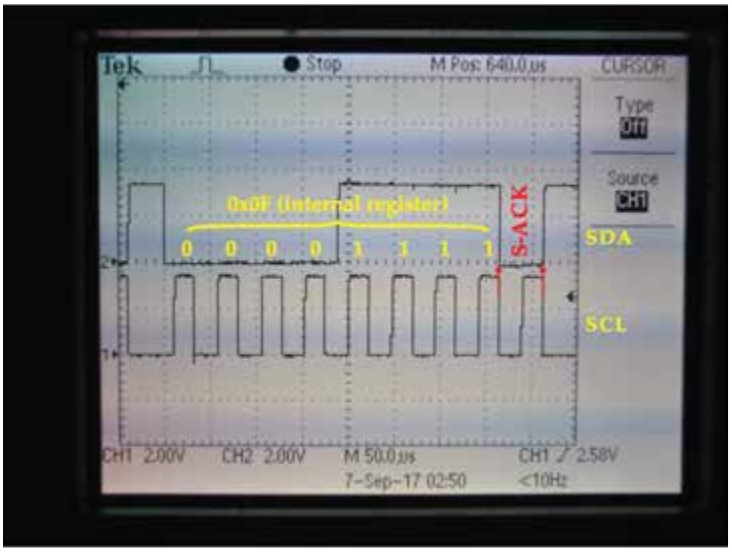


Figure 4.27: Debug I2C communication protocol: second byte transfer.

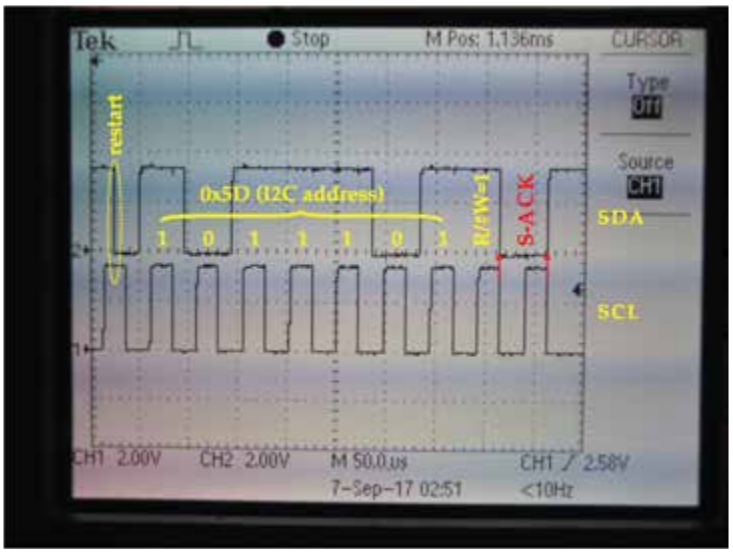


Figure 4.28: Debug I2C communication protocol: third byte transfer.

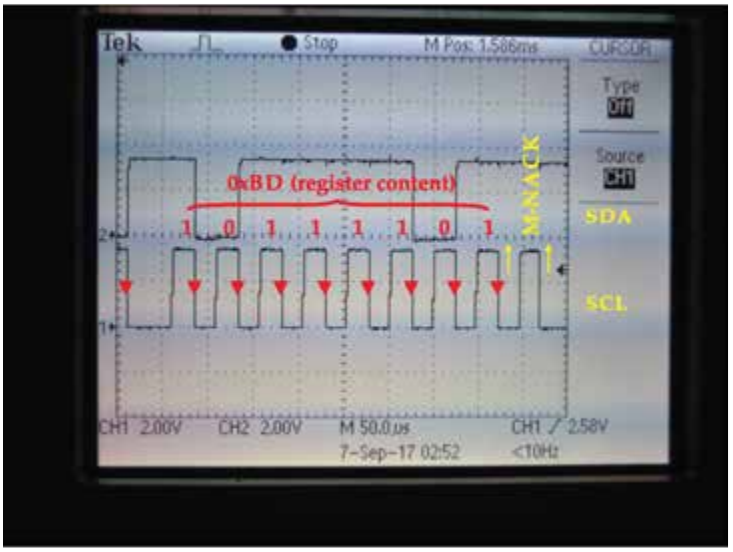


Figure 4.29: Debug I2C communication protocol: fourth byte transfer.

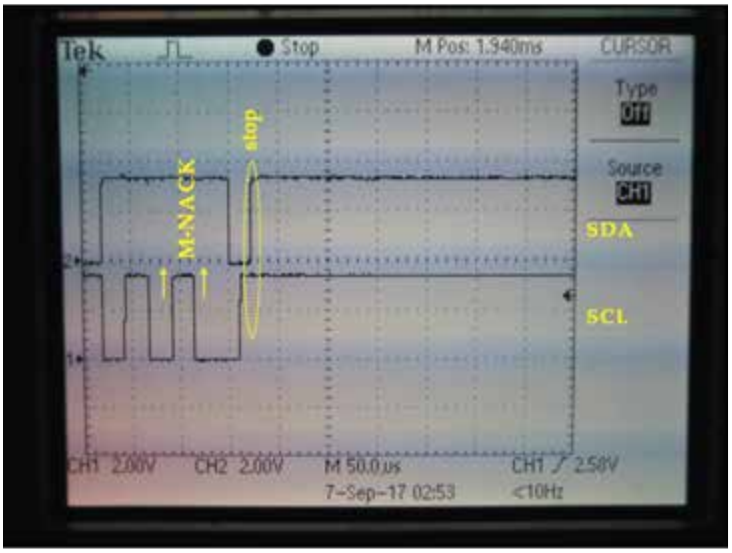


Figure 4.30: Debug I2C communication protocol: stop condition.

the data from the slave device (i.e., the fourth transferred byte) it transmits and no acknowledge bit, denoted *Master NACK (M-NACK)*.

4.4 USER INTERFACE DESIGN IN C PROGRAMMING LANGUAGE

To control the real-time monitoring system implemented before, we need a terminal console which will be used for sending `'D'` character every time a new dataset is acquired. In Figure 4.31a, the user types `'D'` character on the Termite console and presses Enter. The character is sent to the MCU and the latter device transfers back to the computer 5 B. The three former represent the atmospheric pressure, while the latter two represent the temperature (starting from the least significant byte in both cases). The MCU also appends to the dataset the CR (`0x0D`) and LF (`0x0A`) characters so that the next dataset will be printed on the column 1 of the next line of the console, as depicted by Figure 4.31b. In Figures 4.31c,d, the user acquires the content of the `WHO_AM_I` register by transmitting the `'I'` character, as the sensor is currently operating in I2C mode, and receives back the value `0xBD`.

With the pressure and temperature data represented in a raw form of information, the user cannot reach a sense of realization. The raw data should be stored into the computer system and then offline converted in values such as, bar and °C, in order to provide a sense of realization. Thereby, it is often desirable to proceed to the development of a user interface running a personal computer, which provides a real-time conversion of the raw pressure and temperature. A possible

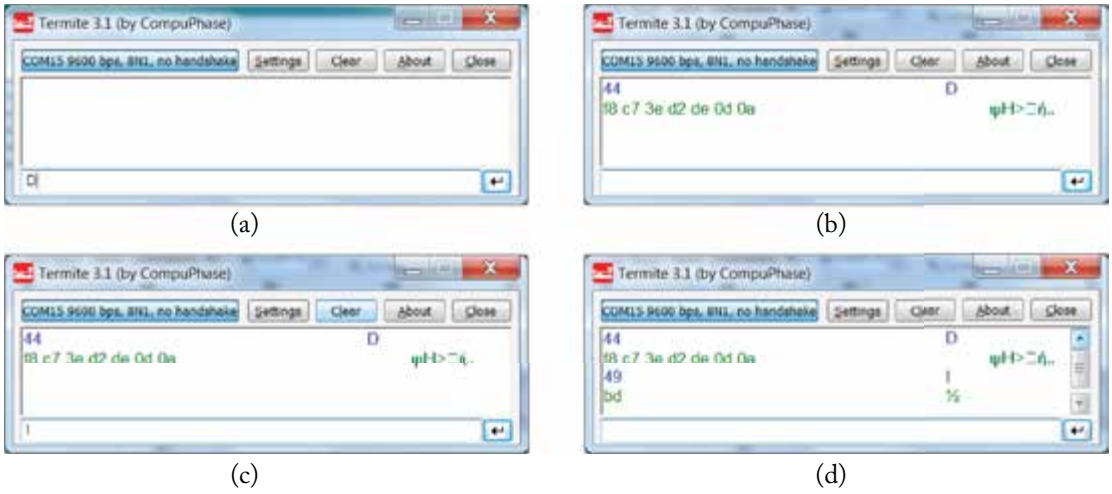


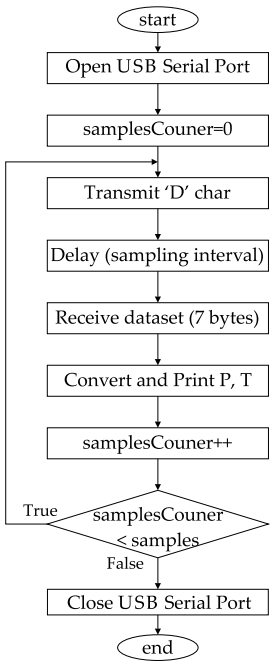
Figure 4.31: Control of the real-time monitoring system through the Termite console.

C language code along with the associated flowchart is given in Figure 4.32. The code acquires a predefined number of samples, defined 100 in this example. The code first opens the USB Serial port and then, within a loop, performs the following: (a) transmits `'D'` character, (b) delays for 500 msec, (c) receives a dataset of 7 bytes from the MCU, (d) converts pressure and temperature by taking into account the guidelines provided by the manufacturer [82], and (e) prints the latter values in bar and °C, respectively.

The above code invokes the functions within the `rs232.h` header file so as to communicate with the selected USB Serial port. The latter header file is given in the book appendix and it is developed under the guidelines for *Serial Communications in Win32* [83], while the main code has been compiled[3] with the *MinGW—Minimalist GNU for Windows* open source programming tool. If we run the program in windows command prompt console, we will obtain measurements similar to those presented in Figure 4.33.

While the converted pressure and temperature samples provide the user with a sense of realization compared to the abstract raw samples, it is sometimes useful to provide a real-time graphical representation of the measurement data. For example, the above measurements where obtained from the experimental procedure depicted in Figure 4.34, which applies to barometric altimetry. Using the wooden construction of Figure 4.34a, the board system was moved from one position (Figure 4.34b) to a higher vertical position (Figure 4.34c). Because the ambient air pressure is inversely proportional to the elevation, we address this setup to observe how the barometric sensor (employed by the system) responds to a change in absolute height of approximately

[3]To compile the code, open a command prompt console in windows and type the path where the source file is found (e.g., `CD <SPACE> C:\WorkingFolder <ENTER>`. Then type `GCC <FILENAME>.C`. This will create the executable `a.exe`. Type `a <ENTER>` to run the program. The MinGW can be downloaded from `http://www.mingw.org/`.



(a)

```
#include <stdio.h>
#include <Windows.h>
#include "rs232.h"

// Change the constants below if needed
#define COM 14          // n-1 (i.e., 14 refers to COM15)
#define BAUD 9600       // 8N1 mode defined within rs232.h
#define SAMPLES 100     // No of acquired samples
#define INTERVAL 500    // Sampling interval in msec
#define expectedData 7 // data obtained from USB port

void Run (int device, int baudRate, int samples, int interval)
{
 int samplesCounter;
 unsigned char Data[]={"D"};
 int Data_length=sizeof(Data)/sizeof(unsigned char);
 unsigned char receive_buf[expectedData];
 DWORD rcvBytes;
 float PRESSURE,TEMPERATURE;

  OpenComport(device, baudRate); // Open USB Serial Port

  for (samplesCounter=0;samplesCounter<samples;samplesCounter++)
  {
   rcvBytes=0;
  // Transmit 'D' Char
   SendBuf(Data, Data_length-1);
   Sleep(interval);
  // Receive 'D' Char
   rcvBytes = PollComport(receive_buf, expectedData);
  // Covert PRESSURE
   PRESSURE = (float) ( (receive_buf[2]<<16) + (receive_buf[1]<<8)
         + receive_buf[0] );
   PRESSURE = PRESSURE / 4096.0 / 1000.0;
   printf("P=%f bar\t", PRESSURE);
  // Covert TEMPERATURE
   if (receive_buf[4] & 0x80){
  receive_buf[4]^=0xFF;
  receive_buf[3]^=0xFF;
  TEMPERATURE = (float) ((receive_buf[4]<<8) + receive_buf[3]+1) ;
   }
   else{
  TEMPERATURE = (float) ( (receive_buf[4]<<8) + receive_buf[3] ) ;
   }
   TEMPERATURE = 42.5+( -TEMPERATURE / 480);
   printf("T=%.1f \xF8\x43\n", TEMPERATURE); //xF8: degrees symbol
  }

 // Close USB Serial Port
  CloseComport();
}

int main()
{
 Run (COM, BAUD, SAMPLES, INTERVAL);
return(0);
}
```

(b)

Figure 4.32: User interface in C language for real-time monitoring of the acquired P and T.

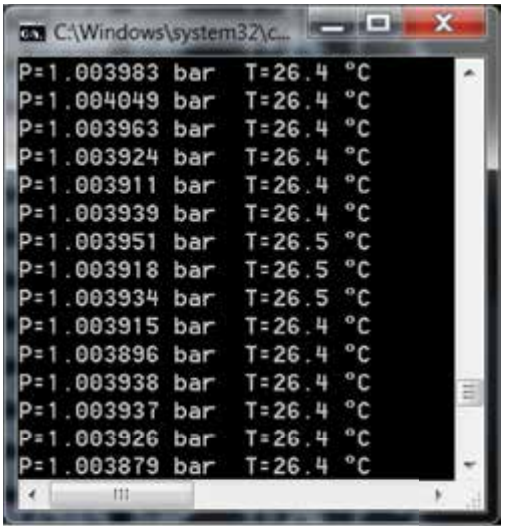


Figure 4.33: Real-time monitoring of the converted P, T samples in windows command prompt.

half a meter. This change is clearer with the graphical representation depicted in Figure 4.34d, which was obtained with *gnuplot*;[4] that is, a freely distributed graphing utility that works on several operating systems.

Figure 4.35 presents the updated code for providing a graphical monitoring (lines in red color). The program has also been updated for saving the converted P, T values within a text file named `data.txt` (code lines in green color), thereby portraying a data acquisition system, as well. The code lines in blue color are used for converting floating point variables into strings, needed for printing values to both gnuplot and text files. It should be noted that the `for` loop is the same as before, but also holds the new code lines. The gnuplot feature invokes a function incorporated within `gnuplot.h` header file which can be found in the Appendix. The latter function implements a LIFO feature, which allows no more than 50 samples to constantly be printed on the window.

Figure 4.36 illustrates the form that is used for saving the pressure and temperature data. The file incorporates two columns where the former holds the pressure measurements, in bar, using six fractional digits. The latter holds the temperature measurements in °C, using one fractional digit. No labels are incorporated in the text file so that it could straightforwardly be uploaded to program (such as, the Matlab) for offline data analysis.

[4]The gnuplot can be downloaded from `http://www.gnuplot.info/`.

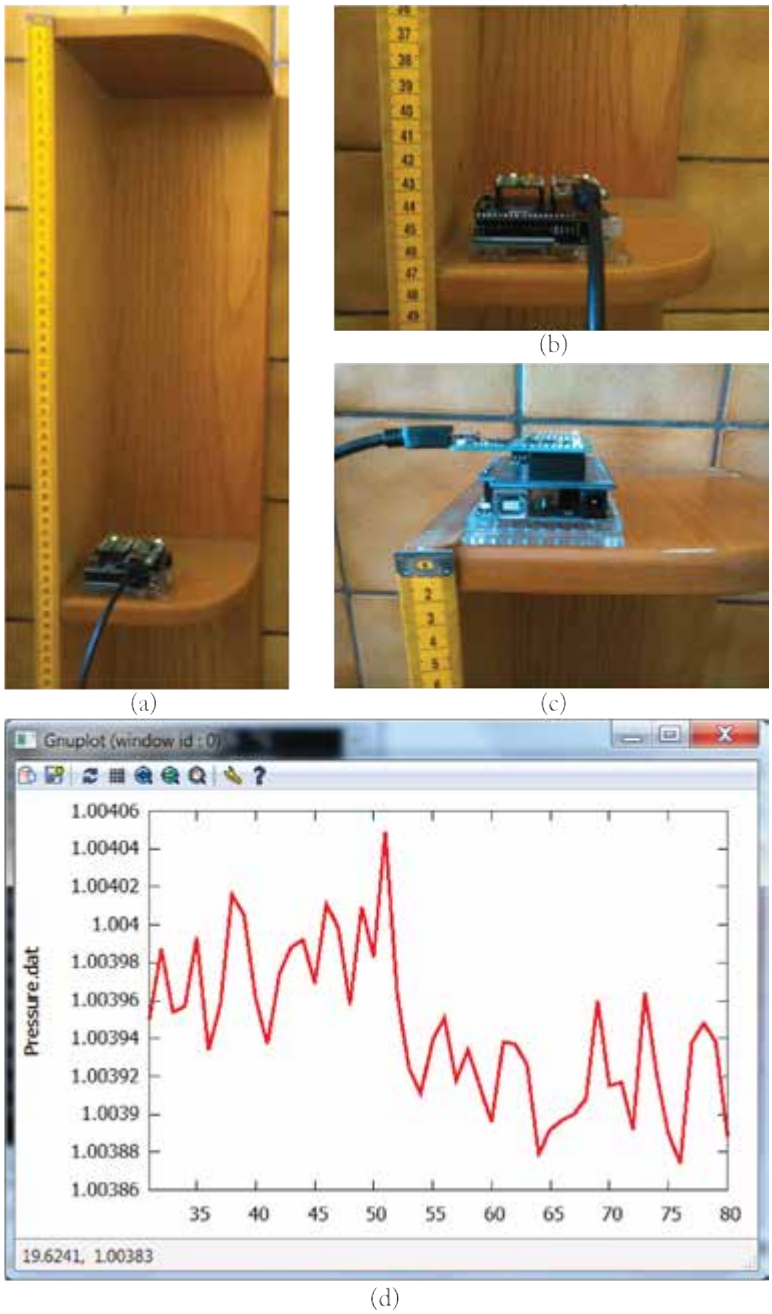


Figure 4.34: Real-time graphical monitoring of the converted P, T samples in gnuplot.

```
#include <stdio.h>
#include <Windows.h>
#include "rs232.h"
#include "gnuplot.h"

// Change the constants below if needed
#define COM 14          // n-1 (i.e., 14 refers to COM15)
#define BAUD 9600       // 8N1 mode defined within rs232.h
#define SAMPLES 100     // No of acquired samples
#define INTERVAL 500    // Sampling interval in msec
#define expectedData 7 // data obtained from USB port

void Run (int device, int baudRate, int samples, int interval)
{
 //int samplesCounter; // this line is moved to "gnuplot.h"
 unsigned char Data[]={"D"};
 int Data_length=sizeof(Data)/sizeof(unsigned char);
 unsigned char receive_buf[expectedData];
 DWORD rcvBytes;
 float PRESSURE,TEMPERATURE;
 unsigned char Pressure[20], Temperature[20]; // for GNUPLOT and .txt files
 // PRINT TO FILE
 FILE *fp;
 fp = fopen("data.txt","a"); //append data to "Pressure.txt" (create file the first time)
 fclose(fp);
 fp = fopen("data.txt","r+");
 fseek (fp, 0, SEEK_END);
 // PRINTO TO GNUPLOT
 FILE *gp1, *gp1_pressure; // For each gnuplop window create two FILEs:
 FILE *gp2, *gp2_temperature;   // 1) gnuplot window, 2) latest measurements to be plotted
 unsigned char fileA[] = "Pressure.dat"; // FILE for the latest measurements to be plotted
 unsigned char fileB[] = "Temperature.dat";
 fclose(stderr); // suppress  warning messages from gnuplot
 // pressure plot window
 gp1 = popen("gnuplot -persist","w"); // the pipe descriptor
 gp1_pressure = fopen(fileA,"w");
 fclose(gp1_pressure);
 // temperature plot window
 gp2 = popen("gnuplot -persist","w");
 gp2_temperature = fopen(fileB,"w");
 fclose(gp2_temperature);

  OpenComport(device, baudRate); // Open USB Serial Port

  for (samplesCounter=0;samplesCounter<samples;samplesCounter++)
  {

    ...same for loop...

   // Float to String conversion
   sprintf(Pressure, "%1.6f", PRESSURE);
   sprintf(Temperature, "%2.1f", TEMPERATURE);
   // PRINT TO FILE
   fprintf(fp,"%s\t%s\n",Pressure,Temperature);
   // PRINT TO GNUPLOT
   sprintf(Sample, "%d", samplesCounter); // create x axes
   gnuplot(gp2, gp2_temperature, fileB, Sample, Temperature);
   gnuplot(gp1, gp1_pressure, fileA, Sample, Pressure);
  }

 // Close USB Serial Port
  CloseComport();
 // .txt FILE
  fclose(fp);
 // GNUPLOT
  fclose(gp1);
  fclose(gp2);
}

int main()
{
 Run (COM, BAUD, SAMPLES, INTERVAL);
return(0);
}
```

Figure 4.35: Code for data acquisition and real-time graphical monitoring.

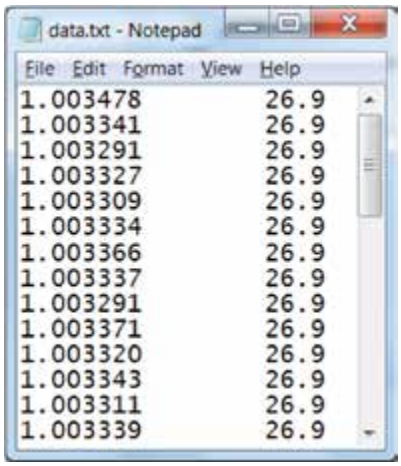


Figure 4.36: The text file incorporating the measurement data (`data.txt`).

APPENDIX A

Firmware and Software

This appendix incorporates the overall firmware and software code of the data acquisition and real-time monitoring system presented in the book.

A.1 FIRMWARE CODE

Hereafter is the adaptable firmware code of the data acquisition and real-time monitoring system presented in the book (top-level code and employed header files).

A.1.1 RTMS.c

```
#define AVRGCC
#define AVR

#include "C:\<replace-with-path>\PINOUT.h"
#include "C:\<replace-with-path>\DATATYPES.h"
#include "C:\<replace-with-path>\IO.h"
#include "C:\<replace-with-path>\DELAYS.h"
#include "C:\<replace-with-path>\hwINTERFACE.h"
#include "C:\<replace-with-path>\drvLPS25HB.h"


int main()
{
 bit8 i, usbVar, sensorVar, LPS25HB_DATA[5];

 for(;;)
 {
  usbVar=0;
  usbVar=getcharFTDI();

  if(usbVar=='S') // SPI who_am_I test
  {
    sensorVar = LPS25HB_WhoAmI_SPI();
    putcharFTDI(sensorVar);
```

```
  }
  if(usbVar=='I')
  {
    sensorVar = LPS25HB_WhoAmI();
    putcharFTDI(sensorVar);
  }
  if(usbVar=='D')
  {
   LPS25HB_convert(LPS25HB_DATA);
   for(i=0;i<5;i++)
   {
    putcharFTDI(LPS25HB_DATA[i]);
   }
   putcharFTDI(0x0D);
   putcharFTDI(0x0A);
  }
 }

 return 0;
}
```

A.1.2 PINOUT.h

```
#if defined AVR
#define LED_out outPORTB
#define LED_dir dirPORTB
#define LED_pin 5
//----//
#define outUART outPORTD
#define inUART inPORTD
#define dirUART dirPORTD
#define txUART 1
#define rxUART 0
//----//
#define I2Csda_in inPORTC
#define I2Csda_out outPORTC
#define I2Csda_dir dirPORTC
#define I2Csda_pin 4
#define I2Cscl_in inPORTC
#define I2Cscl_out outPORTC
```

```
#define I2Cscl_dir dirPORTC
#define I2Cscl_pin 5
//----//
#define SPImosi_in inPORTB
#define SPImosi_out outPORTB
#define SPImosi_dir dirPORTB
#define SPImosi_pin 3
#define SPImiso_in inPORTB
#define SPImiso_out outPORTB
#define SPImiso_dir dirPORTB
#define SPImiso_pin 4
#define SPIsck_in inPORTB
#define SPIsck_out outPORTB
#define SPIsck_dir dirPORTB
#define SPIsck_pin 5
#define SPIcs_in inPORTB
#define SPIcs_out outPORTB
#define SPIcs_dir dirPORTB
#define SPIcs_pin 1
#endif

#if defined PIC
#define LED_out outPORTE
#define LED_dir dirPORTE
#define LED_pin 4
//----//
#define outUART outPORTG
#define inUART inPORTG
#define dirUART dirPORTG
#define txUART 1
#define rxUART 2
//----//
#define I2Csda_in inPORTD
#define I2Csda_out outPORTD
#define I2Csda_dir dirPORTD
#define I2Csda_pin 5
#define I2Cscl_in inPORTD
#define I2Cscl_out outPORTD
#define I2Cscl_dir dirPORTD
```

```
#define I2Cscl_pin 6
//----//
#define SPImosi_in inPORTC
#define SPImosi_out outPORTC
#define SPImosi_dir dirPORTC
#define SPImosi_pin 5
#define SPImiso_in inPORTC
#define SPImiso_out outPORTC
#define SPImiso_dir dirPORTC
#define SPImiso_pin 4
#define SPIsck_in inPORTC
#define SPIsck_out outPORTC
#define SPIsck_dir dirPORTC
#define SPIsck_pin 3
#define SPIcs_in inPORTD
#define SPIcs_out outPORTD
#define SPIcs_dir dirPORTD
#define SPIcs_pin 1
#endif

// Constants
#define delay_baud delay_x100usec
//----//
#define LPS25HB_ADDR 0x5D
#define I2Cscl_oneQuarter 3
#define delay_i2C delay_x1usec
//----//
#define delay_spi delay_x1usec
#define SPIsck_halfPeriod 1
```

A.1.3 DATATYPES.h

```
#if defined (MICROC) || defined (AVRGCC)
typedef volatile unsigned char bit8;
typedef volatile unsigned int bit16;
typedef volatile unsigned long bit32;
typedef volatile signed char sbit8;
typedef volatile signed int sbit16;
typedef volatile signed long sbit32;
#endif
```

```
#if defined CCS
typedef volatile unsigned int8 bit8;
typedef volatile unsigned int16 bit16;
typedef volatile unsigned int32 bit32;
typedef volatile signed int8 sbit8;
typedef volatile signed int16 sbit16;
typedef volatile signed int32 sbit32;
#endif
```

A.1.4 IO.h

```
#if defined AVR //ATMEGA328P
#define inPORTB  (*(volatile unsigned char *) (0x23) )   //PINB
#define dirPORTB (*(volatile unsigned char *) (0x24) )   //DDRB
#define outPORTB (*(volatile unsigned char *) (0x25) )   //PORTB
#define inPORTC  (*(volatile unsigned char *) (0x26) )   //PINC
#define dirPORTC (*(volatile unsigned char *) (0x27) )   //DDRC
#define outPORTC (*(volatile unsigned char *) (0x28) )   //PORTC
#define inPORTD  (*(volatile unsigned char *) (0x29) )   //PIND
#define dirPORTD (*(volatile unsigned char *) (0x2A) )   //DDRD
#define outPORTD (*(volatile unsigned char *) (0x2B) )   //PORTD
#endif

#if defined PIC //PIC18F87J50
#define inPORTA   (*(volatile unsigned char *) (0xF80) ) //PORTA
#define inPORTB   (*(volatile unsigned char *) (0xF81) ) //PORTB
#define inPORTC   (*(volatile unsigned char *) (0xF82) ) //PORTC
#define inPORTD   (*(volatile unsigned char *) (0xF83) ) //PORTD
#define inPORTE   (*(volatile unsigned char *) (0xF84) ) //PORTE
#define inPORTF   (*(volatile unsigned char *) (0xF85) ) //PORTF
#define inPORTG   (*(volatile unsigned char *) (0xF86) ) //PORTG
#define inPORTH   (*(volatile unsigned char *) (0xF87) ) //PORTH
#define inPORTJ   (*(volatile unsigned char *) (0xF88) ) //PORTJ
#define outPORTA  (*(volatile unsigned char *) (0xF89) ) //LATA
#define outPORTB  (*(volatile unsigned char *) (0xF8A) ) //LATB
#define outPORTC  (*(volatile unsigned char *) (0xF8B) ) //LATC
#define outPORTD  (*(volatile unsigned char *) (0xF8C) ) //LATD
#define outPORTE  (*(volatile unsigned char *) (0xF8D) ) //LATE
#define outPORTF  (*(volatile unsigned char *) (0xF8E) ) //LATF
```

```
#define outPORTG  (*(volatile unsigned char *) (0xF8F) ) //LATG
#define outPORTH  (*(volatile unsigned char *) (0xF90) ) //LATH
#define outPORTJ  (*(volatile unsigned char *) (0xF91) ) //LATJ
#define dirPORTA  (*(volatile unsigned char *) (0xF92) ) //TRISA
#define dirPORTB  (*(volatile unsigned char *) (0xF93) ) //TRISB
#define dirPORTC  (*(volatile unsigned char *) (0xF94) ) //TRISC
#define dirPORTD  (*(volatile unsigned char *) (0xF95) ) //TRISD
#define dirPORTE  (*(volatile unsigned char *) (0xF96) ) //TRISE
#define dirPORTF  (*(volatile unsigned char *) (0xF97) ) //TRISF
#define dirPORTG  (*(volatile unsigned char *) (0xF98) ) //TRISG
#define dirPORTH  (*(volatile unsigned char *) (0xF99) ) //TRISH
#define dirPORTJ  (*(volatile unsigned char *) (0xF9A) ) //TRISJ
#endif

#if defined AVR
/* Output Macros */
#define pinSET(dir,data,pin)      ( (data) |= (1<<pin) , (dir) |= (1<<pin) )
#define pinCLEAR(dir,data,pin)    ( (data) &= ~(1<<pin) , (dir) |= (1<<pin) )
#define pinTOGGLE(dir,data,pin)   ( (data) ^= (1<<pin) , (dir) |= (1<<pin) )
#define portWRITE(dir,data,value) ( (data) = (value), (dir) = 0xFF )
/* Input Macros */
#define pinREAD(dir,data,pin)     ( (dir) &= ~(1<<pin), ( (data)
                                  & (1 << pin) ) )
#define portREAD(dir,data,value)  ( (dir) = 0x00, (data) = (value) )
#define inputPIN(dir,pin)         ( (dir) &= ~(1<<pin) )
#endif

#if defined PIC
/* Output Macros */
#define pinSET(dir,data,pin)      ( (data) |= (1<<pin) , (dir)
                                  &= ~(1<<pin) )
#define pinCLEAR(dir,data,pin)    ( (data) &= ~(1<<pin) , (dir) &=
                                  &= ~(1<<pin) )
#define pinTOGGLE(dir,data,pin)   ( (data) ^= (1<<pin) , (dir) &= ~(1<<pin) )
                                  &= ~(1<<pin) )
#define portWRITE(dir,data,value) ( (data) = (value), (dir) = 0x00 )
/* Output Macros */
#define pinREAD(dir,data,pin)     ( (dir) |= (1<<pin), ( (data) &
                                  (1 << pin) )
```

```
#define portREAD(dir,data,value)  ( (dir) = 0xFF, (data) = (value) )
#define inputPIN(dir,pin)         ( (dir) |= (1<<pin) )
#endif
```

A.1.5 DELAYs.h

```
//### delay_x100usec ###//

#if defined PIC
void delay_100usec(void)
{
  #if defined MICROC
  asm{
  #endif
  #if defined CCS
  #asm
  #endif
            movlw  50
            movwf  0x00
   loop2a:  movlw  4
            movwf  0x01
   loop1a:  nop
            nop
            decf   0x01,1
            bnz    loop1a
            decf   0x00,1
            bnz    loop2a
  #if defined MICROC
  }
  #endif
  #if defined CCS
  #endasm
  #endif
}
#endif

#if defined AVR
void delay_100usec(void)
{
 asm ("        push r16      ");
```

```
 asm ("        push r17      ");
 asm ("        ldi r17, 80   ");
 asm ("loop2a: ldi r16, 4    ");
 asm ("loop1a: nop           ");
 asm ("        dec r16       ");
 asm ("        brne loop1a   ");
 asm ("        nop           ");
 asm ("        dec r17       ");
 asm ("        brne loop2a   ");
 asm ("        pop  r17      ");
 asm ("        pop  r16      ");
}
#endif

// function to invoke the assembly code
void delay_x100usec(bit16 dly)
{
  bit16 i;
  for (i=dly; i!=0; --i)
  {
    delay_100usec();
  }
}

//### delay_x10usec ###//

#if defined PIC
void delay_10usec(void)
{
  #if defined MICROC
  asm{
  #endif
  #if defined CCS
  #asm
  #endif
            movlw  5
            movwf  0x00

   loop2b:  movlw  4
```

```
            movwf   0x01
   loop1b:  nop
            nop
            decf    0x01,1
            bnz     loop1b

            decf    0x00,1
            bnz     loop2b
  #if defined MICROC
  }
  #endif
  #if defined CCS
  #endasm
  #endif
}
#endif

#if defined AVR
void delay_10usec(void)
{
 asm ("        push r16      ");
 asm ("        push r17      ");
 asm ("        ldi r17, 8    ");
 asm ("loop2b: ldi r16, 4    ");
 asm ("loop1b: nop           ");
 asm ("        dec r16       ");
 asm ("        brne loop1b   ");
 asm ("        nop           ");
 asm ("        dec r17       ");
 asm ("        brne loop2b   ");
 asm ("        pop  r17      ");
 asm ("        pop  r16      ");
}
#endif
// function to invoke the assembly code
void delay_x10usec(bit16 dly)
{
  bit16 i;
  for (i=dly; i!=0; --i)
```

```
  {
    delay_10usec();
  }
}

//### delay_x1usec ###//

#if defined PIC
void delay_1usec(void) //12 cycles
{
  #if defined MICROC
  asm{
  #endif
  #if defined CCS
  #asm
  #endif
         movlw  2
         movwf  0x01
   loop: nop
         nop
         decf   0x01,1
         bnz    loop
         nop
  #if defined MICROC
  }
  #endif
  #if defined CCS
  #endasm
  #endif
}
#endif

#if defined AVR
void delay_1usec(void) //12 cycles
{
 asm ("        push r16      ");
 asm ("        ldi r16, 3    ");
 asm ("loop1c: nop           ");
 asm ("        dec r16       ");
```

```
 asm ("        brne loop1c    ");
 asm ("        pop  r16       ");
}
#endif
// function to invoke the assembly code
void delay_x1usec(bit16 dly)
{
  bit16 i;
  for (i=dly; i!=0; --i)
  {
    delay_1usec();
  }
}
```

A.1.6 hwINTERFACE.h

```
//# UART Functions #//

void putcharFTDI(bit8 value)
{
 bit8 i;
  pinCLEAR(dirUART,outUART,txUART);
  delay_baud(1);
  for (i=0;i<8;i++)
  {
    if( (value) & 0x01)
    {
     pinSET(dirUART,outUART,txUART);
    }
    else
    {
     pinCLEAR(dirUART,outUART,txUART);
    }
    delay_baud(1);
    value >>= 1;
  }
  pinSET(dirUART,outUART,txUART);
  delay_baud(1);
}
```

```
bit8 getcharFTDI()
{
 bit8 i, value=0;
  while (pinREAD(dirUART,inUART,rxUART));
  delay_x10usec(1);
  //pinCLEAR(LED_dir,LED_out,LED_pin); // debug pin; conflict with SCK
  for (i=0;i<8;i++)
  {
   delay_baud(1);
   if (pinREAD(dirUART,inUART,rxUART))
   {
    value>>=1;
    value |= 0x80;
    //pinSET(LED_dir,LED_out,LED_pin);
   }
   else
   {
    value>>=1;
    value &= 0x7F;
    //pinCLEAR(LED_dir,LED_out,LED_pin);
   }
  }
  delay_baud(1);// skip stop bit
  //pinSET(LED_dir,LED_out,LED_pin);
 return value;
}


//# I2C Functions #//
void I2Cstart(void)
{
   delay_i2C(I2Cscl_oneQuarter);
   inputPIN(I2Csda_dir,I2Csda_pin);
   inputPIN(I2Cscl_dir,I2Cscl_pin);
   delay_i2C(I2Cscl_oneQuarter);
   pinCLEAR(I2Csda_dir,I2Csda_out,I2Csda_pin);
   delay_i2C(I2Cscl_oneQuarter);
   pinCLEAR(I2Cscl_dir,I2Cscl_out,I2Cscl_pin);
   delay_i2C(I2Cscl_oneQuarter);
```

```
}

void I2Cstop(void)
{
  pinCLEAR(I2Csda_dir,I2Csda_out,I2Csda_pin);
  delay_i2C(I2Cscl_oneQuarter);
  inputPIN(I2Cscl_dir,I2Cscl_pin);
  delay_i2C(I2Cscl_oneQuarter);
  inputPIN(I2Csda_dir,I2Csda_pin);
  delay_i2C(I2Cscl_oneQuarter);
}

bit8 I2C_Tx(bit8 value)
{
 bit8 ack=0, i=0;
   for (i=0;i<9;i++)
   {
      if (i<8)
      {
         if(value & 0x80)
         {
          inputPIN(I2Csda_dir,I2Csda_pin);
         }
         else
         {
          pinCLEAR(I2Csda_dir,I2Csda_out,I2Csda_pin);
         }
         delay_i2C(I2Cscl_oneQuarter);
         inputPIN(I2Cscl_dir,I2Cscl_pin);
         delay_i2C(I2Cscl_oneQuarter);
         delay_i2C(I2Cscl_oneQuarter);
 pinCLEAR(I2Cscl_dir,I2Cscl_out,I2Cscl_pin);
         delay_i2C(I2Cscl_oneQuarter);
         value = value<<1;
         if (i==7)
         {
          inputPIN(I2Csda_dir,I2Csda_pin);
          delay_i2C(I2Cscl_oneQuarter);
         }
```

```
      }
      else
      {
       ack = pinREAD(I2Csda_dir,I2Csda_in,I2Csda_pin);
       inputPIN(I2Cscl_dir,I2Cscl_pin);
       delay_i2C(I2Cscl_oneQuarter);
   delay_i2C(I2Cscl_oneQuarter);
       pinCLEAR(I2Cscl_dir,I2Cscl_out,I2Cscl_pin);
       delay_i2C(I2Cscl_oneQuarter);
   delay_i2C(I2Cscl_oneQuarter);
   delay_i2C(I2Cscl_oneQuarter);  // distinguish byte transfer
      }
    }
 return ack;
}

bit8 I2C_Rx(bit8 value)
{
 bit8 I2C_readData=0, i=0;
   for (i=0;i<8;i++)
   {
  delay_i2C(I2Cscl_oneQuarter);
      inputPIN(I2Cscl_dir,I2Cscl_pin);
  while(!pinREAD(I2Cscl_dir,I2Cscl_in,I2Cscl_pin)); // clock streching
  delay_i2C(I2Cscl_oneQuarter);
      if (pinREAD(I2Csda_dir,I2Csda_in,I2Csda_pin))
      {
       I2C_readData = (I2C_readData|0x01);
      }
      else
      {
       I2C_readData = (I2C_readData&0xFE);
      }
      if(i==7)
      {
       I2C_readData = I2C_readData;
      }
      else
      {
```

```
        I2C_readData = I2C_readData << 1;
       }
       delay_i2C(I2Cscl_oneQuarter);
       pinCLEAR(I2Cscl_dir,I2Cscl_out,I2Cscl_pin);
       delay_i2C(I2Cscl_oneQuarter);
    }
    if(value == 0)
    {
     pinCLEAR(I2Csda_dir,I2Csda_out,I2Csda_pin);
    }
    else
    {
     inputPIN(I2Csda_dir,I2Csda_pin);
    }
    delay_i2C(I2Cscl_oneQuarter);
    inputPIN(I2Cscl_dir,I2Cscl_pin);
    delay_i2C(I2Cscl_oneQuarter);
    delay_i2C(I2Cscl_oneQuarter);
    pinCLEAR(I2Cscl_dir,I2Cscl_out,I2Cscl_pin);
    delay_i2C(I2Cscl_oneQuarter);
    delay_i2C(I2Cscl_oneQuarter);
    delay_i2C(I2Cscl_oneQuarter);
 return (I2C_readData);
}

bit8 I2C_wrByte(bit8 reg, bit8 value, bit8 I2Caddr)
{
 bit8 ack=1;
    I2Cstart();
    ack=I2C_Tx((I2Caddr<<1) & 0xFE);
    if(ack==0)
    {
     ack = I2C_Tx(reg);
     if(ack==0)
     {
      ack = I2C_Tx(value);
     }
    }
    I2Cstop();
```

```
 return ack;
}

bit8 I2C_rdByte(bit8 reg, bit8 I2Caddr)
{
 bit8 ack=1;
 bit8 regValue=1; // S-NACK if not responding
   I2Cstart();
   ack = I2C_Tx((I2Caddr<<1) & 0xFE);
   if(ack==0)
   {
    ack = I2C_Tx(reg);
    if(ack==0)
    {
     I2Cstart();
     ack = I2C_Tx((I2Caddr<<1) | 0x01);
     if(ack==0)
     {
      regValue = I2C_Rx(1); //read data & send M-NACK
     }
    }
   }
   I2Cstop();
 return regValue;
}

//# SPI Functions #//
bit8 SPIbyte(bit8 SPIdata) // SPI Mode 3
{
 bit8 i=0;
 bit8 readdata = 0x00;
 pinSET(SPIsck_dir,SPIsck_out,SPIsck_pin);


   for (i=0;i<8;i++)
   {
       delay_spi(SPIsck_halfPeriod);
       pinCLEAR(SPIsck_dir,SPIsck_out,SPIsck_pin);
       if (SPIdata & 0x80)
```

```
        {
         pinSET(SPImosi_dir,SPImosi_out,SPImosi_pin);
        }
        else
        {
         pinCLEAR(SPImosi_dir,SPImosi_out,SPImosi_pin);
        }
        SPIdata = SPIdata << 1;
        if(i>0)
        {
          readdata = readdata << 1;
        }
        if (pinREAD(SPImiso_dir,SPImiso_in,SPImiso_pin))
        {
        readdata = (readdata|0x01);
        }
        else
        {
         readdata = (readdata&0xFE);
        }
        delay_spi(SPIsck_halfPeriod);
        pinSET(SPIsck_dir,SPIsck_out,SPIsck_pin);
   }
   pinCLEAR(SPImosi_dir,SPImosi_out,SPImosi_pin);

 delay_spi(SPIsck_halfPeriod);
 pinSET(SPIsck_dir,SPIsck_out,SPIsck_pin);
 return (readdata);
}
```

A.1.7 drvLPS25HB.h

```
bit8 LPS25HB_WhoAmI(void);
void LPS25HB_convert(bit8 *buf);
bit8 LPS25HB_WhoAmI_SPI(void);

// I2C routines
bit8 LPS25HB_WhoAmI(void)
{
 bit8 I2C_value;
```

```
  I2C_value = I2C_rdByte(0x0F, LPS25HB_ADDR);
 return I2C_value;
}

void LPS25HB_convert(bit8 *buf)
{
 bit8 i, I2C_value;
  I2C_wrByte(0x20, 0x80, LPS25HB_ADDR);
  I2C_wrByte(0x21, 0x01, LPS25HB_ADDR);
  delay_x100usec(100);
  do
  {
    I2C_value = I2C_rdByte(0x21, LPS25HB_ADDR);
  } while (I2C_value != 0);
  for(i=0;i<5;i++)
  {
    I2C_value = I2C_rdByte(0x28+i, LPS25HB_ADDR);
    buf[i]=I2C_value;
  }
  I2C_wrByte(0x20, 0x00, LPS25HB_ADDR);
}

// SPI test
bit8 LPS25HB_WhoAmI_SPI(void)
{
 bit8 SPI_value;
  pinCLEAR(SPIcs_dir,SPIcs_out,SPIcs_pin);
  SPI_value = SPIbyte ( (0x0F|0x80) );
  SPI_value = SPIbyte(0x00); // dummy byte
  pinSET(SPIcs_dir,SPIcs_out,SPIcs_pin);
 return SPI_value;
}
```

A.2 SOFTWARE CODE

Hereafter is the software C code of the data acquisition and real-time monitoring system presented in the book, which provides a graphical monitoring of the acquired pressure as well as temperature. Measurements are saved to the text file data.txt generated in the working folder.

A.2.1 RTMS_DAQ.c

```
#include <stdio.h>
#include <Windows.h>
#include "rs232.h"
#include "gnuplot.h"

// Change the constants below if needed
#define COM 14          // n-1 (i.e., 14 refers to COM15)
#define BAUD 9600       // 8N1 mode defined within rs232.h
#define SAMPLES 100     // No of acquired samples
#define INTERVAL 500    // Sampling interval in msec
#define expectedData 7 // data obtained from USB port

void Run (int device, int baudRate, int samples, int interval)
{

 unsigned char Data[]={"D"};
 int Data_length=sizeof(Data)/sizeof(unsigned char);
 unsigned char receive_buf[expectedData];
 DWORD rcvBytes;
 float PRESSURE,TEMPERATURE;
 unsigned char Pressure[20], Temperature[20];
 // PRINT TO FILE
 FILE *fp;
 fp = fopen("data.txt","a");
 fclose(fp);
 fp = fopen("data.txt","r+");
 fseek (fp, 0, SEEK_END);
 // PRINTO TO GNUPLOT
 FILE *gp1, *gp1_pressure;
 FILE *gp2, *gp2_temperature;
 unsigned char fileA[] = "Pressure.dat";
 unsigned char fileB[] = "Temperature.dat";
 fclose(stderr);
 // pressure plot window
 gp1 = popen("gnuplot -persist","w");
 gp1_pressure = fopen(fileA,"w");
 fclose(gp1_pressure);
 // temperature plot window
```

```
gp2 = popen("gnuplot -persist","w");
gp2_temperature = fopen(fileB,"w");
fclose(gp2_temperature);


 OpenComport(device, baudRate);

 for (samplesCounter=0;samplesCounter<samples;samplesCounter++)
 {
  rcvBytes=0;
  SendBuf(Data, Data_length-1);
  Sleep(interval);
  rcvBytes = PollComport(receive_buf, expectedData);
  // Covert PRESSURE & TEMPERATURE
  PRESSURE = (float) ( (receive_buf[2]<<16) + (receive_buf[1]<<8) +
                       receive_buf[0] );
  PRESSURE = PRESSURE / 4096.0 / 1000.0;
  printf("P=%f bar\t", PRESSURE);
  // Calculate Temperature
  if (receive_buf[4] & 0x80){
     //two's complement below
     receive_buf[4]^=0xFF;
     receive_buf[3]^=0xFF;
     TEMPERATURE = (float) ((receive_buf[4]<<8) + receive_buf[3]+1) ;
  }
  else{
     TEMPERATURE = (float) ( (receive_buf[4]<<8) + receive_buf[3] ) ;
  }
  TEMPERATURE = 42.5+( -TEMPERATURE / 480);
  printf("T=%.1f \xF8\x43\n", TEMPERATURE); //xF8: symbol of degrees
  // Float to String conversion
  sprintf(Pressure, "%1.6f", PRESSURE);
  sprintf(Temperature, "%2.1f", TEMPERATURE);
  // PRINT TO FILE
  fprintf(fp,"%s\t%s\n",Pressure,Temperature);
  // PRINT TO GNUPLOT
  sprintf(Sample, "%d", samplesCounter); // create x axes
  gnuplot(gp2, gp2_temperature, fileB, Sample, Temperature);
  gnuplot(gp1, gp1_pressure, fileA, Sample, Pressure);
```

```
  }

  CloseComport();
  // .txt FILE
  fclose(fp);
  // GNUPLOT
  fclose(gp1);
  fclose(gp2);
}

int main()
{
 Run (COM, BAUD, SAMPLES, INTERVAL);
return(0);
}
```

A.2.2 rs232.h

```
// Define 8N1 settings
#define STOP_BIT ONESTOPBIT
#define PARITY_BIT NOPARITY
#define DATA_BITS 8

char COM_No[32][12] = {"\\\\.\\COM1",  "\\\\.\\COM2",
                       "\\\\.\\COM3",  "\\\\.\\COM4",
                       "\\\\.\\COM5",  "\\\\.\\COM6",
                       "\\\\.\\COM7",  "\\\\.\\COM8",
                       "\\\\.\\COM9",  "\\\\.\\COM10",
                       "\\\\.\\COM11", "\\\\.\\COM12",
                       "\\\\.\\COM13", "\\\\.\\COM14",
                       "\\\\.\\COM15", "\\\\.\\COM16",
                       "\\\\.\\COM17", "\\\\.\\COM18",
                       "\\\\.\\COM19", "\\\\.\\COM20",
                       "\\\\.\\COM21", "\\\\.\\COM22",
                       "\\\\.\\COM23", "\\\\.\\COM24",
                       "\\\\.\\COM25", "\\\\.\\COM26",
                       "\\\\.\\COM27", "\\\\.\\COM28",
                       "\\\\.\\COM29", "\\\\.\\COM30",
                       "\\\\.\\COM31", "\\\\.\\COM32"};
HANDLE COMn;
```

```
int OpenComport(int COM_id, int BAUD_RATE)
{
  COMn = CreateFile ( COM_No[COM_id],
                      GENERIC_READ|GENERIC_WRITE,
                      0,
                      NULL,
                      OPEN_EXISTING,
                      0,
                      NULL);
  if(COMn==INVALID_HANDLE_VALUE) {
    printf("\nUnable to open selected COM port\n");
    return(1);
  }
  // COM settings
  DCB COM_settings = {0};
  COM_settings.DCBlength = sizeof(COM_settings);
  GetCommState(COMn, &COM_settings);
  // Basic settings: BAUD, PARITY, etc.
  COM_settings.BaudRate = BAUD_RATE;
  COM_settings.ByteSize = DATA_BITS;
  COM_settings.StopBits = STOP_BIT;
  COM_settings.Parity   = PARITY_BIT;
  //COM_settings.EvtChar    = RxEvtChar;
  SetCommState(COMn, &COM_settings);
  // Communication settings: TIMEOUTS in msec
  COMMTIMEOUTS COM_timeouts = { 0 };
  COM_timeouts.ReadIntervalTimeout = 50;
  COM_timeouts.ReadTotalTimeoutConstant    = 50;
  COM_timeouts.ReadTotalTimeoutMultiplier  = 10;
  COM_timeouts.WriteTotalTimeoutConstant   = 50;
  COM_timeouts.WriteTotalTimeoutMultiplier = 10;
  SetCommTimeouts(COMn, &COM_timeouts);
 return(0);
}

int SendBuf(unsigned char *buf, int size)
```

```
{
 int n;
  if(WriteFile(COMn, buf, size, (LPDWORD)((void *)&n), NULL))
  {
    return(n);
  }
 return(1);
}

int PollComport(unsigned char *buf, int size)
{
 int n;
 BOOL  Status;
  if(size>4096)  size = 4096;
  ReadFile(COMn, buf, size, (LPDWORD)((void *)&n), NULL);
 return(n);
}

void CloseComport(void)
{
  CloseHandle(COMn);
}
```

A.2.3 gnuplot.h

```
// Max number of samples plotted to gnuplot window
#define gp1BUFLENGTH 50

// buffer used as LIFO within 'gnuplot' function
char gp1BUF[gp1BUFLENGTH*100];

// X-axis holding current sample
char Sample[15];
int samplesCounter;

void gnuplot(FILE *gpFile, FILE *gpPlotFile, unsigned char *gpPlotFileName,
             unsigned char *Xayis, unsigned char *Yaxis);


void gnuplot(FILE *gpFile, FILE *gpPlotFile, unsigned char *gpPlotFileName,
```

```
              unsigned char *Xayis, unsigned char *Yaxis)
{
  int fileLENGTH_A, fileLENGTH_B;
      gpPlotFile = fopen(gpPlotFileName,"a");
      fprintf(gpPlotFile,"%s %s\n",Xayis,Yaxis);
      fclose(gpPlotFile);

     // LIFO for printing gp1BUFLENGTH number of data on GNUPLOT
      if (samplesCounter >= gp1BUFLENGTH)
      {
        gpPlotFile = fopen(gpPlotFileName,"rb");
        fseek(gpPlotFile, 0, SEEK_END);
        fileLENGTH_B = ftell(gpPlotFile);
        fseek(gpPlotFile, 0, SEEK_SET);
        fgets (gp1BUF, 64, gpPlotFile);

        fileLENGTH_A = ftell(gpPlotFile);

        fseek(gpPlotFile, 0, SEEK_CUR);
        fread(gp1BUF, sizeof(char), fileLENGTH_B-fileLENGTH_A, gpPlotFile);
        fclose(gpPlotFile);

        gpPlotFile = fopen(gpPlotFileName,"wb");
        fwrite(gp1BUF, sizeof(char), fileLENGTH_B-fileLENGTH_A, gpPlotFile);
        fclose(gpPlotFile);
      }
      // Print data to GNUPLOT
      if(samplesCounter!=0 && samplesCounter<gp1BUFLENGTH)
      {
          fprintf(gpFile, "set nokey\n"); //set legend at fixed position
          fprintf(gpFile, "set xrange [0:%d]\n",samplesCounter);
          fprintf(gpFile, "replot '%s' using  1:2 with lines linecolor "
                          "'red' lw 2\n",gpPlotFileName);
      }
      else
      {
          fprintf(gpFile, "set nokey\n"); //set legend at fixed position
          fprintf(gpFile, "set ylabel '%s'\n",gpPlotFileName);
  // Insert gpPlotFileName as y-axis label
```

```
            fprintf(gpFile, "set xrange [%d:%d]\n",samplesCounter-
                            (gp1BUFLENGTH-1), samplesCounter);
            if (samplesCounter==0) fprintf(gpFile, "plot '%s' using  1:2 "
                "with lines linecolor 'red' lw 2\n",gpPlotFileName);
            else fprintf(gpFile, "replot '%s' using  1:2 with lines "
                "linecolor 'red' lw 2\n",gpPlotFileName);
        }
        fflush(gpFile);
}
```

Abbreviations

μC	Microcontroller
ACC	Accumulator
ACK	Acknowledge
ADC	Analog-to-Digital Converter
ALU	Arithmetic Logic Unit
ASCII	American Standard Code for Information Interchange
B	Byte
CF	Carry Flag
CLK	Clock
CPHA	Clock Phase
CPOL	Clock Polarity
CPU	Central Processing Unit
CS	Chip Select
DAC	Digital-to-Analog Converter
DAQ	Data Acquition
DIP	Dual In-line Package
EEPROM	Erasable Programmable Read-Only Memory
GPR	General Purpose Register
I2C	Inter-Integrated Circuit
IC	Integrated Circuit
ICD	In-Circuit Debugger
ICSP	In-Circuit Serial Programming
IDE	Integrated Development Environment
IF	Interrupt Flag
I/O	Input/Output
IoT	Internet of Things
ISA	Instruction Set Architecture
ISP	In-System Programming
ISR	Interrupt Sevice Routine
KB	Kilobyte
LED	Light-Emitting Diode
LIFO	Last-In-First-Out
LSB	Least Significant Bit

MB	Megabyte
MCU	Microcontroller Unit
MISO	Master In-Slave Out
M-NACK	Master No Acknowledge
MOSI	Master Out-Slave in
MSB	Most Significant Bit
NC	No Connect
NVM	Non-Volatile Memory
OPCODE	Operation Code
OS	Operating System
OSC	Oscillator
P	Pressure
PC	Program Counter
PCB	Printed-Circuit Board
PBL	Project-Based Learning
PWM	Pulse Width Modulation
RAM	Random-Access Memory
R	Resistor
ROM	Read-Only Memory
RTC	Real-Time Clock
RTOS	Real-Time Operating System
S-ACK	Slave Acknowledge
SCL	Serial Clock
SDA	Serial Data
SP	Stack Pointer
SPI	Serial Peripheral Interface
SR	Stack Register
T	Temperature
UART	Universal Asynchronous Receiver/Transmitter
USB	Universal Serial Bus
UV	Ultraviolet
WREG	Working Register
XTAL	Crystal
ZF	Zero Flag

References

[1] J. O. Hamblen, A. Parker, and G. A. Rohling, An instructional laboratory to support microprogramming, *IEEE Transactions on Education*, Vol. 33, Issue 4, pp. 333–336, 1990. DOI: 10.1109/13.61085. 2, 75

[2] A. del Río, J. J. Rodríguez-Andina, and A. A. Nogueiras-Meléndez, Learning microcontrollers with a CAI-oriented multi-micro simulation environment, *IEEE Transactions on Education*, Vol. 44, Issue 2, pp. 197–211, 2001. DOI: 10.1109/13.925846. 2, 8

[3] D. E. Bolanakis, E. Glavas, and G. A. Evangelakis, A multidisciplinary educational board system for microcontrollers: Considerations in design for technically accurate custom-made platforms, in *Proc. of the 1st International Symposium on Information Technologies and Applications in Education (ISITAE:IEEE Press)*, pp. 391–395, Kunming, PR, China, 2007. DOI: 10.1109/isitae.2007.4409311. 2

[4] H. Jack and N. Barakat, A student owned microcontroller board, in *Proc. of the ASEE Annual Conference and Exposition*, pp. 1–11, Chicago, IL, 2006. 2

[5] J. F. D'Souza, A. D. Reed, and C. K. Adams, Selecting microcontrollers and development tools for undergraduate engineering capstone projects, *Computers in Education Journal*, Vol. 5, Issue 1, pp. 54–66, 2014. 5

[6] J. W. Pritchard and M. Mina, Modern embedded systems as a platform for problem solving in freshman engineering: What is the best option?, in *Proc. of the 120th ASEE Annual Conference and Exposition*, pp. 1–12, Atlanta, GA, 2013. 5

[7] MikroElektronika. `http://www.mikroe.com` 5

[8] Microchip. `http://www.microchip.com` 5

[9] Olimex. `http://www.olimex.com` 5

[10] Arduino. `https://www.arduino.cc` 5

[11] Atmel. `http://www.atmel.com/avr` 5

[12] Adafruit. `https://www.adafruit.com` 6

[13] SparkFun. `https://www.sparkfun.com` 6

[14] D. E. Bolanakis, T. Laopoulos, and K. T. Kotsis, Fixed-point arithmetic for a mcro-computer architecture course, *IEEE Technology and Engineering Education*, Vol. 9, No. 1, pp. 1–7, 2016. 8

[15] J. Duntemann, *Assembly Language: Step by Step*, John Wiley & Sons Inc., New York, 1992. 8

[16] S. Mackenzie, A structured approach to assembly language programming, *IEEE Transactions on Education*, Vol. 31, No. 2, pp. 123–128, 1988. DOI: 10.1109/13.2296. 8

[17] D. E. Bolanakis, G. A. Evangelakis, E. Glavas, and K. T. Kotsis, A teaching approach for bridging the gap between low-level and higher-level programming using assembly language learning for small microcontrollers, *Computer Application in Engineering Education*, Vol. 19, Issue 3, pp. 525–537, 2011. DOI: 10.1002/cae.20333. 8, 41, 64

[18] V. Konstantakos, K. Kosmatopoulos, S. Nikolaidis, and T. Laopoulos, Measurement of power consumption in digital systems, *IEEE Transactions on Instrumentation and Measurement*, Vol. 55, Issue 5, pp. 1662–1670, 2006. DOI: 10.1109/tim.2006.880311. 8

[19] N. Kavvadias, P. Neofotistos, S. Nikolaidis, C. A. Kosmatopoulos, and T. Laopoulos, Measurements analysis of the software-related power consumption in microprocessors, *IEEE Transactions on Instrumentation and Measurement*, Vol. 53, Issue 4, pp. 1106–1112, 2004. DOI: 10.1109/tim.2004.830784. 8

[20] J-H. Park, M. Kim, B-N. Noh, and J. B. D. Joshi, A similarity based technique for detecting malicious executable files for computer forensics, in *Proc. of the IEEE International Conference on Information Reuse and Integration*, pp. 188–193, Waikoloa Village, HI, 2006. DOI: 10.1109/iri.2006.252411. 8

[21] D. E. Bolanakis, K. T. Kotsis, and T. Laopoulos, Arithmetic operations in assembly language: Educators' perspective on endianness learning using 8-bit microcontrollers, in *Proc. of the 5th IEEE International Workshop on Intelligent Data Acquisition and Advanced Computing Systems (IDAACS)*, pp. 600–604, Rende, Italy, 2009. DOI: 10.1109/idaacs.2009.5342909. 8

[22] D. E. Bolanakis, G. A. Evangelakis, E. Glavas, and K. T. Kotsis, Teaching the addressing modes of the M68HC08 CPU by means of a practicable lesson, in *Proc. of the 11th IASTED International Conference on Computers and Advance Technology in Education (CATE)*, pp. 446–450, Crete, Greece, 2008. 8

[23] N. He, H-W. Huang, and Y. Qian, Teaching touch sensing technologies through project-based learning, in *Proc. of the IEEE Frontiers in Education Conference (FIE)*, pp. 1–7, Erie, PA, 2016. DOI: 10.1109/fie.2016.7757625. 8

[24] A. K. R. Segundo, J. A. N. Cocota Junior, R. Q. Hilário, V. de O. Gomide, and D. V. M. Ferreira, Low cost SCADA system for education, in *Proc. of the IEEE Global Engineering Education Conference (EDUCON)*, pp. 536–542, Tallinn, Estonia, 2015. DOI: 10.1109/educon.2015.7096022. 8

[25] H. V. Mekali and P. Patil, Project based learning for a course on advanced microcontroller: Experiment and results, in *Proc. of the IEEE International Conference on MOOC, Innovation and Technology in Education (MITE)*, pp. 128–131, Patiala, India, 2014. DOI: 10.1109/mite.2014.7020255. 8

[26] S. Chandrasekaran, A. Stojcevski, G. Littlefair, and M. Joordens, Project-oriented design-based learning: Aligning students' views with industry needs, *International Journal of Engineering Education*, Vol. 29, Issue 5, pp. 1109–1118, 2013. 8

[27] C. Mercer and D. Leech, Inexpensive miniature programmable magnetic stirrer from reconfigured computer parts, *Journal of Chemical Education*, Vol. 95, Issue 6, pp. 816–818, 2017. DOI: 10.1021/acs.jchemed.7b00184. 8

[28] P. L. Urban, Open-source electronics as a technological aid in chemical education, *Journal of Chemical Education*, Vol. 91, Issue 5, pp. 751–752, 2014. DOI: 10.1021/ed4009073. 8

[29] F. Bouquet, J. Bobroff, M. Fuchs-Gallezot, and L. Maurines, Project-based physics labs using low-cost open-source hardware, *American Journal of Physics*, Vol. 85, Issue 3, pp. 216–222, 2017. DOI: 10.1119/1.4972043. 8

[30] B. Huang, Open-source hardware—microcontrollers and physics education—integrating DIY sensors and data acquisition with Arduino, in *Proc. of the 122nd ASEE Annual Conference*, pp. 1–13, Seattle, WA, 2015. DOI: 10.18260/p.24542. 8

[31] S. Analytis, J. A. Sadler, and M. R. Cutkosky, Creating paper robots increases designers' confidence to prototype with microcontrollers and electronics, *International Journal of Design Creativity and Innovation*, Vol. 5, Issues 1–2, pp. 48–59, 2017. DOI: 10.1080/21650349.2015.1092397. 8

[32] T. Fields, Building a mechatronics design course around quadcopters, in *Proc. of the 55th AIAA Aerospace Sciences Meeting*, pp. 1–13, Grapevine, TX, 2017. DOI: 10.2514/6.2017-0511. 8

[33] W. J. Esposito, F. A. Mujica1, D. G. Garcia, and G. T. A. Kovacs, The lab-in-a-box project: An Arduino compatible signals and electronics teaching system, in *Proc. of the IEEE Signal Processing and Signal Processing Education Workshop (SP/SPE)*, pp. 301–306, Salt Lake City, UT, 2015. DOI: 10.1109/dsp-spe.2015.7369570. 9

[34] D. E. Bolanakis, A. K. Rachioti, and E. Glavas, Nowadays trends in microcontroller education: Do we educate engineers or electronic hobbyists? Recommendation on a multiplatform method and system for lab training activities, in *Proc. of the IEEE Global Engineering Education Conference (EDUCON2017)*, Athens, Greece, pp. 73–77, 2017. DOI: 10.1109/educon.2017.7942826. 9

[35] A. Merkouris, K. Chorianopoulos, and A. Kameas, Teaching programming in secondary education through embodied computing platforms: Robotics and wearables, *ACM Transactions on Computing Education*, Vol. 17, Issue 2, pp. 9:1–9:22, 2017. DOI: 10.1145/3025013. 9

[36] A. Garrigós, D. Marroquí, J. M. Blanes, R. Gutiérrez, I. Blanquer, and M. Cantó, Designing Arduino electronic shields: Experiences from secondary and university courses, in *Proc. of the IEEE Global Engineering Education Conference (EDUCON)*, Athens, Greece, pp. 934–937, 2017. DOI: 10.1109/educon.2017.7942960. 9

[37] M. Resnick, J. Maloney, A. Monroy-Hernández, N. Rusk, E. Eastmond, K. Brennan, and Y. Kafai, Scratch: Programming for all, *Communications of the ACM*, Vol. 52, Issue 11, pp. 60–67, 2009. DOI: 10.1145/1592761.1592779. 9

[38] C. Vandevelde, J. Saldien, C. Ciocci, and B. Vanderborght, Overview of technologies for building robots in the classroom, in *Proc. of the 4th International Conference on Robotics in Education*, pp. 122–130, Lodz, Poland, 2013. 9

[39] Ardublock. `http://blog.ardublock.com/` 9

[40] S4A. `http://s4a.cat` 9

[41] J. S. He, S. Ji, and P. O. Bobbie, Internet of Things (IoT)-based learning framework to facilitate STEM undergraduate education, in *Proc. of the ACM SouthEast Conference*, pp. 88–94, Georgia, 2017. DOI: 10.1145/3077286.3077321. 9

[42] H. Mäenpää, S. Varjonen, A. Hellas, S. Tarkoma, and T. Männistö, Assessing IoT projects in university education: A framework for problem-based learning, in *Proc. of the 39th International Conference on Software Engineering*, pp. 37–46, Buenos Aires, Argentina, 2017. DOI: 10.1109/icse-seet.2017.6. 9

[43] J. O. Hamblen and G. M. E. van Bekkum, An embedded systems laboratory to support rapid prototyping of robotics and the Internet of Things, *IEEE Transactions on Education*, Vol. 56, Issue 1, pp. 121–128, 2013. DOI: 10.1109/te.2012.2227320. 9

[44] A. Özgür, S. Lemaignan, W. Johal, M. Beltran, M. Briod, L. Pereyre, F. Mondada, and P. Dillenbourg, Cellulo: Versatile handheld robots for education, in *Proc. of the ACM International Conference on Human-robot Interaction (HRI)*, pp. 119–127, Vienna, Austria, 2017. DOI: 10.1145/2909824.3020247. 9

[45] F. Mondada, M. Bonani, F. Riedo, M. Briod, L. Pereyre, P. Retornaz, and S. Magnenat, Bringing robotics to formal education: The thymio open-source hardware robot, *IEEE Robotics and Automation Magazine*, Vol. 24, Issue 1, pp. 77–85, 2017. DOI: 10.1109/mra.2016.2636372. 9

[46] 12Blocks. http://onerobot.org 9

[47] Blockly. http://developers.google.com/blockly 9

[48] PICkit™3 in-circuit debugger/programmer, Microchip Technology Inc., 2013. 11

[49] *MPLAB X IDE User's Guide*, Microchip Technology Inc., 2011–2015. 11

[50] MOD-ZIGBEE-PIR sensor development board, Olimex Ltd., Bulgaria, 2015. 11

[51] F. Leens, An introduction to I²C and SPI protocols, *IEEE Instrumentation and Measurements Magazine*, Vol. 12, Issue 1, pp. 8–13, 2009. DOI: 10.1109/mim.2009.4762946. 61

[52] Atmel AVR4027: Tips and tricks to optimize your C code for 8-bit AVR microcontrollers. Atmel Corporation, 2011. 63

[53] *AVR Instruction Set Manual*, Atmel Corporation, 2016. 64

[54] *PICmicro™ Mid-range MCU Family Reference Manual* (Section 29), Microchip Technology Inc., 1997. 64

[55] Microchip technology to acquire Atmel. http://www.atmel.com/about/news/release.aspx?reference=tcm:26--82057 64

[56] E. W. Dijkstra, Go to statement considered harmful, *Communications of the ACM*, Vol. 11, No. 3, pp. 147–148, 1968. DOI: 10.1007/978-3-642-59412-0_21. 68

[57] Atmel AVR4027: Tips and tricks to optimize your C code for 8-bit AVR microcontrollers, Atmel Corporation, 2011. 71, 72

[58] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed., Prentice Hall, 2006. 78

[59] Termite: A simple RS232 terminal. https://www.compuphase.com/software_termite.htm 83

[60] 8-bit AVR microcontroller with 4/8/16/32K bytes in-system programmable flash—ATmega48PA/88PA/168PA/328P, Atmel Corporation, 2009. 84, 89

[61] Arduino_Uno_Rev3-schematic. https://www.arduino.cc/en/uploads/Main/Arduino_Uno_Rev3-schematic.pdf 84, 89

[62] 8-bit AVR microcontroller with 8/16/32K bytes ISP flash and USB controller—ATmega8U2/16U2/32U2, Atmel Corporation, 2010. 86

[63] A. K. Rachioti, D. E. Bolanakis, and E. Glavas, Teaching strategies for the development of adaptable (compiler, vendor/processor independent) embedded C code, in *Proc. of the 15th IEEE International Conference on Information Technology Based Higher Education and Training (ITHET)*, pp. 1–7, Istanbul, Turkey, 2016. DOI: 10.1109/ithet.2016.7760726. 88

[64] R. N. Carney and J. R. Levin, Pictorial illustrations still improve students' learning from text, *Educational Psychology Review*, Vol. 14, No. 1, pp. 5–26, 2002. 88

[65] R. E. Mayer and J. K. Gallini, When is an illustration worth ten thousand words? *Journal of Educational Psychology*, Vol. 82, No. 4, pp. 715–726, 1990. DOI: 10.1037/0022-0663.82.4.715. 88

[66] D. E. Bolanakis, E. Glavas, G. A. Evangelakis, K. T. Kotsis, and T. Laopoulos, Documenting knowledge to the undergraduate education of professional engineers: A case study in microcontroller education, in *Proc. of the 40th Annual Conference of the European Society for Engineering Education (SEFI)*, pp. 1–7, Thessaloniki, Greece, 2012. 88

[67] D. E. Bolanakis, E. Glavas, and G. A. Evangelakis, An integrated microcontroller-based tutoring system for computer architecture laboratory course, *International Journal of Engineering Education*, Vol. 23, No. 4, pp. 785–798, 2007. 88

[68] D. E. Bolanakis, E. Glavas, G. A. Evangelakis, K. T. Kotsis, and T. Laopoulos, *Microcomputer Architecture: Low-level Programming Methods and Applications of the M68HC908GP32*, Self-publishing, Createspace, 2012. 88

[69] PIC18F87J50 FamilyData sheet: 64/80-Pin high-performance, 1-Mbit flash USB microcontrollers with nanoWatt technology, Microchip Technology Incorporated, 2007. 89

[70] Introduction to clicker 2 for PIC18FJ. `https://www.mikroe.com/` 89

[71] Arduino Uno click shield—schematics. `https://www.mikroe.com/` 89

[72] Click boards. `https://shop.mikroe.com/click/` 89

[73] About Notepad++. `https://notepad-plus-plus.org/` 90

[74] MC9S08GB60A Rev.2 data sheet (HCS08 microcontrollers). Freescale Semiconductor Inc. (NPX delivery), 2008. 96

[75] AVR Libc home page. `http://www.nongnu.org/avr-libc/` 99

[76] mikroC PRO for PIC. https://www.mikroe.com/mikroc/#pic 99

[77] PCH command-line C compiler for microchip PIC18 devices. http://www.ccsinfo.com/product_info.php?products_id=PCH_full 99

[78] MAX220-MAX249 +5V-Powered, multichannel RS-232 drivers/receivers, Maxim Integrated Products Inc., 2015. 110

[79] FT2232H dual high speed USB to multipurpose UART/FIFO IC, Future Technology Devices International Limited, 2016. 110

[80] LPS25HB MEMS pressure sensor: 260–1260 hPa absolute digital output barometer, STMicroelectronics, 2015. 119, 124

[81] D. E. Bolanakis, *MEMS Barometers Towards Vertical Position Detection: Background Theory, System Prototyping and Measurement Analysis*. Morgan & Claypool Publishers, 2017. DOI: 10.2200/s00769ed1v01y201704mec003. 126

[82] AN4450 application note: Hardware and software guidelines for use of LPS25H pressure sensor, STMicroelectronics, 2016. 136

[83] Serial communications in Win32. https://msdn.microsoft.com/en-us/library/ms810467.aspx 136

Author's Biography

DIMOSTHENIS E. BOLANAKIS

Dimosthenis E. Bolanakis was born in Crete, Greece (1978) and graduated with a degree in Electronic Engineering (2001) from ATEI Thessalonikis. He received an M.Sc. (2004) in Modern Electronic Technologies and a Ph.D. (2016) in Education Sciences (focusing on Remote Experimentation), both from University of Ioannina. He has (co)authored more than 30 papers (mainly on Research in Engineering Education) and he is the author of *MEMS Barometers toward Vertical Position Detection: Background Theory, System Prototyping, and Measurement Analysis* (Morgan & Claypool, May 2017), and co-author of *Microcomputer Architecture: Low-level Programming Methods and Applications of the M68HC908GP32* (Createspace/Self-publishing, 2012). During the period 2012–2014 he joined European System Sensors S.A. (`http://www.esenssys.com`) corporation that specialized in the design of MEMS sensors. Since 2014 he has been occupied as *Special Lab & Teaching Personnel* at Hellenic Air Force Academy (Informatics & Computers section). His design/research interests include: μC-based & FPGA-based digital hardware design; MEMS sensors system-level design and measurement analysis (using Matlab script code); C/LabVIEW software co-development for the hardware interfacing; and research in engineering education.

He currently lives in Athens (Greece) together with his wonderful wife, Katerina, and their two delightful kids, Manolis and Eugenia.